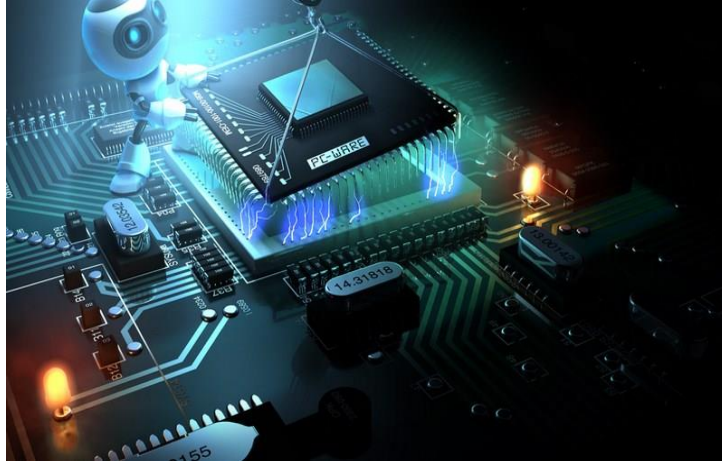


ARQUITECTURA DE COMPUTADORAS



VIDEOS

Del profe Kefalidis 2020 - [Arquitectura 2020 de Kefalidis](#)

Del profe Kefalidis 2021 - [Arquitectura 2021 Youtube](#)

De snoopy 2020 - [Videos de Snoopy Arquitectura 2020](#)

ÍNDICE

ÍNDICE	1
Introducción a la Arquitectura de las Computadoras	7
GENERACIONES DE COMPUTADORAS	8
PARTE 2: Representación de la información	13
REPASO SISTEMAS NUMERACION	14
Operaciones con números binarios. Complementos. Aritmética digital. Concepto de aritmética de punto flotante y fijo.	15
FLUJO DE LA INFORMACIÓN EN EL COMPUTADOR	16
FORMATO DE NÚMEROS EN LAS MÁQUINAS	17
La Coma Fija	17
La Coma Flotante	20
CODIFICACIÓN DE LA INFORMACIÓN	21
CONCEPTO	21
REPRESENTACIÓN LÓGICA DE LA INFORMACIÓN EN LA COMPUTADORA (CONCEPTUALMENTE)	23
- Codificación de la información numérica y alfabética: Código binario puro, código BCD, otros códigos numéricos, códigos alfanuméricos. Códigos redundantes: concepto, tipos.	24
CODIFICACIÓN DE LA INFORMACIÓN EN LA MÁQUINA	24
CODIFICACIÓN INFORMACIÓN NUMÉRICA	24
Códigos Ponderados	25
Códigos No Ponderados	26
CODIFICACIÓN DE LA INFORMACIÓN NO NUMÉRICA	28
Agrupamientos binarios: bit, byte, palabra.	29
SISTEMAS ALFANUMÉRICOS	30
CODIGOS REDUNDANTES	33
Códigos Autodetectores	34
Códigos Autocorrectores	35
Hamming	36
Circuitos Digitales / Combinacionales	40
REPASO DE MAT SUPERIOR	40
Formas Normales	42
Sistemas digitales combinacionales	43
Circuitos combinacionales	43
Circuitos secuenciales	44
AUTOMATA FINITO	45
Funciones de transición	46
DIAGRAMA DE TRANSICIÓN	46
BIESTABLES FLIP-FLOP	48
BIESTABLE RS	49
BIESTABLE RS - ASÍNCRONO (LATCH)	49
RS Síncrono (temporizado)	52
FLIP-FLOP D (Data)	52
FLIP FLOP JK	54
FLIP FLOP T (Toggle o Trigger)	55

MONOESTABLE	55
LA BÁSCULA	56
Biestable Maestro Esclavo	57
Anotaciones Extra	59
Circuito Integrado CI555 para funcionar como Timer	59
Consideraciones de diseño para los circuitos secuenciales	59
Ejemplo ejercicio ascensor:	60
BUSES Y REGISTROS	62
Registros	62
TIPOS DE REGISTROS	62
OPERACIONES ELEMENTALES DE REGISTROS	64
BUSES	65
Registros especiales	67
ABACUS	69
MEMORIA	69
REGISTRO DE INSTRUCCIÓN	69
REGISTRO ACUMULADOR	70
CONCEPTOS	71
Explicación de los registros y sus tamaños	71
SEÑALES DE GOBIERNO DE ABACUS	72
UAL - Unidad Aritmético Lógica	75
Operadores combinacionales y secuenciales	78
Operador combinacional	78
Operadores con acumulador (secuencial)	78
Operaciones Lógicas	80
El semisumador	81
SUMADOR COMPLETO	82
SUMADOR EN SERIE - SECUENCIAL	84
SUSTRACTOR	85
SUMADOR PARALELO	86
Lógica de Gobierno de los desplazamientos	87
Unidad Aritmetico-Logica para ABACUS	89
SUM	90
SUS	91
ORX/XOR	92
Números Binarios con Signo	93
Sobrecapacidad o desbordamiento	95
Ejemplo: Para detectar Carry	96
Gráfico de Morris Mano	99
ALU DE UNA DIRECCIÓN SEGÚN MEINADIER	99
Coma flotante	100
Primera fase: alinear las mantisas	101
Segunda fase: ajustar signos	102
Tercera fase: normalización del resultado	103

Funcionamiento de un operador de suma flotante	103
Algoritmo:	104
Multiplicación secuencial por suma - desplazamiento en ABACUS	105
Mejoras en la multiplicación	107
Multiplicador celular paralelo	107
División en ABACUS	110
DIVISIÓN CON RESTAURACIÓN	111
División sin restauración	112
Consideraciones finales de ALU	112
MEMORIAS	113
CLASIFICACION DE MEMORIAS	115
JERARQUÍA DE MEMORIAS	117
Niveles	117
NIVEL 0 - Registros del procesador	118
NIVEL 1 - Memorias caché	118
NIVEL 2 - MEMORIA RAM	121
NIVEL 3 - MEMORIA FLASH	123
NIVEL 4 - Discos magnéticos	123
NIVEL 4 - Discos SSD	124
NIVEL 5 - Discos ópticos	124
NIVEL 6 - Cintas Magnéticas	125
Evolución de la memoria RAM	125
Memoria de Núcleo de Ferrita	125
LECTURA NÚCLEO DE FERRITA	127
ESCRITURA NÚCLEO DE FERRITA	128
DESCRIPCIÓN DE LAS SOLUCIONES CON ANILLOS DE FERRITA	129
Implementación 2D	129
Escritura 2D	130
Lectura 2D	131
Implementación 3D	132
DIFERENCIA MEMORIAS ESTÁTICAS Y DINÁMICAS	134
Organizaciones de memoria basadas en semiconductores (Biestables)	137
MEMORIA 2D	137
VENTAJAS	141
DESVENTAJAS	141
MEMORIA 3D	141
VENTAJAS	144
DESVENTAJAS	144
MEMORIA 2 Y 1/2 D	145
VENTAJAS	148
DESVENTAJAS	148
MÉTODOS DE DIRECCIONAMIENTO	148
1.Direcccionamiento Normal (Directo y Absoluto)	149
2.Direcccionamiento Inmediato	150

3.Direccionamiento Indirecto	151
4.Direccionamiento por Base y Desplazamiento	154
5.Direccionamiento por Referencia al Programa	155
6.Direccionamiento por Página (o por Yuxtaposición)	156
7.Direccionamiento Indexado	157
UNIDAD DE CONTROL	160
Concepto	160
ABACUS	162
RUTA DE DATOS DE ABACUS	162
ESQUEMA DE VON NEUMANN GENERAL	162
DIMENSIÓN DE LA RUTA DE DATOS	163
SUMA EN ABACUS COMPLETO	164
Fase 1	165
Fase 2	167
Fase 3	169
Fase 4	170
CRONOGRAMA COMPLETO	171
Almacenamiento en abacus	173
Salto incondicional	175
RESUMEN SEÑALES GOBIERNO ABACUS	177
Registros de la ruta de datos y su interconexión	177
ORGANIZACIÓN DE LOS REGISTROS DE LA RUTA DE DATOS	177
Tipos de interconexiones	179
EL SECUENCIADOR	180
El secuenciamiento de las instrucciones	181
Diseño y construcción del órgano secuenciador	181
Ejemplo para la suma	181
Modelos de secuenciadores	182
Secuenciador cableado	183
SECUENCIADOR SÍNCRONO	183
SECUENCIADOR ASÍNCRONO	184
Tratamiento de instrucciones	186
DISTRIBUIDOR DE FASES	188
Decodificación de la instrucción	190
Biestable de estado	191
Esquema general de Abacus	192
Secuenciador de 2 FASES	193
Trazado de Cronogramas	194
Cronograma	194
Ecuaciones lógicas	198
Ejemplo SUM	198
Lectura	199
Salida del registro M (palabra memoria) al bus M	199
SECUENCIADOR MICROPROGRAMADO	204

Codificación de las microinstrucciones	207
Técnicas de codificación en MICROPROGRAMADOS	207
CODIFICACIÓN POR CAMPO	208
CODIFICACIÓN POR TIPO DE INSTRUCCIÓN	209
TIPOS DE DIRECCIONAMIENTO DE LAS MICROINSTRUCCIONES	210
DIRECCIONAMIENTO SECUENCIAL	210
DIRECCIONAMIENTO EXPLÍCITO	211
Acompasamiento	212
MICROPROGRAMACIÓN HORIZONTAL Y RITMO FIJO	212
MICROPROGRAMACIÓN RITMO VARIABLE	213
CISC y RISC	213
CISC - Computador con Set de Instrucciones Complejas	214
RISC - Reduced instruction set computing	215
Comparación	218
Conclusiones	220
LENGUAJE DE MÁQUINA	221
Definición del profe	221
Elementos informativos de una instrucción	222
Clasificación de las arquitecturas	222
Conjunto de instrucciones de un microprocesador	223
Nivel de microprogramación según la arquitectura	223
Tipos de instrucciones - Formato (Convención)	224
Canales	227
Modos - Simultaneidad de los procesos de E/S	229
1.Modos bloqueado → técnica muy rudimentaria	229
2.Modos por prueba de estado	230
3.Modos por interrupción de programa	231
4. Modos automáticos, por suspensión de programa o MODO CANAL	231
5.Modos de encadenamiento automático de las transferencias	233
Interfaz	234
Concentración por multiplexaje	237
Concentración por línea omnibus	238
Técnicas de transferencia	239
Transferencia programada	240
Transferencia por instrucción forzada	241
Transferencia por robo de ciclo	243
Transferencia por acceso directo a memoria (DMA)	245
Comparación de las técnicas	246
Enlace programado en Abacus	247
TIPOS DE OPERACIONES 4 (CUATRO)	248
Canal automático para abacus	250
Encadenamiento automático	254
Noción de programa canal	256
Programa canal	256

Dispositivos de Entrada Salida - PERIFÉRICOS	261
UNIDADES DE ENTRADA	261
TECLADO	261
Tipos de teclado	262
RATÓN	263
UNIDADES DE SALIDA	264
MONITORES	264
TIPOS DE MONITORES	264
IMPRESORAS	265
TIPOS DE IMPRESORAS	266
UNIDADES DE MEMORIA MASIVA AUXILIAR	268
1. Discos Magnéticos	269
2. Discos Estado Solido (SSD)	270
Arquitectura 8086	272
Terminologías	272
Historia de Intel y sus Procesadores 8086/88	273
Arquitectura en modo de bloques 8086	273
Unidad de interfaz c/bus	274
Unidad de ejecución	275
Ciclo fetch o de búsqueda	275
Registros de propósito General	277
Registros de Direcciones de Datos (Apuntadores)	279
Registros Banderas (Flags)	281
Registros de direcciones de segmentos	282
Modos de direccionamiento en 8086	285
Resumen de los Modos de Direccionamiento	289

Introducción a la Arquitectura de las Computadoras

Computadora

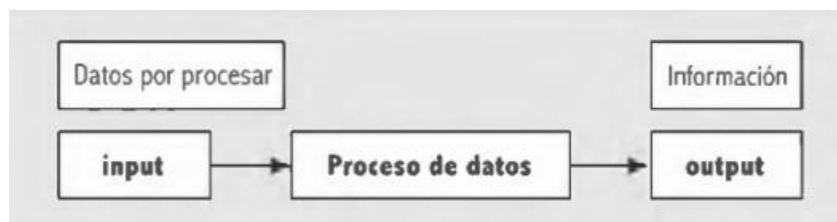
Máquina digital electrónica programable para el tratamiento automático de la información, capaz de recibirla, operar sobre ella mediante procesos determinados y suministrar los resultados de tales operaciones

Arquitectura de computadoras

La manera lógica de organizar y distribuir los componentes de un computador con el objeto de lograr un procesamiento óptimo.

- Concepto de sistema de Información y procesamiento de datos.

SISTEMA DE INFORMACIÓN: Conjunto coordinado de **elementos**, **datos** y **procesos**, cuya interacción permite la obtención de la **información** (la información son datos procesados con significado en un contexto real, es decir no son elementos aislados, que permite al usuario la toma de decisiones).



Evolución del PROCESAMIENTO DE DATOS:

Procesar datos: realizar alguna operación sobre ellos. Por lo tanto, los "datos de entrada al proceso" (input) se transformarán en "resultados" (output), en general, luego de que alguna de estas operaciones los afecte:

- Cálculo.
- Comparación.
- Clasificación (ordenamientos, intercalaciones).
- Transformación (por ejemplo, de datos numéricos).
- Transferencia.
- Almacenamiento (para usarlos con ulterioridad).
- Consulta.



El conjunto de los resultados constituye **información**, ya que los datos elaborados o procesados están incluidos en un contexto válido para quien los interpreta y no son elementos aislados. Esto implica que el objetivo del **procesamiento de datos** es obtener **información**.

- Computadoras digitales y sistemas digitales:

Una computadora está constituida por una memoria, que almacena programas y datos en forma temporal, y un órgano ejecutor de estos programas conocido como unidad central de proceso (UCP).

Tanto las instrucciones de un programa como los datos que ingresan en la memoria desde el exterior lo hacen en un código alfanumérico de representación de caracteres.

COMPUTADORA DIGITAL	SISTEMA DIGITAL
→ Es un sistema digital, también denominado “sistema de procesamiento de información discreta”. ❖ Es una interconexión de módulos digitales. ❖ Propiedad más importante: GENERALIDAD. ❖ El usuario puede determinar y cambiar los programas y datos de acuerdo a una necesidad específica. ❖ Pueden realizar una amplia variedad de tareas. El computador digital opera con pasos discretos controlados por las señales de tiempo.	Es un sistema lógico secuencial construido con flip-flops y compuertas. <u>El computador digital de uso general</u> es el ejemplo más conocido de sistema digital. (voltímetros digitales, máquinas calculadoras, etc). Típico de un sistema digital es su manejo de elementos discretos de información. Ejemplo: Un módulo digital típico sería la unidad procesadora de un computador digital.

GENERACIONES DE COMPUTADORAS

RECORDATORIO: SON 5 (CINCO)

Generación	Años	Características	Resuelven
0 <i>Mecánico</i>	hasta 1945	Sistemas mecánicos y electromecánicos	No siguen instrucciones (ábacos, calculadoras mecánicas, etc.)
1 <i>Pseudo-mecánico</i>	1945 - 1955	Tubos al vacío, tableros	Una sola tarea
2 <i>Transistores</i>	1955 - 1965	Transistores y sistemas por lotes	Varias tareas
3 <i>Arranca integrado</i>	1965 - 1980	Circuitos integrados y multiprogramación	Varias tareas

4 Microtecno	desde 1980	Microprocesadores y Computadores personales	Varias tareas
--------------	------------	---	---------------

Características generales

Generación 0 (hasta 1945)

No siguen instrucciones. Sistemas basados en relés.
Ejemplos: Calculadoras mecánicas

Primera generación (1945 - 1955) Pseudomecánico

Las computadoras estaban constituidas de **válvulas de vacío**, que disipaban gran cantidad de calor y ocupaban una amplia superficie. Ejecutaban algunos **miles de instrucciones por segundo (X1000)**.

Manchester Mark I ya utiliza la arquitectura de John Von Neuman.

Almacenaban de **10 a 20 Kbytes** en su memoria.

Un solo grupo diseñaba, construía, programaba, operaba y mantenía cada máquina.

Toda la programación se hacía en lenguaje de máquina (alambrado tableros por ejemplo)

Valvulas como punto de memoria.

Los medios de entrada/salida eran tarjetas y cintas perforadas.

La memoria principal tenía un tiempo de acceso muy grande en comparación con los tiempos de procesamiento de la **UCP (Unidad Central de Procesamiento)**.

Las tareas se ejecutaban de manera **secuencial UNA TRAS OTRA**.



Características Primera Generación

- Amplia superficie
- Miles de instrucciones por segundo
- 10 a 20 KB en su memoria
- Entradas/Salidas tarjetas perforadas
- Memoria principal tenía un tiempo de acceso muy grande en comparación con la UCP
- Tareas secuencial

Segunda generación (1955 - 1965) Transistores

Las computadoras de la 2da generación estaban constituidas por **transistores**, más confiables, de menor tamaño, y menor disipación de calor y más rápidos que las válvulas de vacío para **cambiar de estado**. Se pasan a alcanzar velocidades de procesamiento de **centenares de miles de instrucciones por segundo (X100.000)**.

Ya empiezan a aparecer grupos de programadores, encargados de la parte de hardware.

Mainframes en salas acondicionadas

Posibilitaron la **simultaneidad** entre un **cálculo** y una **operación de E/S**, aunque el encadenamiento de los trabajos seguía siendo **secuencial**. Las operaciones de E/S se realizaban usando como soporte de almacenamiento **unidades de cinta magnética**, más rápidas que las lectoras de tarjetas y las impresoras.

Concepto de **procesamiento por lotes**, ya que era necesario esperar a que un lote de trabajos cargados en la cinta magnética se procese **por completo** para obtener los resultados de cada trabajo.



El transistor viene a funcionar como compuerta lógica.

Características Segunda Generación

- Constituida por transistores
- Velocidades de centenares de miles de instrucciones x segundo
- Surge el concepto de “procesamiento por lotes”, era necesario esperar a que un lote de trabajos cargados en la cinta magnética se procese por completo para obtener un resultado de trabajo.
- Simultaneidad entre cálculo y operaciones de E/S PERO el encadenamiento de los trabajos seguía siendo secuencial.
- Operaciones E/S se realizaban usando como soporte de almacenamiento unidades de Cinta Magnética
- Aparecen lenguajes de Alto Nivel como Cobol y Fortran (1959)

Tercera generación (1965 - 1980) Integrados

Surge con la aparición de la tecnología de **circuitos integrados**, osea miles de componentes electrónicos incluidos en una sola **pastilla de silicio**. Estos circuitos eran de **pequeña y**

mediana escala de integración (SSI y MSI), y permitieron el incremento de la velocidad interna de la computadora y la reducción de energía que usaban.

Ejemplo de IBM 360 altamente compatible.



Características de Tercera Generación

- Circuitos integrados, miles de componentes electrónicos incluidos en una sola pastilla de silicio.
- Estos circuitos eran de pequeña y mediana escala de integración (SSI) y (MSI), permitían incrementar la velocidad interna de la computadora y reducción de energía.
- Permiten explotar eficazmente las **simultaneidades** latentes mediante la **multiprogramación**: Varios programas residen en forma simultánea en memoria en estado de ejecución. En un instante solo uno de ellos usa la CPU, pero los otros pueden efectuar operaciones de E/S en forma simultánea. Esto significa, cuando el programa que ejecuta la CPU se detiene en espera de una operación de E/S, otro programa puede ocupar su lugar dejando al primero suspendido y evitando así tiempos inactivos de CPU.
- **La carga por lotes de los trabajos fue sustituida por la “carga continua”**: A medida que llegan son puestos en una cola de espera en un disco magnético y el **sistema operativo** es el que se encarga de cargarlos en memoria para ejecutarlos según su nivel de prioridad. Los resultados son transferidos luego al disco, hasta que son extraídos por la impresora. No tenemos que esperar a que se procese todo un lote de programas para obtener resultados.
- **Teleprocesamiento, trabajar a distancia**. Los trabajos de los terminales remotos quedan incluidos en una cola de espera con prioridades, exactamente igual que aquellos que se cargan localmente.
- **Usuario puede intervenir en el procesamiento, es interactivo**. El diálogo se gestiona usando un terminal con menús o cuestionarios que el sistema realiza preguntas y el usuario responde. Trajo mayor flexibilidad en cuanto a la puesta a punto o en la modificación de programas. A fin de atender un gran número de procesos o usuarios, las computadoras pasaron a asignar en forma alternada una parte del tiempo

de CPU a cada uno, lo que producía un efecto de aparente simultaneidad de ejecución.

- El concepto de **máquina virtual**, que simplifica la labor del programador, ya que éste no conoce más que una máquina ficticia creada y controlada por el sistema operativo, que no presenta limitaciones de configuración.
- Se introducen los **discos duros**.

Cuarta generación (1980 - 2010)

Las tecnologías de circuitos integrados pasan a ser **LSI (Large Scale Integration)** con una mayor miniaturización y más circuitos dentro del chip, lo que permitió incluir una CPU completa que se denominó **microprocesador**.



Características de Cuarta Generación

- Comienzan a existir las PC hogareñas, con una nueva gama de periféricos para permitir la conectividad entre ellas.
- Se introduce la GUI (Graphical User Interface) - Macintosh
- El **procesamiento** se realiza en mayor medida en **tiempo real**.
- Se introducen los **sistemas interactivos**, se hace posible la consulta y actualización de datos. Nuevas utilidades para mejorar la interacción usuario-máquina: enseñanza asistida por computadora, consultas telefónicas por voz, etc.
- **Scope enfocado al hardware cambia hacia el software** ya que los avances en el hardware parecían agotados.
- Aparece la arquitectura de chip RISC
- Sistemas operativos en red y sistemas operativos distribuidos
- ARM compiten contra los Core de Intel (ordenadores portátiles)

QUINTA GENERACIÓN - 2010 - HOY

Computación cuántica

Tiene sus principios fundamentales en la mecánica cuántica, conocido como **principio de superposición de estados**.

La unidad de información es el **cubit, qubit o bit cuántico**.

Tienen compuertas lógicas cuánticas para llevar a cabo operaciones lógicas cuánticas.

Es necesario mantener los dispositivos cercanos al cero absoluto.



PARTE 2: Representación de la información

Contenido

- Sistemas numéricos
 - Sistemas de numeración y cambio de base
 - Aritmética binaria
 - Sistemas de codificación y representación de los números
- Codificación binaria
 - Representación binario de datos e instrucciones
 - Características de los espacios de representación
 - Aspectos de los sistemas de representación
- Sistemas alfanuméricos
 - Características de los códigos
 - Principales sistemas de codificación
- Códigos redundantes
 - Características de los códigos
 - Códigos detectores
 - Códigos correctores

REPASO SISTEMAS NUMERACION

SISTEMAS DE NUMERACIÓN

Un sistema de numeración es un **sistema de escritura para expresar números**. Es la **notación matemática** para representar números de un determinado conjunto u otros símbolos de manera consistente. Proporciona una **representación** algebraica y aritmética.

Para empezar conviene tener presente que, en cualquier sistema de representación, un mismo conjunto de símbolos puede tener distintos significados según la convención que se siga.

Un sistema de numeración en base **b** utiliza para representar los números de un alfabeto compuesto por **b** símbolos.

Ejemplo: Los diez símbolos decimales pueden usarse para distintas convenciones de representación 170527 puede significar:

- El número decimal ciento setenta mil quinientos veintisiete.
- La fecha de nacimiento 17 de mayo de 1927
- 17 horas 05'27", etc.

Este multisignificado también existe para las representaciones binarias numéricas que se dan dentro de las máquinas.

Ejemplo: una combinación binaria como 10000110, puede significar:

- El número decimal equivalente 134 si corresponde a un número natural
- El número decimal equivalente 86 si es un número BCD
- El número que identifica una celda de memoria, etc.

Números con precisión finita: Otra consideración a tener en cuenta en la **representación y operaciones de números en una máquina** ya que está relacionada que los mismos solo pueden constar de una cantidad fija de dígitos físicamente.

Módulo: es el **número máximo de enteros diferentes** que pueden representarse en un registro físico. Es útil para poner formalmente de manifiesto la **capacidad limitada de representación** de los dispositivos físicos usados.

Naturaleza Posicional del Sistema Decimal

La denominación “decimal” o de “base diez”, se refiere a los diez dígitos o símbolos (0 a 9) que combinados permiten simbolizar los números, conforme a una convención que atribuye un valor individual y otro posicional a cada símbolo.

Cada dígito indica cuántos subconjuntos (0,1,...,9) que agrupan igual número de elementos (uno, diez, cien, mil, ...) se han utilizado para formar un número decimal. Existen tantos tipos distintos de subconjuntos como dígitos contenga un número.

Por ejemplo,

Los símbolos 107 implican que el número de elementos representados está formado por un subconjunto que agrupa cien, mas siete subconjuntos que solo contienen un elemento cada uno, no existiendo ningún subconjunto de diez elementos.

$$107 \Rightarrow 1.100 + 0.10 + 7.1 \Rightarrow 1.10^2 + 0.10^1 + 7.10^0$$

Características de un Sistema Numérico Posicional:

- Consta de un número finito de símbolos distintos, número que define la “base” o “raíz” de cada sistema.
- Cada símbolo aislado representa un número específico de unidades.
- Existe un símbolo (cero) para indicar la ausencia de elementos a representar.
- Los símbolos pueden ordenarse en forma monótona creciente.
- Formando parte de un número compuesto por varios símbolos, un mismo símbolo **tiene una significación o “peso” distinto según su posición.**
- La posición extrema derecha corresponde a unidades (peso uno), a partir de ella, cada posición tiene el peso de la que está a su derecha multiplicada por la base.

Operaciones con números binarios. Complementos. Aritmética digital. Concepto de aritmética de punto flotante y fijo.

Naturaleza Posicional del Sistema Binario

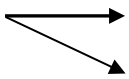
Con los dos símbolos: 1 y 0, se puede representar cualquier número, resultando así un sistema numérico en *base dos* o *binario*.

Cada uno de los símbolos que forman un número binario se llama “dígito binario” o BIT (de “binary digit”).

$$1101011 = 1.2^0 + 1.2^1 + 0.2^2 + 1.2^3 + 0.2^4 + 1.2^5 + 1.2^6 = 107$$

Luego $1101011_b = 107_d$ (“b” binario y “d” decimal)

Base: Determina la **cantidad de símbolos distintos para representar los números**, “los símbolos son finitos pero los números no”.

Complemento 

A la base (en binario complemento a 2)

A la base -1 (En binario complemento a 1)

Complemento a la base: Es lo que le falta al número para llegar a la base, es decir, la base – el número, el 1 seguido de tantos ceros como dígitos tenga el número. Potencia de la base inmediatamente superior al número.

Complemento a la base -1: se define como el complemento a la base, pero restando 1, es decir, se obtiene restando 1 a la base. En el complemento a 1 los unos se cambian por ceros, y los ceros por unos.

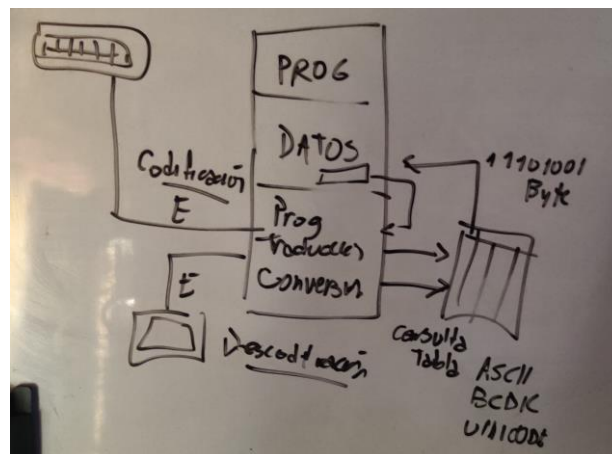
Tipos de Datos: Las computadoras digitales manejan 4 tipos básicos de datos.

- Números enteros con signo (representación numérica).
- Números reales con signo (representación numérica).
- Número de decimales codificados en binario BCD (representación numérica).
- Caracteres (representación alfanumérica).

El número de bits que se usan para representar cada uno de estos tipos de datos y el **significado de cada uno varía** según el tipo de dato a representar y el sistema computacional que se utilice (HW y SW).

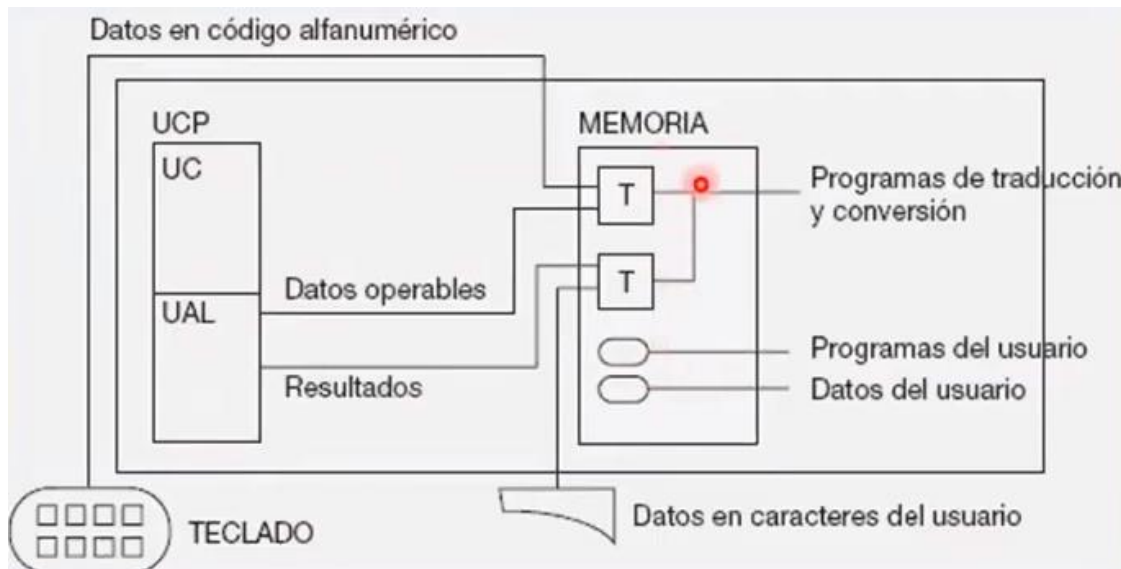
FLUJO DE LA INFORMACIÓN EN EL COMPUTADOR

Un **proceso de comunicación** puede tener lugar y diversas formas e involucra el movimiento de información a través de un canal definido en el sistema.



Codificación
Decodificación

Teclado → Programa
Programa → Pantalla



Viene una instrucción por parte del teclado, y entra en la sección de “Programas dedicados a la Traducción y Conversión” (puede ser un Compilador, Traductor o Ensamblador) a donde se busca su coincidencia con una tabla de caracteres que puede ser ASCII, EBCDIC, UNICODE, etc y lo **CODIFICA** a **CODIGO DE MAQUINA (Byte)** que es lo que entiende el procesador para luego ejecutarse, para buscar en el “sector de Datos”.

A la inversa cuando luego de encontrado el Dato, pasa por el sector de traducción, se utiliza la tabla y a la salida sale en éste caso del ejemplo, la letra “E” aparecería en la pantalla del Ordenador.

FORMATO DE NÚMEROS EN LAS MÁQUINAS

Un operador aritmético no puede procesar más que números que le sean presentados según uno o unos *formatos* bien definidos.

Componentes del Formato:

- **Dimensión:** Las máquinas de palabra, que trabajan generalmente en sistema binario, admiten formato de *longitud fija*, según que la información ocupe una palabra, media palabra, dos palabras, etc., se hablará de *longitud simple*, *media longitud*, *doble longitud*, etc. Las máquinas de carácter, que operan en sistema decimal, trabajan generalmente con formatos de *longitud variable*, bajo los cuales un número va codificado con un número variable de caracteres.
- **Formato:** analizaremos tres tipos de formato, el *formato fijo* o formato en coma fija y el *formato flotante* o formato en coma flotante, para los números binarios, por otro lado, los formatos de longitud variable, para los números decimales.

La Coma Fija

Es la manera más natural de escribir un número binario en una palabra de memoria: Los números se consideran como enteros y se le deja al programador el cuidado de situar la coma. Pueden representarse los números negativos de acuerdo con uno de los tres convenios.

1. Signo-magnitud
2. Complemento a 1
3. Complemento a 2

La forma más simple de representación que emplea un bit de signo es la representación **signo-magnitud**. En una palabra de N bits, los N-1 bit menos significativos almacenan la magnitud del entero; mientras que el bit más significativo indica el signo del número, siendo 0 la representación positiva y 1 la negativa.

$$00010010 = +18$$

$$10010010 = -18$$

Ahora, una palabra de 8 bits puede representar valores en un rango -127 a +127.

Desventajas:

- La adición y la sustracción requieren considerar ambos signos y sus magnitudes relativas para poder llevar a cabo la operación.
- Existen 2 representaciones para el cero.

$$00000000 = 0_{10}$$

$$10000000 = -0_{10}$$

Hay otras dos representaciones que utilizan el bit más significativo para el signo. Difieren una de la otra y de la representación *signo-magnitud*, en la manera que los otros bits se interpretan. Estas representaciones se conocen como *complemento a 1* y *complemento a 2*. Ambas permiten algoritmos más eficientes para la adición y sustracción. La representación de *complemento a 2*, además, tiene una sola representación para el cero.

Para llevar a cabo la operación de **complemento a 1** sobre el conjunto de dígitos binarios solo hay que reemplazar los dígitos 0 con dígitos 1 y los dígitos 1 con dígitos 0.

Así:

$$X = 01010001$$

$$\text{Complemento a 1 de } X = 10101110$$

Ejemplo Resta como Suma de complemento

$$A = 1001$$

$$B = 0100$$

$$!B = 1011$$

$$A - B = 101$$

$$A + !B =$$

$$\begin{array}{r}
 1001 \\
 + \quad 1011 \\
 1|0100 \\
 + \quad 1 \\
 \hline
 101
 \end{array}$$

La **representación en complemento a 1** de los dígitos binarios es como sigue. Los enteros positivos se representan de la misma manera como en la representación *signo-magnitud*. Un entero negativo está representado por el *complemento a 1* de un entero positivo con la misma magnitud.

Por ejemplo:

$$\begin{aligned}
 18 &= 00010010 \\
 -18 &= \text{Complemento a 1 de } 18 = 11101101
 \end{aligned}$$

Puesto que todos los enteros positivos tienen el bit más significativo igual a 0, todos los números negativos tienen necesariamente el bit más significativo igual a 1. De esta manera, el bit de más a la izquierda sigue funcionando como bit de signo.

La operación de **Complemento a 2** consiste en dos pasos:

1. Llevar a cabo la operación de *complemento a 1*.
2. Tratar el resultado como un entero binario sin signo, sumarle 1.

La representación en **complemento a 2** de enteros positivos es la misma como en las representaciones de *signo-magnitud* y *complemento a 1*. Un número negativo es representado por el complemento del entero positivo con la misma magnitud. Por ejemplo:

$$\begin{aligned}
 18 &= 00010010 \\
 \text{Complemento a 1} &= 11101101 \\
 &\quad + \quad 1 \\
 -18 &= \text{Complemento a 2 de } 18 = 11101110
 \end{aligned}$$

Esta representación tiene una anomalía no encontrada con *signo-magnitud* o *Complemento a 1*. El patrón de bits 1 seguido por N-1 ceros es su propio *complemento a 2*. Para mantener consistencia en el bit de signo, a este patrón de bits se le asigna el valor -2^N .

Por ejemplo, para las palabras de 8 bits:

$$\begin{aligned}
 -128 &= 10000000 \\
 \text{Complemento a 1} &= 01111111 \\
 &\quad + \quad 1 \\
 10000000 &= -128
 \end{aligned}$$

Con esta interpretación, todos los enteros positivos tienen el bit más a la izquierda en 0 y todos los negativos tienen el bit más a la izquierda en 1. De esta manera el bit más significativo sigue funcionando como bit de signo.

En las representaciones de *Complemento a 1* y *Complemento a 2* encontramos que el negativo del negativo de ese número es el mismo ($-(-18) = 18$)

Si se representaran números binarios sin signo, para n bits se obtendrían 2^n combinaciones; vale aclarar que todas estas combinaciones son positivas. Ahora bien, cuando se representan números con signo, la mitad del rango corresponde a números positivos y la otra mitad a números negativos (signo-magnitud y complemento a la base menos 1), es decir, desde $-2^{n-1} + 1$ hasta $2^{n-1} - 1$; para el caso de complemento a la base es necesario añadir un elemento más a la parte negativa, ya que existe una sola representación para el 0.

En las operaciones en coma fija se dan algunas peculiaridades. La suma de dos números fijos puede ocasionar lo que se llama *desbordamiento o sobrepasamiento de capacidad*.

La **multiplicación de dos números de n cifras da un número de $2n$ cifras**, como máximo, que será representado en coma fija de doble longitud.

	0	1	1	0	1	0
+	0	1	0	0	1	0
1	0	0	1	1	0	0

↗ signo
 ↘ desbordamiento
 0

Características de la representación en coma fija:

- *Rango*: se determina por la diferencia entre el mayor y el menor elemento.
- *Precisión*: es la distancia entre dos números consecutivos en una serie numérica.
- *Error*: se obtiene dividiendo la precisión por 2.

La Coma Flotante

236×10^3 - El 3 (exponente) indica la posición de la coma

Acumulador no trabaja con coma, razón por la cual está obligado a utilizar la representación de coma flotante (notación científica para hacer los cálculos).

Si queremos trabajar con número decimales, tenemos que trabajar con un formato de tipo S M E - Signo (de la Mantisa), Mantisa (Numero), Exponente

De esta forma trabajamos con coma, sin meterlo dentro de la ALU.

Con la notación en coma fija puede representarse un rango de números enteros centrados en el cero. También pueden representarse números decimales asumiendo una coma fija. El problema de esta representación es que los números muy grandes o muy pequeños requieren mucho espacio. Esta limitación se soluciona mediante otra representación llamada **coma flotante**, la cual consiste en representar los números bajo la forma: $SM \times \alpha^E$ donde

S: el signo del número

M: la mantisa del número

E: Es el exponente del número

α : se escoge generalmente igual a la base del sistema de numeración

Esta notación es corriente en la escritura cuando representamos 125000000 por 125×10^6 . Tal representación no es única, puesto que también puede escribirse: $12,5 \times 10^7$ ó 1250×10^5 , etc. De entre estas representaciones se selecciona una, **llamada normalizada**, que permite conservar el mayor número de cifras significativas. Para normalizar un número se desplaza su mantisa hacia la izquierda o derecha según corresponda hasta que el primer dígito sea significativo, y se reduce o aumenta el exponente, respectivamente, en un número igual al número de desplazamientos que han sido necesarios. (en hipótesis de que el exponente expresa una potencia de 2).

0	0	0	1	2	5
0	0	1	2	5	0
0	1	2	5	0	0
1	2	5	0	0	0

Mantisa

0	6
0	5
0	4
0	3

Exponente

Operación de Normalización

CODIFICACIÓN DE LA INFORMACIÓN

CONCEPTO

La **codificación** consiste en establecer una ley de correspondencia, llamada **CÓDIGO**, entre las informaciones por representar y las posibles configuraciones binarias, de tal manera que a cada información corresponda una y generalmente solo una, configuración binaria.

Es un proceso o sistema que permite establecer una ley de correspondencia entre 2 alfabetos A y B.

En sentido genérico **CÓDIGO** significa: **Sistema de signos y de reglas que permite formular y comprender un mensaje.**

El proceso de **CODIFICAR** significa: Transformar, mediante las reglas de un código, la formulación de un mensaje.

Llamamos **CODIFICACIÓN** al proceso de convertir un símbolo complejo en un grupo de símbolos más simples. Ejemplo: convertir una letra del alfabeto en un código de cinco bits.

DECODIFICAR: Aplicar inversamente las reglas de su código a un mensaje codificado para obtener la forma primitiva de este.

DECODIFICACIÓN es el proceso inverso al de codificación, se convierte a un código donde la cantidad de símbolos es menor, pero cada uno contiene más información.

Ejemplo el código más antiguo del que se tiene conocimiento es el código del César que tenía una ROT13 - Rotación 13. Es decir, ésta es la función, es la **LEY DE CORRESPONDENCIA** por ejemplo para el mismo alfabeto con una ley distinta.

TRANSCODIFICACIÓN: Aplicación de un cambio de código a una información ya codificada. Ejemplo: EBCDIC a ASCII.

BITS: Teniendo en cuenta que las computadoras manejan un lenguaje binario analizaremos los **DÍGITOS BINARIOS** como caracteres para comunicación de datos.

La condición binaria es la que posee una calidad BIVALUADA. En el sistema binario de numeración esas condiciones están representadas por los dígitos 0 y 1. Se denomina **BIT** (contracción de **BINARY DIGIT**) al dígito binario, independientemente del valor asignado (0 o 1). Un **BIT** es la unidad de información más pequeña posible, que no puede tomar más que dos valores, generalmente 1 ó 0.

REPRESENTACIÓN DE LOS NÚMEROS EN LA MÁQUINA (Analógicamente)

Señales Lógicas y Analógicas

El procesamiento de la información en un calculador electrónico consiste en tratar, transferir y memorizar señales eléctricas.

En los *calculadores analógicos* se utilizan **señales de variación continua**. Por ejemplo, una magnitud cualquiera será representada por una tensión eléctrica, tensión que deberá mantenerse, durante el procesamiento, tan proporcional como sea posible a la magnitud representada.

En los *calculadores digitales*, la **información procesada que es de tipo binario**, está materializada por señales eléctricas de dos estados, que pueden hacerse muy distintos uno del otro, por ejemplo un nivel de tensión para el estado 1 y otro, muy diferente para el estado 0.

Ventaja de la transmisión digital vs lógica

Inmediatamente resalta una **primera ventaja de la representación digital** respecto de la analógica: la información digital se transmite mejor. Es bastante fácil realizar sistemas con dos estados estables, capaces de memorizar un bit. Por el contrario memorizar el valor de una señal eléctrica es mucho más difícil (se memorizan muy temporalmente las tensiones eléctricas cargando condensadores).

Lógica de Nivel

Puede representarse la información digital elemental sobre una línea eléctrica, manteniendo una tensión A para materializar el 1 lógico y una tensión B para el 0 lógico. En realidad no son valores precisos de la tensión, sino franjas alrededor de estos valores (figura 1).

Se llama **“zona prohibida”** a la zona intermedia, en ella, el valor lógico de la señal eléctrica es indeterminado.

Se dirá que se trabaja con **lógica positiva** cuando la tensión que representa el **1 lógico** es superior a la tensión del **0 lógico** y, en caso contrario, se trabajará con **lógica negativa**. La mayoría de las veces el 0 lógico será la masa o la tensión 0 voltios (figura 2). La elección de los *niveles* de tensión resulta de la tecnología escogida para la realización de los circuitos.

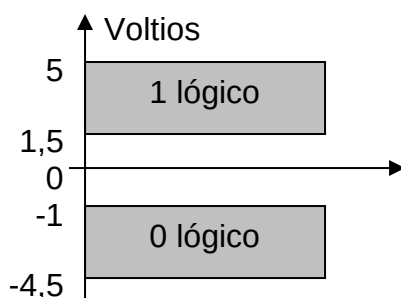


Figura 1

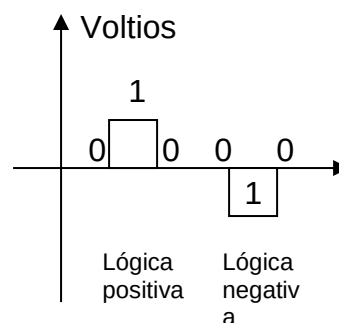
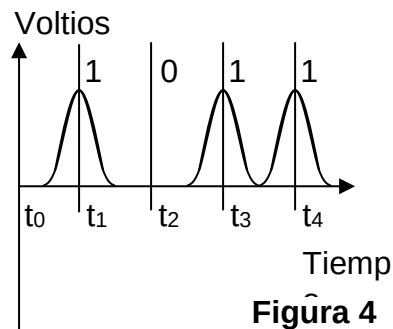
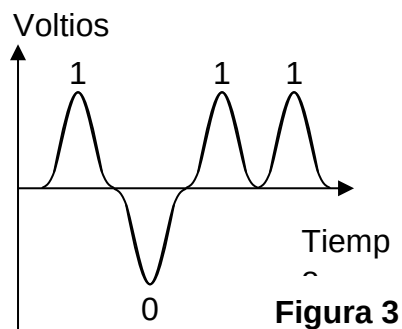


Figura 2

Se dice que se trabaja con una **lógica de impulsos** cuando la magnitud eléctrica representativa de la información dura un tiempo muy corto. Una primera técnica consiste en utilizar un impulso positivo para representar un 1, un impulso negativo para el 0, y la tensión 0 para la ausencia de información (figura 3).

La mayoría de las lógicas de impulsos emplean solamente impulsos positivos (o negativos) e imponen leer las informaciones en instantes dados, t_1 , t_2 , t_3 , etc.

Si un “**top**” coincide con un impulso, se trata de un 1 lógico, en caso contrario es un 0 lógico (figura 4).



Convencionalmente se anotan las líneas de nivel por una semiflecha, las líneas de pulsos por una flecha. Una señal eléctrica analógica puede variar a lo largo del tiempo de una manera continuada, gradual, sin saltos bruscos, entre dos valores extremos, que determinan un rango, y en un instante dado puede tener un valor dentro de dicho rango.

REPRESENTACIÓN LÓGICA DE LA INFORMACIÓN EN LA COMPUTADORA (CONCEPTUALMENTE)

Los dígitos binarios llevan, ambos, la misma cantidad de información, ya que la presencia de uno significa la ausencia del otro.

Un proceso fundamental en la codificación binaria es determinar la cantidad de **BITS** necesarios para representar las informaciones. De manera que podamos identificar una entre varias posibles.

Como un bit puede ser 1 o 0, podremos utilizarlos para seleccionar una información entre dos; con dos bits, una entre cuatro; tres bits una entre ocho; etc. Las posibilidades aumentan como potencias de dos:

Un bit	→	$2^1 = 2$ elecciones
Dos bits	→	$2^2 = 4$ elecciones
Tres bits	→	$2^3 = 8$ elecciones

Si quisiéramos conocer cuantos bits necesitamos para una de ocho situaciones utilizamos logaritmos en base 2, así: $\log_2 8 = 3$.

En general el número de bits (I) que necesitaremos para poder codificar una determinada cantidad de informaciones (N) estará determinado por:

$$I = \log_2 N$$

Como ejemplo, para poder representar en forma binaria los 26 caracteres de nuestro alfabeto necesitaríamos:

$$I = \log_2 26 = 4,7 \text{ bits} \longrightarrow I = 5 \text{ bits}$$

- Codificación de la información numérica y alfabética: Código binario puro, código BCD, otros códigos numéricos, códigos alfanuméricos. Códigos redundantes: concepto, tipos.

CODIFICACIÓN DE LA INFORMACIÓN EN LA MÁQUINA

Analizaremos los siguientes puntos:

1. Codificación de la Información Numérica

- A. Códigos Ponderados
- B. Códigos No Ponderados

Clasificación del profe:

- A. Conversion de numeros de distintas bases
- B. Conversion de numeros de punto flotante

2. Codificación de la Información No Numérica

- A. Codificación de los Caracteres
 - a. Caracteres Imprimibles
 - b. Caracteres No Imprimibles
- B. Codificación de las Instrucciones
- C. Redundantes
 - A) Detección
 - B) Corrección

CODIFICACIÓN INFORMACIÓN NUMÉRICA

Estudiaremos la representación de los símbolos del sistema decimal de numeración mediante símbolos binarios (BIT).

¿Cuántos bits necesitamos para representar los 10 símbolos del sistema decimal de numeración?

$$\Rightarrow I = \log_2 10 = 3,162 \dots \equiv 4$$

Esto nos indica que cualquier código que utilicemos para representar los números del sistema decimal precisará, como mínimo, 4 Dígitos Binarios (Bits). Cuando el código utilice más dígitos que los que necesita lo denominaremos **CÓDIGO REDUNDANTE**.

Códigos Ponderados

Se denomina código ponderado a aquel que respeta, para la representación de cada dígito decimal, el “peso” que corresponde a cada dígito binario de acuerdo a la posición que ocupa.

Ejemplos:

- BCD (8421)
- AIKEN (2421)
- 8-4-2-1.

Nota I: En cualquiera de los códigos numéricos (ponderados y no ponderados) un número decimal se codificará cada dígito decimal por separado.

Ejemplo: el número 345 se codificará

	3	4	5		
En BCD	0011	0100	0101	————	$345_{(10)} = 001101000101_{(BCD)}$
En AIKEN	0011	0100	1011	————	$345_{(10)} = 001101001011_{(AIKEN)}$

Nota II: Si bien los símbolos utilizados se corresponden con los del sistema binario, la representación codificada no tiene nada que ver con el SISTEMA BINARIO DE NUMERACIÓN.

- **BCD puro (Pesos 8, 4, 2, 1)**

El código **BCD** (DECIMAL CODIFICADO BINARIO) respeta para cada dígito decimal el “peso” que corresponde a cada dígito binario de acuerdo a la posición que ocupa, teniendo en cuenta el “peso” asignado en el sistema binario de numeración.

- **2, 4, 2, 1 (no autocomplementario)**

Nota III: El código 2, 4, 2, 1 permite dos combinaciones de los dígitos 2, 3, 4, 5, 6 y 7. Cualquiera es válida, más aún, en un mismo número de varios dígitos iguales pueden usarse codificaciones distintas.

- **AIKEN (Pesos 2, 4, 2, 1):** Construcción de la tabla de código Aiken. Es autocomplementario.

Aiken				
	2	4	2	1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1

2	4	2	1
---	---	---	---

1	0	0	0
1	0	0	1

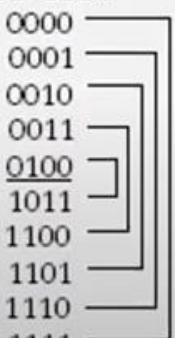
4	0	1	0	0
5	1	0	1	1
6	1	1	0	0
7	1	1	0	1
8	1	1	1	0
9	1	1	1	1

1	0	1	0
0	1	0	1
0	1	1	0
0	1	1	1

Aquellos números que poseen doble representación, deberá optarse por la que permita invertir un solo dígito respecto a su anterior. Por Ejemplo para 5 base 10 codificar como 1001 en 2421

Valor decimal	BCD 2421
0	0000
1	0001
2	0010
3	0011
4	0100
5	1011
6	1100
7	1101
8	1110
9	1111

"Código autocomplementario"



- **Exceso en Tres**

La representación en el código exceso de 3, cada dígito decimal se representa como en BCD pero excedido en 3. Ejemplo la representación del 2 es la BCD del 5. Es un código autocomplementario, obsérvese que 0111 (4) es el complemento restringido de 1000 (5), o que 0011 (0) es el complemento restringido de 1100 (9).

Códigos No Ponderados

En estos códigos la representación de cada dígito decimal es en principio arbitraria o responde a características que no son el "peso" de acuerdo a la posición que ocupan. Ejemplos GRAY.

- **Construcción de la tabla del Código de GRAY**

1. Se colocan los dos bits (0 y 1) y se traza una línea debajo de ellos (espejo), se copian los bits como si se reflejaran en dicho espejo.

0
1

1
0

2. Se completa la siguiente columna con 1 por debajo del espejo y con 0 por encima del mismo.

0 0
0 1
—
1 1
1 0

3. Se repite la operación de reflejado con los cuatro números obtenidos.

0 0
0 1
1 1
1 0
—
1 0
1 1
0 1
0 0

4. Se completa la tercera columna con 1 debajo del espejo y 0 por encima.

0 0 0
0 0 1
0 1 1
0 1 0
—
1 1 0
1 1 1
1 0 1
1 0 0

5. Se vuelve a realizar la operación hasta completar los diez dígitos.

0	0	0	0		0	0	0	0	0
0	0	0	1		1	0	0	0	1
0	0	1	1		2	0	0	1	1
0	0	1	0		3	0	0	1	0
0	1	1	0		4	0	1	1	0
0	1	1	1		5	0	1	1	1
0	1	0	1		6	0	1	0	1
0	1	0	0		7	0	1	0	0
1	1	0	0		8	1	1	0	0
1	1	0	1		9	1	1	0	1

ALGUNOS CÓDIGOS NUMÉRICOS MÁS USUALES

Dígito Decimal	B C D (8421)	Exceso de tres	A I K E N (2421)	8 4 -2 -1	BIQUINARIO	Código de Gray
0	0000	0011	0000	0000	0100001	0000
1	0001	0100	0001	0111	0100010	0001
2	0010	0101	0010	0110	0100100	0011
3	0011	0110	0011	0101	0101000	0010
4	0100	0111	0100	0100	0110000	0110
5	0101	1000	1011	1011	1000001	0111
6	0110	1001	1100	1010	1000010	0101
7	0111	1010	1101	1001	1000100	0100
8	1000	1011	1110	1000	1001000	1100
9	1001	1100	1111	1111	1010000	1101

ALGUNOS COMENTARIOS SOBRE LOS CÓDIGOS

1. Los códigos Exceso de 3, el 2421 y 84-2-1 son autocomplementarios. o sea que el complemento a 9 del número decimal se obtiene cambiando los ceros por unos y los unos por ceros.
2. El código Biquinario es un CODIGO REDUNDANTE donde cada dígito decimal se representa con cinco ceros y dos unos.
3. El código de Gray o BINARIO REFLEJADO tiene la particularidad que de un dígito decimal al siguiente cambia siempre un solo dígito por vez.
4. El código de Gray y Biquinario son códigos NO PONDERADOS.

CODIFICACIÓN DE LA INFORMACIÓN NO NUMÉRICA

Codificación de los Caracteres

Luego de ver como pueden representarse los números, analizaremos ahora como extender el sistema de codificación al conjunto de signos de la máquina de escribir: letras, signos de puntuación, operadores y caracteres especiales.

Condiciones que se Imponen para la Codificación de Caracteres

- A. La representación debe englobar a las de las cifras en una de las formas descriptas (BCD, 2421, etc.) y permitir distinguir las cifras rápidamente de los otros caracteres.
- B. La representación debe permitir añadir nuevos caracteres específicos para una aplicación determinada.
- C. En el caso de las transmisiones, la representación debe incluir un sistema de redundancia que permita la detección de errores.

CARÁCTER

El concepto de caracter aparece como la cantidad de BITS necesarios para representar los diferentes símbolos del alfabeto (letras, cifras, signos de puntuación, etc.).

De acuerdo al código utilizado cada caracter puede codificarse con un número variable de BITS, pero dentro de un sistema todos los caracteres se representan con el mismo número de bits.

El código ASCII se definió inicialmente con 6 bits, esto permitía representar $2^6 = 64$ caracteres. Posteriormente la ANSI (Instituto Nacional Norteamericano de Normas) definió un nuevo ASCII (que se mantiene como norma) de 7 bits. Esta nueva definición permite codificar 128 caracteres; haciéndolo más apto, fundamentalmente para la transmisión donde parte de los “caracteres” codifican funciones de control.

El código EBCDIC creado por IBM utiliza 8 bits para representar cada caracter.

PALABRA

La palabra es un conjunto de caracteres, fijo o variable, según el caso, que la computadora trata como unidad.

Es una unidad de información de rango superior al caracter. Generalmente contiene un número entero de caracteres.

La palabra es la unidad de información procesada por la máquina.

BYTE

El término BYTE (octeto), se utiliza para describir un conjunto de 8 bits consecutivos que se toman como unidad.

Agrupamientos binarios: bit, byte, palabra.

El BYTE muchas veces no es práctico para manipular datos en problemas de programación, recurriendo al concepto de PALABRA (WORD), de acuerdo a la siguiente relación:

1 BYTE	= 8 bits
1 PALABRA	= 2 Bytes
1 DOBLE PALABRA	= 4 Bytes
1 CUADRUPLE PALABRA	= 8 Bytes
1 DECABYTE	= 10 Bytes

<u>Otras agrupaciones de bytes:</u>	PÁRRAFO	= 16 Bytes
	PÁGINA	= 256 Bytes
	SEGMENTO	= 64 Kbytes

CODIGO EBCDIC (Expanded Binary Code Decimal Interchange Code)

Este código fué diseñado y utilizado exclusivamente por IBM. Su importancia radica en que sirvió como base para los códigos posteriores normalizados.

Utiliza 8 dígitos binarios para representar cada caracter. La correspondencia entre las informaciones por representar la correspondiente secuencia binaria se encuentra en tablas con distintos formatos.

Con respecto a los caracteres alfabéticos y signos de puntuación no tiene características que lo destaquen, salvo que podemos encontrar pequeñas alteraciones al ser utilizados en países con distintos alfabetos.

Representación de la Información Numérica (EBCDIC)

Cada dígito decimal es representado internamente con 8 bits, distribuidos de la siguiente forma:

ZONA	DIGITO
4 Bits	4 Bits

SISTEMAS ALFANUMÉRICOS

CODIGO ASCII (American Standard Code for Information Interchange)

ZONA: Son las columnas

DIGITO: Son las filas

La tabla simplificada contaba con 7 dígitos (zona de 3, digito de 4) en vez de 8 que es la que se utilizó después.

La palabra está dividida en 2, zonas y dígitos.

Se trata de un código que utiliza 7 bits (SIMPLIFICADO/REDUCIDA, después vino la completa de 8 bits, 4 y 4) para representar cada carácter. Parte de las configuraciones binarias son utilizadas para codificar funciones de control.

La tabla contempla **3 áreas:**

- Sector de caracteres especiales
- Nros y símbolos matemáticos
- Caracteres alfanuméricos

La mayor parte de las máquinas actuales utilizan este código normalizado, pero generalmente agregar un octavo bit, que les permite extender el código para representaciones no previstas o para utilizarlo como bit de control de paridad en las transmisiones.

El éxito de este código se basa en que cumple con todas las condiciones impuestas para la codificación de caracteres:

- Utiliza relaciones para establecer el código.
- Los valores correspondientes a las letras de alfabeto y a los restantes caracteres, siguen una secuencia binaria continua, la computadora no tiene que dejar su propio lenguaje binario para realizar operaciones secuenciales con esos caracteres.

- Agrupamiento de las funciones de control, con solo analizar los dos primeros bits de una combinación cualquiera codificada, la computadora puede determinar si se trata de una función de control (dos ceros) o de un carácter (uno de los dos no es cero).

Sirve para mostrar el proceso interno de un computador, en la parte más interna.

Representación de los Números

En este código, al igual en el EBCDIC, la representación de la información numérica contempla representar los símbolos del sistema decimal de numeración en forma binaria. Cada dígito se codifica con 7 bit que también se encuentran asociados en dos grupos **d** Y **DIGITO**.

La zona es siempre 011 y dígito corresponde a la representación BCD del número.

Aquí surge el mismo inconveniente visto en el código EBCDIC, la dificultad para realizar operaciones matemáticas con los números representados con la ZONA, la solución es la misma que la indicada para el código anterior. Como la mayor parte de las máquinas utilizan 8 bits para cada dígito no surgen conflictos al producirse el EMPAQUE.

CODIFICACION DE LAS INSTRUCCIONES DE MAQUINA

STOP	0000
SUM	0001
SUS	...
DIV	...
MUL	...
SAI	...

Darle un código a cada uno de ellas, estos códigos se definen a la hora de definir la arquitectura de la máquina.

La codificación está grabado en la BIOS para trabajar con esas instrucciones de base.

La codificación de las instrucciones se desarrollará en conjunto con la Arquitectura del procesador.

Una instrucción de máquina viene dado con un registro de instrucción.

Código de operación | Condicion de direccionamiento | Direccion del Operando

Quiero sumar en modo directo el contenido de la dirección de memoria X con el contenido del valor del Acumulador de la ALU.

Delimitación de strings

De longitud fija: Cada dato ocupa un número determinado de bytes fijo.

- Byte (Zona, Dígito)

De longitud variable: Cada dato tiene en su inicio un campo (cabecera) que es el que indica el la longitud de bytes.

Indicador	Palabra	Indicador	Palabra
4 Bits	...	2 Bits	...

CÓDIGO BCD

Los codigos BCD son convenciones que permiten la representación de números decimales (0 a 9) en bloques binarios de 4 bits. Se denomina así porque los pesos en cada bloque coinciden con el valor de los 4 primeros pesos del sistema binario puro.

Esta es la razon por la que un BCD puro se lo suele llamar **BCD 8421**.

Para el pasaje de base10 a binario, se tienen que consumir ciclos para hacer las divisiones. Mientras que el nro 123(10 a BCD es: 0001 0010 0011, está pensado para la

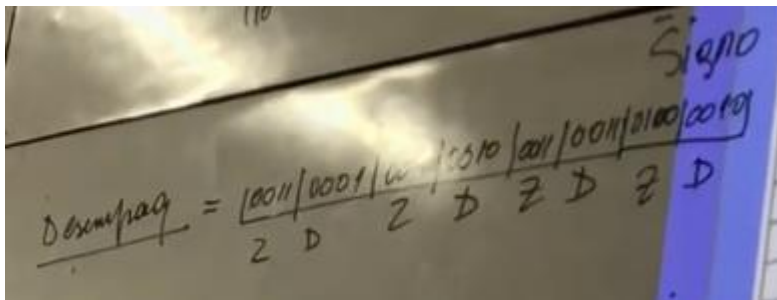
Ejemplo: 123(10

B - Positivo

D - Negativo

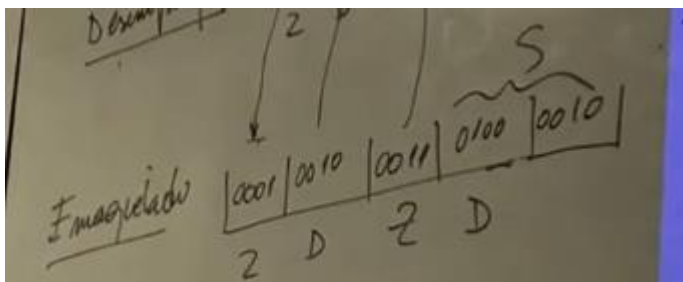
DESEMPAQUETADO:

- En este caso el signo se pone al final, y en el ejemplo se utilizó la tabla ASCII, y se ocupa el valor de B.



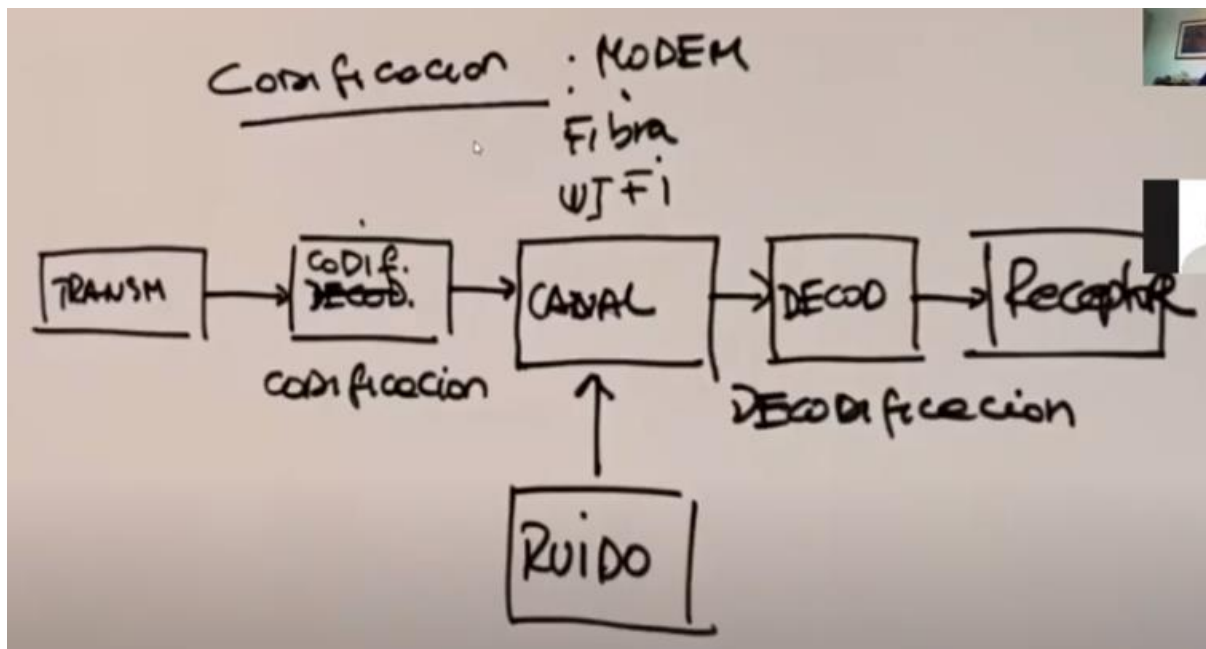
EMPAQUETADO:

- Acá no se utiliza la ZONA, porque se utiliza solo el valor directo del valor numérico. Se ocupan de a 4 bits, porque es Empaquetado.



Siempre se busca transmitir en cantidad de par de paquete, entonces si tenés 7, entonces tenes que completar con 0s.

CODIGOS REDUNDANTES



En el **CANAL** se puede meter **RUIDO** que produce una distorsión en la información. Para mejorar la eficiencia se usan codificadores y decodificadores en ambos extremos, que permitan transformar el mensaje para que el receptor pueda detectar y corregir errores.

Consiste en agregarle una redundancia, de tal manera que el receptor a través de una **FUNCIÓN $f(x)$** pueda **verificar** si la información que **recibió es CORRECTA**, nuevamente existe una codificación para poder realizar este chequeo.

Cuando se teme una modificación de la información durante su transmisión o su almacenamiento en memoria, se va al empleo de códigos que permitan detectar errores -**códigos autodetectores**- o corregirlos -**códigos autocorrectores**-.

Ello implica que éstos códigos utilizan de un número de dígitos binarios superior al estrictamente necesario para codificar la información.

Es difícil pensar en un equipo que funcione sin fallas durante un tiempo indefinido. Para cualquier máquina se define un **Tiempo Medio Entre Fallas (MTBF)**, el objetivo de los desarrollos tecnológicos es incrementar ese tiempo.

Teniendo en cuenta esa premisa, es de esperar que la información pueda verse alterada en transcurso de la transmisión o almacenamiento, si nuestro equipo tiene la posibilidad de detectar o, mejor aún, corregir esas modificaciones es evidente que aumentará la confiabilidad del mismo. Estos códigos utilizan un número de bits superior al estrictamente necesario para codificar la información.

Los códigos redundantes se aplican en la transmisión de información dentro del computador, y fuera del computador (**periféricos**). Por lo que se utilizan distintos códigos para cada caso.

Códigos Autodetectores

Código en el que mediante un determinado número de bits de redundancia se puede detectar si la información recibida es correcta o no. El ejemplo mas clásico de este tipo de códigos es de CONTROL DE PARIDAD.

- **Control de Paridad**

Este código, si bien no permite detectar errores dobles, es el más utilizado debido a su simplicidad y a que en los ordenadores la probabilidad de que ocurra un error es muy pequeña, por lo tanto que ocurran 2 es mucho menos probable.

Consiste en agregar a los bits de información transmitidos un bit mas (generalmente el primero de la izquierda), que hace que la cantidad de unos transmitidos sea **PAR (PARIDAD PAR) o IMPAR (PARIDAD IMPAR)**.

Se puede hacer que se roten los bits 2 veces, o que también el bit de paridad se pueda cambiar dando un falso positivo.

PARIDAD PAR de 1:

Compuerta XOR - Porque le agrega la paridad par de 1.

- **Paridad vertical simple o a nivel de carácter:** Es cuando se agrega 1 bit al octeto para conseguir la paridad.
- **Paridad horizontal y vertical a nivel de bloque o CÓDIGO ENTRELAZADO:** Estás trabajando sobre los bits de un bloque, no sobre los bits de palabra.

Ejemplo para un bloque de 48 bits, quiere decir 6 filas de 8 bits. Paridad par.

Paridad Horizontal	Bit1	Bit2	Bit3	Bit4	Bit5	Bit6	Bit7	Bit8	
0	0	1	1	0	0	1	0	1	mnas
1	1	0	1	1	1	0	0	1	inal, s
0	1	0	0	1	0	1	0	1	o al f
1	0	0	1	1	1	0	0	0	nás qu
1	1	0	0	1	0	1	1	1	de tod
1	0	0	1	0	1	0	1	0	las co
0	1	1	0	0	1	1	0	0	partir
									Paridad Vertical

Filas: Caracter

Paridad horizontal es la paridad de cada fila.

Paridad vertical es la cantidad de cada columna.

Entonces por eso en la diagonal, se utiliza un bit para mantener la paridad entre ambas columna/filas amarillas.

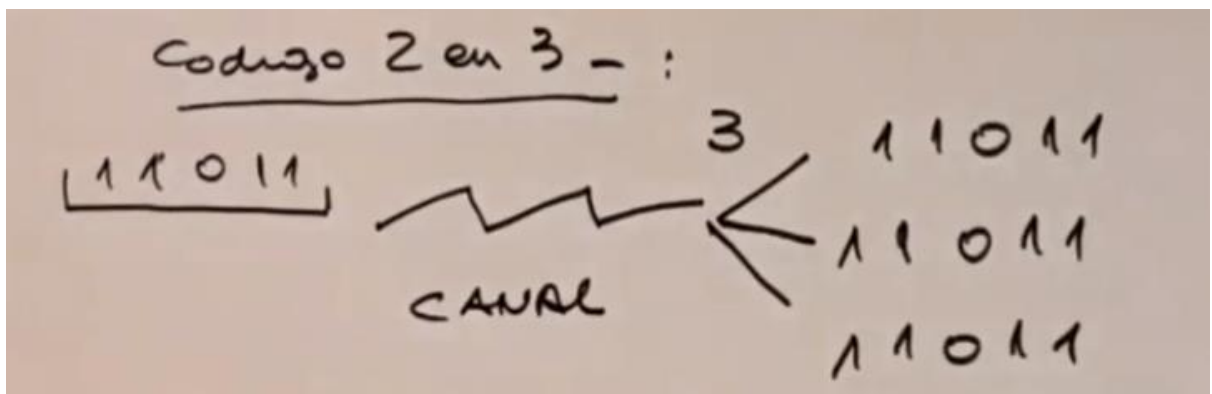
SI hay más de un error por fila o columna, el código ya no funciona.

Código entrelazado, permite detectar y corregir un caracter.

Códigos Autocorrectores

- **Control 2 en 3**

Para transmitir una información cualquiera de “n” bits, se envían 3 veces esos “n” bits, en forma sucesiva.



El receptor de la información, al efectuar el análisis de la misma, se le puede presentar tres situaciones distintas:

- 1) Las tres son idénticas. La información se toma como correcta.
- 2) Dos son iguales y una distinta. El código se comporta como **AUTOCORRECTOR**, selecciona una de las dos iguales y la toma como correcta.
- 3) Las tres son distintas. El código se comporta como **AUTODETECTOR**, la máquina detecta que hay error pero no puede determinar cual es la información correcta, por lo que solicita **RETRANSMITIR**.

No solamente detecta sino que también solicita la retransmisión por lo que es autocorrector. Si el canal funciona mal, lo que va a pasar es que se va a bloquear el canal y va a ralentizar el sistema.

Hamming

Codificación

Estos códigos permiten detectar y corregir uno o más errores producidos durante la transmisión para palabras de cualquier número de bits. Analizaremos el método para construir un código de Hamming para corregir un solo error. Se utiliza mucho en transmisiones donde la retransmisión es muy penalizada, como por ejemplo en el Wifi, ya que si se utilizara en exceso la retransmisión el ancho de banda sería copado solamente con retransmisiones.

Primero hablemos de Distancia

Distancia de Hamming

$D_m = 2X + 1$

$D_m = 2 * 0 + 1 = 1$
Distancia HAMMING = 1

Mensaje	
0	Válido
1	Válido

Se debe modificar 1 bit

$D_m = 2 * 1 + 1 = 3$

Distancia HAMMING = 2

Mensaje		
0	0	Válido
0	1	Invalido
1	0	Invalido
1	1	Válido

Se pueden detectar errores de 1 bit
(Caso bit Paridad)

$D_m = 2 * 3 + 1 = 7$
Distancia HAMMING = 3

Mensaje			
0	0	0	Válido
0	0	1	Invalido
0	1	0	Invalido
0	1	1	Invalido
1	0	0	Invalido
1	0	1	Invalido
1	1	0	Invalido
1	1	1	Válido

Detecta y corrige errores de 1 bit

Esto es solo el concepto de distancia que es la cantidad de bits que tengo que cambiar para pasar de un código a otro.

Como por ejemplo, para pasar de **0 0 0** → **1 1 1** cambiamos 3 símbolos. Pero en el medio también nos quedan las “configuraciones inválidas”, de los posibles “errores” que puede tener un mensaje, hablando de paridad.

Distancia de Hamming entre dos Códigos

Se denomina Distancia de Hamming entre dos códigos de igual longitud, al número de símbolos que se deben alterar para pasar de una configuración válida a otra. Es decir, el número de bits en que difieren dos configuraciones binarias válidas de igual longitud.

Distancia de Hamming

Distancia	Longitud Total (n)	Longitud Información (k)	Notación (n,k)	Eficiencia sobre Canal	Detección	Corrección int((m-1)/2)
m	$(2^m)-1$	$(2^m)-1-m$	(n,k)	k/n	(m-1)	$(m-1)/2$
1	1	0	(1,0)	0,0%	0	0
2	3	1	(3,1)	33,3%	1	0
3	7	4	(7,4)	57,1%	2	1
4	15	11	(15,11)	73,3%	3	1
5	31	26	(31,26)	83,9%	4	2
6	63	57	(63,57)	90,5%	5	2
7	127	120	(127,120)	94,5%	6	3
8	255	247	(255,247)	96,9%	7	3
9	511	502	(511,502)	98,2%	8	4
10	1023	1013	(1023,1013)	99,0%	9	4
11	2047	2036	(2047,2036)	99,5%	10	5
12	4095	4083	(4095,4083)	99,7%	11	5

La notación en la fila 3, por ejemplo, es el (7,4) significa que son **4 bits de información** y **7 bits en total (longitud total)**, lo que nos queda **3 bits de paridad**.

Eficiencia de canal, es la cantidad de bits de información dividido longitud total.

El primer paso consiste en determinar, para un código de “i” dígitos binarios de información, cuantos bits de de control de paridad “p” son necesarios para detectar y corregir un error único. Si consideramos que tenemos i dígitos de información y p bits de control y con la premisa que los casos que se puede presentar son de ningún error o un solo error por vez, tendremos $i + p + 1$ condiciones que deben identificarse usando los p bits de control de paridad. Teniendo en cuenta que con p bits podemos formar 2^p combinaciones, entonces la cantidad de bits de control de paridad debe ser tal que satisfaga que:

$$2^p \geq i + p + 1$$

Mediante esta expresión podemos construir la siguiente tabla:

i	1	2	3	4	5	6	7	8	9	10	11	12	...	Bits de información
p	2	3	3	3	4	4	4	4	4	4	4	5	...	Bits de control
n	3	5	6	7	9	10	11	12	13	14	15	17	...	Bits del mensaje

La forma en que se distribuyen los bits de información y los de control, para conformar el mensaje, es en principio arbitraria.

Analizaremos una distribución en el mensaje de la siguiente forma, considerando un mensaje de 4 bits de información y que por lo tanto necesitará 3 bits de control:

Posición Mensaje	7	6	5	4	3	2	1
	i_3	i_2	i_1	p_2	i_0	p_1	p_0

Los bits de control se colocan en las posiciones que corresponden a las potencias de dos.

Se desea que los dígitos de control (en nuestro ejemplo 3) que indican el resultado del test de paridad sobre los dígitos de paridad, den (en binario) la posición del dígito erróneo.

Para determinar el valor que tomará cada bit de paridad se construye una tabla de combinaciones binarias de tantas columnas como la cantidad de bits de paridad determinada para la codificación. Para el caso anterior serían 3 columnas (p_0 , p_1 y p_2).

	p_2	p_1	p_0	
0	0	0	0	Cada bit de paridad controla la posición que coincide con un 1 en su columna en la tabla binaria. Es decir:
1	0	0	1	
2	0	1	0	
3	0	1	1	p_0 controla las posiciones: 1, 3, 5 y 7
4	1	0	0	p_1 controla las posiciones: 2, 3, 6 y 7
5	1	0	1	p_2 controla las posiciones: 4, 5, 6 y 7
6	1	1	0	
7	1	1	1	

Para mayor cantidad de bits de paridad se agregan más columnas a la tabla. El bit de paridad debe hacer que la cantidad de bit unos controlados sea par.

Para el caso de un byte (8 bits) de información, corresponde un mensaje de 12 bits.

Control y Corrección

Cuando el mensaje es enviado, el receptor calcula el valor de los bits de control de la siguiente forma:

$$\begin{aligned}
 c_0 &= 1 \oplus 3 \oplus 5 \oplus 7 \\
 c_1 &= 2 \oplus 3 \oplus 6 \oplus 7 \\
 c_2 &= 4 \oplus 5 \oplus 6 \oplus 7
 \end{aligned}$$

Los números corresponden a la posición de cada bit utilizado para el cálculo de los bits de control (ejemplo: “3” corresponde al bit ubicado en la posición 3) y el símbolo designa la operación OR exclusiva.

Si el bit de control es igual a 0 es correcto, si es 1 es incorrecto. La posición del error se determina según los valores obtenidos en los bits de control ordenados en forma decreciente, es decir c_2 , c_1 y c_0 . Si todos los bits de control son igual a cero no se detecta error en el mensaje.

EJEMPLO: Información a transmitir:

1 1 0 1

$$p = 3 \quad (2^p \geq i + p + 1)$$

7	6	5	4	3	2	1
1	1	0	—	1	—	—
i_3	i_2	i_1	p_2	i_0	p_1	p_0

$p_0: 1, 3, 5, 7 \Rightarrow \underline{0}, 1, 0, 1$ (para mantener la paridad para toma el valor 0)

$p_1: 2, 3, 6, 7 \Rightarrow \underline{1}, 1, 1, 1$ (para mantener la paridad para toma el valor 1)

$p_2: 4, 5, 6, 7 \Rightarrow \underline{0}, 0, 1, 1$ (para mantener la paridad para toma el valor 0)

Mensaje transmitido:

7	6	5	4	3	2	1
1	1	0	0	1	1	0

Supongamos que al receptor llega el siguiente mensaje: 1101110

Al calcular, el receptor, los bits de paridad determina:

$$c_0 = 1 \oplus 3 \oplus 5 \oplus 7 = 0 \oplus 1 \oplus 0 \oplus 1 = 0 \quad \text{Correcto}$$

$$c_1 = 2 \oplus 3 \oplus 6 \oplus 7 = 1 \oplus 1 \oplus 1 \oplus 1 = 0 \quad \text{Correcto}$$

$$c_2 = 4 \oplus 5 \oplus 6 \oplus 7 = 1 \oplus 0 \oplus 1 \oplus 1 = 1 \quad \text{Incorrecto}$$

1 0 0 ($c_2 = 4$ indica la posición del error).

$c_2 \quad c_1 \quad c_0$

7	6	5	4	3	2	1
1	1	0	0	1	1	0

Error en la posición 4 (se cambia el valor del bit)

Circuitos Digitales / Combinacionales

REPASO DE MAT SUPERIOR

Compuertas lógicas

Nombre	Símbolo gráfico	Función algebraica	Tabla verdad															
AND		$F = A \cdot B$ or $F = AB$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	F	0	0	0	0	1	0	1	0	0	1	1	1
A	B	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$F = A + B$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	F	0	0	0	0	1	1	1	0	1	1	1	1
A	B	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
NOT		$F = \overline{A}$ or $F = A'$	<table><tr><th>A</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	F	0	1	1	0									
A	F																	
0	1																	
1	0																	
NAND		$F = \overline{(AB)}$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	1	0	1	1	1	0	1	1	1	0
A	B	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$F = \overline{(A + B)}$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	1	0	1	0	1	0	0	1	1	0
A	B	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																

Nombre	Símbolo gráfico	Función algebraica	Tabla verdad															
Buffer		$F = x$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	x	F	0	0	1	1									
x	F																	
0	0																	
1	1																	
XOR		$F = xy' + x'y$ $= x \oplus y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
$\overline{\text{XOR}}$		$F = xy + x'y'$ $= x \odot y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

MINITÉRMINOS Y MAXITÉRMINOS

Una variable binaria puede aparecer ya sea en su forma (X) o en su forma complementada (X'). Si consideramos dos variables binarias X e Y con su operador AND, ya que cada variable puede aparecer en cualquier forma, hay cuatro combinaciones posibles: X'Y', X'Y, XY' y XY. Cada uno de esos cuatro términos AND se denomina un **Minitérmino** de un Producto normalizado. En forma semejante pueden combinarse **n** variables para formar **2ⁿ** minitérminos o términos mínimos.

De manera similar, las **n** variables formando un término OR, con cada variable tildada o no tildada, darán **2ⁿ** combinaciones posibles, denominadas **Maxitérminos** de las sumas normalizadas. Cada maxitérmino es el complemento de su correspondiente minitérmino y viceversa.

A las funciones de Boole expresadas como Suma de Minitérminos o Producto de Maxitérminos se dice que está en **forma canónica**.

SUMA DE MINITERMINOS Y PRODUCTO DE MAXITÉRMINOS

Suma de Minitérminos:

Son los minitérminos cuya suma define la función de boole, es decir, que son aquellos que dan los 1 de la función en una tabla de verdad. Como la función puede ser 1 ó 0 para cada minitérmino, y ya que hay 2^n minitérminos, se pueden calcular las funciones posibles que pueden formarse con n variables como 2^{2^n} .

Minitermino o Producto Canónico: Es el producto de las variables en juego o sus negaciones individuales que hacen que el producto valga uno.

Producto de Maxitérminos:

Cada una de las 2^{2^n} funciones de n variables binarias pueden expresarse como un producto de maxitérminos, o sea, son aquellos que dan los 0 de la función en una tabla de verdad.

Maxitermino o Suma Canónica: Es la suma de las variables en juego o sus negaciones individuales, que hacen que la suma valga cero.

ENTONCES

La **Forma Normal Disyuntiva (FND)** se obtiene a partir de la tabla de verdad, en las filas donde la función vale 1, tomando el producto de todas las variables, reemplazando así los 0 por sus respectivas negadas y los 1 por sus correspondientes variables sin negar. Luego se realiza la suma de los minitérminos así determinados.

2. Una forma normal disyuntiva (FND) es una disyunción de conjunciones de literales:

$$(L_{11} \text{ y } L_{12} \text{ y } \dots \text{ y } L_{1n}) \text{ o } (L_{21} \text{ y } L_{22} \text{ y } \dots \text{ y } L_{2n}) \text{ o } (L_{m1} \text{ y } L_{m2} \text{ y } \dots \text{ y } L_{nm})$$

A	B	C	Minitérminos	S
0	0	0	$\overline{A} \overline{B} \overline{C}$	1
0	0	1	$\overline{A} \overline{B} C$	0
0	1	0	$\overline{A} B \overline{C}$	0
0	1	1	$\overline{A} B C$	1
1	0	0	$A \overline{B} \overline{C}$	0
1	0	1	$A \overline{B} C$	1
1	1	0	$A B \overline{C}$	1
1	1	1	$A B C$	0

La **Forma Normal Conjuntiva (FNC)** se obtiene a partir de la tabla de verdad en las filas donde la función vale 0, se forman las sumas de todas las variables reemplazando los unos de cada combinación por sus respectivas variables negadas y los 0 por sus correspondientes variables sin negar. Luego se realiza el producto de los maxitérminos así obtenidos.

1. Una forma normal conjuntiva (FNC) es una conjunción de disyunciones de literales:

$$(L_{11} \text{ o } L_{12} \text{ o } \dots \text{ o } L_{1n}) \text{ y } (L_{21} \text{ o } L_{22} \text{ o } \dots \text{ o } L_{2n}) \text{ y } (L_{m1} \text{ o } L_{m2} \text{ o } \dots \text{ o } L_{nm})$$

A	B	C	Maxitérminos	S
0	0	0	$A+B+C$	1
0	0	1	$A+B+\overline{C}$	0
0	1	0	$A+\overline{B}+C$	0
0	1	1	$A+\overline{B}+\overline{C}$	1
1	0	0	$\overline{A}+B+C$	0
1	0	1	$\overline{A}+B+\overline{C}$	1
1	1	0	$\overline{A}+\overline{B}+C$	1
1	1	1	$\overline{A}+\overline{B}+\overline{C}$	0

Formas Normales

FNC - Forma Normal Conjuntiva - con Maxiterminos

Conjunción de disyunciones de los literales de las entradas $(A+B \cdot C \cdot A)$

FND - Forma Normal Disyuntiva (**Más utilizada**) - con Miniterminos

Disyunción de las conjunciones de los literales de las entradas $(A \cdot B + C \cdot A)$

A	B	Minterminos
0	0	$\bar{A} \cdot \bar{B}$
0	1	$\bar{A} \cdot B$
1	0	$A \cdot \bar{B}$
1	1	$A \cdot B$

A	B	Maxiterminos
0	0	$A + B$
0	1	$A + \bar{B}$
1	0	$\bar{A} + B$
1	1	$\bar{A} + \bar{B}$

Mintermino o Producto Canónico: Es el producto de las variables en juego o sus negaciones individuales que hacen que el producto valga uno.

Maxitermino o Suma Canónica: Es la suma de las variables en juego o sus negaciones individuales, que hacen que la suma valga cero.

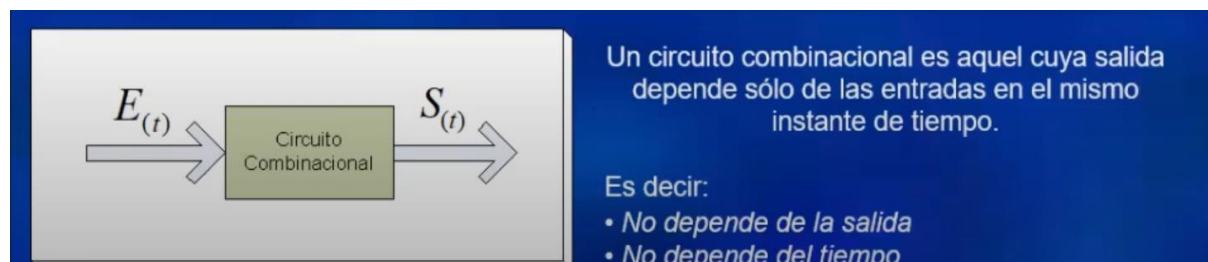
Sistemas digitales combinacionales

Circuitos combinacionales

Son aquellos en los que sus valores de salida dependen de las combinaciones de valores en sus **entradas** en el **mismo instante de tiempo T**. Este tipo de circuitos **NO tiene en cuenta la variable Tiempo**, es decir, que a menos que se redefina su estructura interna, un circuito combinacional entrega siempre el mismo valor de salida para una combinación dada de entradas.

La **salida es función de la entrada en el mismo instante de tiempo**.

Es combinacional porque plantea todas los posibles valores de entrada que a la salida cumplen con una determinada restricción planteada en el problema.



Circuitos secuenciales

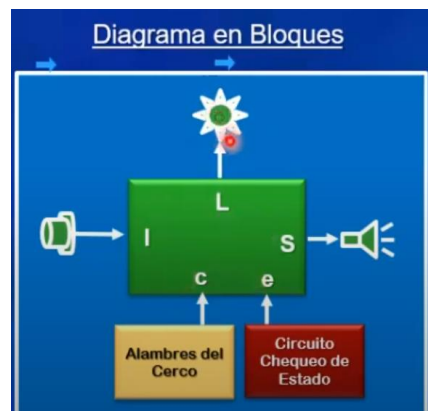
Los circuitos secuenciales funcionan sobre la **base del tiempo y de la salida**.

Un circuito es secuencial cuando sus salidas dependen del valor de las entradas y del estado anterior del circuito representado, que puede tener una o más salidas. Por lo tanto, la representación de estos circuitos debe tener en cuenta que las funciones de salida deben contemplarse como entradas; por ello se afirma que el **circuito tiene uno o más lazos de realimentación**.

Los circuitos secuenciales usan elementos de memoria (celdas binarias) además de las compuertas lógicas. Las salidas de un circuito secuencial son una función no solamente de las entradas externas sino del presente estado de los elementos de la memoria. El estado de los elementos de memoria, a su vez, es una función de las entradas previas. **Como consecuencia las salidas de un circuito secuencial dependen no solo de las entradas presentes, sino también, de las entradas del pasado y el comportamiento del circuito debe especificarse por una secuencia de tiempos de las entradas y estados internos.** Otra manera más general y útil de ver esto es que la salida actual de un circuito secuencial depende de la entrada actual y del estado actual del circuito.

Ejemplos

- Alarma

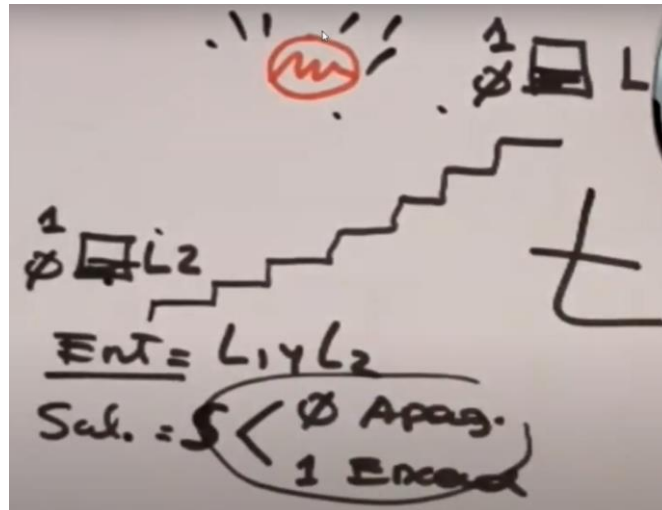




- Lámpara de escalera

Se utilizan para circuitos que deben devolver una salida según la entrada como una escalera.

La escalera con las 2 llaves combinatorias, y la lámpara en el medio.



Un circuito secuencial **sincrónico** es un sistema cuyo comportamiento puede definirse a partir del conocimiento de sus señales en instantes discretos de tiempos.

La sincronización se logra por un dispositivo de tiempo llamado generador maestro de tiempo el cual genera un tren periódico de pulsos de reloj. Los pulsos de reloj se distribuyen a través del sistema de tal manera que los elementos de memoria son afectados solamente con la llegada del pulso de sincronización.

El comportamiento de un circuito secuencial **asincrónico** depende del orden en que cambien las señales de entrada (es decir, cambios se producen únicamente por activación de alguna de las entradas) y puedan ser afectadas en cualquier instante de tiempo dado. Los elementos de memoria comúnmente usados en los circuitos secuenciales asincrónicos son mecanismos retardadores de tiempo. Así, un circuito secuencial asincrónico puede tomarse como un circuito combinatorial con realimentación.

AUTOMATA FINITO

Se denomina autómata de **Estado Finito** a los que satisfacen las siguientes hipótesis:

1. La función $H(t)$ está completamente determinada por la historia previa del autómata. Ésta hipótesis elimina los sistemas no deterministas, por ejemplo aquellos cuyo funcionamiento es aleatorio.
2. Está constituido por un **número finito de elementos**, por consiguiente de un número finito de elementos de memoria, de tal suerte que no puede recordar una historia arbitrariamente larga ni diferentes. Por lo tanto, no puede tomar más que un

número finito de valores distintos, que se llaman estados internos o simplemente estados del autómatas.

3. Definición de Mauro:

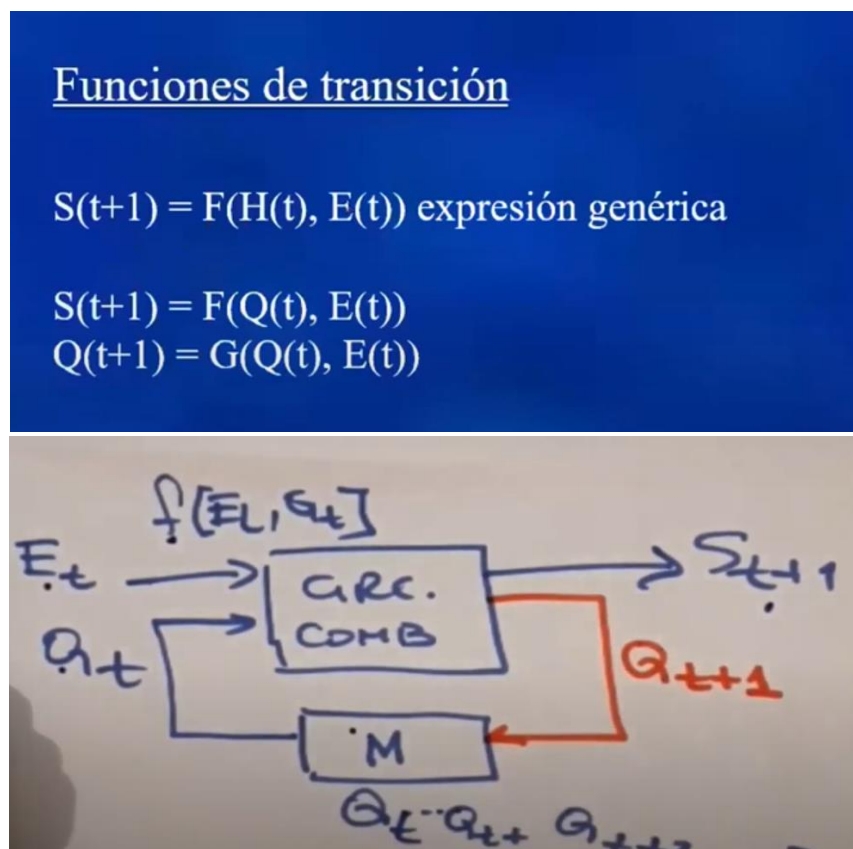
- a. Almacena un conjunto de estados finitos, y su comportamiento (Salida) es una función de los estímulos externos que tiene el autómatas (Entrada) y de los estados (Q) que tiene que atravesar para ofrecer un resultado (Salida).

Funciones de transición

El **estado Q (t+1)** del autómatas en el **instante t+1** no puede depender más que de su estado anterior Q(t) y de lo que recibe del exterior, es decir E(t).

Las funciones de transición se representan en forma de tablas o de diagramas.

Es decir, las salidas dependen **NO SOLO** de las entradas sino que también del **ESTADO** en que estaban las salidas y del tiempo.



F es una **función de transición**, porque vos al generar una Salida S_{t+1} , tenes que también actualizar tu Estado Q_{t+1} mediante M (memoria, que se encarga de almacenar el último salida).

DIAGRAMA DE TRANSICIÓN

Así se ve un **Diagrama de transición**:

Diagrama de transición

Grafo orientado que permite ver como se produce cada cambio de estado en función a una entrada y una posible salida

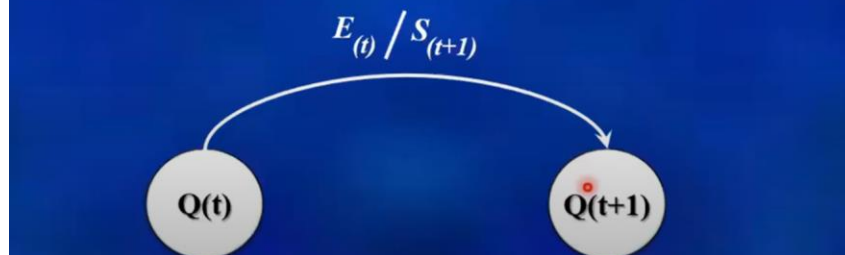


Tabla de Estados o Comportamiento o Funcionamiento

Matriz que permite ordenar en columnas los estados actuales, las entradas, y los estados futuros, en función al diagrama de transición definido.

A	B	x	A^f	B^f
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	1	0
1	0	1	1	1
1	1	0	1	1
1	1	1	0	0

Tabla de Excitación

Es como darle vuelta a la tabla de Estados, porque miramos primero la Estado Actual, Estado futuro, y entradas según el Biestable. Por ejemplo, la Tabla del biestable RS:

<u>Tabla de Excitación</u>			
$Q(t)$	$Q(t+1)$	S	R
0	0	0	1
0	1	1	0
1	0	0	1
1	1	X	0

BIESTABLES FLIP-FLOP

CONCEPTO

Un biestable es un circuito lógico con dos estados (manifestados a la salida) estables.

Un biestable almacena 1 bit.

El biestable es un **circuito realimentado**: la salida se inyecta en la entrada.

Los biestables son los elementos de memoria que se usan en los circuitos secuenciales temporizados. Estos circuitos son celdas binarias capaces de almacenar un bit de información. Un circuito FF tiene dos salidas, una para el valor normal (Q) y uno para el valor del complemento del bit almacenado en él (Q'). La diferencia principal entre los diversos FF está en el número de entradas que poseen y en la manera en la cual las entradas afectan al estado binario.

Características

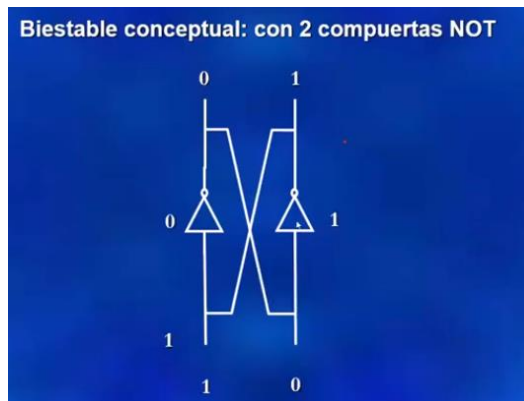
1. El FF es un sistema con dos estados estables (0 y 1). El estado es una situación interna del circuito. El estado de un circuito está determinado por los valores que los biestables pueden estar en ese momento.
2. La salida de un FF queda determinada tanto por las entradas actuales como por las historias de las mismas.
3. Cada FF puede almacenar/recordar un bit de información binaria.
4. El FF tiene dos señales de salida, una es complemento de la otra (Q y Q').
5. El FF se mantiene en uno de los estados mientras no sea excitado por señal alguna.
6. La mayoría de los FF tiene entradas de impulso y salidas de nivel.

Clasificación

- a) **Biestables SÍNCRONOS**: sus entradas pueden actuar sobre el estado del circuito solo en determinados instantes, cuando se activan los pulsos que llegan por otra entrada de sincronismo conocida como reloj o "clock".
- b) **Biestables ASÍNCRONOS**: sus entradas pueden actuar en cualquier instante para cambiar el estado del circuito, y en consecuencia, cambiar el valor de sus salidas.

Flip - Flop	Asincrónico	Sincrónico
RS	SI	SI
JK	SI	SI
D	-	SI
T	SI	SI
MS	-	SI

Biestable conceptual: con 2 compuertas NOT



Se comporta como un **punto de memoria**.

BIESTABLE RS

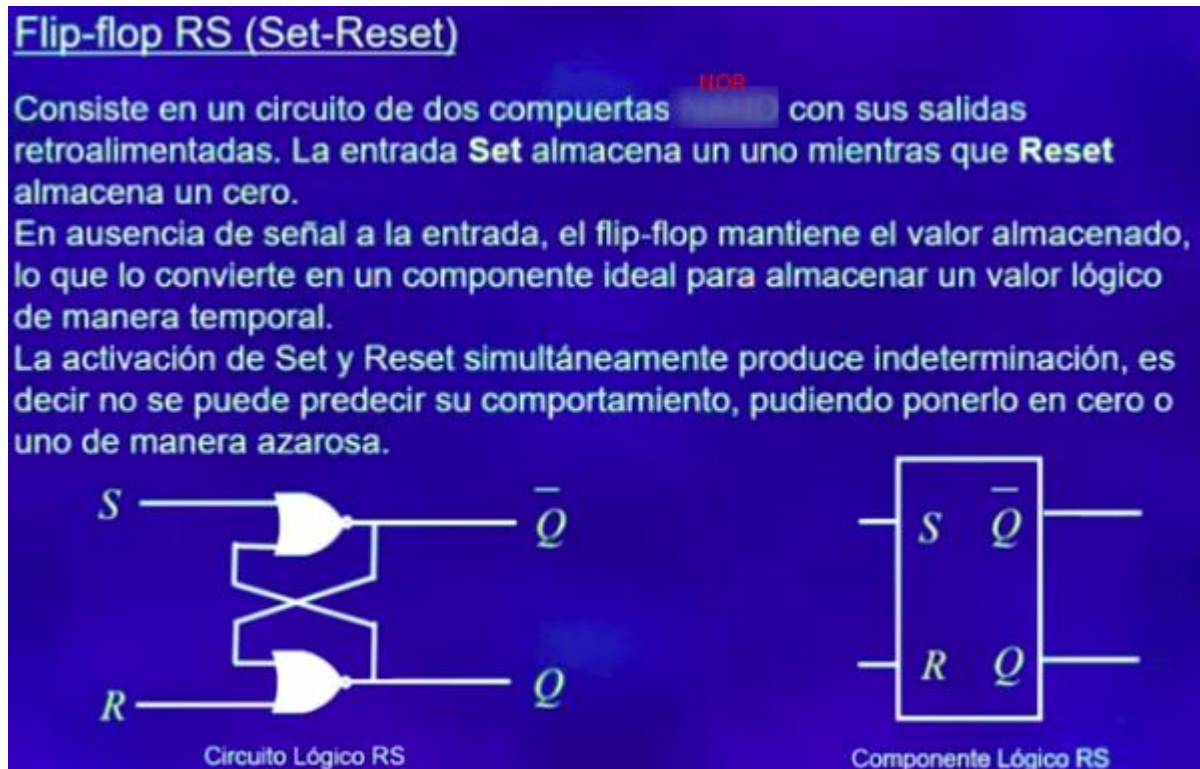
BIESTABLE RS - ASÍNCRONO (LATCH)

Flip-flop RS (Set-Reset)

Consiste en un circuito de dos compuertas **NOR** con sus salidas retroalimentadas. La entrada **Set** almacena un uno mientras que **Reset** almacena un cero.

En ausencia de señal a la entrada, el flip-flop mantiene el valor almacenado, lo que lo convierte en un componente ideal para almacenar un valor lógico de manera temporal.

La activación de Set y Reset simultáneamente produce indeterminación, es decir no se puede predecir su comportamiento, pudiendo ponerlo en cero o uno de manera azarosa.



Circuito Lógico RS

Componente Lógico RS

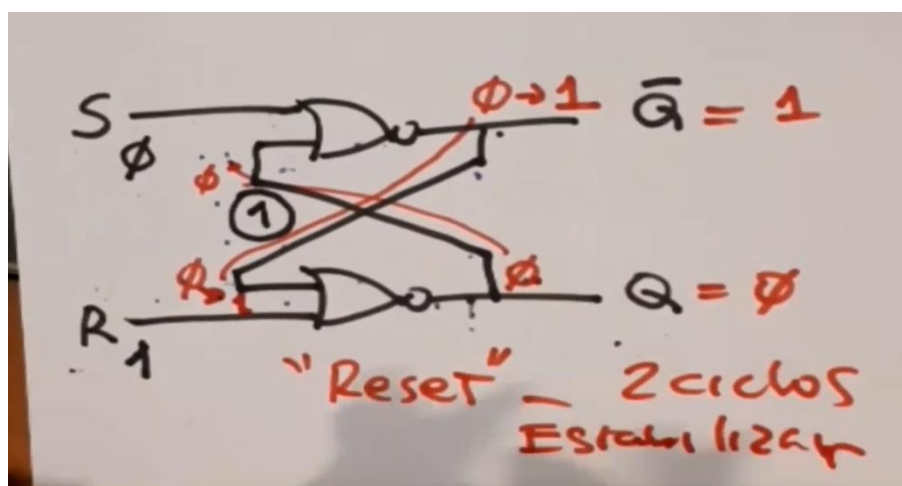
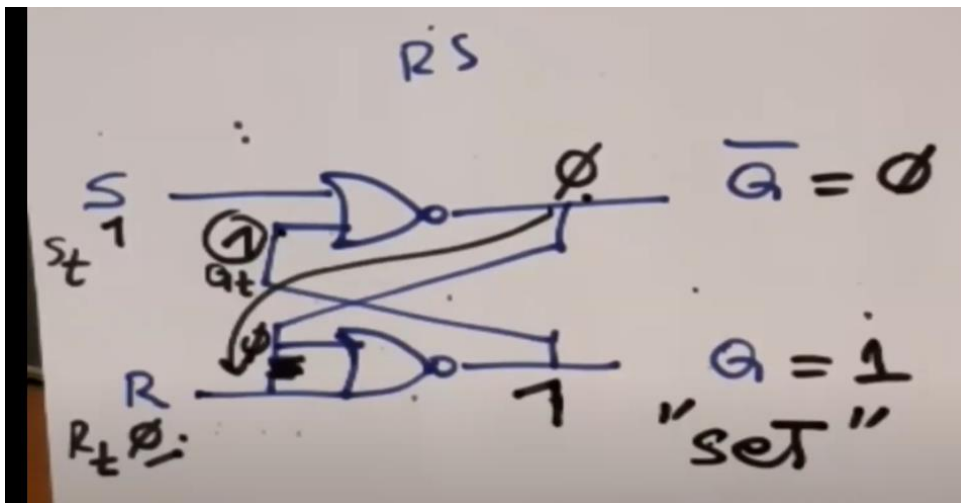
Es un circuito LATCH, porque no existe ningún clock que esté gobernando su funcionamiento. Solamente responde a la pulsación en la entrada en S o R para cambiar de estado.

Si se manda 1 y 1 en S y R para este Biestable RS, queda un estado "Indeterminado".

Tabla de comportamiento/estado/funcionamiento

	S	R	Q(t)	Q(t+1)
Mantiene el valor almacenado	0	0	0	0
	0	0	1	1
Puesta a Cero del flip-flop (Reset)	0	1	0	0
	0	1	1	0
Puesta a Uno del flip-flop (Set)	1	0	0	1
	1	0	1	1
Pone a Cero o Uno (Indeterminado)	1	1	0	?
	1	1	1	?

Ejemplos de entradas y salidas:



Mejor dibujado:

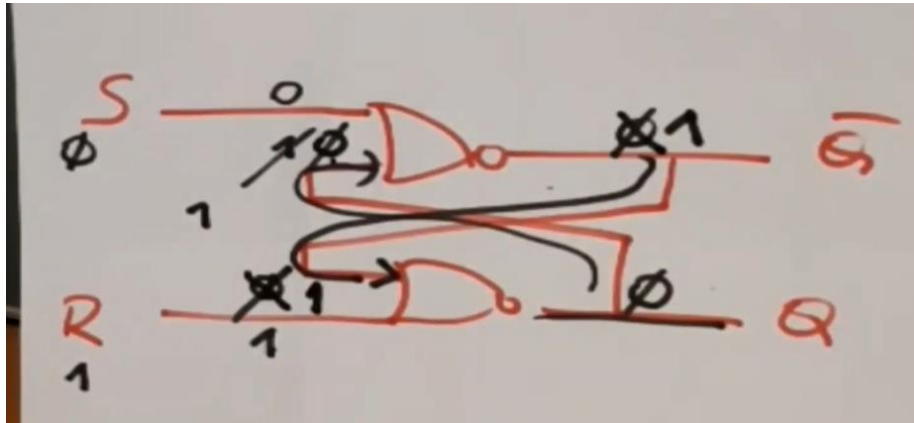


Tabla de Excitación

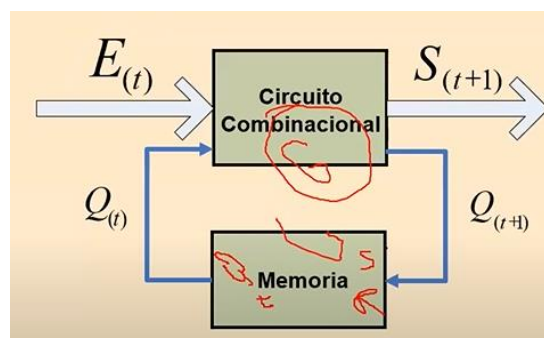
Tabla de Excitación

Para lograr determinar la función combinacional G del Circuito Secuencial, encargada de actualizar el estado del autómata, es necesario deducir los valores de entrada de los Flip Flops necesarios para producir cada transición de t a $t+1$, definido en el diagrama de transición.

Por tal motivo la Tabla de Transición es un resumen de la tabla de comportamiento donde, las entradas están planteadas como los estados en t y $t+1$, mientras que las salidas por las entradas de cada Flip Flop encargado de registrar dichos estados.

→ “con la tabla de excitación queremos deducir ante un cambio de estado qué acción quieres darle al flip flop para que manifieste el cambio de estado”

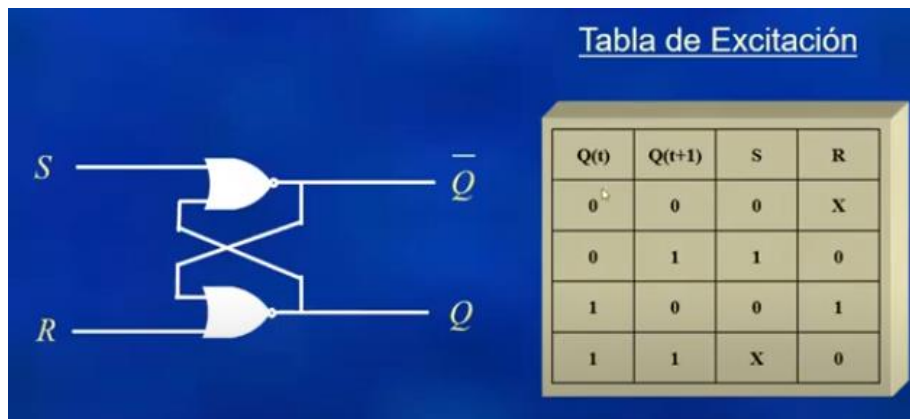
→ “se hace esta inversión (de que las entradas pasan a ser los estados, y los estados pasan a ser las entradas), porque se quiere deducir, del circuito, ante una nueva entrada y un estado anterior, qué tiene que accionar sobre las entradas del flip flop para actualizar el estado. Es decir, conocer cómo el circuito tiene que accionar el set y el reset para actualizar el estado de la memoria”



Matriz que permite ordenar en columnas los estados actuales, las entradas, los estados futuros, y las entradas a los elementos de memoria Q_t que registran cada estado del autómata.

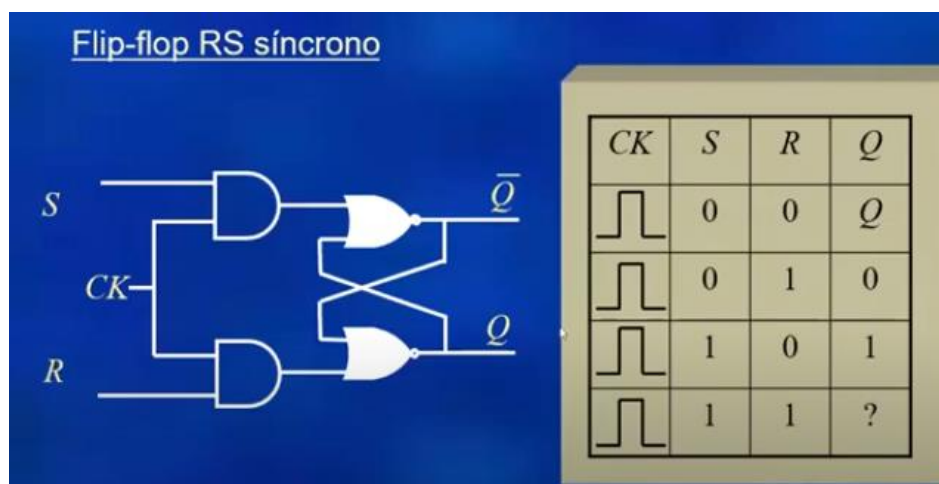
Ante un cambio de estado del flip-flop, qué tendríamos que modificar para obtener dichos estados.

Las entradas son los Estados, las salidas con las Entradas del Flip-flop.



Se suelen utilizar dentro de los diseño de los FPGA (Chips programables). Pero los latches son muy complicados de trabajarlos debido a su indeterminación, se prefieren los síncronos.

RS Síncrono (temporizado)



FLIP-FLOP D (Data)

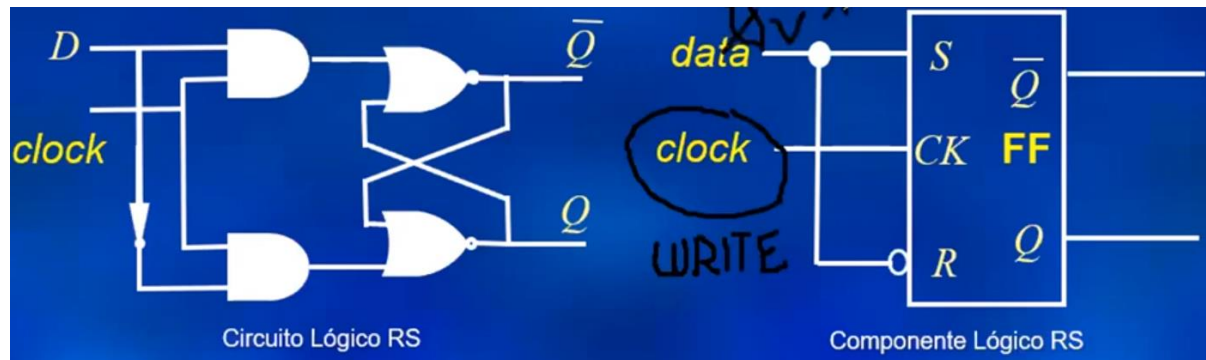
Para evitar la indeterminación que posee el flip-flop RS cuando ambas entradas son igual a 1, se opta por unir la entrada del Set a la del Reset a través de una compuerta lógica NOT. De ésta manera el flip-flop D almacena el valor lógico presente en su entrada (cero o uno según corresponda).

Lo que tiene de distinto, es la unión entre Set a la del Reset a través de una compuerta lógica NOT.

¿Por qué es necesario el Clock?

Debido a que la ausencia de entrada en D se interpreta como Cero lógico, es necesario su diseño de tipo SINCRONICO para evitar esta situación de RESET espontaneo. La entrada de Sincronismo Clock es interpretada como CARGA en bancos de memoria.

Circuito:



Cuando se intente utilizar como Unidad de Memoria, entonces el clock funciona como un "Write", y mantiene la información hasta la próxima interacción.

D _t	Q _t	Q _{t+1}
0	0	0
0	1	0
1	0	1
1	1	1

Tabla de Comportamiento

Q _t	Q _{t+1}	D
0	0	
0	1	
1	0	
1	1	

Tabla de Excitación

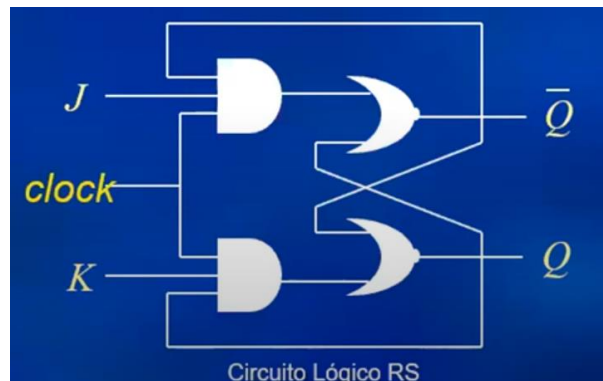
FLIP FLOP JK

Es una modificación del flip-flop RS donde $J=Set$ y $K=Reset$.

Agrega una nueva funcionalidad al circuito para el caso de indeterminación del RS.

Si $J=K=1$ **procede a complementar el contenido** lógico almacenado.

La retroalimentación entre $Q!$ con J y Q con K se activan solo en el caso que $J=K=1$, produciendo que las salidas actúen sobre las entradas complementando el valor almacenado.



J	K	Q_t	Q_{t+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Tabla de Comportamiento

Q_t	Q_{t+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

Tabla de Excitación

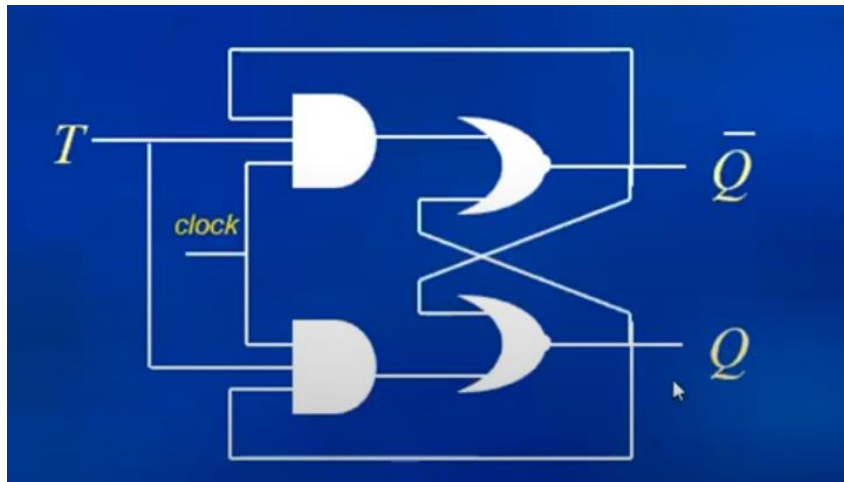
FLIP FLOP T (Toggle o Trigger)

Es una modificación del flip-flop JK, versión de 1 sola entrada del FF JK.

Este biestable cambia de estado alternando entre 0 y 1 cuando la entrada $T=1$ y clock = True.

Cuando $T=0$ mantiene el último estado almacenado.

Construido con compuertas NOR



Q_t	Q_{t+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

Tabla de Excitación

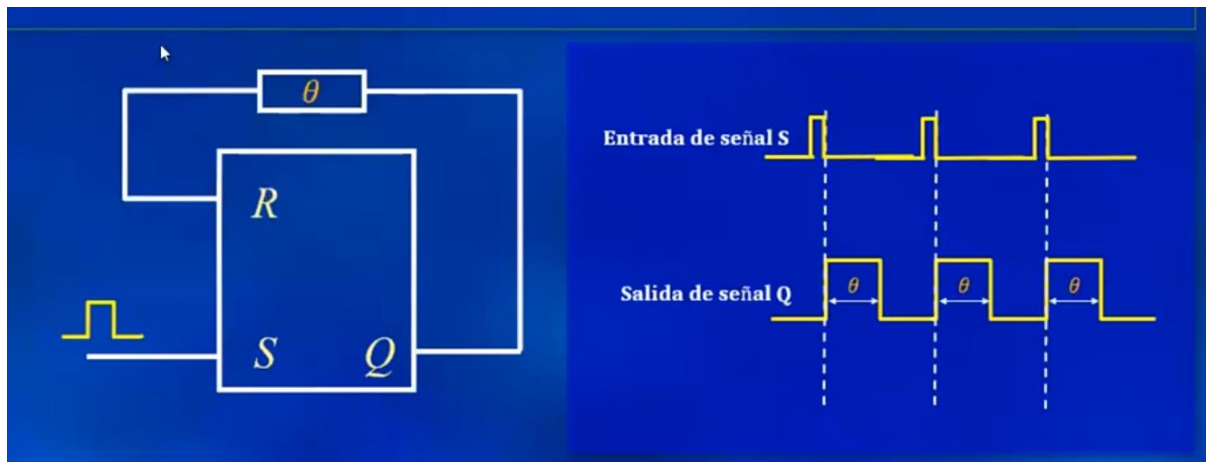
T	Q_t	Q_{t+1}
0	0	0
0	1	1
1	0	1
1	1	0

Tabla de Comportamiento

Los latches utilizan menos componentes electrónicos, menos compuertas, pero en el mundo físico pueden llegar a presentar un comportamiento errático.

MONOESTABLE

Es un sistema de DOS ESTADOS CON SOLO UNA POSICIÓN ESTABLE.



Cuenta con un temporizador, una amplitud “phi” que lo que hace es permitir que la señal del set se mantenga en el tiempo, hasta que pasado el período establecido, permita que la señal retroalimente al R (reset) y vuelva a establecer el estado como 0.

Se regula el reset

El estado estable es el 0.

El estado inestable es el 1.

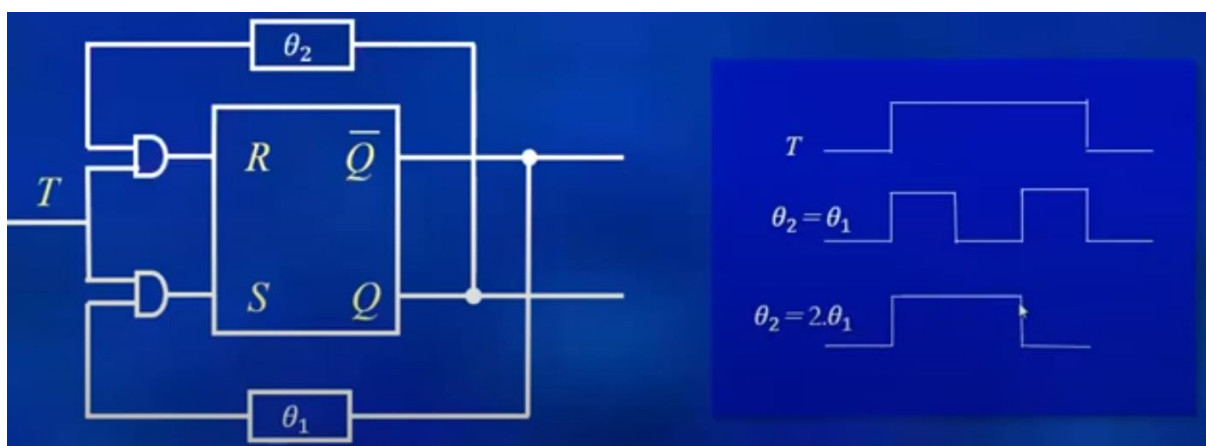
Ejemplos de monoestable:

Luces intermitentes, regular el tiempo que tarda en apagarse la luz, ajustar el tiempo de la tostadora, etc.

LA BÁSCULA

Es un sistema con NINGUNO de sus DOS ESTADOS ESTABLES.

Puede realizarse uniendo las dos entradas de un biestable JK y en electrónica se los conoce como **ASTABLE**. También se los puede implementar con el CI 555.

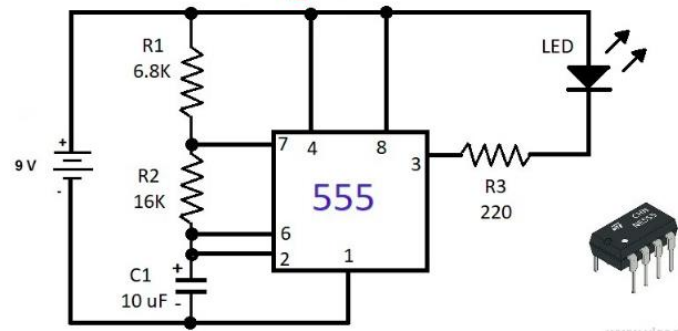


Entonces en el gráfico de la derecha, se puede ver como variando los tiempos de retardo se puede conseguir en caso de ser IGUALES se hace SET, RESET, SET, RESET.

Mientras que en caso de ser el reset el DOBLE del set, se hace el SET se mantiene y luego RESET.

Ejemplo de Báscula/Oscilador:

Luz intermitente con circuito integrado 555



SE REGULA EL SET Y EL RESET con Timers

El monoestable es un circuito de dos estados, de los cuales solo uno es estable (pudiendo ser el cero o el uno); en tanto que el astable es un circuito que no posee ningún estado estable. Este último es el que Meinadier lo define como "báscula" y una de sus aplicaciones es por ejemplo, el famoso (por ser muy usado) circuito integrado 555.

Nota:

El circuito integrado 555 se utiliza principalmente:

- **Modo Astable:** En este modo, el 555 opera como un oscilador. Puede generar una señal de salida en forma de onda cuadrada que oscila entre un estado alto y un estado bajo a una frecuencia determinada. Este modo se utiliza comúnmente para crear **generadores de pulsos**, y otros dispositivos de control de tiempo.
- **Modo Monoestable:** Cuando se aplica un pulso de disparo (trigger) en su pin correspondiente, el 555 produce un pulso de salida de duración específica. Este modo se utiliza para generar un retardo controlado por tiempo, por ejemplo, en **sistemas de control y automatización. También se utiliza en sistemas de seguridad, como alarmas, donde se necesita generar un pulso que active una señal de alarma durante un tiempo determinado.**

Biestable Maestro Esclavo

Un FF maestro esclavo se construye con dos FF por separado. Este consiste en un FF maestro, un FF esclavo y un inversor.

- Durante la **subida del pulso de reloj** se habilita el maestro y se deshabilita el esclavo. La información de entrada es transmitida hacia el FF maestro.
- Cuando **el pulso baja nuevamente a cero** se deshabilita el maestro lo cual evita que lo afecten las entradas externas y se habilita el esclavo. Entonces el esclavo pasa al mismo estado del maestro.

Lo mismo sucederá si el esclavo tiene una información binaria almacenada y el CP pasa a 0, entonces se habilitará el maestro y todas las entradas que afecten al biestable maestro no afectarán al esclavo mientras el CP se encuentre en 0.

El esclavo es el que tiene el estado final de todo el biestable.

Entonces, en un flip – flop maestro esclavo, es posible variar la salida y la información de entrada con el mismo pulso de reloj. Puede ocurrir por ejemplo que la entrada S venga de la salida de otro flip – flop maestro esclavo que fuera conmutado con el mismo pulso de reloj.

La combinación maestro esclavo puede construirse para cualquier tipo de flip-flop agregando un flip-flop RS temporizado con un reloj invertido para formar un esclavo.

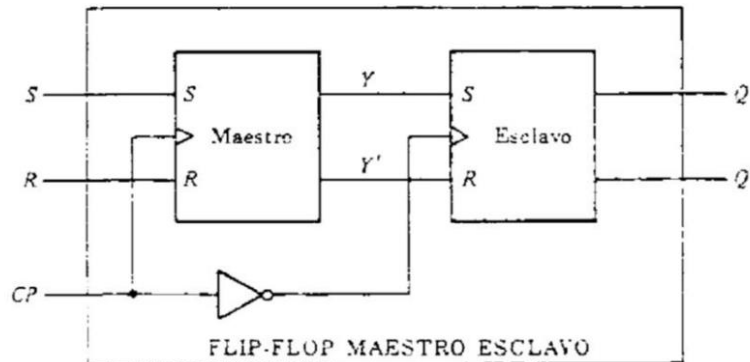


Figura 6-9 Diagrama lógico de un flip-flop maestro esclavo

Este diseño maestro-esclavo se utiliza para evitar **problemas de estabilidad** y transiciones no deseadas que pueden ocurrir en los flip-flops simples cuando la señal de reloj cambia cerca de la entrada. El flip-flop maestro-esclavo garantiza que el estado del flip-flop no cambie en medio de una operación crítica y, por lo tanto, **mejora la fiabilidad de los circuitos secuenciales**.

El flip-flop maestro-esclavo se utiliza ampliamente en la construcción de registros, **contadores**, y otros circuitos secuenciales en sistemas digitales y microprocesadores, donde es **esencial controlar con precisión cuándo se actualiza la información almacenada**.

Videos pa entender:

<https://www.youtube.com/watch?v=INMDcd4ud30>

https://www.youtube.com/watch?v=t2LZtaNck_g

Anotaciones Extra

Circuito Integrado CI555 para funcionar como Timer

CI 555

- **GND** : Polo negativo de la alimentación,
- **Disparo** : Establece el inicio del tiempo de retardo si el 555 es configurado como monoestable. Este proceso de disparo ocurre cuando esta patilla tiene menos de $1/3$ del voltaje de alimentación. Este pulso debe ser de corta duración, pues si se mantiene bajo por mucho tiempo la salida se quedará en alto hasta que la entrada de disparo pase a alto otra vez.
- **Salida** : Aquí veremos el resultado de la operación del temporizador, ya sea que esté conectado como monoestable, estable u otro.
- **Reinicio** : Si se pone a un nivel por debajo de 0.7 Voltios, pone la patilla de salida a nivel bajo. Si por algún motivo esta patilla no se utiliza hay que conectarla a alimentación para evitar que el temporizador se reinicie.
- **Control de voltaje** : Cuando el temporizador se utiliza en el modo de controlador de voltaje, el voltaje en esta patilla puede variar casi desde $V_{cc} - 1.7\text{ V}$ hasta casi 0 V (aprox. 2 V menos). Así es posible modificar los tiempos.
- **Umbral** : Es una entrada a un comparador interno que se utiliza para poner la salida a nivel bajo.
- **Descarga** : Utilizado para descargar con efectividad el condensador externo utilizado por el temporizador para su funcionamiento.
- **Voltaje de alimentación (VCC)** : es el terminal donde se conecta el voltaje de alimentación que va de 4.5 V hasta 16 V .

Consideraciones de diseño para los circuitos secuenciales

Consideraciones de diseño:

1. Hacer un diagrama de estado identificando las variables entrada (control) y salida. En el diagrama: un estado es un círculo, un flecha es una transición de un estado a otro. Indicar en cada arco la entrada / Salida producida.
2. El número de flip-flops necesarios para el circuito es el número de bits que tienen los estados.
3. Se realiza la tabla de estados y la tabla de excitación para cada flip-flop.
4. Se diseña el circuito combinacional para cada entrada de cada flip-flop usando mapas de Karnaugh.
5. Se implementa el circuito secuencial.

Ejemplo ejercicio ascensor:

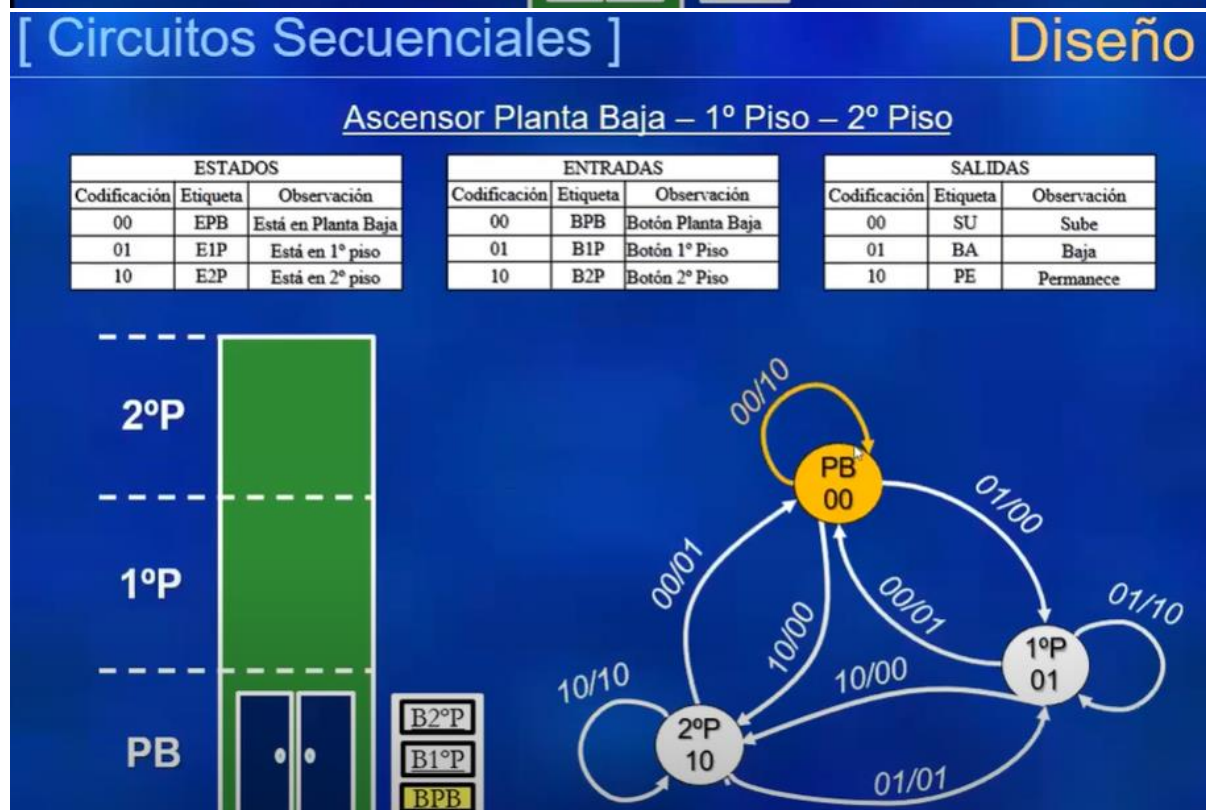
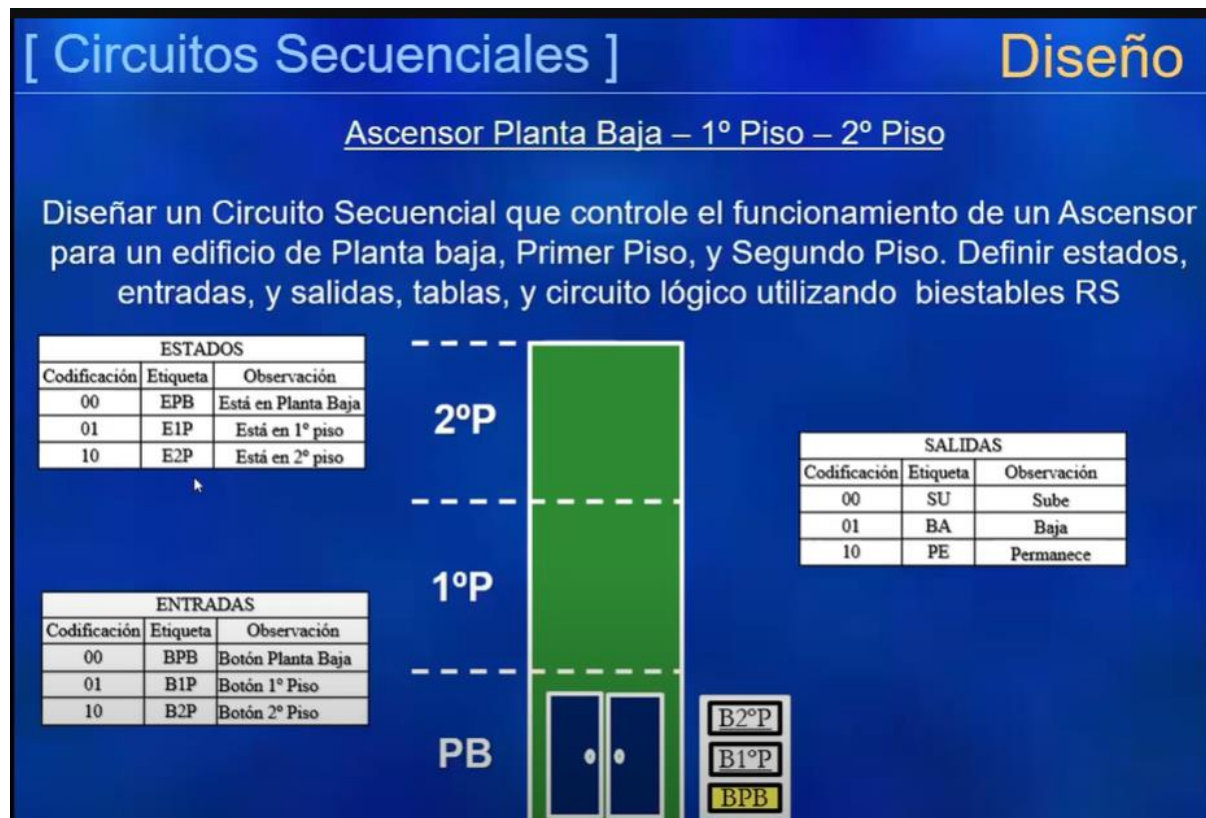


Tabla de Estados y Salidas

Estado Actual		Entrada		Estado Futuro		Salida	
$q0$	$q1$	$e1$	$e0$	$q1'$	$q0'$	$s0$	$s1$
0	0	0	0	0	0	1	0
0	0	0	1	0	1	0	0
0	0	1	0	1	0	0	0
0	1	0	0	0	0	0	1
0	1	0	1	0	1	1	0
0	1	1	0	1	0	0	0
1	0	0	0	0	0	0	1
1	0	0	1	0	1	0	1
1	0	1	0	1	0	1	0

Estado Actual		Entrada		Estado Futuro		Salida	
$q0$	$q1$	$e1$	$e0$	$q1'$	$q0'$	$s0$	$s1$
0	0	0	0	0	0	1	0
0	0	0	1	0	1	0	0
0	0	1	0	1	0	0	0
0	1	0	0	0	0	0	1
0	1	0	1	0	1	1	0
0	1	1	0	1	0	0	0
1	0	0	0	0	0	0	1
1	0	0	1	0	1	0	1
1	0	1	0	1	0	1	0

Está en EPB, Presiona Botón BPB, Permanece

Luego, se realiza la tabla de excitación

Estado Actual		Entrada		Estado Futuro		Salida	
$q0$	$q1$	$e1$	$e0$	$q1'$	$q0'$	$s0$	$s1$
0	0	0	0	0	0	1	0
0	0	0	1	0	1	0	0
0	0	1	0	1	0	0	0
0	1	0	0	0	0	0	1
0	1	0	1	0	1	1	0
0	1	1	0	1	0	0	0
1	0	0	0	0	0	0	1
1	0	0	1	0	1	0	1
1	0	1	0	1	0	1	0

Está en EPB, Presiona Botón B1P, sUBE

BUSES Y REGISTROS

Registros

Como mínimo debe contener la lógica necesaria para su lectura y grabación para poder inicializarlo. A través de una lectura, poder transmitir la información al órgano que lo requiera.

Un registro representa un arreglo de flip-flops organizados a través de una lógica combinacional y/o secuencial que lo capacita para realizar operaciones específicas sobre la información binaria contenida. Un registro es una función digital, que suele utilizarse para retener información binaria temporalmente. La principal característica de los registros es que poseen una gran velocidad de acceso a los datos y permiten almacenar los datos rápidamente, pero cuentan con poca capacidad de almacenamiento, se encuentran en el microprocesador que guardan la información temporalmente.

Permiten operaciones elementales:

- Puesta a cero
- El complemento

Permiten operaciones no-elementales (dependen del cableado):

- Desplazamiento a izquierda o derecha
- Incremento o decremento

Operaciones entre registros:

- Transferir el contenido
- Sumar o restar contenidos

Puede verse como una Memoria, la lógica asociada al mismo permite la carga y lectura de palabras en función a una dirección o ubicación dada por la **lógica de decodificación de direcciones**.

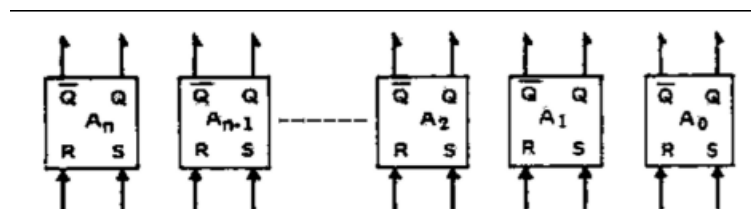


Fig. 68 Registro con biestables

TIPOS DE REGISTROS

Registros de datos: Usados para guardar números enteros. Antiguamente existía un único registro donde se guardaba toda la información en números binarios enteros, llamado **acumulador**.

Registros de memoria: Son usados para guardar exclusivamente direcciones de memoria. Muy usados en la Arquitectura Harvard ya que muchas veces las direcciones tienen un tamaño de palabra distinto que la de datos, se manejan por buses diferentes.

Registros de propósito general (GPR): Pueden guardar tanto datos como direcciones. Son fundamentales en la arquitectura Von Neuman. La mayor parte de las computadoras modernas usa GPR. Que sobre la marcha se le puede agregar un atributo particular.

Registros de coma flotante: Son registros para cálculos algebraicos, pero tienen un manejo de coma flotante tal que permite manejar los números decimales.

Registros de constantes: Registros de valores creados por hardware de sólo lectura. Alimentados por la batería del computador. Ejemplo: Valor de frecuencia del clock.

Registros de propósito específico: Guardan información del estado del sistema, como el puntero de pila o el registro de estado.

Estructuras de pilas o registros: Ejemplo, saltos a subrutinas en un contador de programa con organización de tipo pila. Se tienen que guardar para poder retornar al punto desde donde se había hecho la llamada.

Registro de marcha de tensión, permite que arranque o se apague el computador.

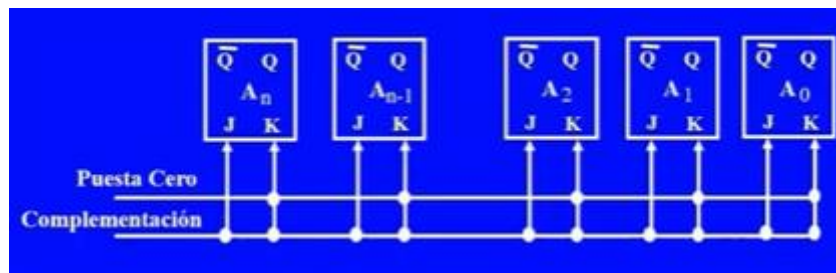
Diagrama conceptual con RS (ejemplos):



Gobierno de las operaciones → lógica asociada a qué función va a cumplir el registro además de almacenar la información. Como desplazamiento, contar, descontar, etc.

Gobierno de las entradas/salidas → Referido a la lógica combinacional, le permite recibir información a través del BUS o a quién enviar a quién lo necesite. **SI O SI tiene que estar para que esto sea considerado un Registro. → necesaria para que se saque información**

OPERACIONES ELEMENTALES DE REGISTROS



Puesta a cero: Se manda un impulso para todas las entradas en K, y forzaría a hacer todos un Reset.

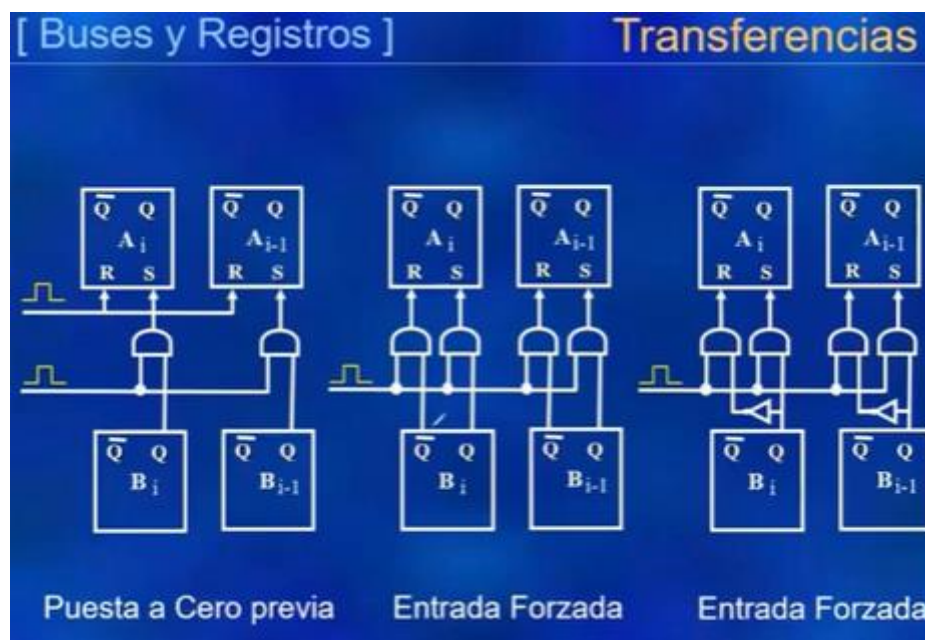
Complementación: Se envía una señal de doble 1 en J y K para realizar una complementación.

Todo esto se va a producir en el tiempo que le lleve producir en el tiempo que dure el impulso (tita).

Transferencias en paralelo

Una operación de transferencia entre registros es una operación básica en sistemas digitales y consiste en la transferencia de la información almacenada de un registro a otro. Puede transferirse la información de un registro B a otro registro A:

- Por dos impulsos sucesivos, uno de puesta a cero y el otro de transferencia de los 1
- Por introducción forzada de los 0 y de los 1.



Transferencia desde el registro B hacia el registro A.

Incursionar significa emitir una señal del mismo valor que el reloj.

Con la puesta a cero, lo que nos aseguramos es que los dispositivos electrónicos estén **listos** para la transferencia. Como por ejemplo con registros de núcleos magnéticos que

necesitaban realizar la puesta a cero primero, o en la utilización de capacitores cuya descarga es lenta.

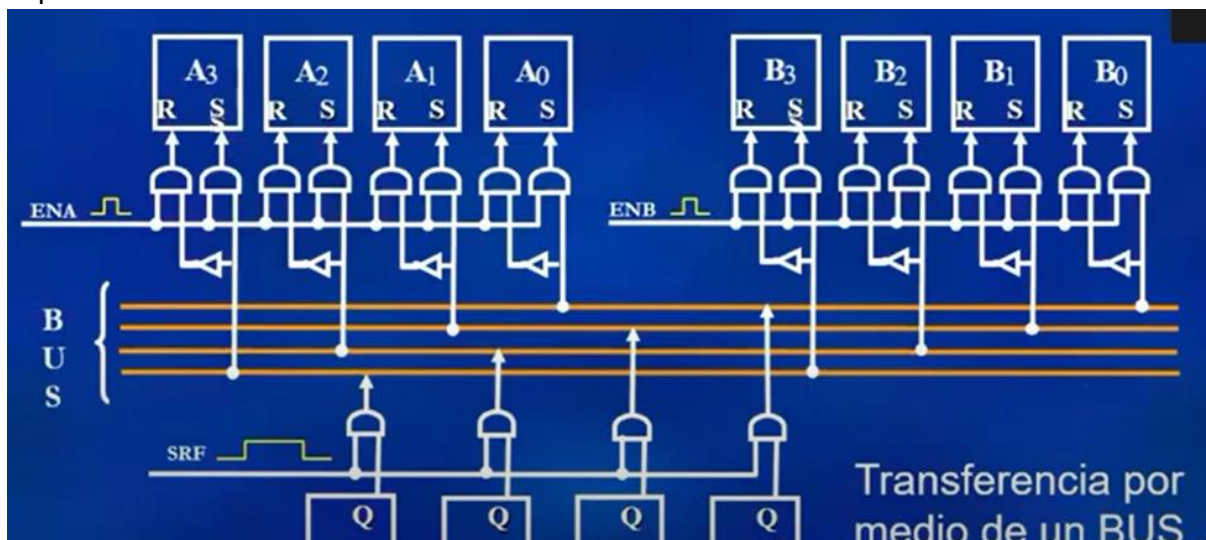
BUSES

BUS es una palabra inglesa que significa “transporte” y consiste en un arreglo de líneas lógicas dedicadas a interconectar órganos del computador entre sí, interconectando varios registros (Registros fuente con Registros destino, que son registros dedicados dentro de cada órgano), con el objeto de transportar configuraciones binarias presentes en sus registros.



Los órganos están casi siempre conectados a un bus como mínimo para comunicarse con otros órganos, pero hay excepciones como las de la ALU, la cual solamente necesita recibir sólo información de datos y no así del bus de direcciones.

Buses son un conjunto de hilos eléctricos, que forman pistas metálicas en los registros impresos.



Cada hilo del BUS va a estar relacionado con 1 y solo 1 biestable del registro.

Unidad de control

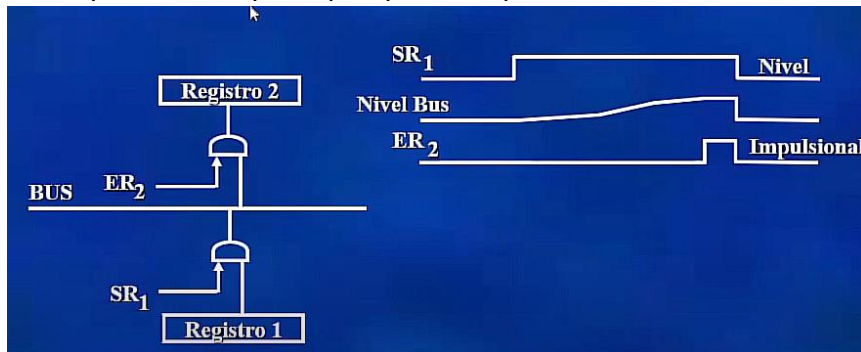
Va a gobernar el tráfico de ese Bus, es el que va a manejar cuándo está disponible el BUS para la transferencia de A a B. Cuando no se puede ocupar, porque está siendo ocupado por el órgano C, etc.

Proceso

Mediante señales de gobierno de nivel permite estabilizar los valores lógicos de Registro A sobre el BUS.

Tiempo de llenado de BUS:

Es lo que se tarda para que quede disponible en todos los hilos.

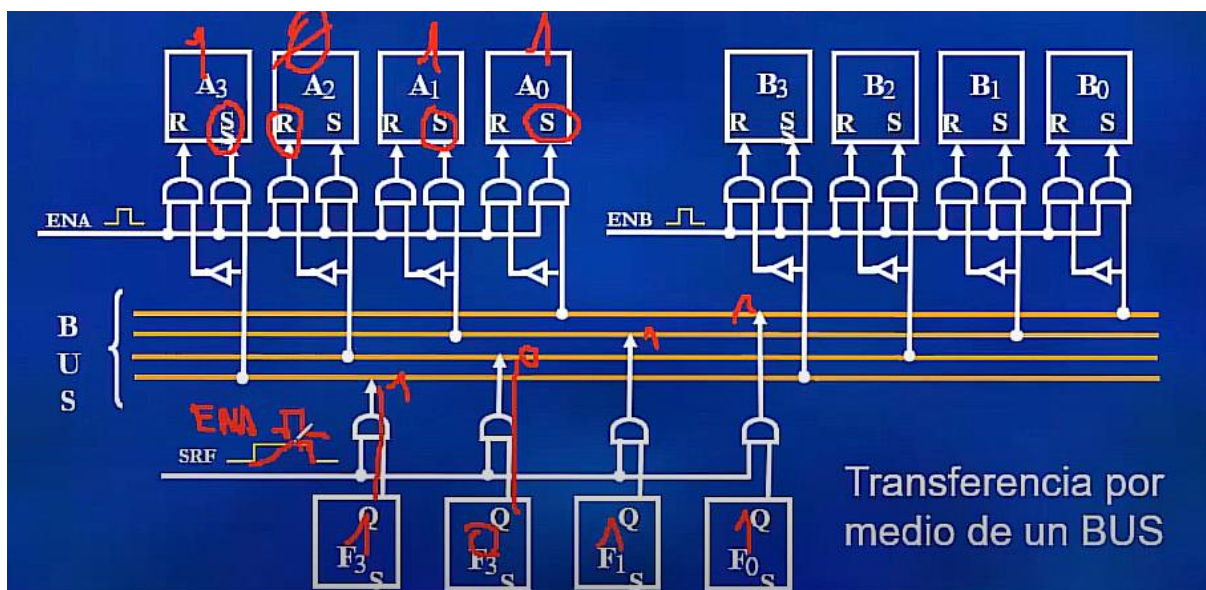


Ejemplo de funcionamiento:

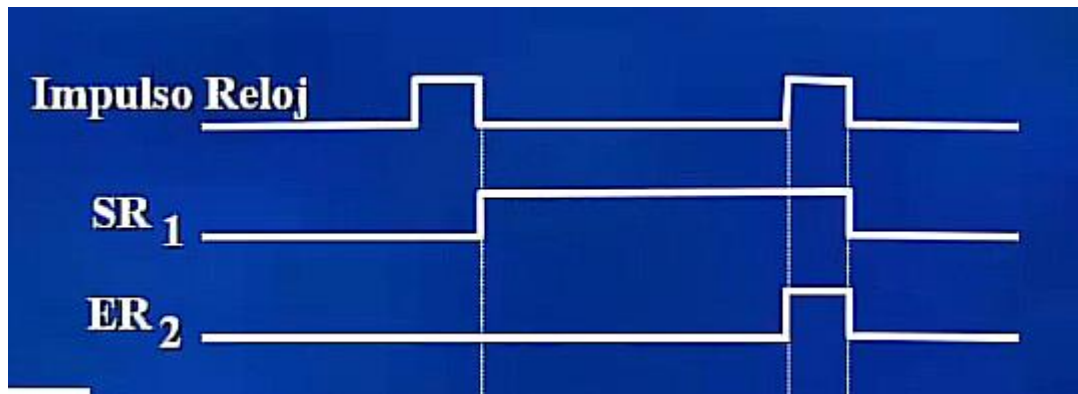
Se necesita una **señal de gobierno DE NIVEL**, mucho más larga, para permitir que el valor del registro Fuente se establezca en los hilos que componen el BUS.



Ahora a través de otra **señal de gobierno de IMPULSIONAL** se procede a ingresar los valores lógicos presentes en el BUS hacia el Registro B destino.



Entre un impulso y otro no puede haber en el medio una impulsión:



Registros especiales

Configuración para Registros de Desplazamiento

Cableando adecuadamente el entorno de un registro de biestables se consigue, en respuesta a un determinado número de impulsos, desplazar el contenido del registro de uno o varios dígitos a la izquierda o derecha. Un registro de desplazamiento es un registro capaz de *desplazar su información binaria* hacia la izquierda o hacia la derecha. La configuración lógica de un registro de desplazamiento consiste en una **cadena de FF conectados en cascada**, con la salida de un FF conectado a la entrada del siguiente. Todos los FF reciben un pulso de reloj común el cual causa el desplazamiento de un estado al siguiente. La salida Q de un FF dado, se conecta a la entrada S, mientras que la salida Q' a la entrada R, del FF a la derecha (en caso de desplazamiento a derecha). Cuando se trabaja con FF D, la salida Q va conectado a la entrada D, del FF de la derecha.

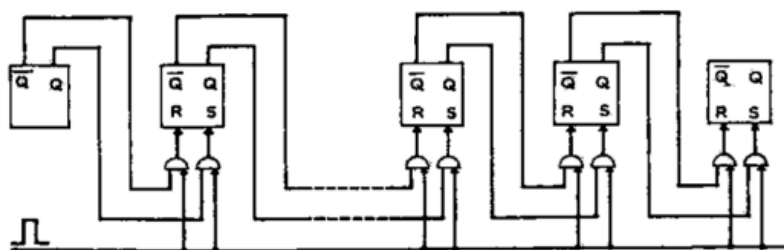
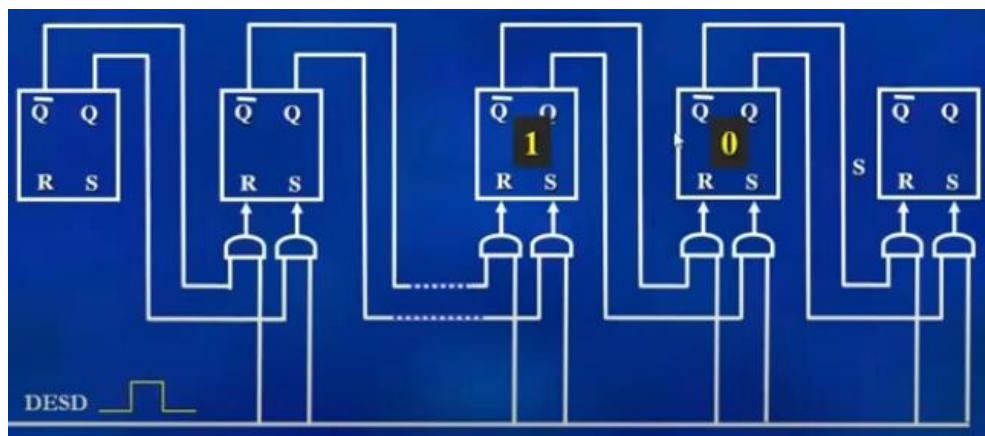


Fig. 74. Registro de desplazamientos a derecha



Configuración para Registros contadores

Son registros cuyo contenido se incrementa en 1 cada vez que se presenta un impulso a la entrada de cuenta. Son capaces de contar el número de impulsos recibidos. Trabaja de la siguiente manera: el impulso ataca al biestable de menor peso posicional. Si está a cero, debe pasar a 1; si está a 1, debe pasar a 0, pero debe propagar el impulso al biestable inmediato con objeto de tomar en cuenta el arrastre. Por consiguiente, el biestable debe ser complementado en todos los casos y *debe propagarse el impulso solamente en aquellos casos en que el biestable estaba a 1 antes de la complementación* (condición para que haya arrastre).

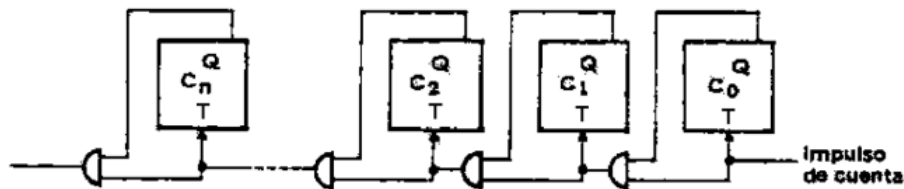
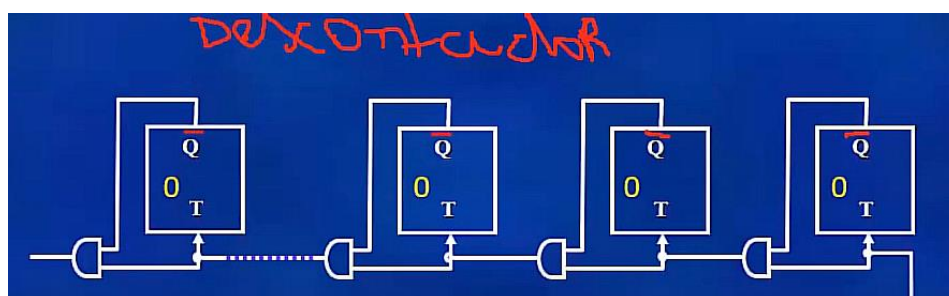


Fig. 76. Contador binario



Descontador

Con el mismo esquema/circuito que el contador, solamente se puede tomar el !Q y de esa manera se van a ir descontando los valores, hasta pasar por ejemplo de 1 1 1 1 hasta 0 0 0 0, y si se llega a conectar la salida nuevamente, se realiza un descontador cíclico.



ABACUS

Está diseñada como una máquina síncrona con un registro de instrucción de UNA dirección (código de operación y dirección del operando) en tal sentido los cálculos se realizan contra el valor almacenado en el registro Acumulador de la ALU.

Es de **UNA DIRECCIÓN**.

El reloj sincroniza todos sus componentes produce un batido por cada Θ segundos. siendo el **ciclo base de memoria** de 2Θ .

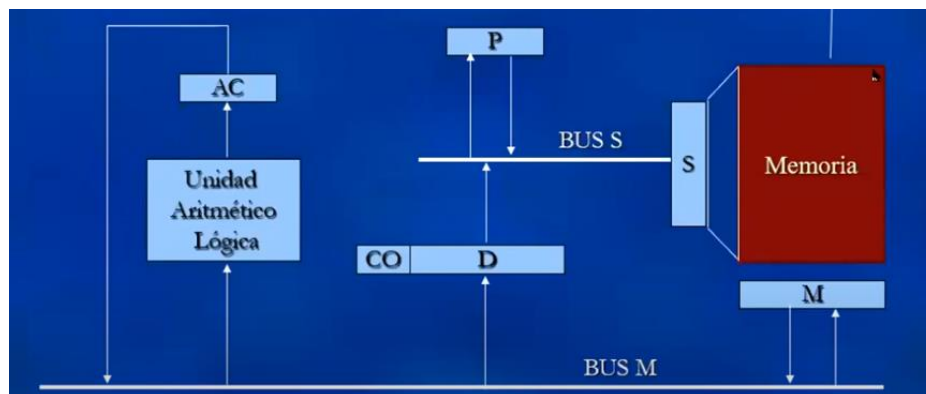
Arquitectónicamente posee:

- Registro en la ALU que se encarga de mantener los resultados parciales.
- Bus M o de instrucción y operandos.
 - Lleva la próxima dirección al registro Instrucciones para ser decodificada. Une a la memoria con el registro de instrucción, cuando es instrucción. Une a la memoria con la ALU, cuando es un operando.
- Bus S de direcciones y operandos.
 - En caso de que ésta deba buscar un operando, el campo DIR proveerá al Bus S el valor de la ubicación del operando en memoria, el que será leído y llevado a la ALU para su cálculo

MEMORIA

La **memoria** principal está compuesta por 2^m Registros o palabras de **n bits**. m sería cantidad de palabras de memoria (para el caso Direcccionamiento Directo) direccionables por S.

- Dividida en Programas y Datos
- Las palabras pueden ser seleccionadas independientemente por su posición en relación al valor del **Registro S** a la hora de Lectura o Escritura.
- Se relaciona directamente con el **Registro M** de intercambio o palabra, y a los restantes órganos a través del **Bus de Datos e Instrucción M**.
- **Registro S** sirve para buscar la Dirección de la Instrucción o la Dirección del Operando.

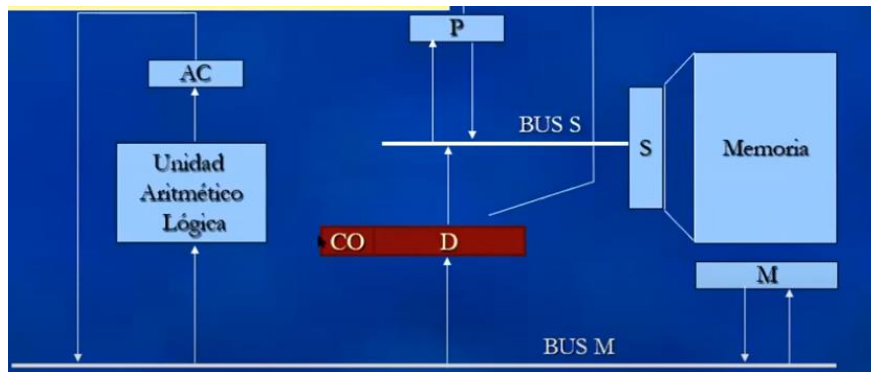


REGISTRO DE INSTRUCCIÓN

I (Registro de instrucción) de **n bits** compuesto por:

- Código de operación: p bits
- Dirección: m bits

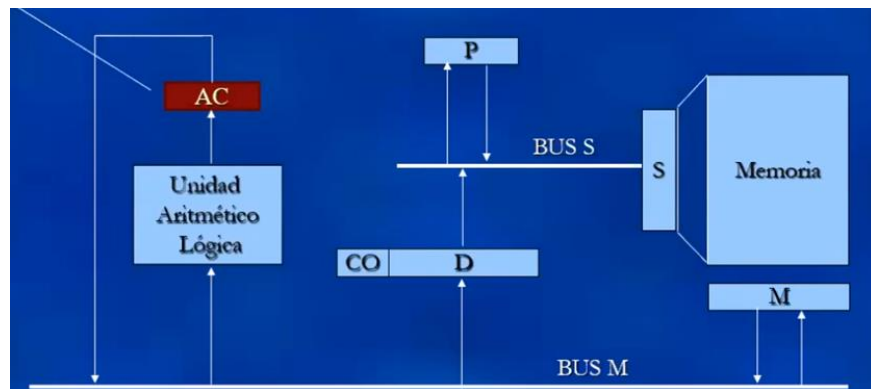
El campo D permitirá el direccionamiento de 2^m Operandos

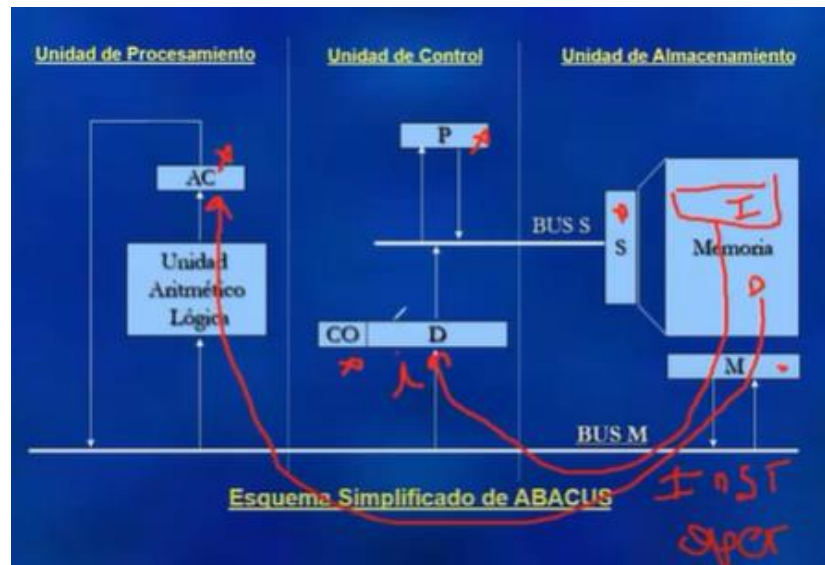


REGISTRO ACUMULADOR

Registro acumulador de **n bits**. Opera contra el contenido del registro M y el resultado es escrito sobre AC sobrescribiendo su contenido.

Ejemplo: Si C.O (Código operación) es SUMA entonces $(M) + (AC) \rightarrow AC$





Si es una instrucción, se va al Registro I para ser decodificado para ser interpretado por la máquina.

Si es un operando, se lleva al registro del acumulador de la ALU.

CONCEPTOS

P - Registro Contador - Que arranca en 0 y va incrementándose en 1 en su valor, así su contenido se transfiere como dirección de la siguiente instrucción I.

Teniendo en cuenta también las instrucciones de salto, saltos condicionales e incondicionales, estas operaciones modifican directamente dicho registro para mantener la consistencia.

I - Registro Instrucción - Recibe una instrucción de memoria, está dividido en COD_OPERACION y DIR_OPERANDO.

BUS S - Maneja solo dirección del Registro I, o también del Registro P (de la siguiente instrucción).

P -> S – en caso de búsqueda de siguiente instrucción

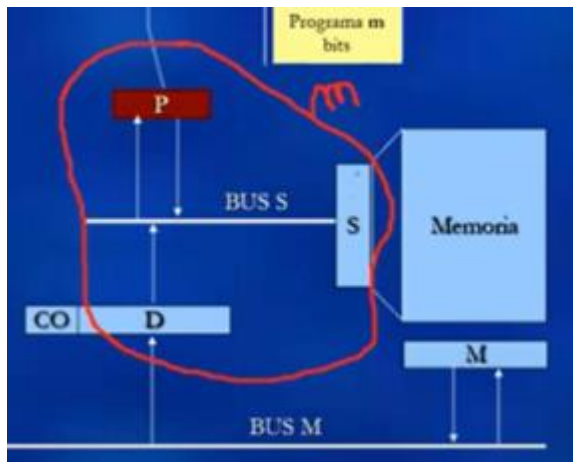
D -> S – en caso de búsqueda del operando de la instrucción en curso para su resolución

S - Registro Selección - Almacena dirección de una operación o de un operando. (Dependiendo del ciclo)

Explicación de los registros y sus tamaños

Registro Instrucción:

- **p** = n° de bits de CO (Código Operación)
- **m** = n° de bits de Dirección



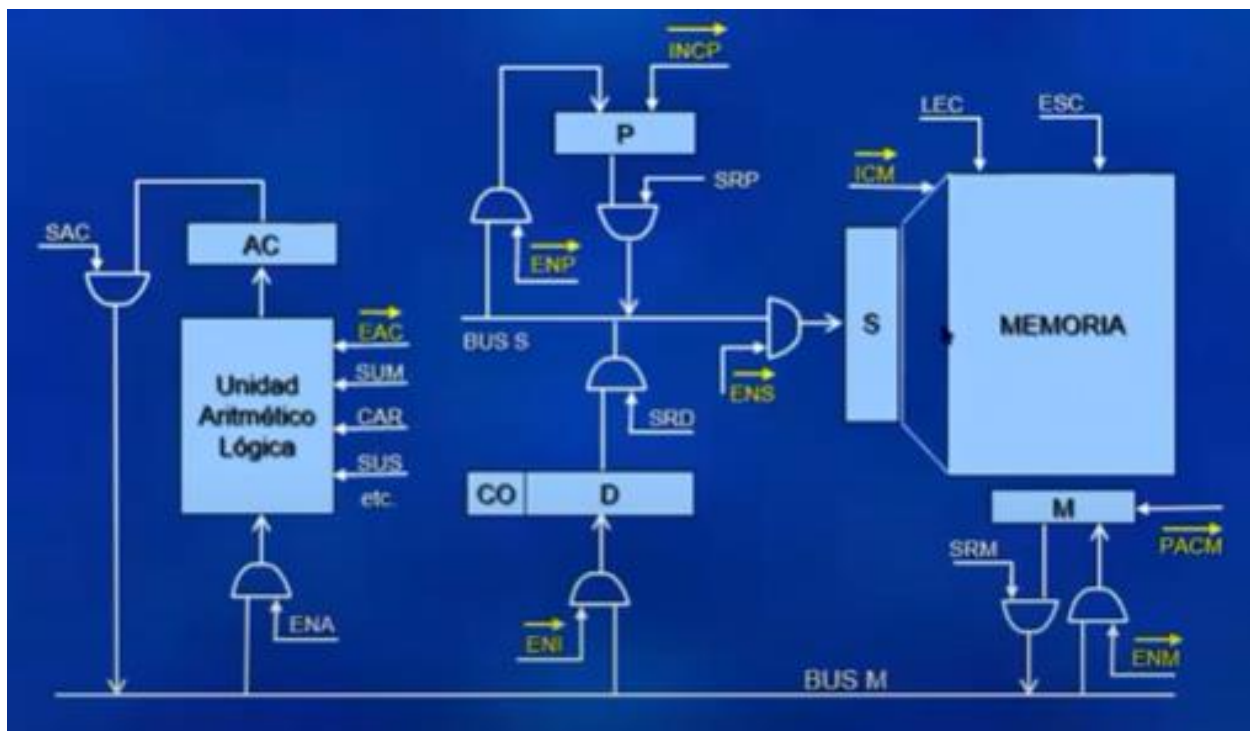
$n = p + m$ - - Que es el tamaño del **Registro I** (el que está en el medio de la pantalla).

SEÑALES DE GOBIERNO DE ABACUS

[UTN 20- Arquitectura de Computadoras - Teoría, ABACUS Señales de Gobierno Unidad Aritmético Lógica](#)

Impulsionales, se les pone una flechita arriba.

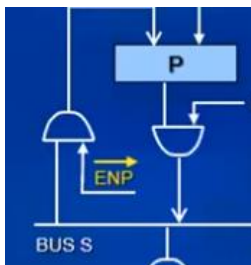
De nivel, se les deja así nomás, sin flechita



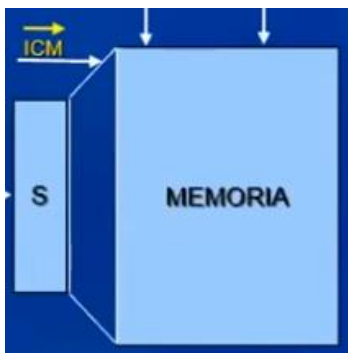
Señales de Transferencia	1. ENS	Impulso	(bus S) → S
	2. PACM	Impulso	0 → M
	3. SRM	Nivel	(M) → bus M
	4. ENM	Impulso	(bus M) → M
	5. ENI	Impulso	(bus M) → I
	6. ENA	Nivel	(bus M) → UAL
	7. SAC	Nivel	(AC) → bus M
	8. ENP	Impulso	(bus S) → P
	9. SRD	Nivel	(D) → bus S
	10. SRP	Nivel	(P) → bus S
	11. INCP	Impulso	(P) + 1 → P

SRP - Salida Registro P - NIVEL - Porque el contenido del contador de programa se tiene que estabilizar en el Bus S.

Aclaración:** El ENP - Entrada Registro P - IMPULSIONAL, es fundamental para cuando se realiza un salto del programa, porque se tiene que sobrescribir el valor de P con la dirección a la que se desea saltar existente en D, por lo cual es necesario primero sensibilizar el BUS S con dicha dirección, y después con la ENP se realiza la sobrescritura.

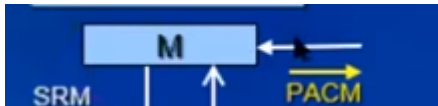


ICM - Inicio Ciclo de Memoria - IMPULSIONAL - es ciclo de memoria de aviso, activa el decodificador de direcciones. Marca el inicio de un ciclo de memoria. Cuando un órgano requiere leer/escribir en la memoria.



La LECTURA o ESCRITURA puede tomar más de un ciclo de memoria, por eso son de nivel, se necesita estabilizar dicho valores.

PACM - Puesta a Cero de Registro M - IMPULSIONAL - Cuando se requiere de un acerado (puesta a cero) rápido. Es impulsional



ENM: Entrada registro memoria: copia del bus M y lo pone en el registro memoria, para guardar.

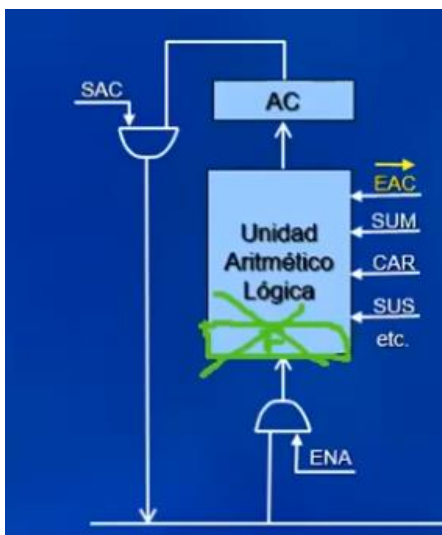
SRM - Salida Registro Memoria - DE NIVEL -

A través de una lectura, la palabra leída se mueve al Registro de Memoria (la estabiliza) y se la pasa al BUS M.

Estas 2 señales son para almacenar un valor (resultado de AC por ejemplo) en la Memoria, o sino SRM que es para estabilizar una dirección u operando en el BUS M.



ENA - Entrada Acumulador - DE NIVEL - por lo cual se utiliza el BUS M como fuente de información para la ALU, porque no se cuenta de un Registro F, entonces no es necesario liberar rápidamente el BUS M porque no existe conexión con Periféricos. De ésta manera se puede ahorrar la existencia de un registro F (Fuente) para la ALU, y se mantiene el valor en BUS M todo lo que dure la operación.



Todas las operaciones de cálculo son de nivel, porque necesita realizar la operación y propagar el cálculo (almacenarlo).

- **SUM**
- **CAR** (Cargar acumulador)
- **SUS, etc.**

EAC - Entrada Registro Acumulador - IMPULSIONAL - Señal de gobierno, mover el resultado del cálculo de la ALU al Acumulador, sobrescriba el resultado del Acumulador.

Las señales que mantienen el valor son las de cálculo mencionadas arriba, entonces esta Impulsional mueve el valor al AC.

SAI - Salto incondicional, y ahí va la dirección del salto.

Es una instrucción para romper con la secuencia del contador del programa.

Acá es lenguaje de máquina, ubicación física.

UAL - Unidad Aritmético Lógica

La UAL es la encargada de ejecutar todo tipo de instrucciones de cálculo.

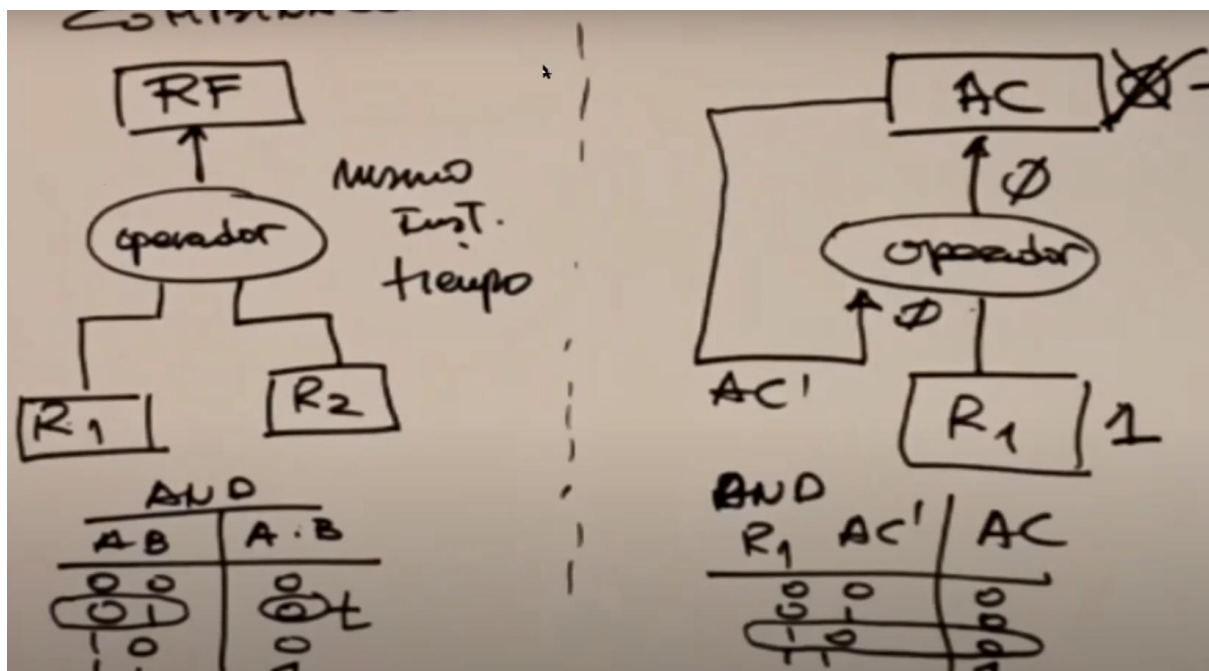
Para cumplir esta función dispone de Operadores o Unidades Funcionales destinada a realizar una o varias operaciones aritméticas o lógicas.

Operador aritmético



Carácter:

- Combinacional
- Secuencial

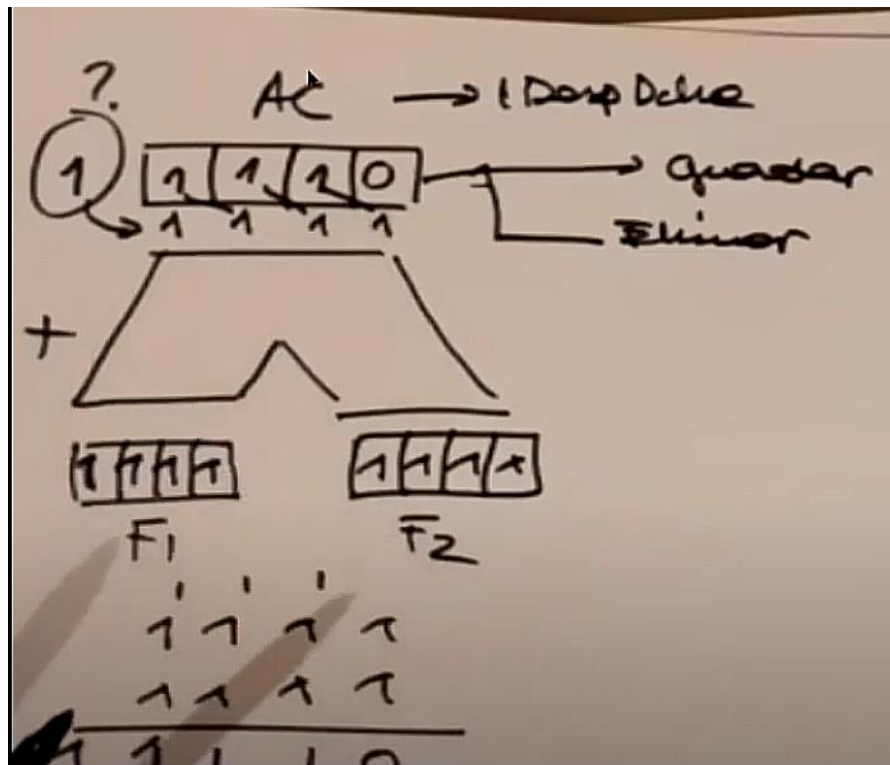


Indicadores del Operador Aritmético:

- Última operación Realizada
- Error de Desbordamiento
- Error de desborde de capacidad
- Error de División por Cero
- Estado Acumulador Positivo
- Estado Acumulador Negativo
- Estado Acumulador Igual a cero

Error de desbordamiento - En caso de desbordamiento se debe realizar el desplazamiento a derecha. Si perdemos el bit de mayor peso, estamos en un problema, por eso no hay problema de perderlo al bit de menor peso.

Ejemplo:



Error de división por cero.

Estados Acumulador:

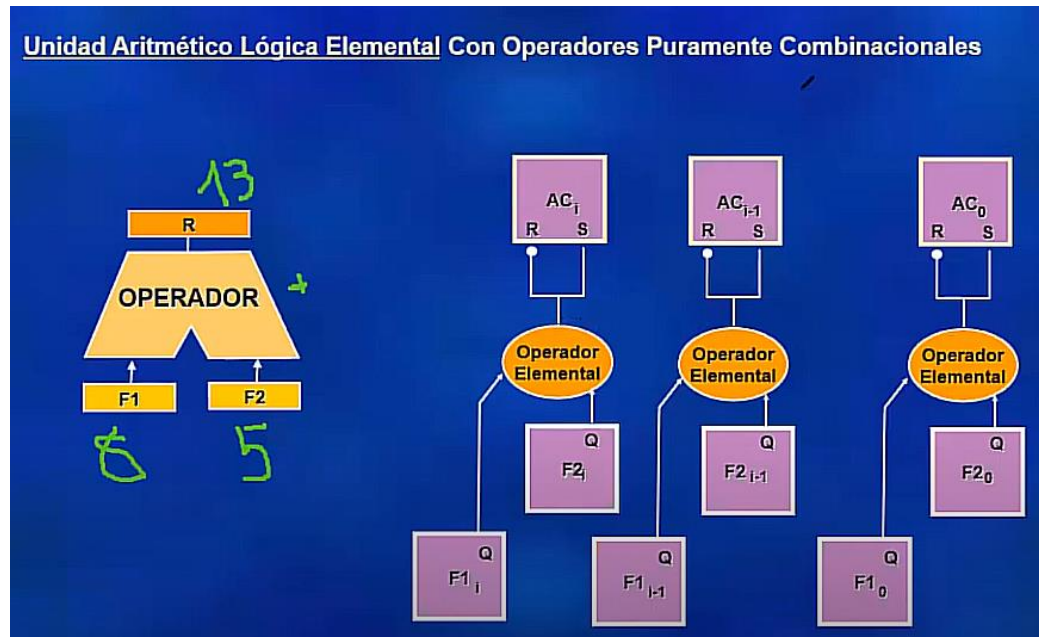
- Positivo
- Negativo
- Igual a cero

Son necesarios para determinar los **saltos**.

Operadores combinacionales y secuenciales

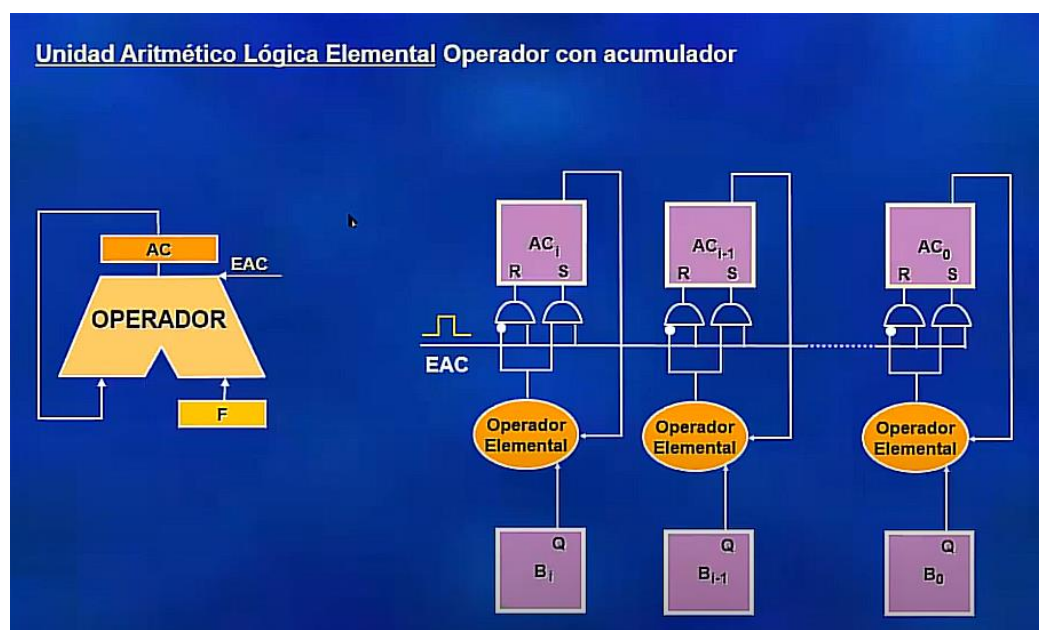
Los operadores pueden clasificarse en función del carácter **combinacional** o **secuencial** de su concepción.

Operador combinacional



Los **operadores combinacionales** ejecutan las operaciones en una sola fase y por esto pueden ser gobernados por señales de nivel, que permanecen activas durante toda la operación. Se realizan en el mismo instante de tiempo.

Operadores con acumulador (secuencial)

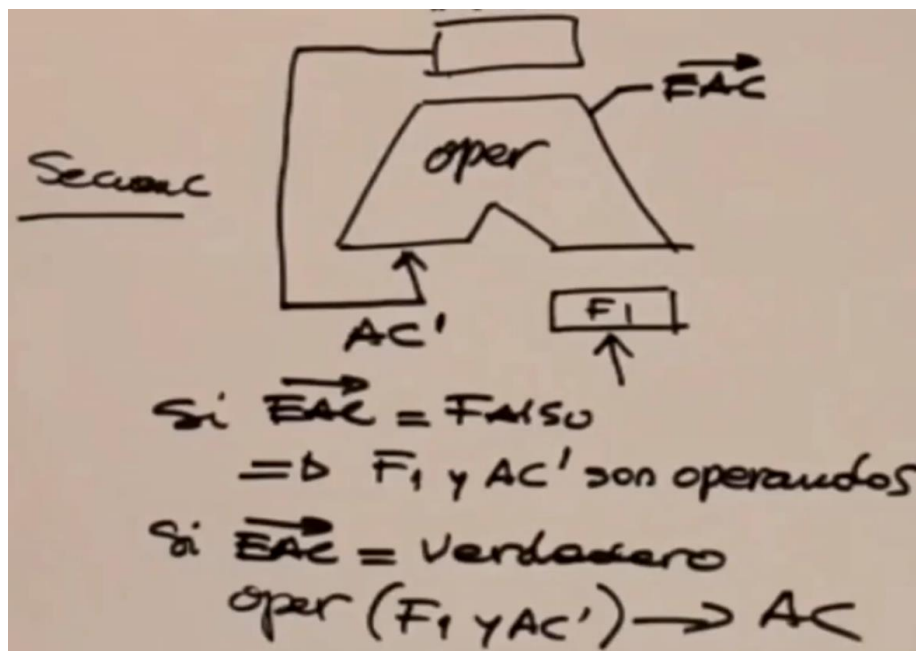


Los **operadores secuenciales** ejecutan las operaciones en varias fases gobernadas por impulsos distribuidos por un dispositivo de control (puede ser la UC o un órgano del

operador). Los operadores constan entonces de dispositivos de memorización de los resultados parciales. La salida es función de tiempo de la entrada. Y depende tanto de la salida del instante anterior en conjunción con la nueva entrada.

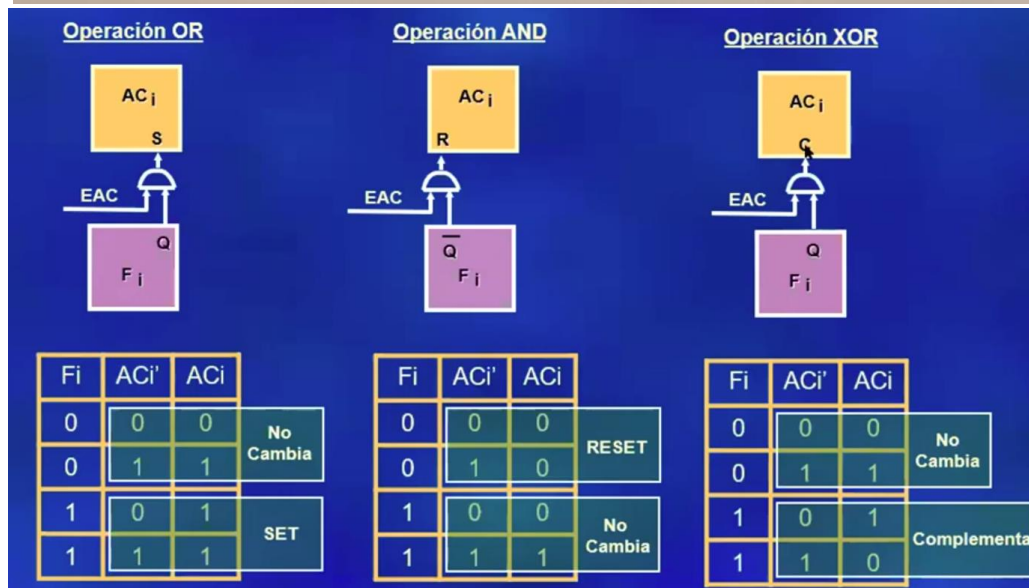
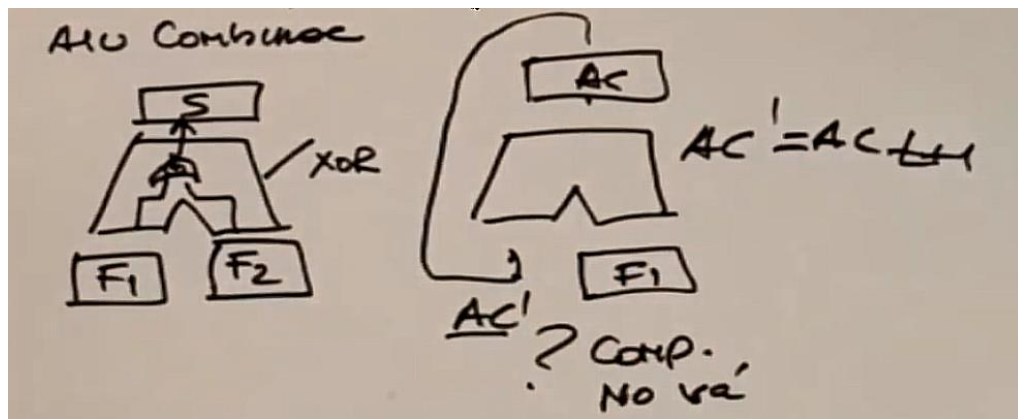
EAC (impulsional): Es importante porque condiciona el funcionamiento actual del acumulador.

- **Falso:** Significa que se está realizando la operación entre el valor anterior del AC y de F1.
- **Verdadero:** Significa que el resultado de la operación de Resultado, debe guardarse en el AC.

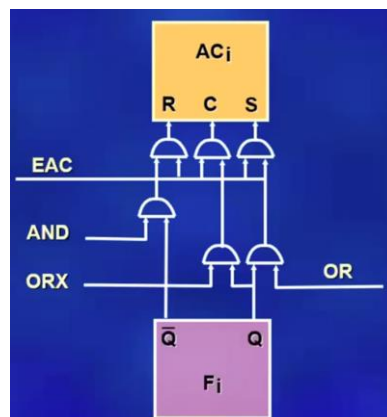


Estas son las configuraciones dentro de la ALU como conecta el F1 con el Acumulador a la hora de realizar las operaciones OR, AND y XOR. (No existe la entrada C, después se arregla)

Operaciones Lógicas



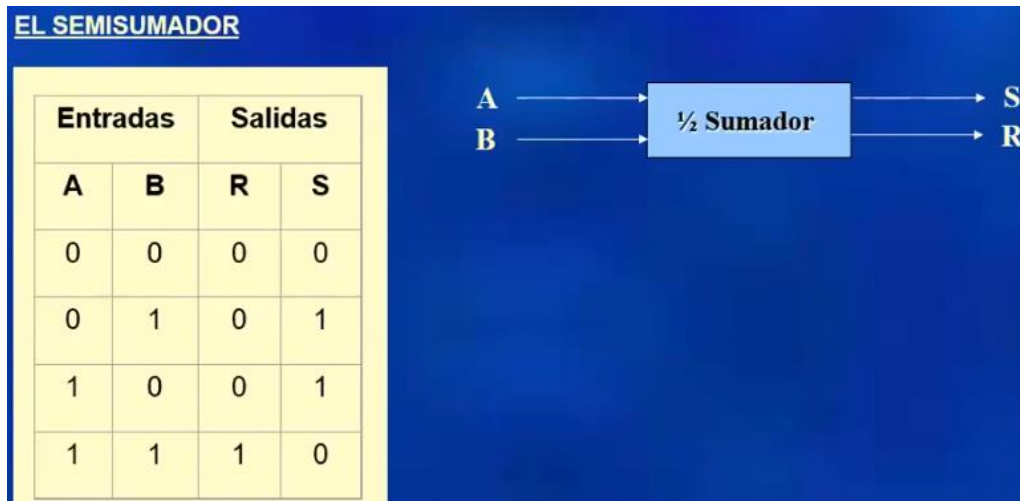
Comentario: La operación XOR es un “RCS”, un flip flop que inventa meindier, pero que en realidad no existe. Lo que se debería hacer como solución es con un JK unir la salida Q con J y K simultáneamente y producir así el complemento



Video: [UTN 20- Arquitectura de Computadoras - Teoria, Unidad Aritmetico Lgica de ABACUS](#)

El semisumador

Sirve solo para iniciar la operación de suma porque no tiene en consideración el Acarreo porque solo funciona con los bits de menor peso, porque no existe etapa anterior.

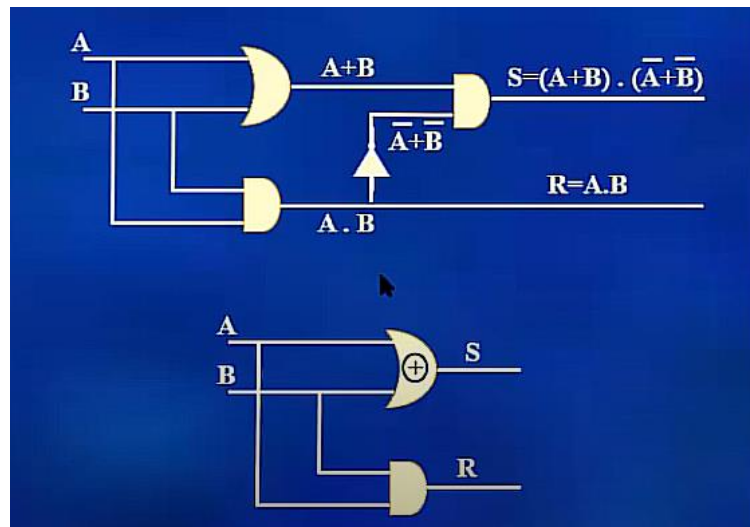


Siempre va a estar ubicado en la primer parte de una suma, porque **NO considera el acarreo de la etapa anterior**.

Ecuaciones

$$\begin{aligned} S &= \bar{A}B + A\bar{B} \\ S &= A \oplus B \\ R &= AB \end{aligned}$$

Circuito (forma complicada y sencilla - simplificada)



SUMADOR COMPLETO

EL SUMADOR

Entradas		Salidas		
A	B	R'	R	S
0	0	0	0	0
0	1	0	0	1
1	0	0	0	1
1	1	0	1	0
0	0	1	0	1
0	1	1	1	0
1	0	1	1	0
1	1	1	1	1

Block diagram of the Sumador:

```

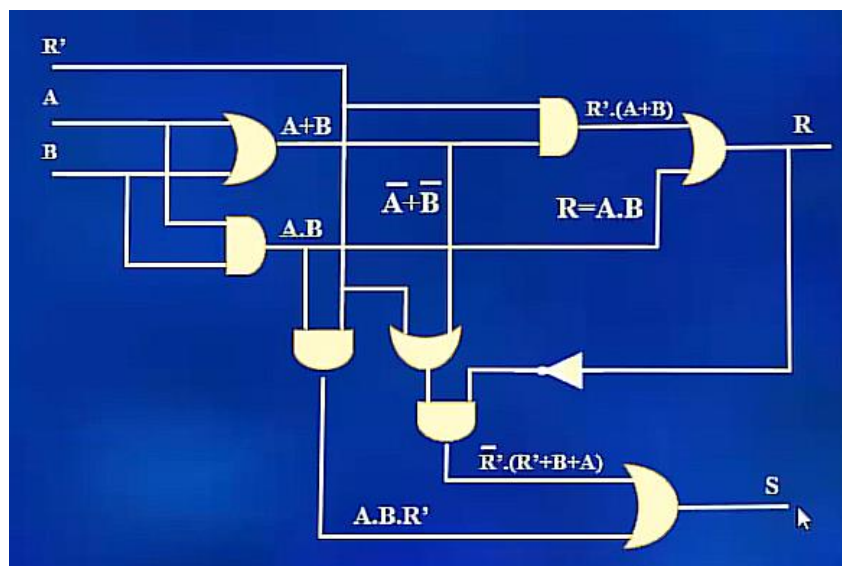
graph LR
    A[ $A(A_n)$ ] --> Sumador
    B[ $B(B_n)$ ] --> Sumador
    R_prime[ $R'(R_{n-1})$ ] --> Sumador
    Sumador --> S[ $S(S_n)$ ]
    Sumador --> R[ $R(R_n)$ ]
  
```

Ecuaciones

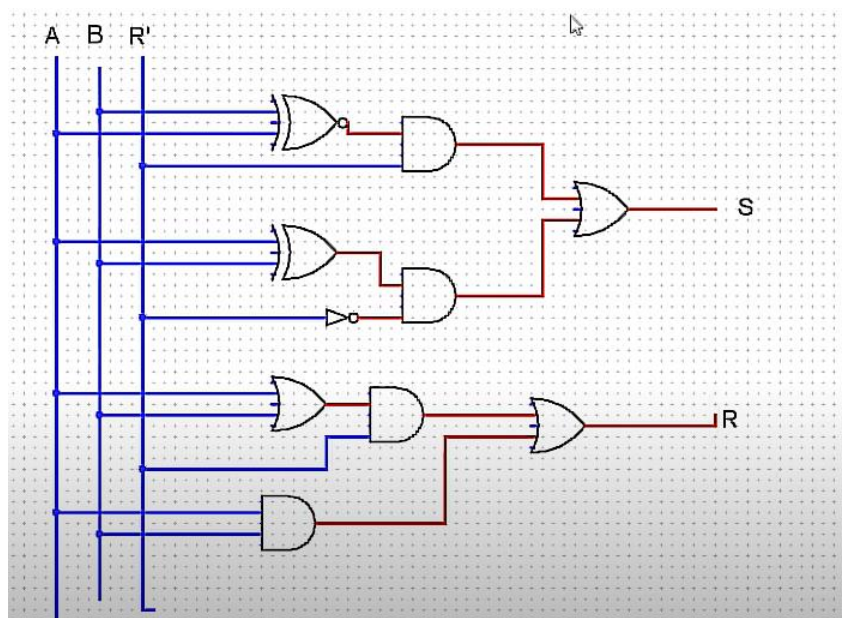
$$\begin{aligned}
 S &= \bar{A}\bar{B}R' + \bar{A}B\bar{R}' + A\bar{B}\bar{R}' + AB\bar{R}' \\
 S &= R'(\bar{A}\bar{B} + AB) + \bar{R}'(\bar{A}B + A\bar{B}) \\
 S &= R'(\overline{A \oplus B}) + \bar{R}'(A \oplus B)
 \end{aligned}$$

$$\begin{aligned}
 R &= \bar{A}BR' + \bar{A}\bar{B}R' + A\bar{B}\bar{R}' + \textcircled{ABR'} \quad \text{DIST x 4 terminos} \\
 R &= (\bar{A}\bar{B}R' + \bar{A}BR') + (\bar{A}\bar{B}R' + A\bar{B}\bar{R}') + (\bar{A}BR' + ABR') \\
 &\quad \text{fact. comun} \\
 R &= \bar{B}R' + \bar{A}R' + AB \\
 R &= \bar{A}R' + \bar{B}R' + AB \\
 R &= R'(A+B) + A \cdot B \\
 R &= (A+B)R' + A \cdot B
 \end{aligned}$$

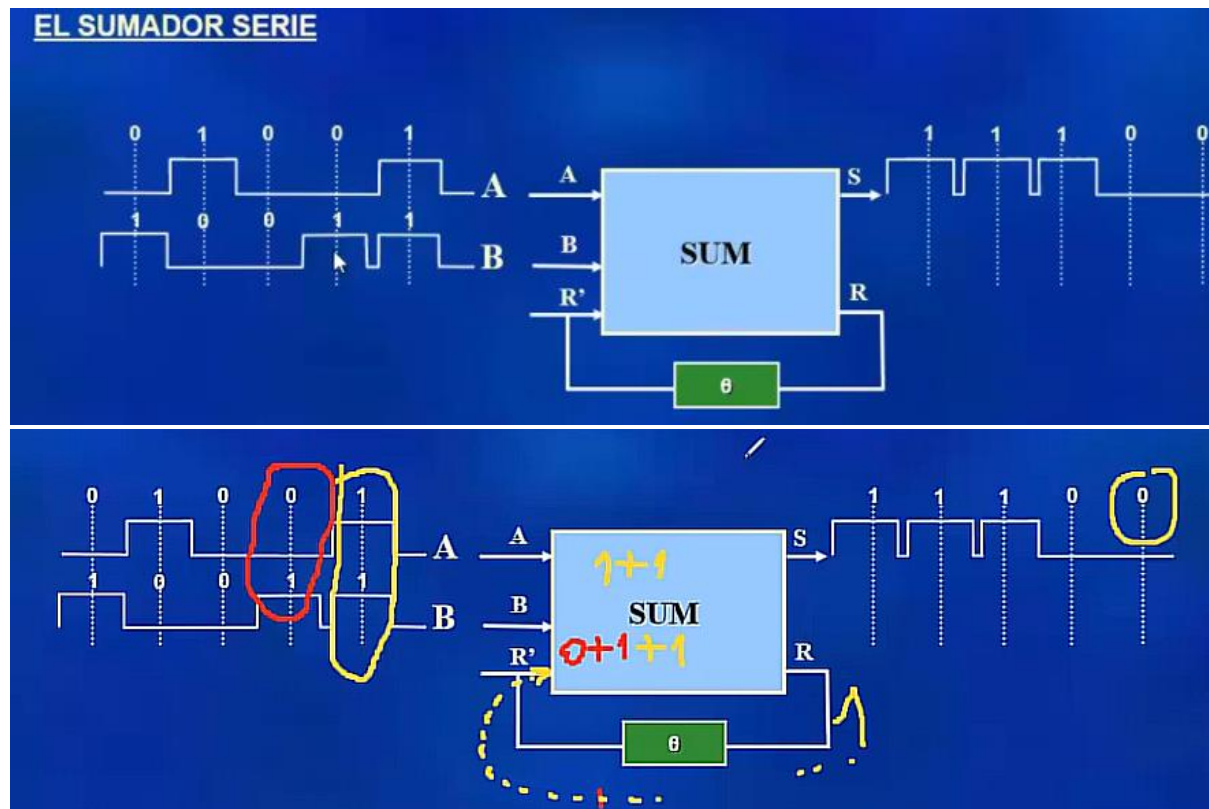
Circuito De meiadier, sin simplificación:



Simplificado:



SUMADOR EN SERIE - SECUENCIAL



Es un circuito sincronizado.

El acarreo va a quedar retardado a la espera del próximo tren de pulso

Ventajas

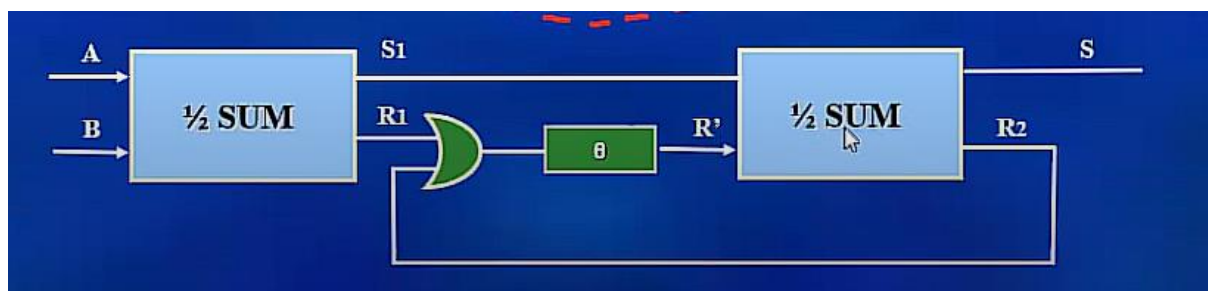
Con un solo elemento sumador, estamos simplificando con un circuito de hardware, muy económicas.

Desventajas

Si la cadena de bits es de 32 va a requerir 32 batidos para poder sumarlos

Consume demasiados ciclos y deja bloqueado el procesador.

De meinadier: de forma más económica, evitando lag que se pueda producir entre los componentes



De meinadier:

7.3 El sumador en serie

A título de primer ejemplo de empleo de la etapa de sumador, vamos a diseñar un sumador en serie. Los sucesivos dígitos de los dos números por sumar se presentan por parejas, los pesos más pequeños a la cabeza, a intervalos regulares, en las dos entradas del sumador. Este se reduce a una etapa de sumador, cuya salida de arrastre R se realimenta sobre la entrada del arrastre R' después de verse sometida a un retardo igual a la diferencia de tiempo entre los instantes de llegada de dos dígitos sucesivos (fig. 64). Este circuito ha sido estudiado en forma de

autómata en el parágrafo 3.3.2. La existencia y la no existencia de la propagación del arrastre en el dispositivo de retardo corresponden a los dos estados internos del autómata.

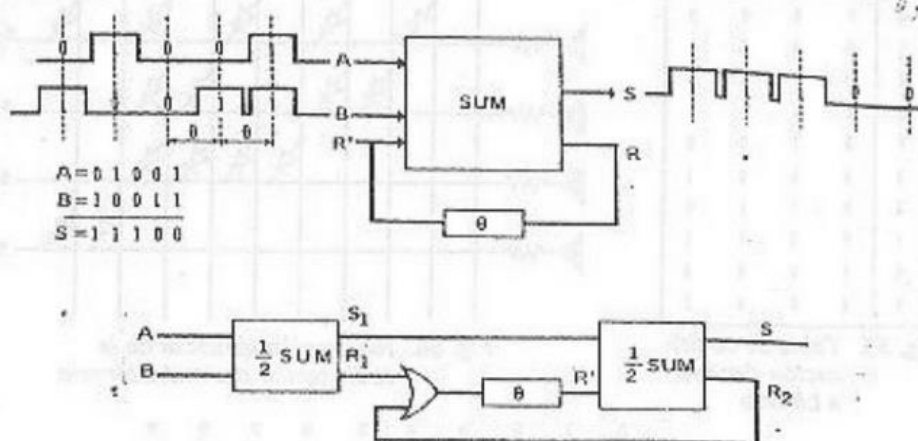


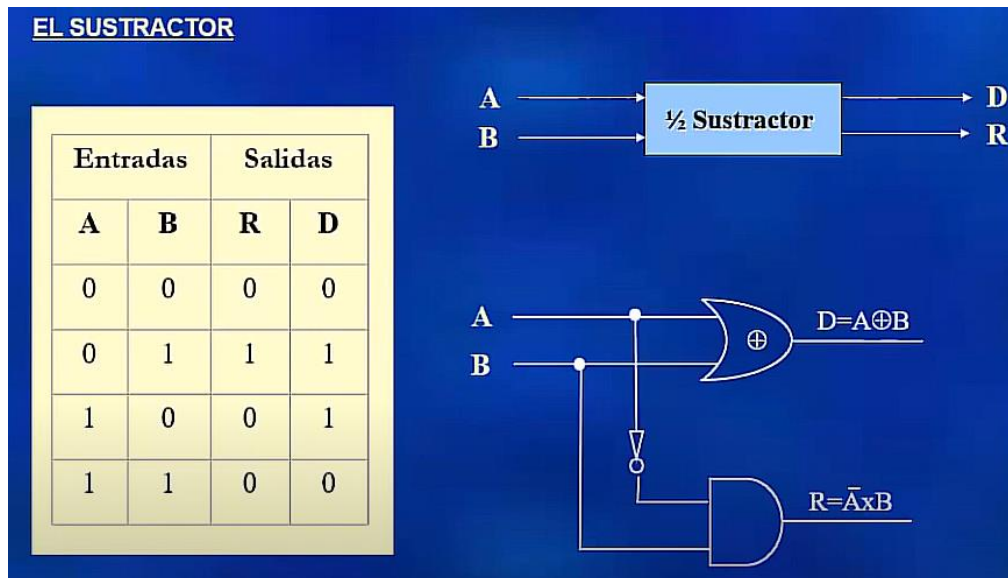
Fig. 64. Sumador serie

SUSTRACTOR

El sumador puede con el complemento, se le puede realizar sin necesidad de este circuito.

$$A - B = A + \bar{B}$$

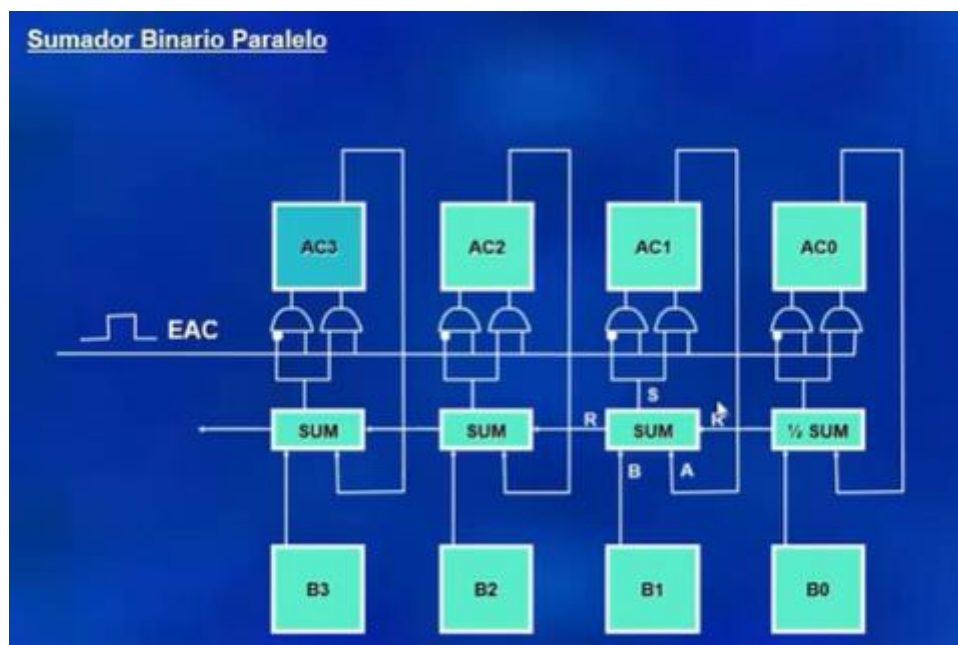
Resta $\rightarrow$ SUMA $\text{COMP}_2(B)$



SUMADOR PARALELO

Un **sumador binario en paralelo** está fundamentalmente constituido por una cadena de etapas de sumadores, en la etapa de sumador de rango i transmite a la de rango $i+1$ el arrastre eventual.

- La etapa del rango 0 puede estar formada solamente por un semisumador.
- El arrastre de la etapa de mayor peso indica desbordamiento si está activada

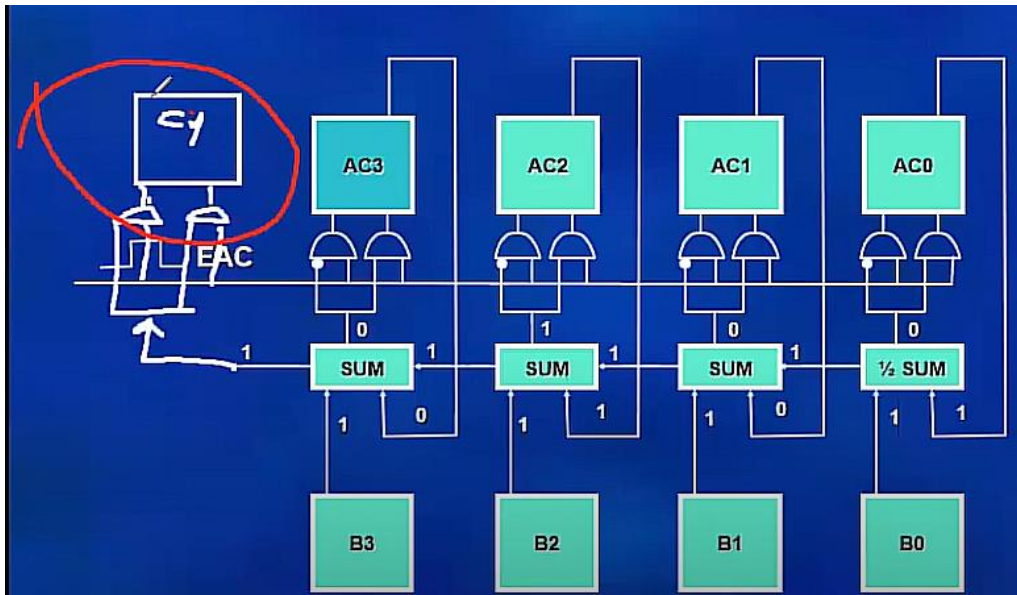


Entre un batido y otro batido puede ya solucionar la operación de la suma, si los componentes se encuentran en un estado disponible.

Es muchísimo más caro por la cantidad de componentes que tiene, propone un esquema de performance mucho más superior.

Se realiza la actualización forzada cuando se activa la señal **EAC**, por lo que se perdió el valor anterior de AC'

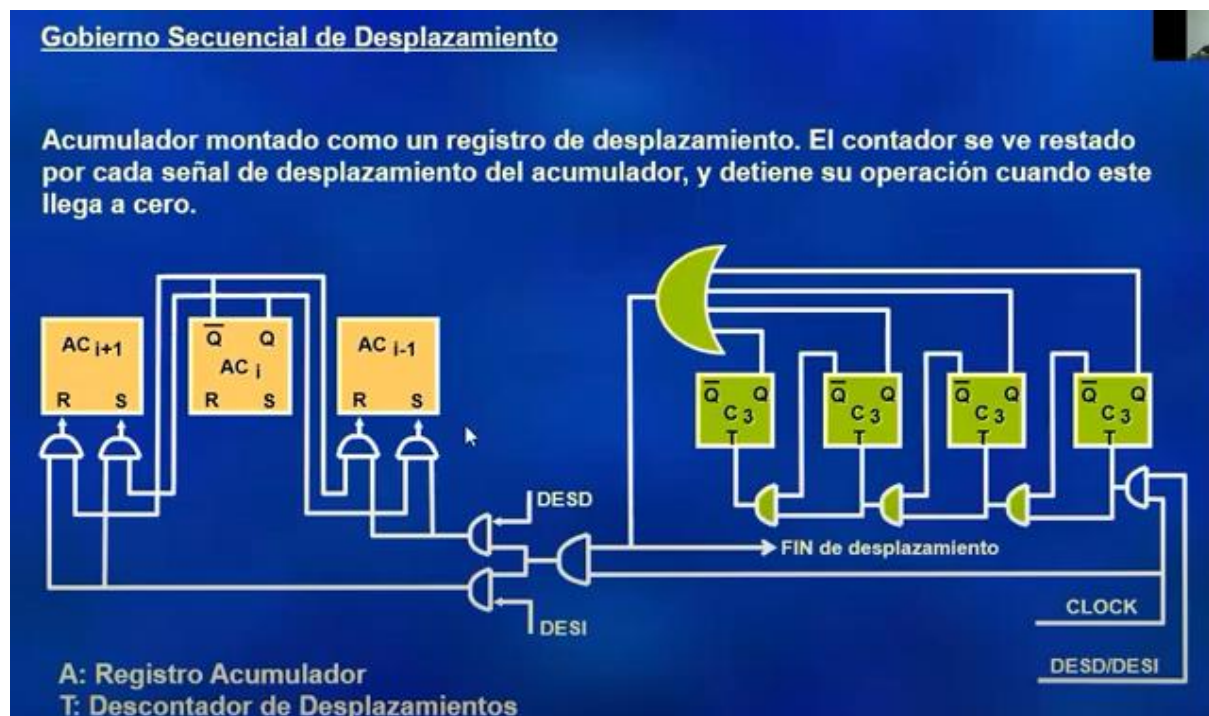
Contemplando el acarreo:



El último sumador debería de tener un "registro" del CARRY por si hay un desborde en la última etapa de la suma.

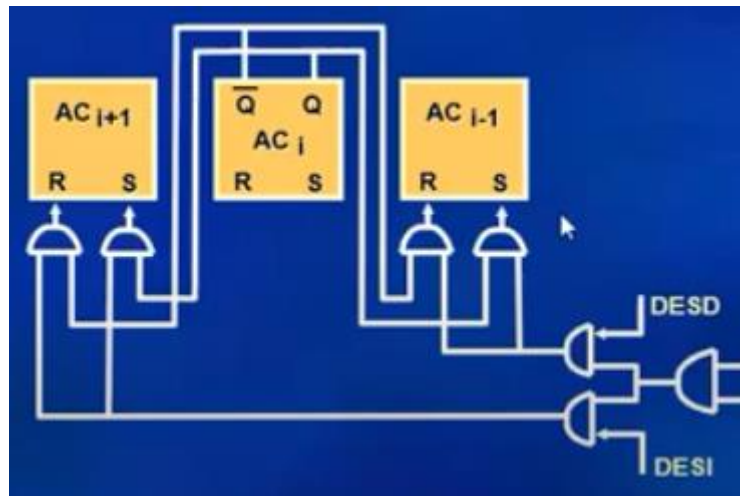
Lógica de Gobierno de los desplazamientos

<https://youtu.be/4qbosoOyaDg?t=4503>

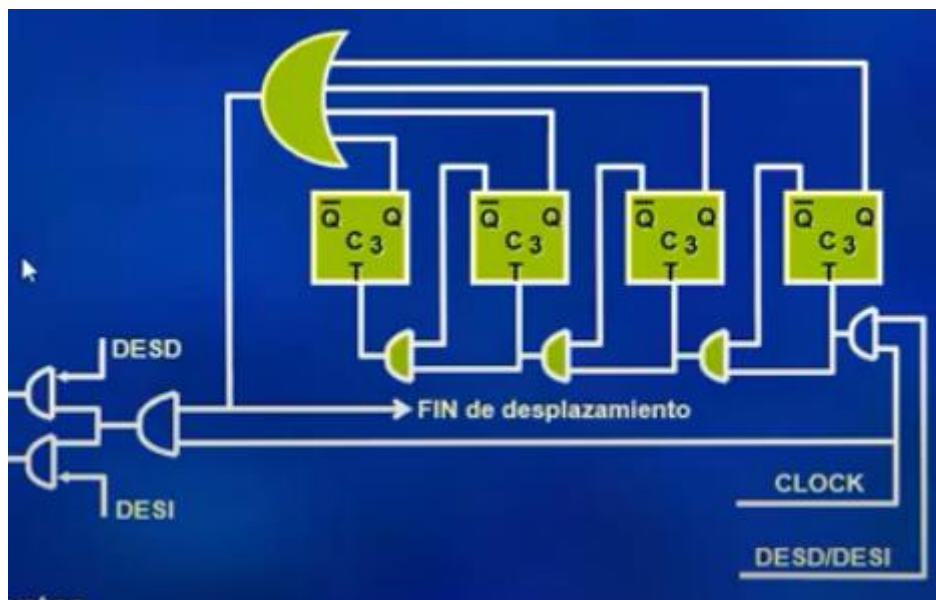


El de la derecha es un módulo **Gobierno de Cantidad de Desplazamiento**, es un contador. Provee cantidad de desplazamientos en binario

Imagen clarita es **Registro de desplazamiento**, según DESD o DESI se moverá hacia derecha o izquierda.

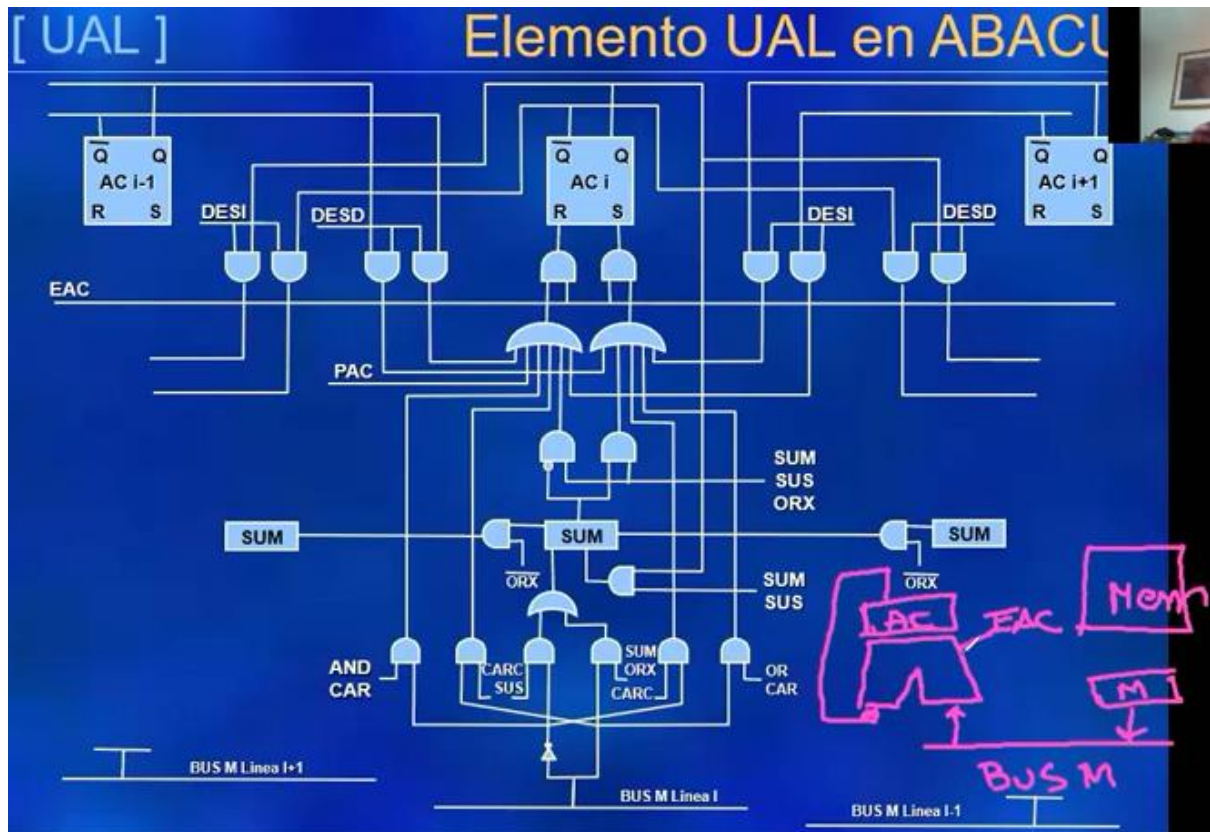


Cuando llega **Gob de Cantidad de Desplazamiento** llega a 0 no permite más interacciones de desplazamiento hasta renovar la cantidad de desplazamientos.

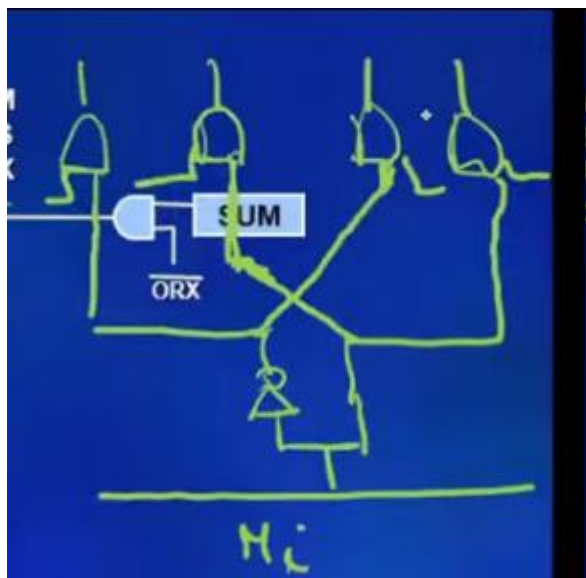


Unidad Aritmetico-Logica para ABACUS

EJEMPLO de **1 Bit** del Acumulador contra 1 Bit del Acumulador M



Tecnica para dibujar:



El **BUS M** está trabajando solo para hacer cálculos aritméticos, porque no se requería interactuar con el exterior. Entonces el BUS M se mantiene activado con el valor del Registro M todo lo que sea necesario hasta que se complete la operación en la ALU. Bits del mismo peso.

De meinadier:

La unidad aritmética de Abacus está montada entre un **bus fuente M**, donde se mantienen los niveles lógicos representativos del **segundo operando**, y un **acumulador AC** que mantiene los niveles lógicos correspondientes al **primer operando** hasta que la señal de muestreo **EAC (señal impulsional)** introduce el resultado en el acumulador.

Consta de un sumador en paralelo y de un conjunto de puertas para ejecutar las operaciones lógicas y distribuir las informaciones de acuerdo con las operaciones aritméticas a efectuar. Además, el acumulador está montado como **registro de desplazamiento**. Las diferentes operaciones están gobernadas por señales lógicas, procedente normalmente del generador central de secuencias del ordenador. Todas estas señales son de nivel, de tal suerte que al cabo de un cierto tiempo después del posicionamiento el resultado de la operación se estabiliza bajo la forma de niveles lógicos, que serán introducidos en el acumulador por el impulso EAC. En la figura anterior se representa el i-ésimo elemento de esta unidad.

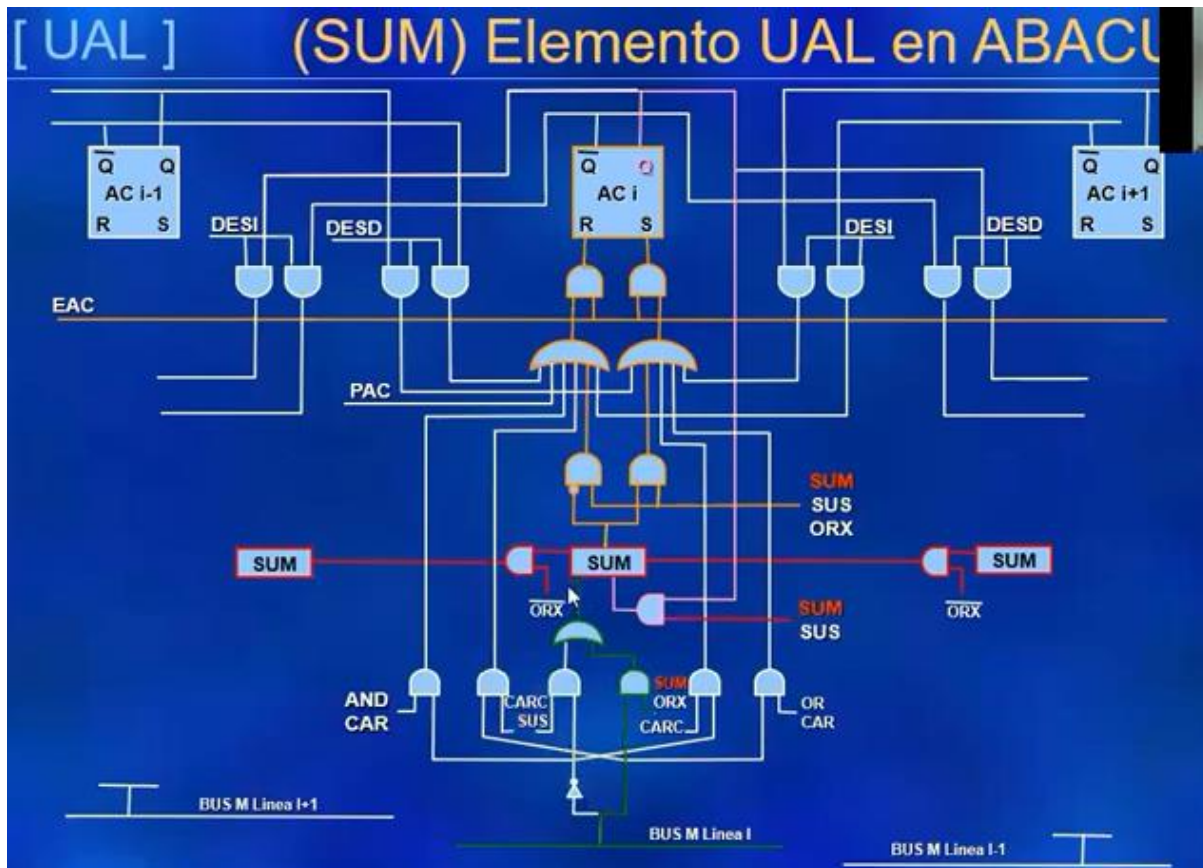
ACLARACIÓN: estas operaciones se harán siempre y cuando haya EAC

Las diferentes operaciones que permite son:

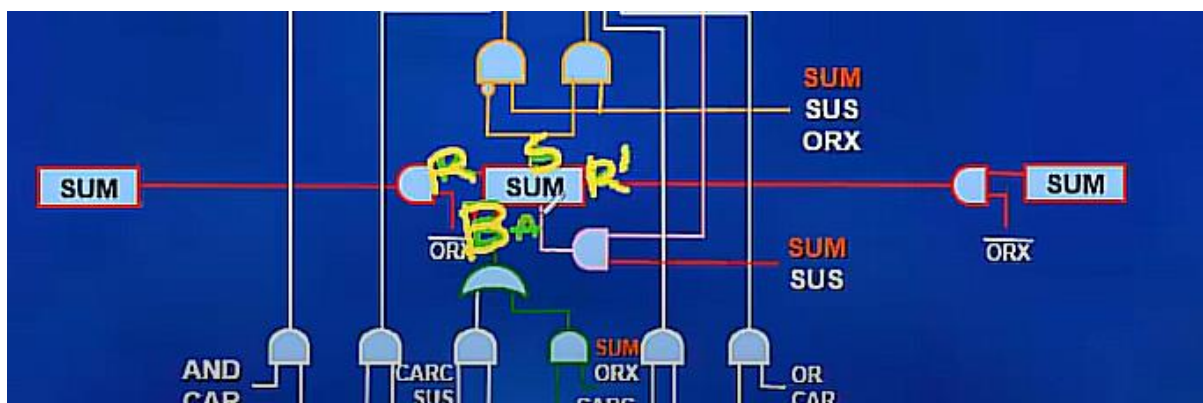
- **CAR - Carga** - contenido del Bus M, al contenido del Acumulador en sus bits de correspondientes peso.
- **CARC - Carga con complemento** - Carga complementada del hilo del BUS al registro de Acumulador. Utilizando para el proceso de **Resta**. Para cargar el valor complementado. → te ahorras componentes haciendo a través de la suma
- **DESI - Desplazamiento a izquierda**
- **DESD - Desplazamiento a derecha**
- **AND** - Utiliza la salida !Q y se conectaba al Reset del biestable del Registro AC.
- **OR** - Utiliza la salida Q y se conecta al Set del biestable del Registro AC.

SUM

Suma al contenido del acumulador AC la información presente sobre el Bus M, ésta adición es operada por el sumador en paralelo. Y la duración de la misma es el tiempo de **Propagación de los arrastres**.

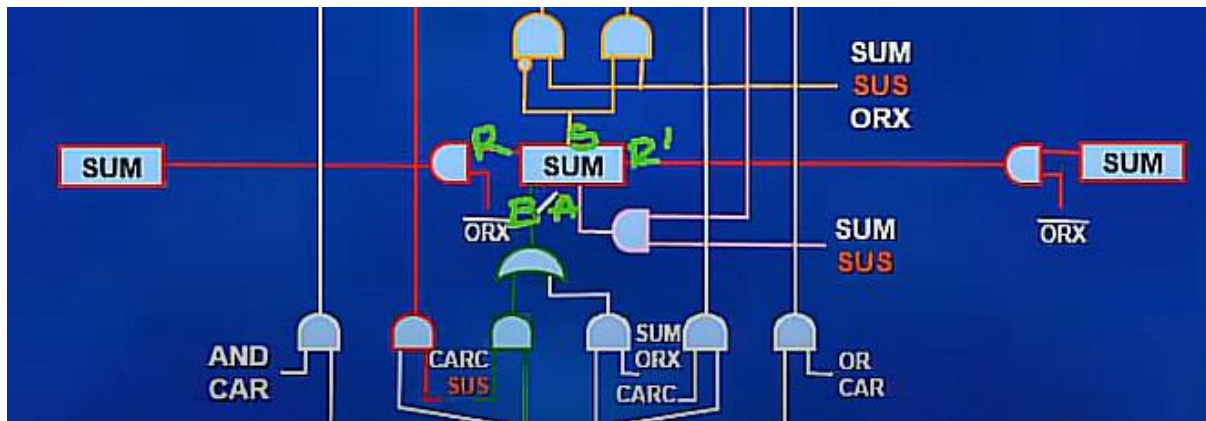


S - Salida
 R - Carry
 R' - Carry estado anterior
 A - Valor entrada desde el AC
 B - Valor entrada desde el Bus M



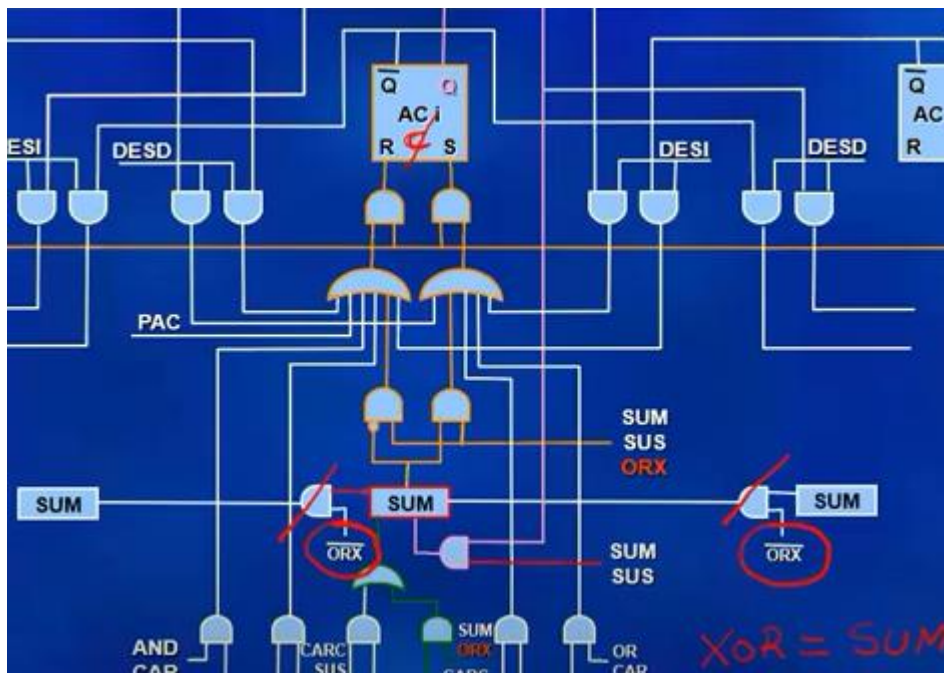
SUS

Sustractor. Complementa la información del Bus M y después suma en el sumador paralelo.



ORX/XOR

Debia ser una compuerta CRS (pero como no existe, entonces se realiza con un RS).
Entonces por eso el XOR lo que se hace, es ocupar la SUM y almacenar el resultado. pero con la señal !XOR lo que hacemos es no considerar el acarreo.



XOR = SUM (sin contemplar acarreo ni etapa anterior/ni actual)

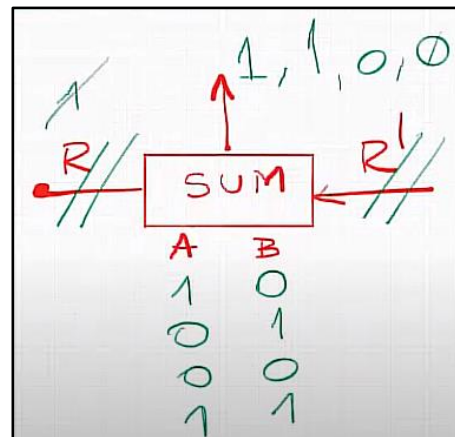
XOR = SUM
SIN
R' y R

Explicación de por qué es igual XOR a no contemplar el acarreo en la suma:

$SumA(A, B)$

A	B	XOR	S	R
0	0	0	0	0
0	1	1	1	0
1	0	1	1	0
1	1	0	0	1

$L = \text{SUM sin } R$
 $XOR = \text{SUM sin } R$



Link a video: [UTN 20- Arquitectura de Computadoras - Teoría, Unidad Aritmético Lógica de ABACUS Punto Flotante](#)

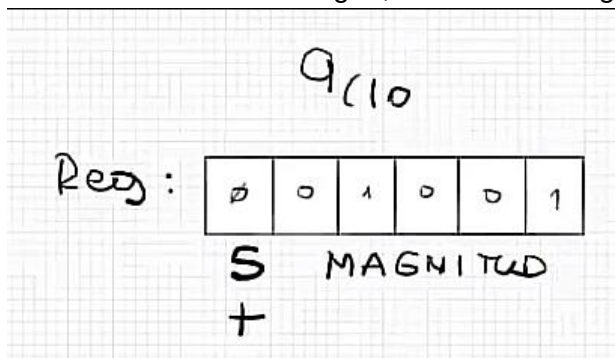
Números Binarios con Signo

El bit de mayor peso, es para indicar el signo

- 0 - Positivo
- 1 - Negativo

Magnitud en este caso es entera (en el futuro puede convertirse en una mantisa para hablar de cifras con coma)

$+9 = 001001 \rightarrow 0$ es el signo, 01001 es la magnitud



Cuando un número binario de punto fijo es **positivo**, el signo **se representa con un 0**, y la magnitud con un número binario positivo.

Si el número es **negativo**, el signo **se representa con un 1**, y el resto puede ser representado de las siguientes maneras:

a) Signo – Magnitud	-9	1	0	0	1	0	0	1
		Signo			Magnitud			
b) Signo-Complemento a 1 (restringido)	-9	1	1	1	0	1	1	0
		Signo			Magnitud			
c) Signo-Complemento a 2 (a la base)	-9	1	1	1	0	1	1	1
		Signo			Magnitud			

Para representarlo como negativo:

- Signo-Complemento a 2 (a la base) para +9 =

$$\begin{array}{r}
 \text{comp}_b +9 = \cancel{1}000000 \\
 - \quad 001001 \\
 \hline
 -9 \mid 110111 \\
 \text{S} \quad \text{MAG.}
 \end{array}$$

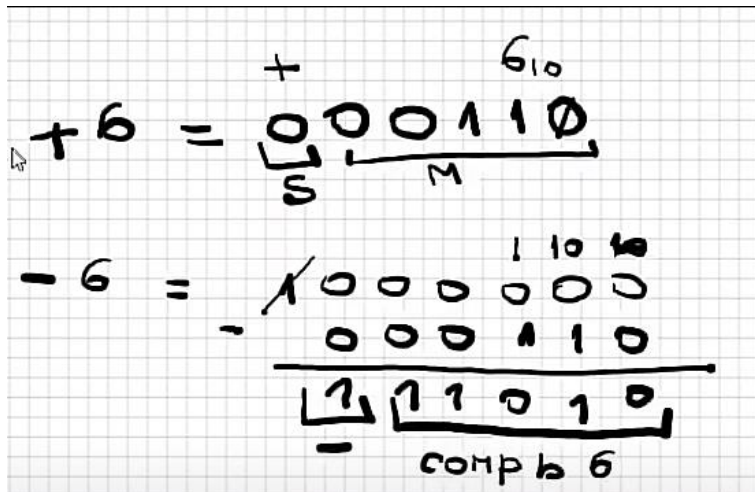
- Signo-Complemento a 1 (restringido) para +9:

$$\begin{array}{r}
 \text{comp}_{b-1} - 111111 \\
 - \quad 001001 \\
 \hline
 110110
 \end{array}$$

Ejemplos:

+	+6	0000110	+	-6	1111010
	+9	0001001		+9	0001001
	+15	0001111		+3	0000011
+	+6	0000110	+	-9	1110111
	-9	1110111		-9	1110111
	-3	1111101		-18	1101110

Otro ejemplo:



$$\begin{array}{r}
 +6 = 000110 \\
 \hline
 \text{S} \quad \text{M}
 \end{array}$$

$$\begin{array}{r}
 -6 = 1000000 \\
 - 000110 \\
 \hline
 111010 \\
 \text{comp } b \ 6
 \end{array}$$

Sobrecapacidad o desbordamiento

Limitación de tamaño de Registro, surgen los problemas con Sobrecapacidad o Desbordamiento. Por lo que hay que saber cómo controlarlo.

Puede haber **Desbordamiento** después de una suma, si ambos números son positivos o negativos.

SOBRECAPACIDAD o DESBORDAMIENTO:

Debido a la longitud fija del tamaño de los registros del computador, estos deben verificar situaciones de sobre capacidad e indicar esta situación en un biestable diseñado a tal efecto.

No puede haber Sobrecapacidad o Desbordamiento después de una suma si: Un número Positivo y otro Negativo

Puede haber Sobrecapacidad o Desbordamiento después de una suma si: Ambos números son positivos o negativos.

La condición de Sobrecapacidad o Desbordamiento se detecta observando el arrastre producido por el bit mas significativo de la Magnitud y el del Bit del Signo. Si son distintos se produce una condición de Sobrecapacidad debiendo indicarlo en Bit Desbordamiento.

Sumador/Sustractor de números algebraicos binarios en punto fijo

Sumador/Sustractor de Numeros Algebraicos Binarios en Punto Fijo

Para poder desarrollar un circuito sumador/ sustractor es necesario:

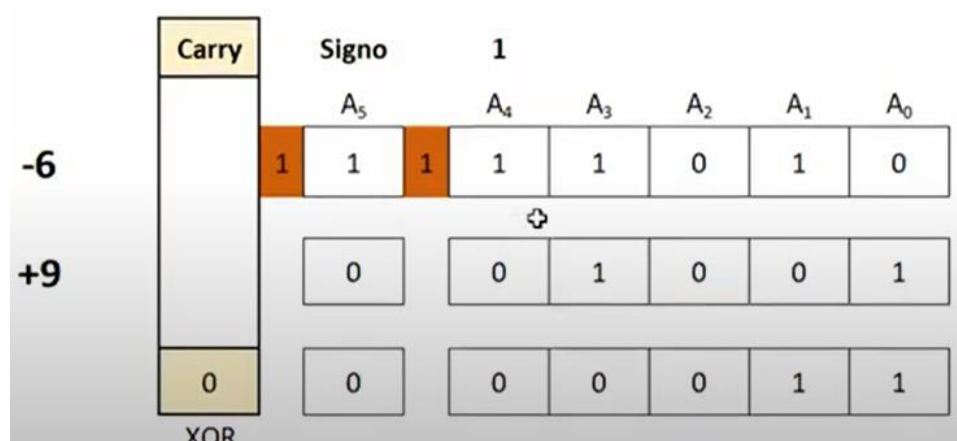
- **Expresar las magnitudes positivas en valor absoluto e indicar en el bit de mayor peso su signo a través de un Cero**
- **Expresar las Magnitudes Negativas por complemento Absoluto, e indicar en el bit de mayor peso su signo a través de un Uno**
- **Verificar situaciones de desborde a través de analizar el arrastre del dígito de mayor peso de la magnitud junto al arrastre de la suma de los signos, detectando situaciones donde alguno de ellos sea igual a UNO**

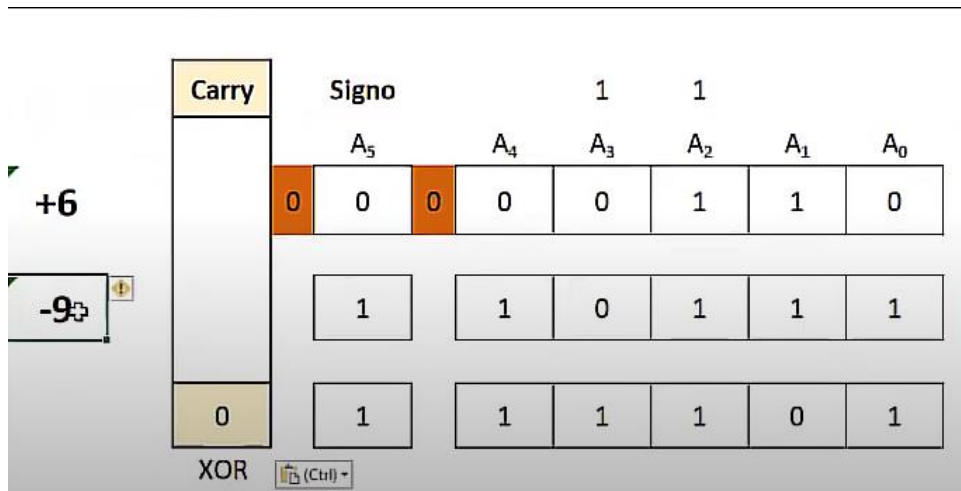
De esta manera es posible plantear un circuito que permita la suma o resta a través de sumadores, ajuste el signo de la operación, y detecte e indique en un biestable la condición de arrastre de la operación.

Ejemplo: Para detectar Carry

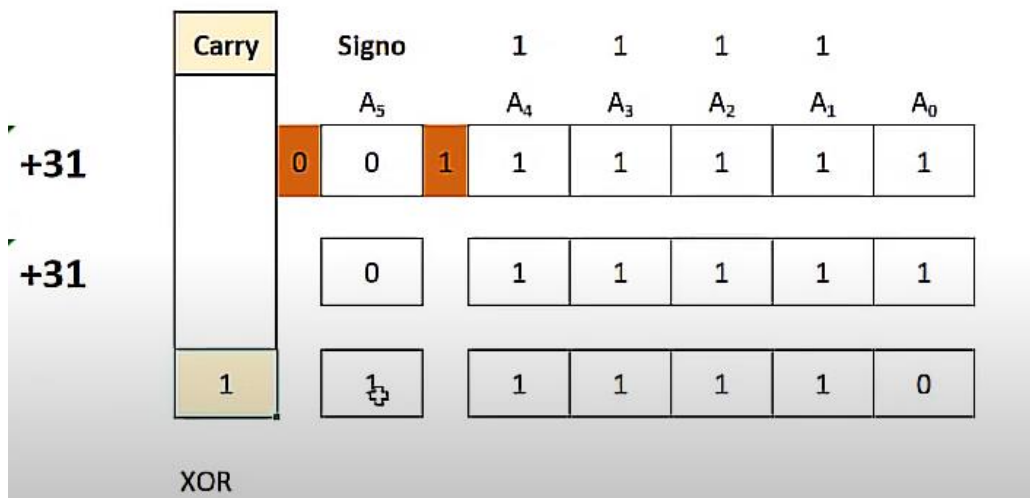
Lo que se hace para detectar un Carry es hacer la suma de las magnitudes, y dicho resultado (el 1 en anaranjado) con la operación XOR con la suma de los bits de signo (el otro 1 en anaranjado) y el resultado indica si hubo Carry o no.

Acarreo de la última etapa de suma de magnitudes **XOR** Acarreo de la suma de los signos

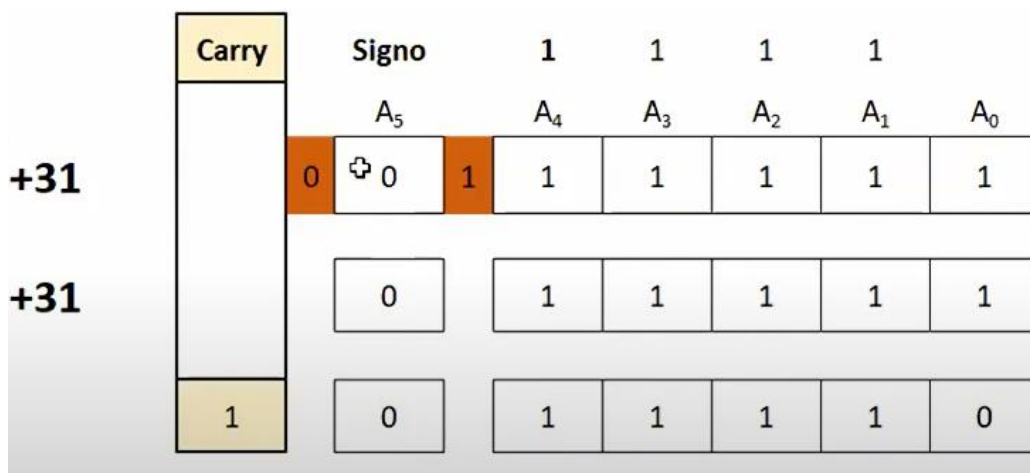




Cuando tenemos del mismo signo:
Antes de la corrección:



Después de la corrección:

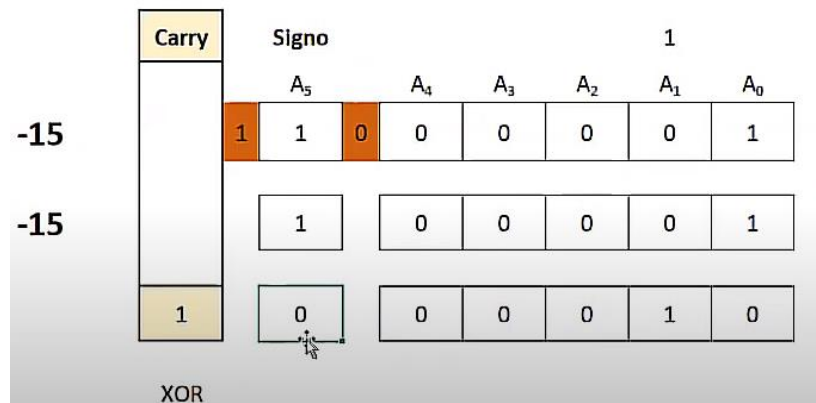


Ejemplo -15 con -15:

Aca se detecta que como el Carry es 1, entonces el **Signo está mal, por lo que se cambia de 1 (anterior, que vino de la etapa de suma de magnitudes), se lo corrige a 0** porque el XOR entre 0 (suma de signos) y 1 (suma de ultima etapa de magnitudes) dió 1, es decir, SI hubo Carry.

Cuando no hay desbordamiento, no consideramos el arrastre anterior, **porque el signo ya está correcto con el valor**. Ver ejemplo de -15 y +31.

Otro ejemplo:



Cambia el signo porque hay carry:

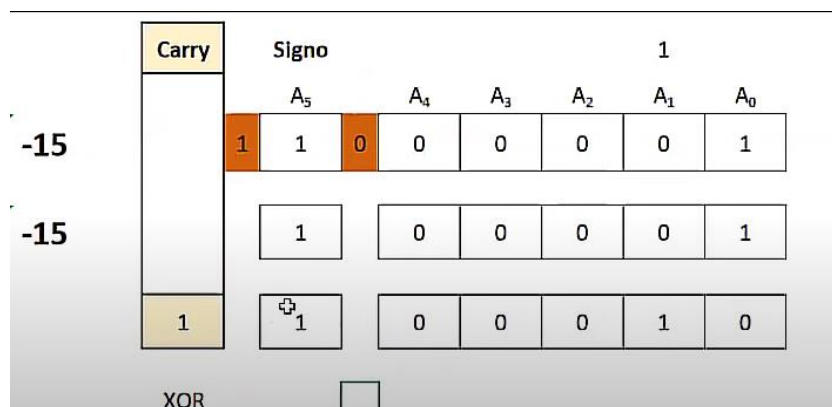
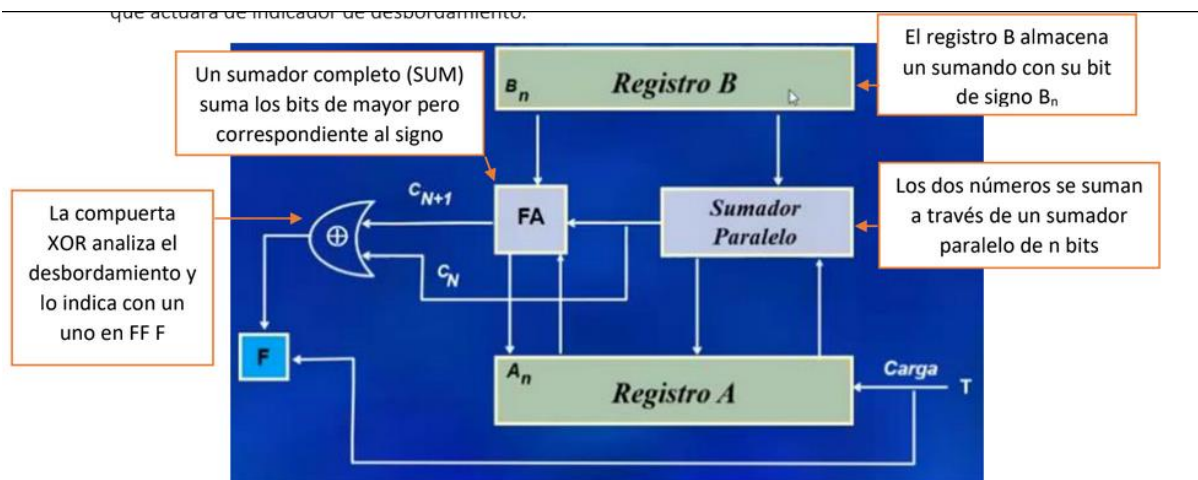


Gráfico de Morris Mano

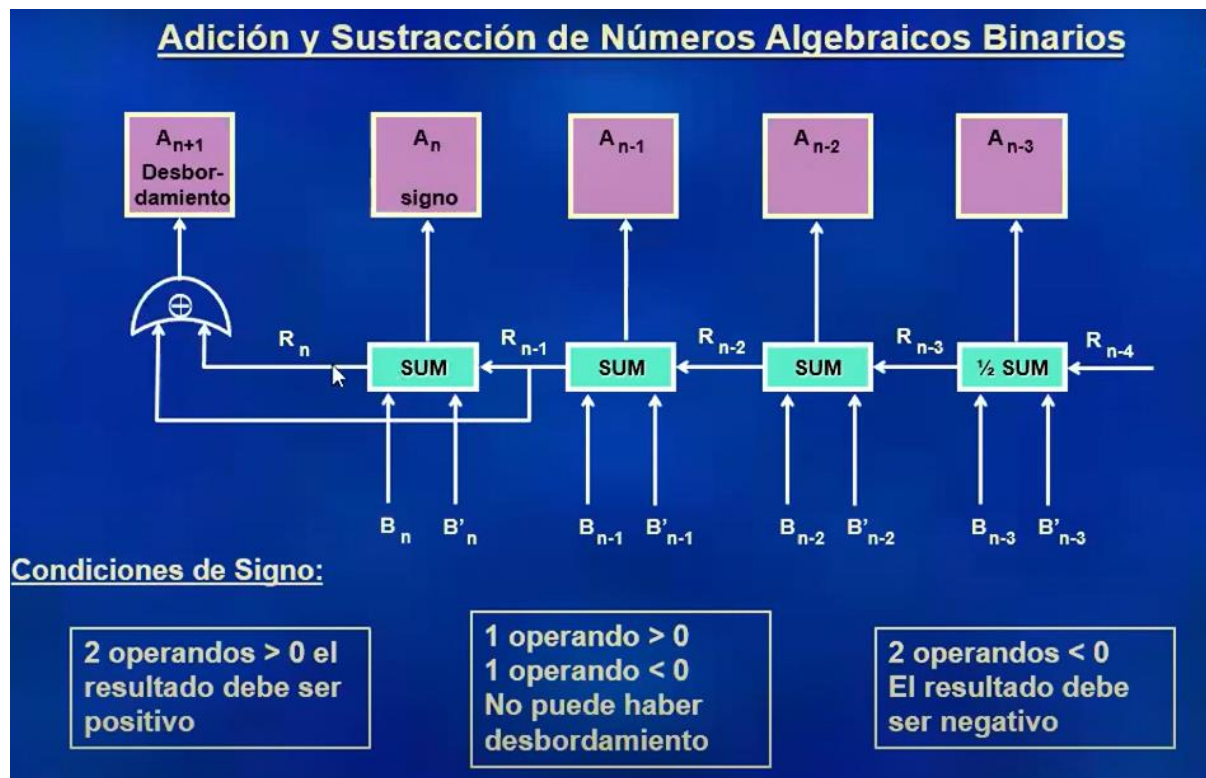


FA es un sumador completo, porque permite el signo de mayor peso separado del resto del registro.

Entonces el resultado del sumador paralelo se puede en caso de desborde.

ALU DE UNA DIRECCIÓN SEGÚN MEINADIER

A continuación se muestra: Considerando según el modelo de Meinadier, son 2 entradas, siendo 1 de ellas el AC en una etapa anterior y la otra el Bus M.



Coma flotante

Para empezar, recordemos que los números en coma flotante se representan en la forma $S.M. a^E$, donde S es el signo del número, M la mantisa y E el exponente y a generalmente vale 2 o 16 en aritmética binaria. Las operaciones se llevan a cabo sobre números normalizados, esto es, sobre números cuyo valor de exponente está ajustado para que la mantisa tenga el mayor número posible de dígitos significativos. Por consiguiente, los operadores en aritmética flotante deben, no solamente ejecutar la operación correspondiente, sino también normalizar el resultado obtenido.



Se considera a un número en coma flotante como normalizado, cuando el valor binario de su signo y el del bit más significativo de su mantisa son distintos

Ejemplo:

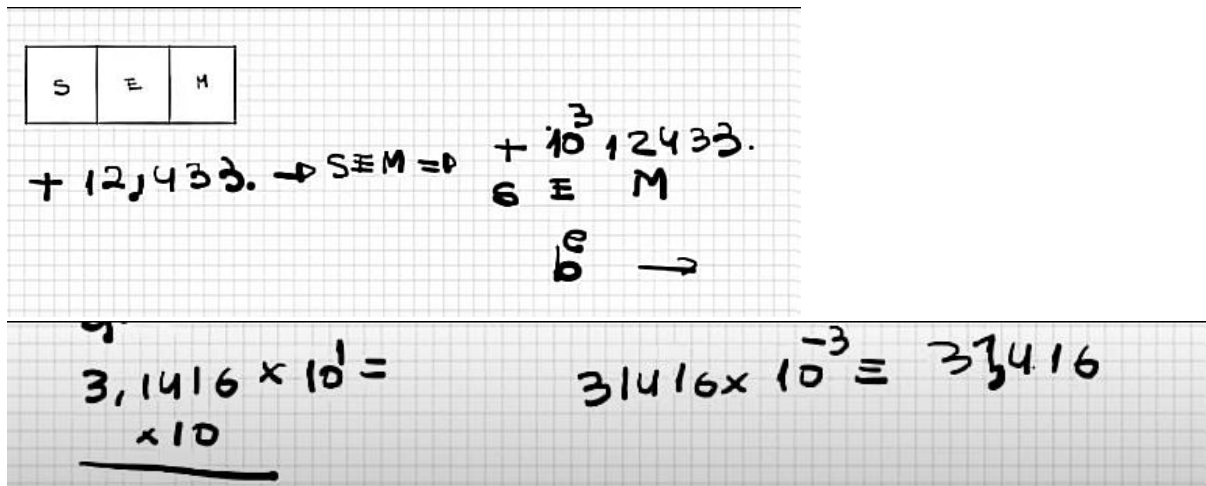


Diagrama de un número en coma flotante:

S	E	M
+	12	433

$\rightarrow S \oplus E = 0$

Normalización:

$$3,1416 \times 10^1 = 31416 \times 10^{-3} = 31416$$

Primera fase: alinear las mantisas

En primer lugar, tomemos conciencia de las dificultades planteadas por estas delicadas operaciones sobre unos ejemplos significativos, descritos, por simplificar, en aritmética decimal.

La primera fase consiste en *alinear las mantisas* con objeto de enfrentar los bits de igual peso. Supongamos a sumar los números $A = 1200 \times 10^{-0}$ y $B = 80000 \times 10^{-2}$, que se escriben, respectivamente:

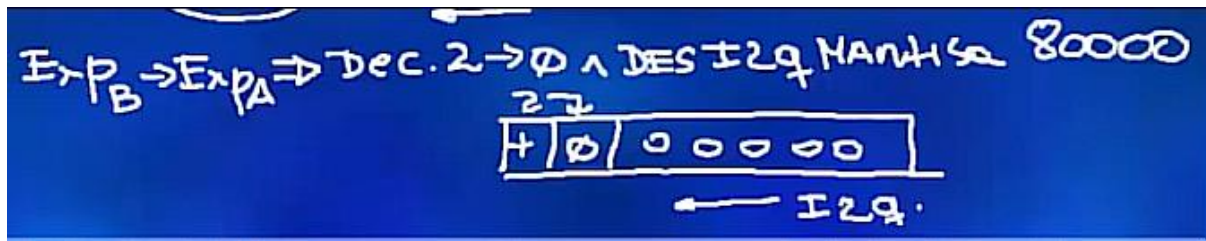
A	+	0	0	1	2	0	0
B	+	2	8	0	0	0	0

En este ejemplo los operandos no están normalizados porque presentan exponentes distintos $\rightarrow$ mantisas no **alineadas**, es decir, no está la coma en la misma posición. En este caso, o el 0 llevamos a 2 o viceversa

A	+	0	0	1	2	0	0	$= 1200 \times 10^0$
B	+	2	8	0	0	0	0	$= 80000 \times 10^2$

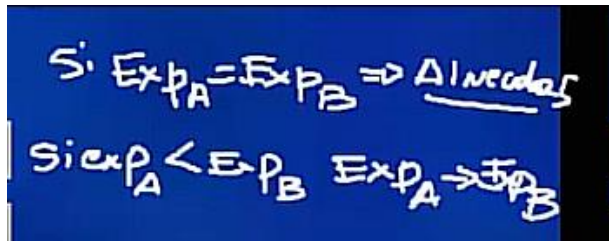
No puede operarse directamente la suma de las mantisas; se comparan los exponentes y se detecta que el primero de los números es más pequeño; sobre este número se ejecuta una doble operación: **incremento del exponente en una unidad y desplazamiento de la mantisa una posición a la derecha**. De nuevo se comparan y se recommienza hasta obtener identidad de exponentes. En ese momento puede efectuarse la suma de las mantisas:

Ejemplo de **alineación incorrecta** (se pierde precisión decimal y bits de mayor peso!)



Lo correcto sería el camino inverso.

Seguir la siguiente regla:



Luego de dos desplazamientos:

A	+	2	0	0	0	1	2
B	+	2	8	0	0	0	0

Segunda fase: ajustar signos

La segunda fase consiste en **ajustar los signos**. Consideremos la sustracción $A - B$. En la representación en valor absoluto y signo, se deberán primeramente comparar las mantisas de A y B, comprobar que la de B es superior a la de A y deducir de ello que la operación pertinente es sustraer la mantisa de A de la mantisa de B y al resultado darle el signo menos:

A	+	2	0	0	0	1	2
-B	+	2	8	0	0	0	0

MB	8	0	0	0	0	
MA	0	0	0	1	2	
-	2	7	9	9	8	8

Debe tomarse así en cuenta en esta fase la clase de operación (suma o sustracción), el signo de los operandos y el resultado de la comparación de mantisas. Nótese que el problema se simplifica si el conjunto signo-mantisa fuera representado en complementación cuando el número sea negativo.

Si está expresado MB como complemento, no es necesario cambiar el signo, PERO trae otros problemas.

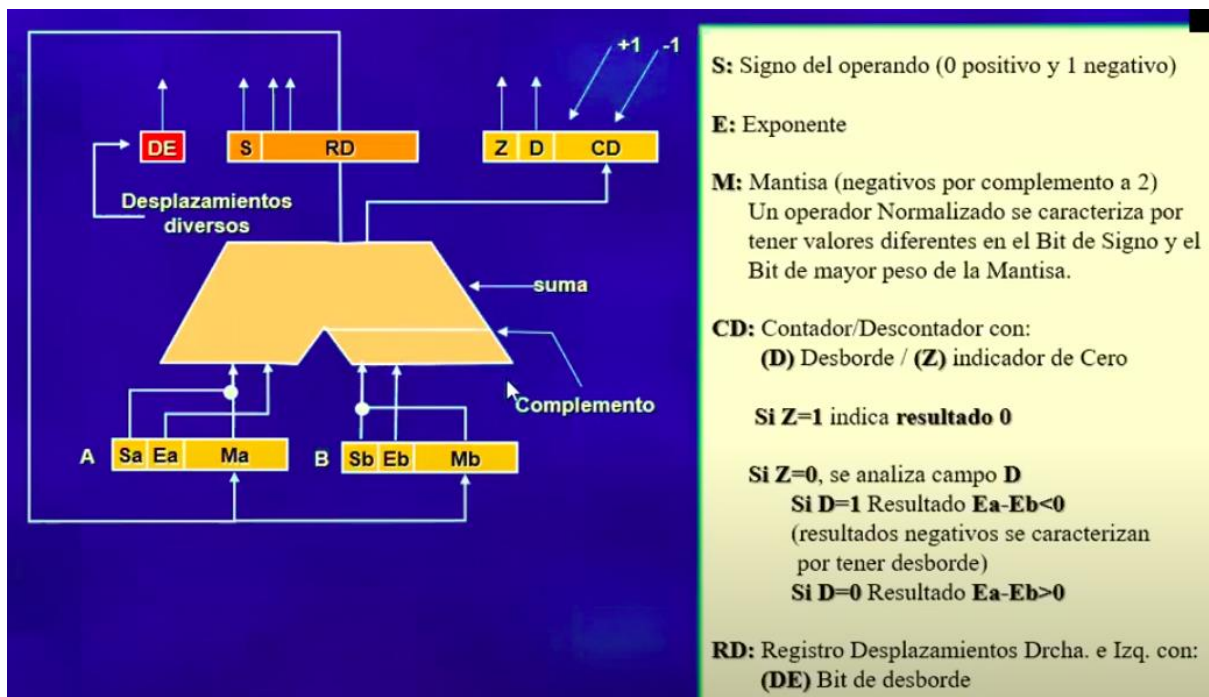
Tercera fase: normalización del resultado

La tercera fase es la **normalización del resultado**. Puede que el resultado bruto de la operación no esté normalizado, por dos razones: (1) **hay desbordamiento de capacidad en la suma**, en cuyo caso es necesario **desplazar a mantisa una posición a la derecha**, aumentando al tiempo en una unidad el exponente; (2) **la diferencia de la mantisas implica la formación de un número n de ceros a la izquierda del resultado**, en cuyo caso es necesario **desplazar la mantisa n posiciones a la izquierda del resultado simultáneamente**. Veamos un ejemplo de suma en ambos casos:

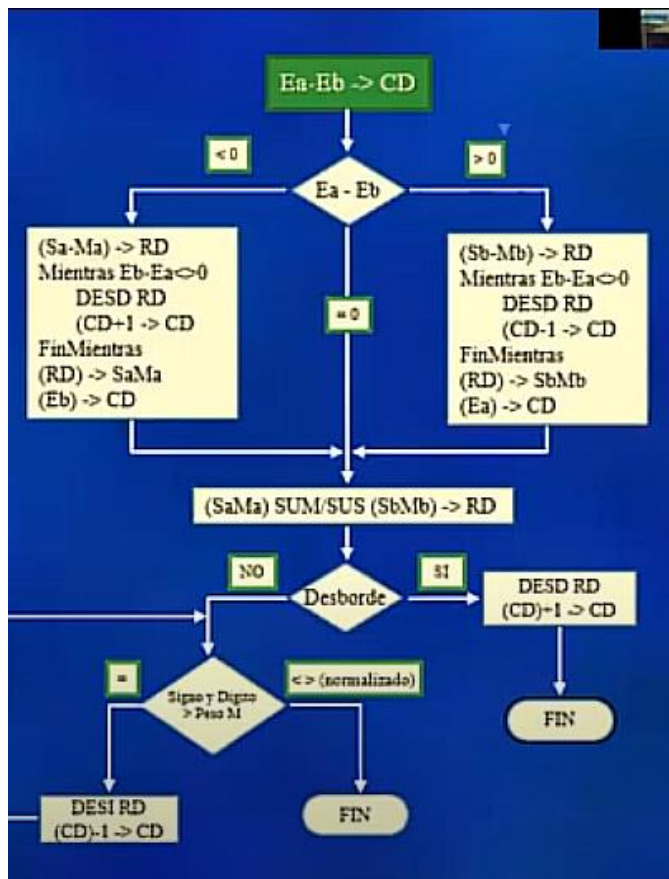
1º Operando	+	2	8	0	0	0	0
2º Operando	+	2	8	0	0	0	0
Operación		1	6	0	0	0	0
Normalización	+	3	1	6	0	0	0

1º Operando	+	2	8	0	0	0	0
2º Operando	+	2	7	9	9	9	9
Operación			0	0	0	0	1
Normalización	+	-2	1	0	0	0	0

Funcionamiento de un operador de suma flotante



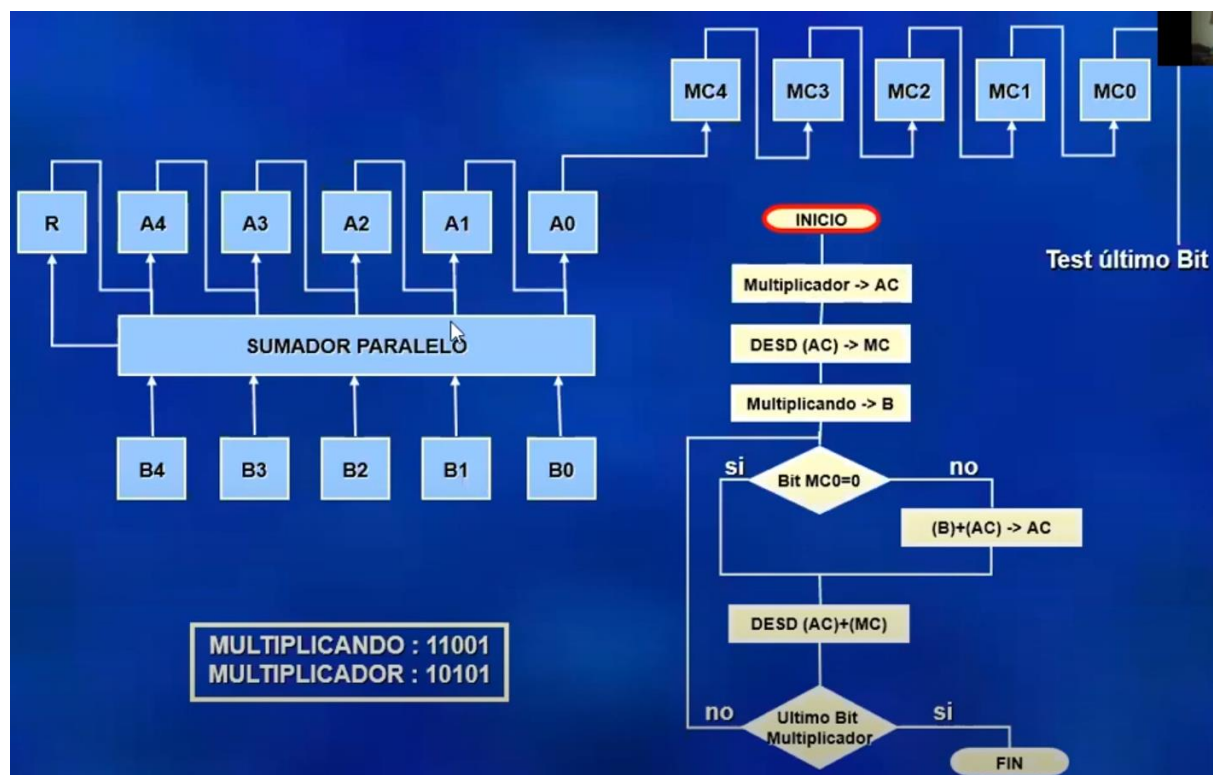
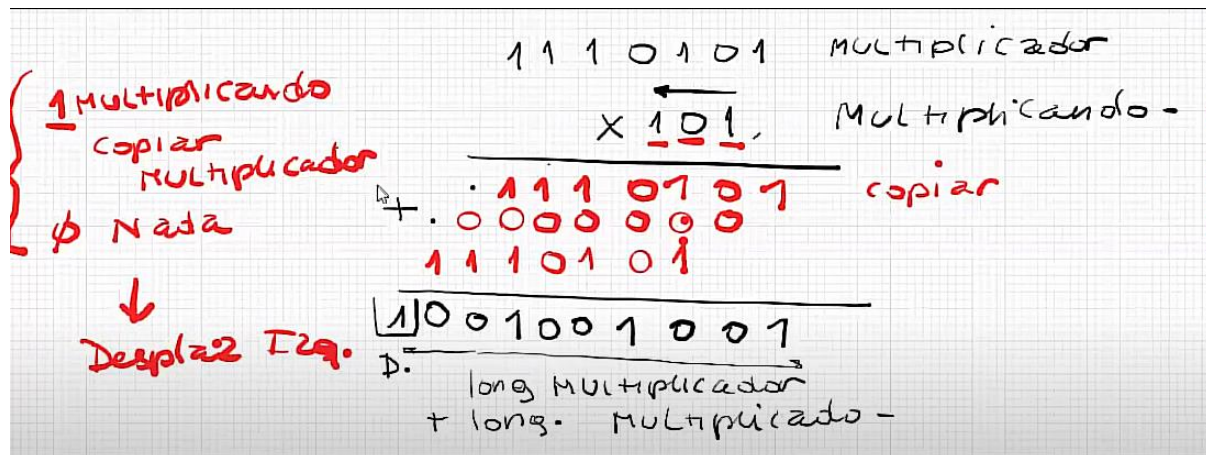
Algoritmo:

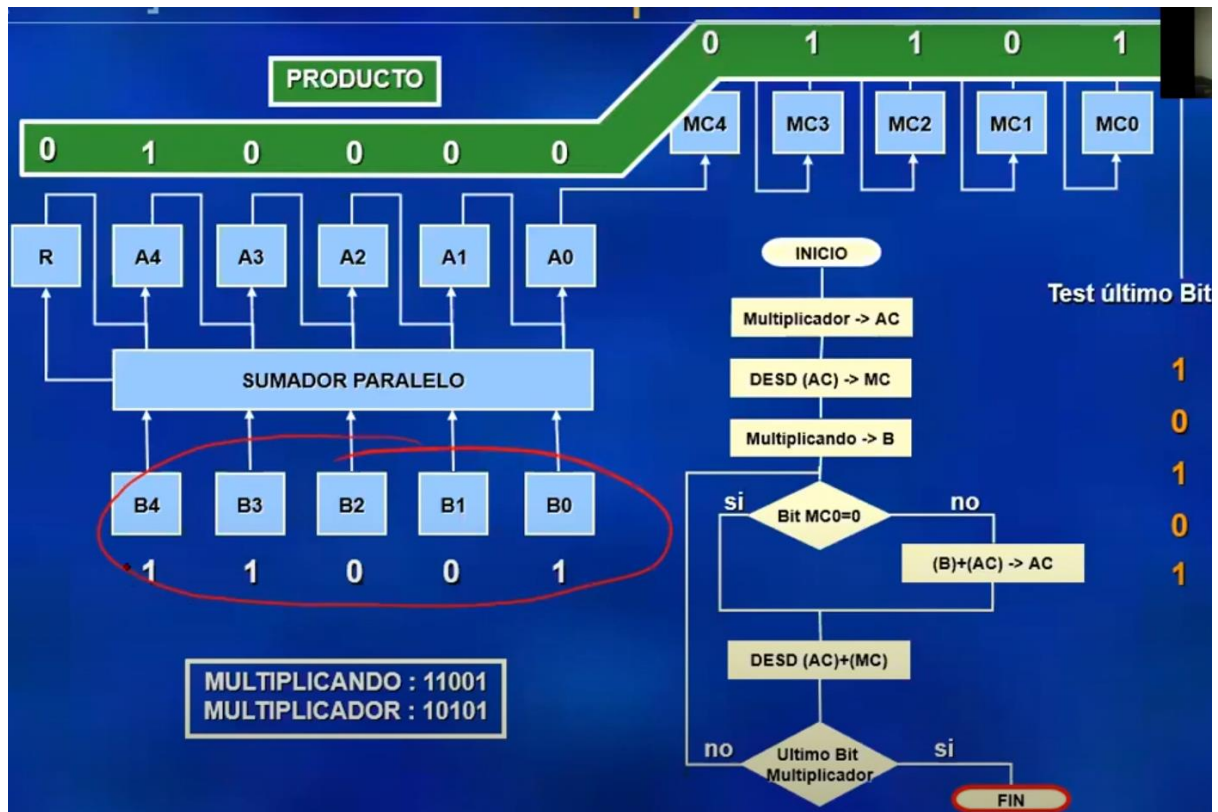


1. Se restan los registros A y B y se lo mueve a registro CD. Si la resta da 0, el bit Z lo indica. Un resultado negativo se caracteriza por producir un 1 en bit D indignado que el exponente de A es menor que el de B
2. Al ser CD negativo, se copia Signo y mantisa de A al registro RD para normalizarlo. La cantidad de desplazamientos necesarios a derecha se encuentra en CD
3. Se procede a ir desplazando a derecha RD hasta que CD llegue a cero (Z) quedando en RD la mantisa de A normalizada respecto a la de B.
4. se actualiza el Registro de A con la mantisa normalizada y se copia el exponente
5. Se realiza la suma

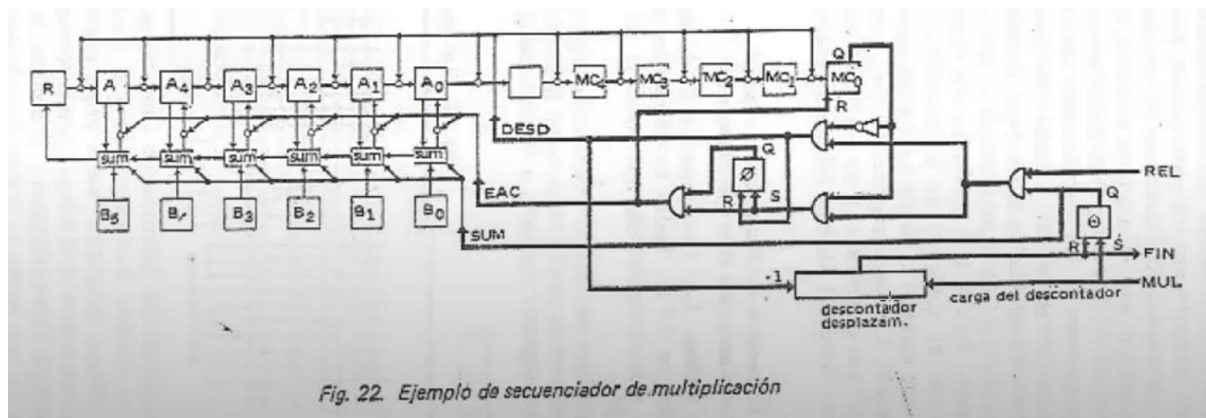
Multiplicación secuencial por suma - desplazamiento en ABACUS

Ejemplo:





Circuito de Meinadier:



Mejoras en la multiplicación

TECNICAS DE MULTIPLICACION RAPIDA

A) Técnica tradicional Suma-Desplazamiento:

Consiste en condensar operaciones repetitivas para evitar el consumo de ciclos de cpu. Se analiza en el multiplicador valores con sucesivos ceros y unos contiguos.

- **Sucesivos Ceros contiguos:** Se dota al acumulador de un circuito que realice en un solo paso tantos desplazamientos como ceros contiguos haya, como si se tratase de un único desplazamiento.
- **Sucesivos Unos contiguos:** Se usa la técnica Suma-Sustracción. Una serie de unos contiguos conlleva a una serie de sumas y desplazamientos. Esta técnica consiste en convertir una serie de sumas y desplazamientos a una adición y una sustracción. Ejemplo:

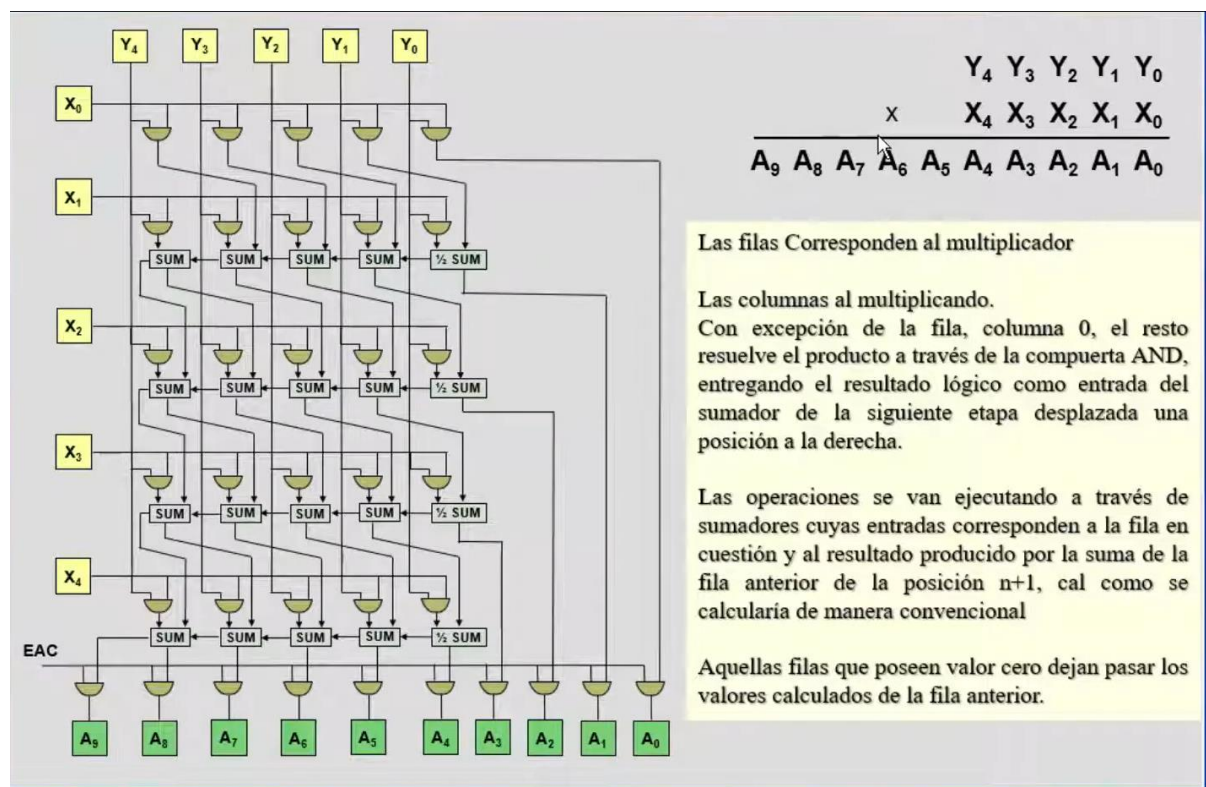
Multiplicando

X 01111100

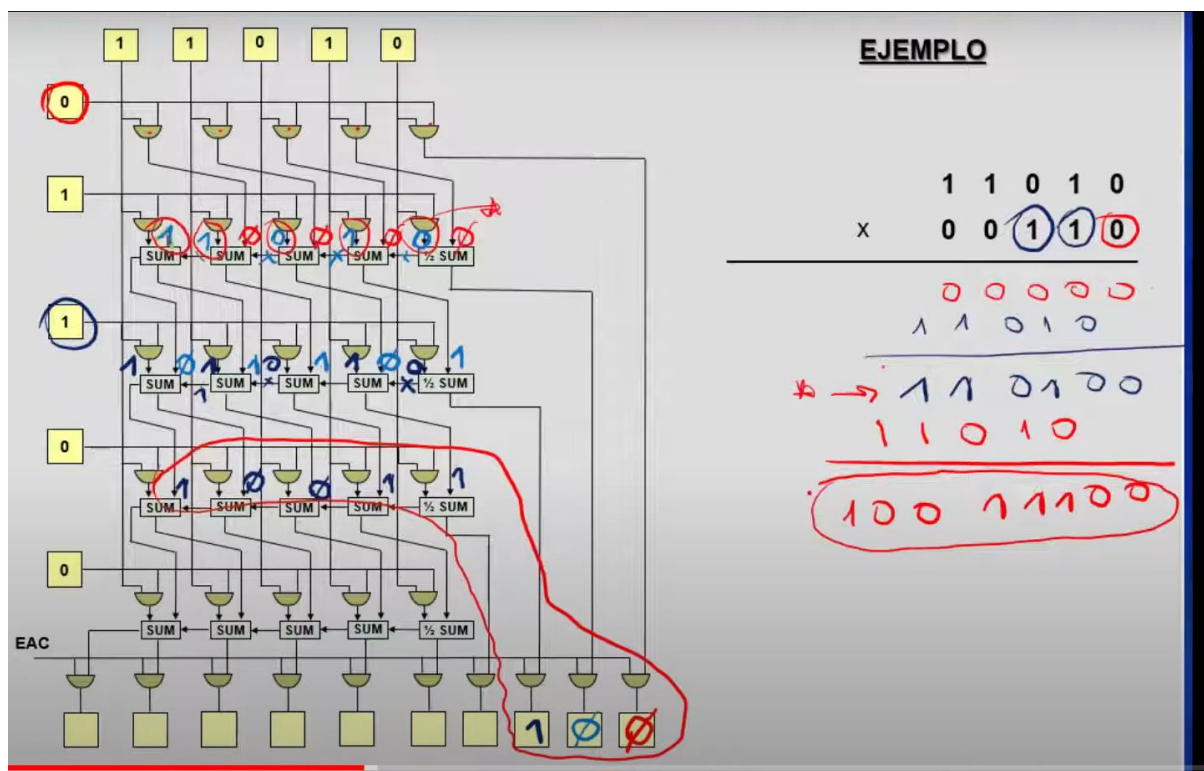
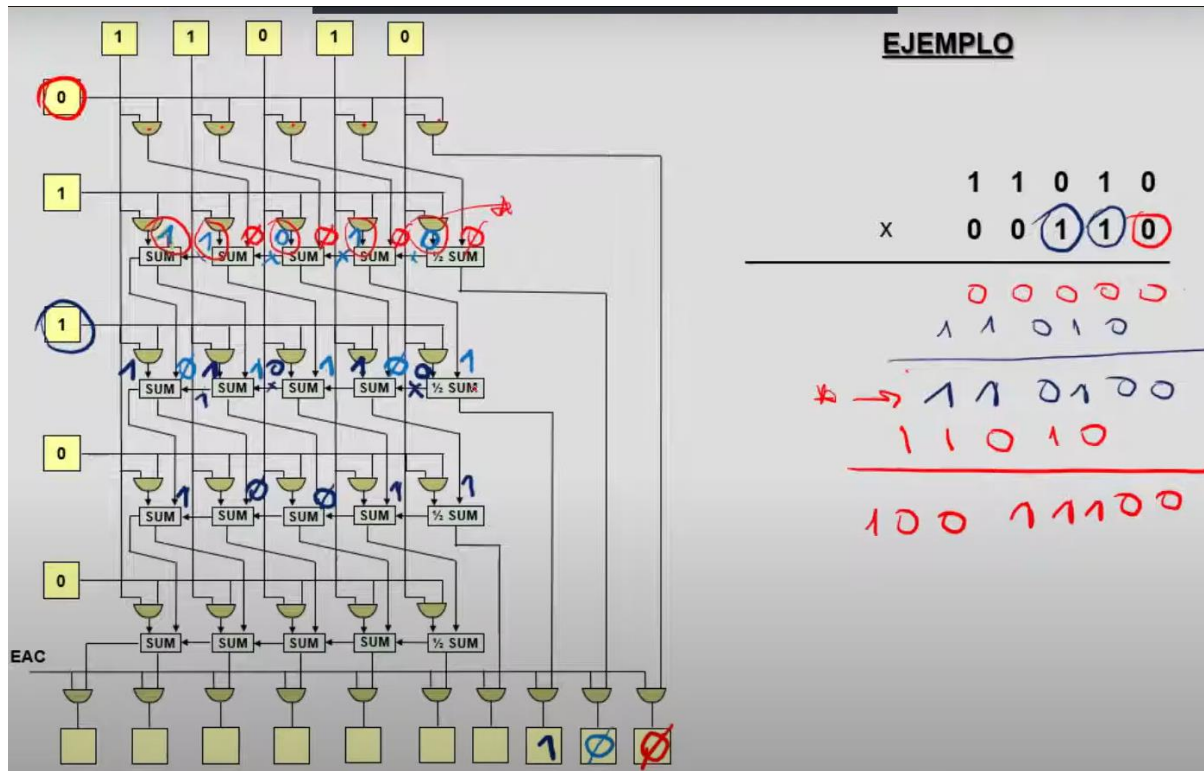
10000000 - 00000100

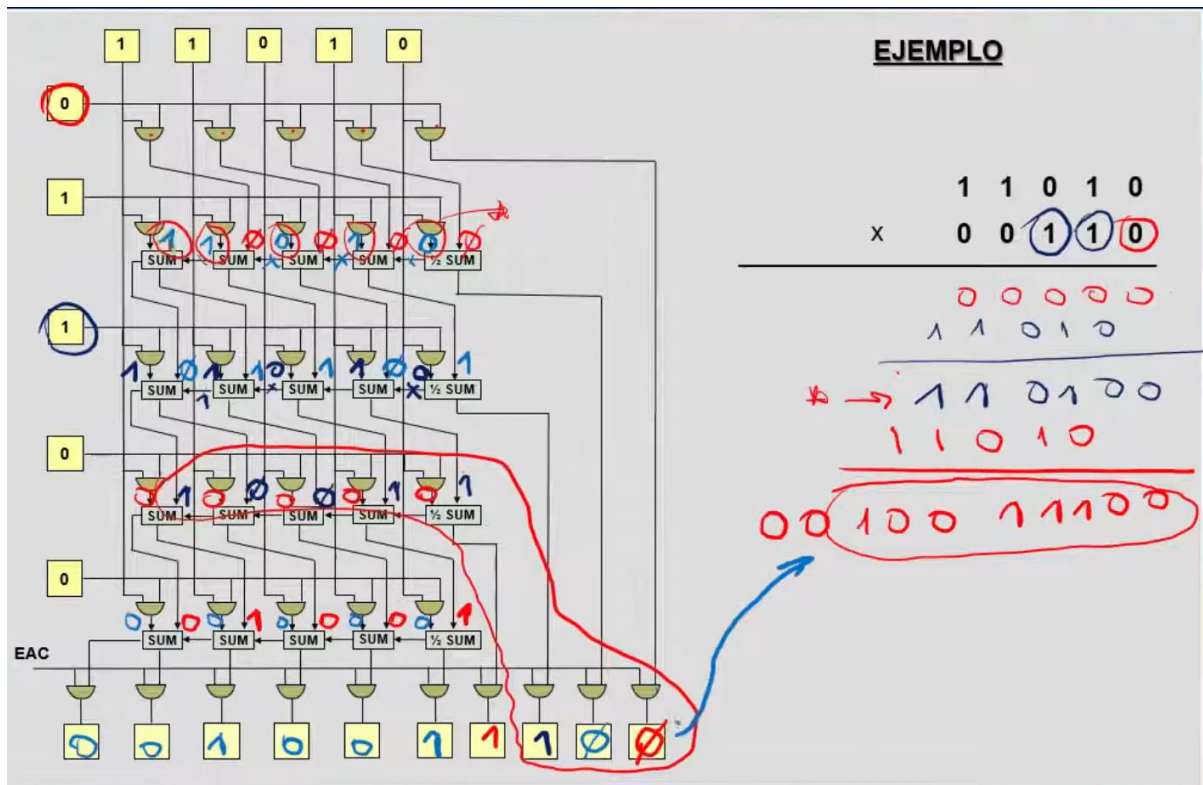
B) Operador Celular para Multiplicación semi-paralela:

Multiplicador celular paralelo



Ejemplo:





Desventaja: cantidad de componentes

$$\text{cont AND} : f \times C = 25$$

$$\text{cent AND} : 25 \times 2 = 50e$$

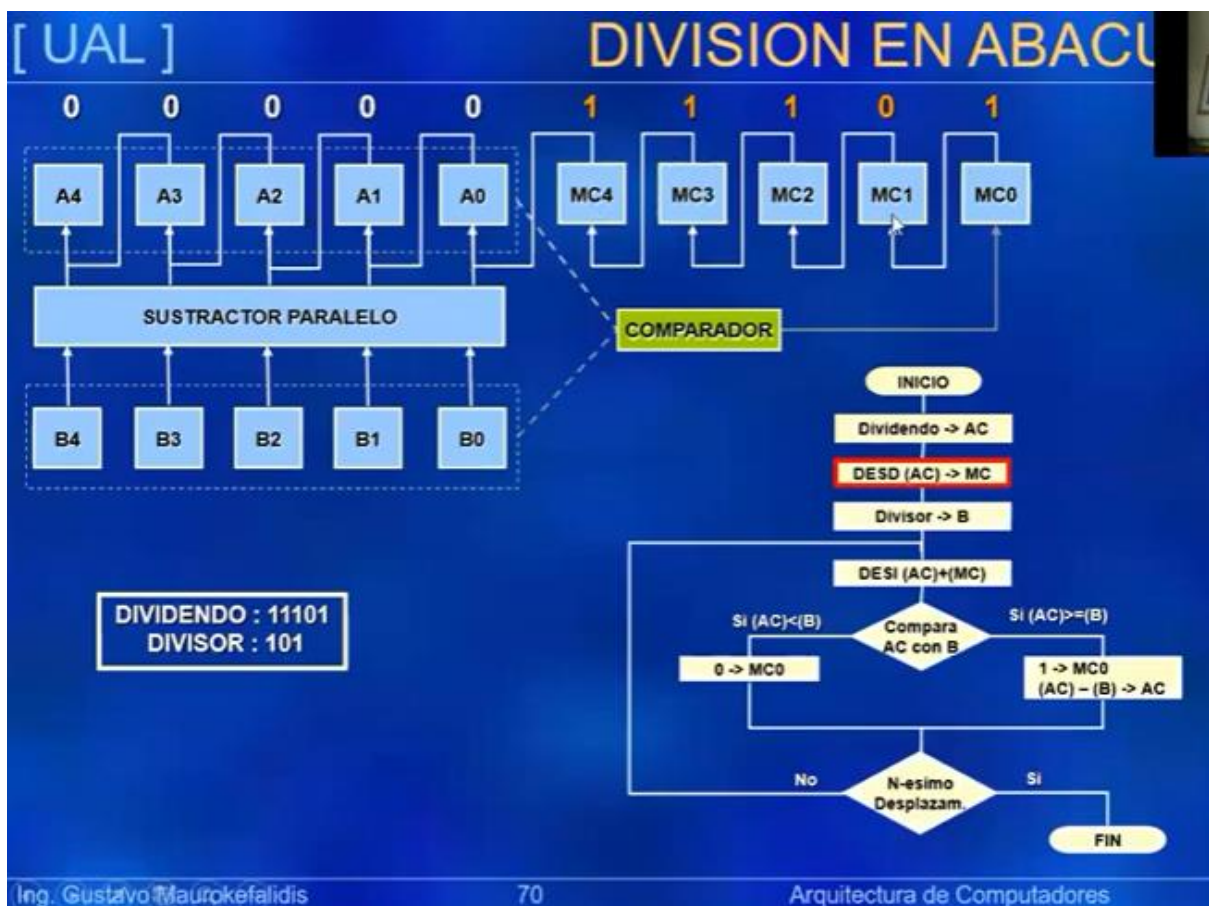
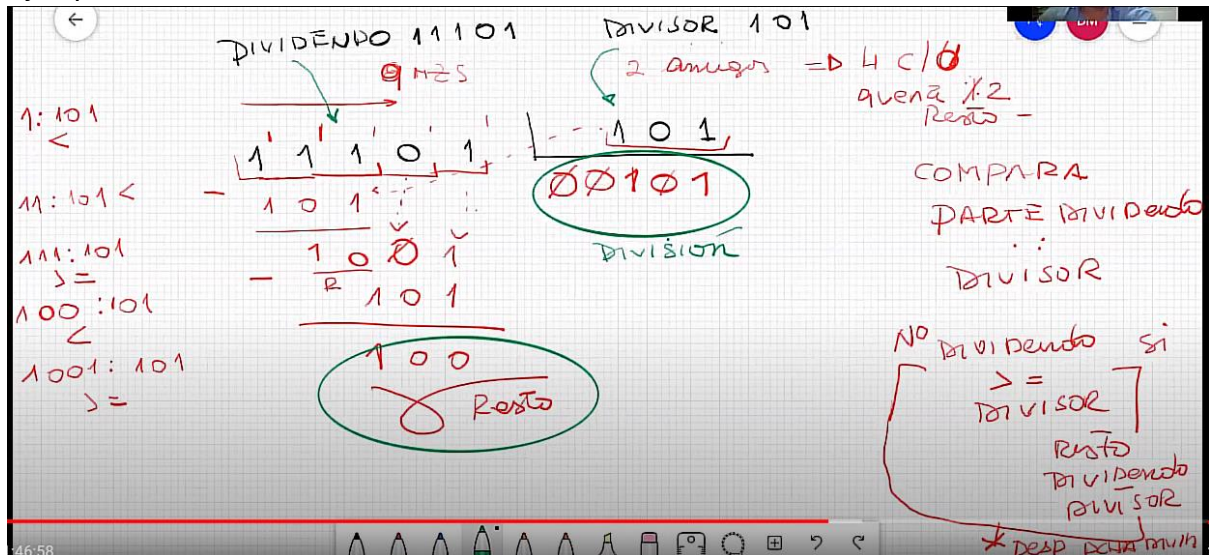
$$\text{c/SUM} : \left(\frac{cf + 1}{5} - 1 \right) \times \left(\frac{col - 1}{4} \right)$$

$$\text{c}'_{1/2SUM} : \left(\frac{cf + 1}{4} - 1 \right)$$

$$cf = \frac{2 \cdot cf + 2 \cdot col}{20}$$

División en ABACUS

Ejemplo:

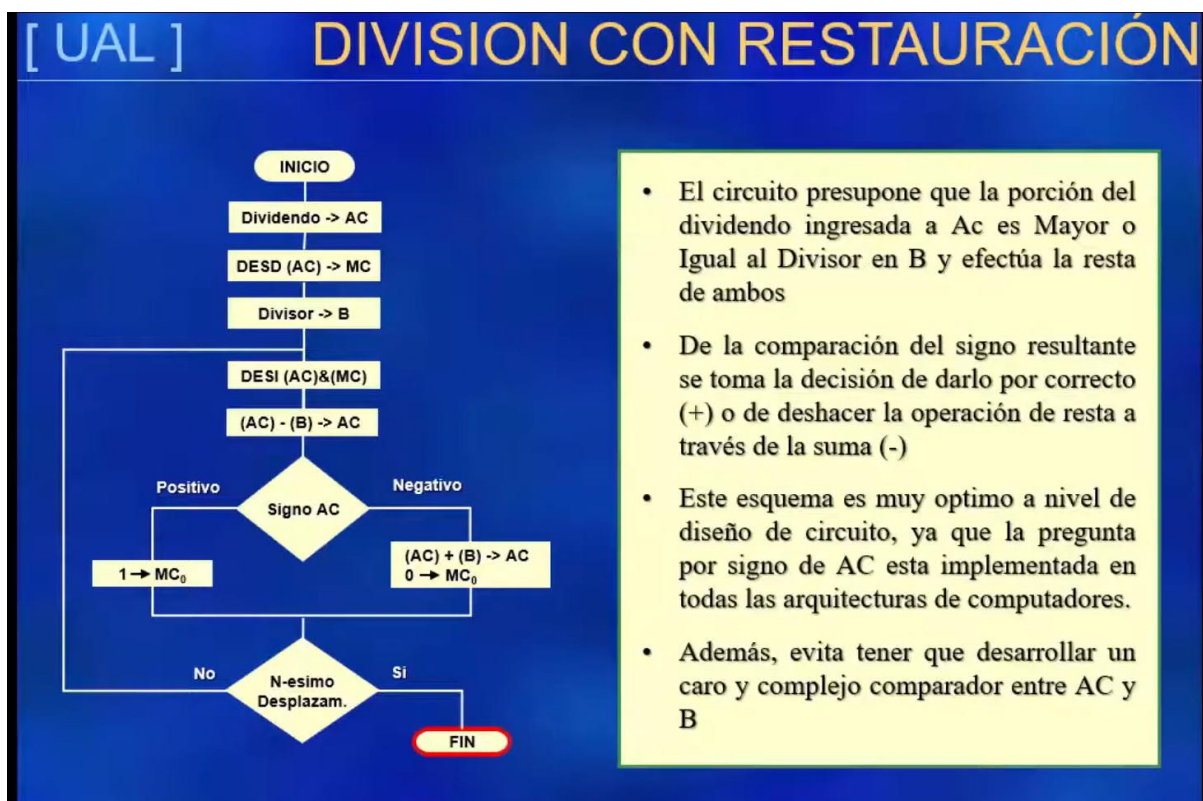
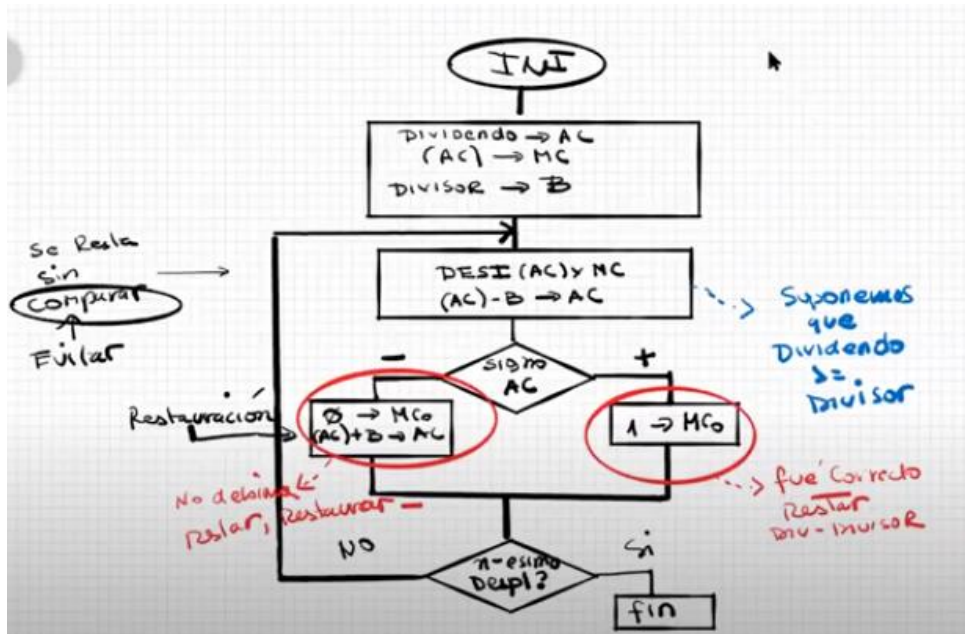


El Dividendo se carga en la derecha (Cociente), y después se va a ir comparando contra el Divisor y se va desplazando hacia la izquierda, hacia dentro del Acumulador.

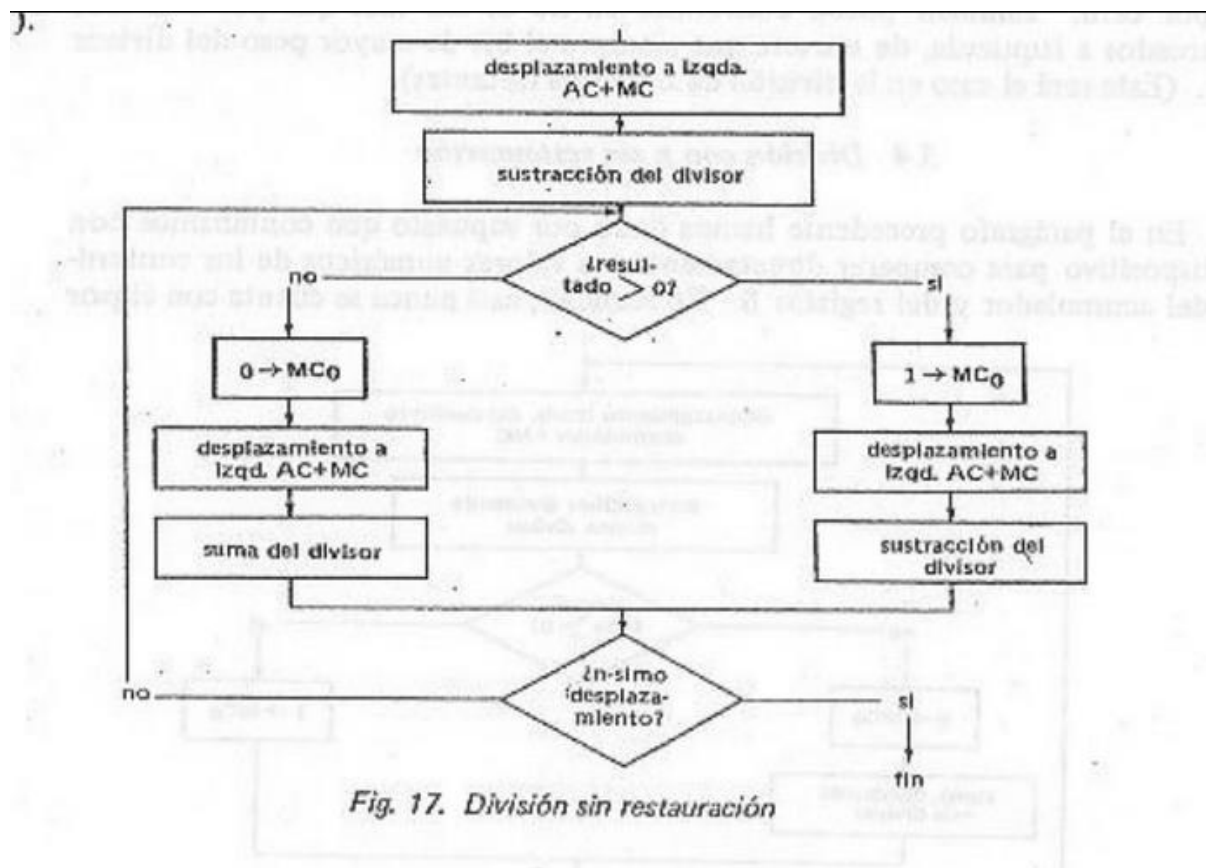
Y en el cociente vamos colocando 0, que equivale a cuando el Dividendo es muy pequeño frente al Divisor, y necesitamos considerar más cifras para poder realizar la división.

En MC0 va colocando 0, se va cargando los resultados parciales de la división.
Y va preguntando constantemente si es la enésimo desplazamiento, para saber si ya termino

DIVISIÓN CON RESTAURACIÓN



División sin restauración



Consideraciones finales de ALU

- En General, el diseño de la ALU juega un papel muy importante en los ciclos de cálculo de computador, incidiendo en gran porcentaje sobre la performance de la arquitectura del computador.
- El tipo y cantidad de operadores cableados o microprogramados será preponderante en la relación costo/performance.
- Observamos que ciertas funciones se han optado por cablear con el tiempo y en función de los resultados obtenidos, tal el caso de las operaciones en coma flotante, que hasta los 90 estaban microprogramadas y era una opción la incorporación de un Co Procesador adicional para estas operaciones que mejoraba notablemente la performance global
- En lo particular, la ALU de ABACUS consiste en un dispositivo que cablea operaciones básicas tales como suma/sus y algunas operaciones lógicas básicas. A su vez desarrolla esquemas que puedan implementar operaciones mas complejas como el manejo de números con coma, y la multiplicación y división algebraica binaria.
- A su vez prescinde de un registro fuente de entrada (Bi obligando al registro palabra de memoria a sostener todo el tiempo los valores lógicos del Bus M en pos del calculo solicitado por la instrucción en curso.

 MEMORIAS

Link: [Clase Memorias 2021 - Video 12](#)

Definición:

Dispositivo electrónico capaz de almacenar de manera temporal una información binaria de tal forma que todo órgano que requiera servirse de él, pueda acceder a la información solicitada en cualquier momento.

Punto de memoria:

Es el soporte físico que permite almacenar la unidad mínima de información binaria. Deberá contar con lógica para permitir la lectura o escritura en él.

Ejemplo:

- Núcleo de ferrita (Antiguo solo para idea)
- Biestable electrónico - El semiconductor, y en las memorias dinámicas: Transistores y capacitores

Se denomina punto de memoria al soporte físico del bit; que es la información más elemental registrada en la totalidad de las memorias (almacenamiento binario). Es decir, *cada punto de memoria almacena un bit*. El punto de memoria puede ser perfectamente definido e individualizado y constará, además del dispositivo de almacenamiento y sus métodos de acceso para lectura y escritura: es el caso del biestable electrónico o del núcleo de ferrita.

Características de la memoria

- **Volatilidad:** se dice que la información almacenada es volátil si corre el riesgo de verse alterada cuando se produce un fallo de la alimentación eléctrica y no volátil en caso contrario. Las memorias constituidas por biestables electrónicos son volátiles, en cambio, las magnéticas son no volátiles.
- **Lectura y Escritura:** son las operaciones básicas de las memorias. Se realiza escritura cuando se registran informaciones en la memoria y lectura cuando se extraen informaciones previamente registradas.
 - La **lectura** puede ser **destructiva** (la información leída se borra de la memoria), por ejemplo, en los núcleos magnéticos o en la RAM Dinámica; en estas se debe re-grabar la información sistemáticamente luego de cada lectura. En caso contrario, la lectura es **no destructiva**.
 - La **escritura** puede exigir o no exigir un borrado previo. Las memorias de semiconductores son de lectura no destructiva y no obligan forzosamente a un borrado previo a la escritura, mientras que en las memorias de núcleos la lectura es destructiva y la escritura exige un borrado previo
- **Direccionamiento:** Para localizar una información dentro de una memoria, la computadora debe saber donde la ha colocado anteriormente. Es la función del direccionamiento.
- **Modo de Acceso:**

- Aleatorio, directo o selectivo: en las memorias con punto de memoria electrónico (biestables) se tiene acceso directamente a cualquier información, cuya dirección sea previamente conocida, en una cantidad de tiempo que es independiente a la posición de la información en la estructura de la memoria.
- Secuencial: en las informaciones guardadas en cinta magnéticas, es necesario desenrollar completamente la cinta, leyendo registro por registro hasta encontrar el buscado. En este tipo de memorias, el tiempo de acceso al registro buscado dependerá fundamentalmente de la posición de este en la cinta. En los discos magnéticos el acceso a la información es selectivo (directo) en cuanto a la elección de la pista, y secuencial en el interior de la pista.
- **Tiempo de Acceso:** Tiempo que transcurre entre el instante en que se lanza una operación de lectura en memoria y el instante en que se dispone, en el RPM, la primera información buscada.
- **Tiempo de Escritura:** Es el tiempo que transcurre entre el momento en que se proveen, en los registros asociados correspondientes de la memoria, la información a guardar y su dirección, y el instante en que la información queda realmente almacenada.
- **Velocidad de Transferencia:** Es la velocidad a la que se pueden transferir datos a, o desde, una unidad de memoria. Para memorias de acceso aleatorio coincide con el inverso del tiempo de ciclo.
- **Caudal:** así se llama al número máximo de informaciones leídas o escritas por unidad de tiempo. Para las memorias de acceso por bloques se supone que las informaciones están situadas en lugares sucesivos de memoria. Se expresa en kilo-informaciones o megainformaciones por segundo.
- **Tiempo de Ciclo de Memoria:** incluye el tiempo de lectura y reescritura.
- **Capacidad:** Es el número de informaciones que la memoria puede almacenar. Teniendo en cuenta el direccionamiento binario, las capacidades de la memoria se expresan habitualmente como potencia de dos.

CLASIFICACION DE MEMORIAS

Tipo de Memoria	Clase	Borrado	Mecanismo de Escritura	Volatilidad
Memoria de Acceso aleatorio (RAM)	Memoria de lectura / escritura	Eléctricamente por Bytes	Eléctricamente	Volátil
Memoria de solo Lectura (ROM)	Memoria de solo Lectura	No Disponible	Mediante Máscaras	No Volátil
ROM Programable (PROM)			Eléctricamente	
ROM Borrable (EPROM)	Luz Ultravioleta, chip completo			
Memoria FLASH	Eléctricamente por bloques			
PROM Borrable eléctricamente (EEPROM)	Eléctricamente por bytes			

MEMORIA DE ACCESO ALEATORIO

Borrado por Bytes - En una memoria no puedes borrar $\frac{1}{2}$ palabra, un ASCII, una cadena de 8 bits, mínima cantidad.

El borrado se hace por palabra, en la memoria central (RAM) tienes que borrar la fila completa. Es volátil, ante la ausencia de electricidad se borra toda la RAM.

MEMORIA DE SOLO LECTURA (ROM)

No permite su borrado, una vez que se grabó, ese chip va a mantener la información hasta que el material se degrade. Mediante máscaras. Ejemplo de CD-ROM, Vinilo (Campo análogo).

Diodo quemado - Cierra el circuito.

Aplicaciones: Queremos que la cuestión atmosférica, que ocupe un programa de arranque mínimo. Mientras que las Flash se pueden borrar.

ROM PROGRAMABLE (PROM)

Mecanismo de escritura a través de un programa, no se hace con máscaras como las ROM. Lo puedes montar en un grabador de PROM, por única vez.

Los programadores, podían grabar sus programas, no dependía de una línea de producción.

ROM Borrable (EPROM)

Se utiliza mucho para leer, se puede reescribir.

Se aplica ultravioleta, a través del proceso químico, puedes borrar toda la info. Que se pueda reescribir la BIOS por ejemplo. Estas graban por Byte.



Pero más que nada se ocupa para lectura.

MEMORIAS FLASH

Se desempeñan muy bien en la lectura.

La escritura produce un desgaste en el dispositivo, lo que puede llevar al futuro a tener pérdida de información.

Bloques, conjunto de palabras. Lo graban de manera eléctrica por bloques. Simulan la estructura de un disco rígido.



EEPROM (Electrical Erasable Programmable ReadOnly Memory)

Son memorias de solo lectura, que pueden ser borrables, de manera eléctrica.

Ya trabajan **por bytes, por palabras**. Lo que más se asemeja a un Disco de Estado Sólido.

Las ventajas y desventajas se verían después en **Periféricos**.

JERARQUÍA DE MEMORIAS

- “Es una sinopsis del conocimiento específico/particular de cada uno de los niveles que integran la jerarquía. Comenzando desde el procesador al exterior, es decir en orden creciente de tiempo de acceso y capacidad.”
- “Es una organización que agrupa tipos de memoria en función a 3 parámetros que la identifican:
 - capacidad
 - tiempo de acceso
 - costo.
- De esta manera, cada nivel establece el **área/campo de aplicación** dentro de la arquitectura, donde se desempeñarán estas memorias con similares características.”
- “La capacidad a medida que bajamos es mayor, los dispositivos almacenan más. El costo es menor hacia abajo. El tiempo de acceso aumenta hacia abajo.” (Hacia donde apunta la flecha significa que aumenta).

Vértice superior denota POCA CAPACIDAD y la base GRAN CAPACIDAD. La pirámide ordenada por Capacidad, Tiempo de Acceso y Costo, en niveles que son Campo de aplicación(Es un área dentro de la cual va a dar una utilidad)



De arriba para abajo, aumenta el tiempo de acceso, pero bajando disminúis costos.

Niveles

NIVEL 0 - De los registros del hardware de la arquitectura.

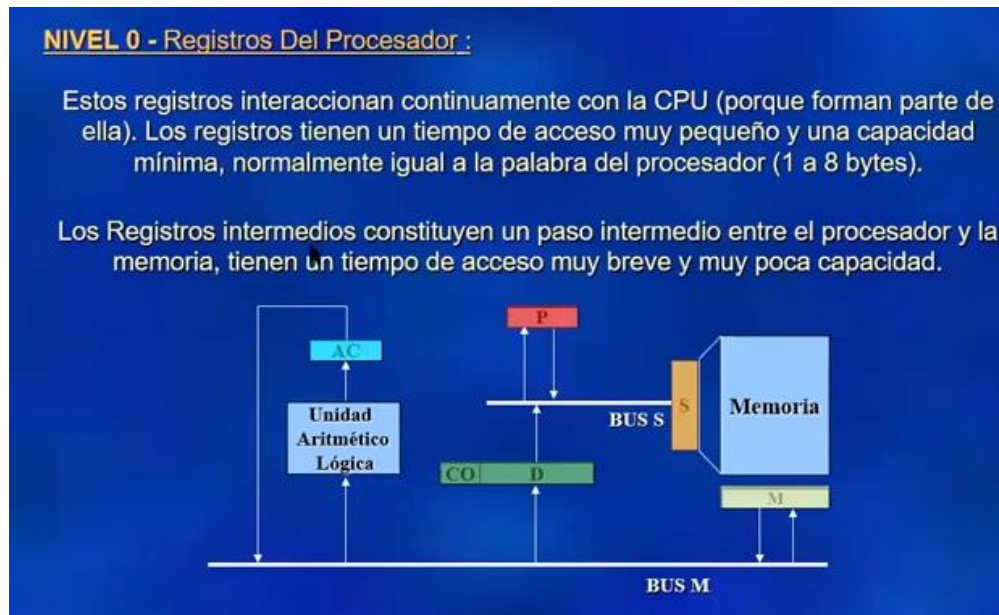
NIVEL 2 - Como órgano que provee información a la Unidad Central de Procesamiento.

NIVEL 4 - Disco de estados sólidos son altamente veloces vs HDD. Tienen diferencia en costos y velocidades.

El campo de aplicación, por más que sea muy rápido. Da apoyo a la arquitectura, no cambia su campo de aplicación.

NIVEL 5 - Backup

NIVEL 0 - Registros del procesador



Registros que interactúan con los BUSES.

“Estos dispositivos tienen que ser muy veloces, el campo de aplicación es para el diseño lógico de la máquina Von Neumann.”

Estos registros son los que interactúan con la unidad central de proceso. Los registros de la CPU son:

- Registro P - Contador de programa
- Registro S - Registro de selección de memoria
- Registro M - Registro palabra Memoria
- Registro AC - Registro Acumulador
- Registro I - Registro de instrucción

“Se habla de costo no porque estos registros se compran, sino que en un diseño industrial son parte de una placa. Se habla de costo por unidad de información pero relativo.”

NIVEL 1 - Memorias caché

Son memorias de pequeña capacidad. Se coloca entre la CPU y la Memoria Central. Muy alta velocidad, amortiguar las asimetrías entre velocidades de tiempos de accesos, entre los diferentes órganos. Liberar a la memoria RAM de estar leyendo/escribiendo continuamente.

Actúa como un sistema de almacenamiento de instrucciones y de datos capaz de comunicarse con el procesador a gran velocidad, evitando así dos cosas muy importantes que podrían reducir el rendimiento.

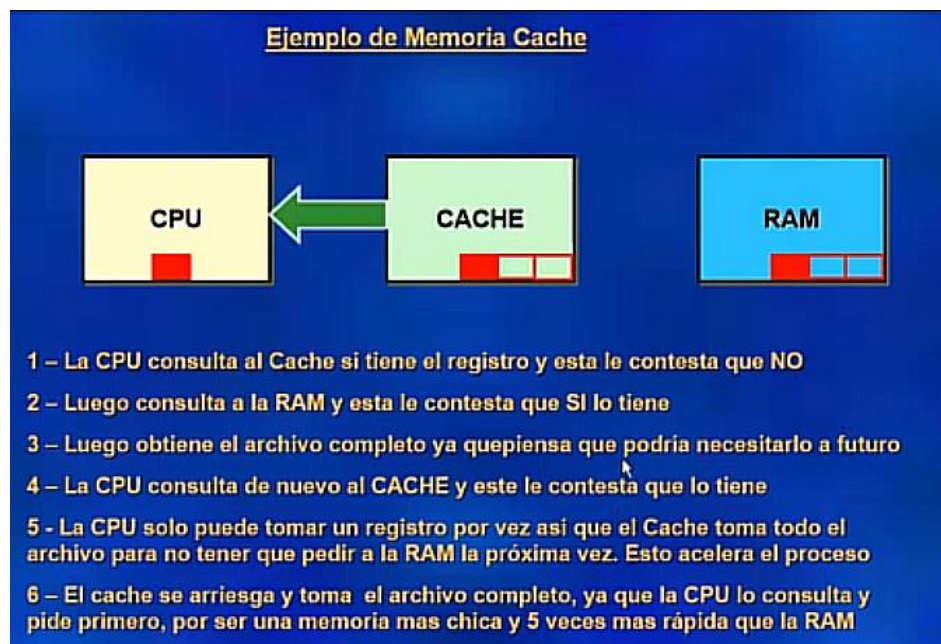
Puede tener **2 niveles**: (Hoy en día hay caché hasta Nivel 4)

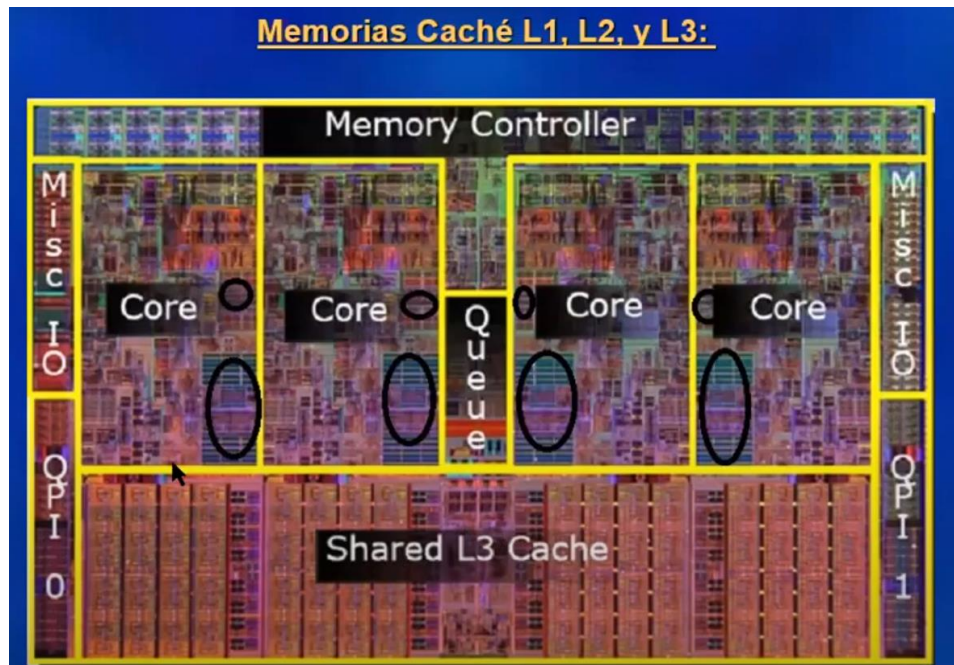
- **Caché on chip**, memoria caché dentro del circuito integrado. Dentro del procesador, permitían cachear datos e instrucciones, estaban cerca de todas las operaciones.
- **Caché on board**, memoria caché en la placa del circuito impreso pero fuera del circuito integrado. De nivel 2, mucho más cara que una RAM, se ponía en un zócalo de la mother. Hacía caché entre toda la arquitectura (RAM, periféricos, etc) y el procesador.

La primera es mucho más rápida que la segunda.

Arquitectura Harvard, a diferencia de Arquitectura Von Neuman, utiliza memorias caché distintas para códigos que para datos.

Ejemplo de funcionamiento:



**L1:****Características:**

- Nivel más básico rápido y caro
- Menos capacidad
- Está más cerca del procesador
- Se caracteriza por cachear cálculos e información
- Libera al núcleo de tener que hacer cálculos, o acceder recurrentemente a la memoria

L2:**Características:**

- Equilibrio entre cercanía, capacidad y velocidad
- Mayor capacidad que una L1
- Tiempo de acceso es más lento que L1

L3, un Caché compartido, Shared:



Característica:

- Permite integrar el caché a nivel de núcleos y no a nivel individual de core
- Comparte información de los núcleos del procesador
- Tiene mayor capacidad que una L2 pero son más lentos

Entonces el camino de la búsqueda de un dato te queda así:

L1 -> L2 -> L3 -> RAM -> Disco

Latencia - Retardo que necesita el sistema hasta que encuentre el dato o instrucción, esta latencia aumenta a medida que pasamos desde las caché L1 hasta la memoria RAM.

La memoria caché es un tipo de memoria específica que está preparada para servir de apoyo al procesador, y que es capaz de trabajar a velocidades muy elevadas. Esta memoria se divide en diferentes niveles como dijimos, pero su función base es la misma: apoyar a la CPU para mejorar su rendimiento.

Para ello actúa como un sistema de almacenamiento de instrucciones y de datos capaz de comunicarse con el procesador a gran velocidad. Normalmente en la memoria caché se almacenan datos e instrucciones que el procesador va a necesitar para completar diferentes tareas a corto plazo. Con ello se evitan dos cosas muy importantes que podrían reducir el rendimiento

NIVEL 2 - MEMORIA RAM

En este nivel residen los programas y los datos.

La CPU lee y escribe datos en él con **menos frecuencia** que en los niveles anteriores.

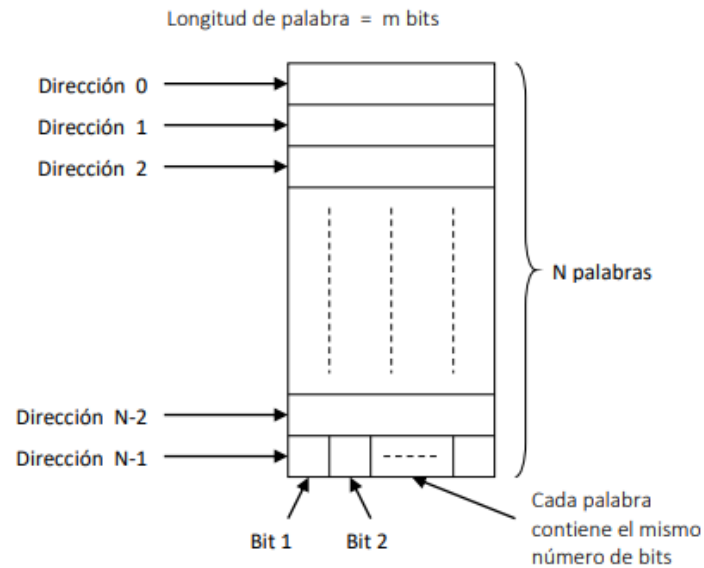
Tiene un tiempo de acceso relativamente **rápido** y **gran capacidad**.

Existen 2 tipos de memorias RAM: **Estáticas y Dinámicas**.



Las **extensiones de memoria central** (los módulos de memoria RAM que compras de los negocios) son memorias de la misma naturaleza que la memoria central que amplían su

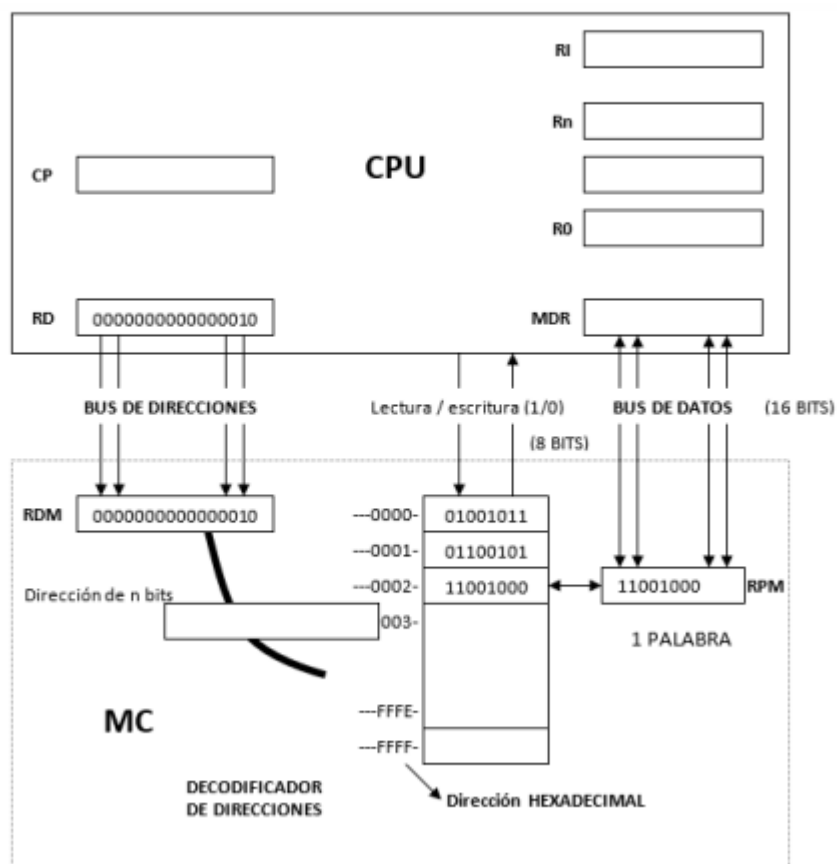
capacidad de forma modular. El tiempo es similar, a lo sumo un poco mayor, al de la memoria central y su capacidad puede ser algunas veces mayor.



Su campo de aplicación es:

- Está afuera de la CPU, es intercambiable, son módulos que van afuera. Tiene una amplia conectividad con la arquitectura a través del bus M (mover instrucciones y datos), el bus S (enviar direcciones y poder ubicar palabras adentro).

En la memoria habrá tantas direcciones como posiciones pueda tener una memoria.



NIVEL 3 - MEMORIA FLASH

NIVEL 4 - Discos magnéticos

Los HDD, son memorias que residen en dispositivos externos al ordenador. En ellas se archivan programas y datos para su uso posterior.

Estas memorias suelen tener **gran capacidad** pero pueden llegar a tener un **tiempo de acceso muy lento**.

Dentro de ellas también se pueden establecer varios niveles de Jerarquía.

Los discos Rígidos. Tener en cuenta las líneas Western Digital Black, Blue, Purple.

Cantidad de revoluciones también tener en cuenta.



Dato

- También se usan como apoyo a la memoria central en caso de que ésta sea insuficiente (**memoria virtual**). (Volcado de la RAM en el disco rígido). La sobreutilización del volcado va a impactar en la performance.

NIVEL 4 - Discos SSD

Están los Discos SSD (solid-state drive) es un tipo de dispositivo de almacenamiento que utiliza memoria no-volátil, como la **memoria flash**, para almacenar datos EN VEZ de platos o discos magnéticos como los HDD.

La mayoría de los SSD utilizan **memoria flash basada en puertas NAND**, que retiene los datos sin alimentación eléctrica.

Las SSD pueden ser construidas a partir de RAM.

Estos dispositivos pueden emplear fuentes de alimentación independientes, como baterías, para mantener los datos después de la desconexión de la corriente eléctrica.



NIVEL 5 - Discos ópticos

Un disco óptico es básicamente una superficie plástica que tiene una lámina metálica. La superficie es alterada por un láser a medida que va girando el disco, en función de la superficie, va magnetizando para colocar 1, y no hay se busca un 0.

Para la lectura:

- El lector del láser, cuando está magnetizado interpreta un 1.
- El lector del láser, cuando no lo está, interpreta un 0.

NIVEL 6 - Cintas Magnéticas

Data cartridge / backup tape / Cinta magnética de datos:

Los sistemas de cinta usan las mismas técnicas de lectura y grabación que los discos. El medio es una cinta de poliéster flexible cubierta por un material magnetizable.

Datos

- Tecnología de almacenamiento magnético secuencial, utilizada por primera vez en el ámbito informático de 1949.
- El dispositivo grabador de estas cintas, era un contenedor plástico que en su interior tiene un par de carretes móviles y una larga cinta magnética. Sobre la cuál se leen, regraban y almacenan bits permanentemente por medio de inducción electromagnética.
- Temperatura o campo magnético podría romper a las cintas magnéticas

Evolución de la memoria RAM

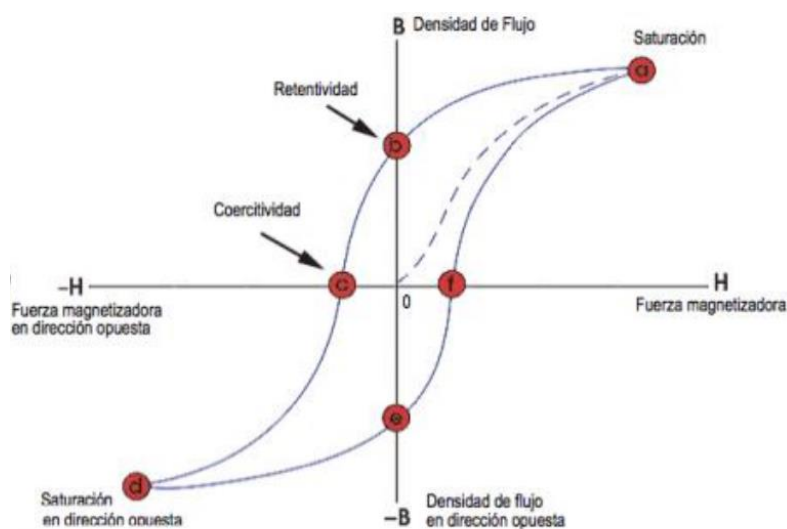
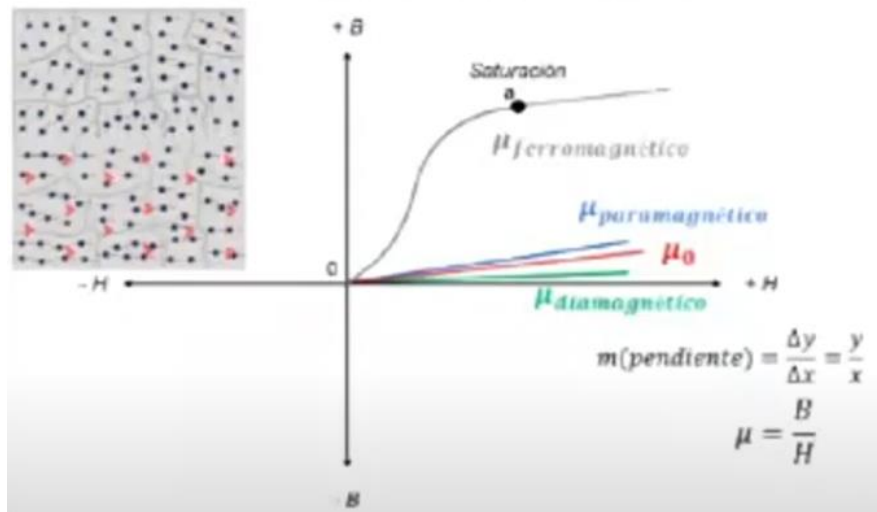
Memoria de Núcleo de Ferrita

El dispositivo básico de almacenamiento en una memoria de núcleos magnéticos consiste en una **pequeña pieza toroidal** (en forma de anillo), llamada núcleo magnético o anillo de ferrita. El núcleo magnético, almacena información binaria, es generalmente una pieza sólida de material cerámico ferromagnético. Se ocupaban hasta los años 50.

Ciclo de histéresis

<http://hyperphysics.phy-astr.gsu.edu/hbasees/Solids/hyst.html>

Núcleo de diversos materiales



La corriente que transita sobre el alambre induce un campo magnético en el material
 H = Corriente eléctrica, y B = Campo magnético

Aumentando H , el campo en B empieza a magnetizarse. El límite de hasta donde se puede magnetizar, depende del material (composición, tamaño, etc), el punto de saturación.

Frente a la ausencia de la corriente eléctrica, entonces se sigue manteniendo un poco de valor de en B , porque queda como un campo remanente, se queda un "imán permanente". B mide la fuerza del campo.

**Datos:**

- Es un anillo para mantener mejor la magnetización del mismo.
- El anillo se va a magnetizar en el sentido de la corriente.
- Desde el plano físico, se tomó ventaja de dicho material para utilizar el ciclo de Histéresis y magnetización

https://youtu.be/9Ptzl_juHbA?t=2221

LECTURA NÚCLEO DE FERRITA

Para averiguar el estado o sentido de magnetización del anillo se enhebra un segundo hilo (que no debe ser paralelo al hilo de escritura para evitar inducciones indirectas entre ambos) que hará las veces de hilo de lectura (“porque tiene otras funciones”).

La lectura es destructiva, por eso, se necesitan **2 ciclos** para hacer **LECTURA y RESTAURACIÓN**.

- Para obtener el estado, se aplica una corriente eléctrica una corriente por el hilo verde de sentido contrario para forzar la **Desmagnetización** del anillo y llevarlo al estado 0.
 - **Si dicho anillo estaba en el estado 1**, al pasar al 0 su magnetización cambia bruscamente.
 - **Es decir: Si estaba en 1** es decir Magnetizado, al pasar a 0, va a hacer que ese cambio de magnetismo, induzca una corriente muy pequeña (pero suficiente) en el **HILO ROJO (LECTURA)** y un circuito lo puede interpretar, que **HABÍA UN 1**.
 - **Si el anillo estaba en 0**, este cambio induce una corriente en el hilo de lectura totalmente apreciable, aunque menor que la necesaria para producir una conmutación (como haría el hilo de escritura). Si el anillo estaba ya en el estado 0, la magnetización apenas cambia y la corriente inducida es mucho menor. Midiendo esta corriente inducida puedo averiguar cual era el estado del anillo. Entonces, si el anillo estaba en 1 se ve forzado por la corriente

negativa a ponerse en 0, en esa bajada de campo magnético se induce una corriente eléctrica por el hilo de lectura.

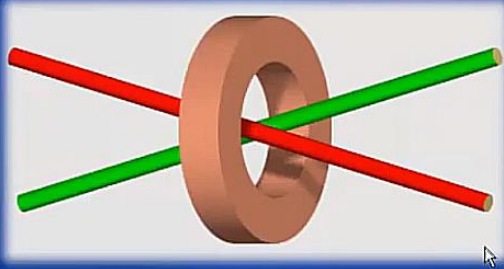
- Es decir: **Si estaba en 0** es decir Desmagnetizado, al aplicar la corriente eléctrica, no hay corriente eléctrica ínfima (no suficiente) en el **HILO ROJO (LECTURA)** y un circuito lo puede interpretar, que **HABÍA UN 0**.

Este **mecanismo de lectura es destructivo**, porque al magnetizarlo para obtener su estado siempre produce su puesta a 0. La manera de solucionar esto es **aplicando un nuevo pulso de escritura cuando el estado inicial era 1**. Así pues, al haber cambiado este estado a 0, lo volvemos a poner en 1 y rápidamente lo dejamos como estaba.

Restauración

Si el anillo está cargado con un 1, aplicaría un corriente eléctrica contraria/negativa por el hilo de escritura tratando de llevar el 1 a 0. Por eso se dice que la memoria en la lectura es destructiva, el primer ciclo lee y el segundo tiene que regenerar.

Anillo de Ferrita: Lectura



Para averiguar el estado o sentido de magnetización del anillo se enhebra un segundo hilo (que no debe ser paralelo al hilo de escritura para evitar inducciones indirectas entre ambos) que hará las veces de hilo de lectura. Para obtener el estado, se aplica una corriente eléctrica que fuerce la magnetización del anillo al estado 0. Si dicho anillo estaba en el estado 1, al pasar al 0 su magnetización cambia bruscamente

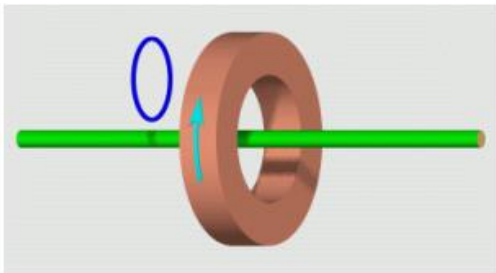
Este cambio induce una corriente en el hilo de lectura totalmente apreciable, aunque menor que la necesaria para producir una conmutación (como haría el hilo de escritura). Si el anillo estaba ya en el estado 0, la magnetización apenas cambia y la corriente inducida es mucho menor. Midiendo esa corriente inducida puedo averiguar cuál era el estado del anillo.

Este mecanismo de lectura es destructivo, porque al magnetizarlo para obtener su estado siempre produce su puesta a 0. La manera de solucionar esto es aplicando un nuevo pulso de escritura cuando el estado inicial era 1. Así pues, al haber cambiado este estado a 0, lo volvemos a poner en 1 rápidamente lo dejamos como estaba.

ESCRITURA NÚCLEO DE FERRITA

Para actuar sobre la magnetización de un anillo de ferrita se le enhebra un hilo conductor y se hace pasar por él una corriente eléctrica. Al aplicar una intensidad suficiente, el anillo se magnetiza en el sentido del campo magnético inducido y, al suprimir dicha corriente inductora, la **magnetización permanece**.

Por el hilo conductor deberíamos aplicar una corriente eléctrica tal que produzca una resistencia sobre el campo magnético que tiene el anillo, de manera tal de orientar todas sus partículas hacia el norte o hacia el sur **magnetizándolo positivamente o negativamente**. Dependiendo del sentido para donde gire la corriente en el anillo será un 0 o 1.



DESCRIPCIÓN DE LAS SOLUCIONES CON ANILLOS DE FERRITA

Implementación 2D

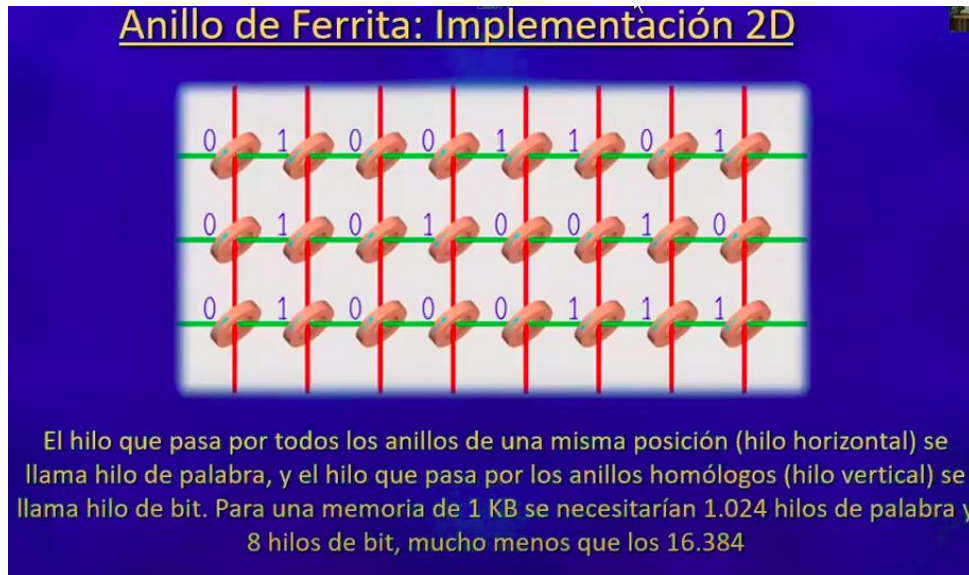


“Solución: teoría de las corrientes coincidentes. Vamos a tener dos tipos de hilos, hilos verticales por cada columna (gobierna el bit de la palabra seleccionada) e hilos horizontales por cada fila (gobierna una palabra completa).” (Memoria 2D) Esto reduce notablemente la cantidad de hilos. Con la misma memoria mencionada anteriormente se pasa de 16.384 hilos a 1032 hilos. En aquellos anillos donde se quiera colocar un 1 se debe mandar una corriente de $\frac{1}{2}$ por toda su fila y además mandar una corriente de $\frac{1}{2}$ por su columna, entonces en aquellos anillos donde coincidan ambas corrientes de $\frac{1}{2}$ se pondrán en 1. Es

importante destacar que en aquellos anillos donde solo tendrán una carga de $\frac{1}{2}$ no se llegaran a magnetizar.

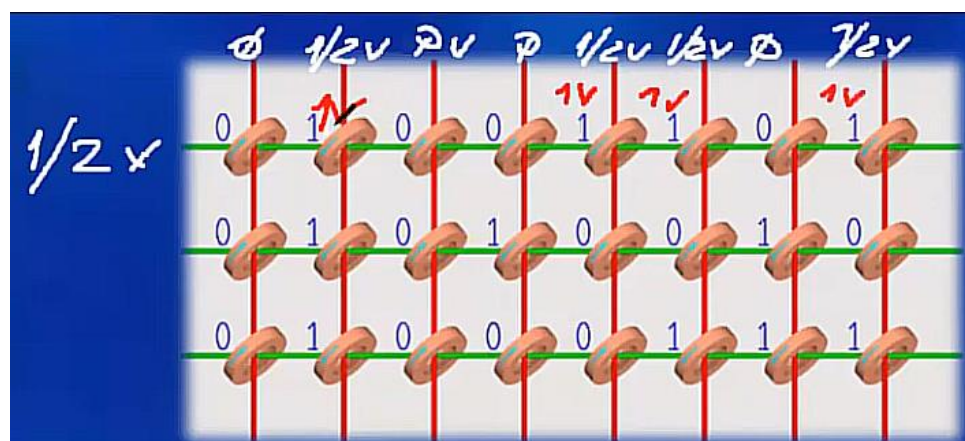
El hilo que pasa por todos los anillos de una misma posición (hilo horizontal) se llama **hilo de palabra**, el hilo que pasa por todos los anillos homólogos (hilo vertical) se llama **hilo de bit**.

Presenta un problemón = 8 (columna) * 1024 (filas) para una memoria de 1 KB



Hilo de palabra = Gobierno la palabra completa, se utiliza para seleccionar la palabra a modificar.

Hilo de bit = Gobierno el bit dentro de la palabra, se utiliza para interactuar con los bits de la palabra seleccionada.



Solamente las que tienen 1V se van a magnetizar. Tienen la corriente eléctrica necesaria para llegar al punto de saturación.

Escritura 2D

Escritura: Primero se pone la palabra en cero. Luego se hace pasar una corriente eléctrica positiva por el hilo de palabra y, simultáneamente, otras iguales por los hilos de bits cuyos anillos hay que poner a "1".

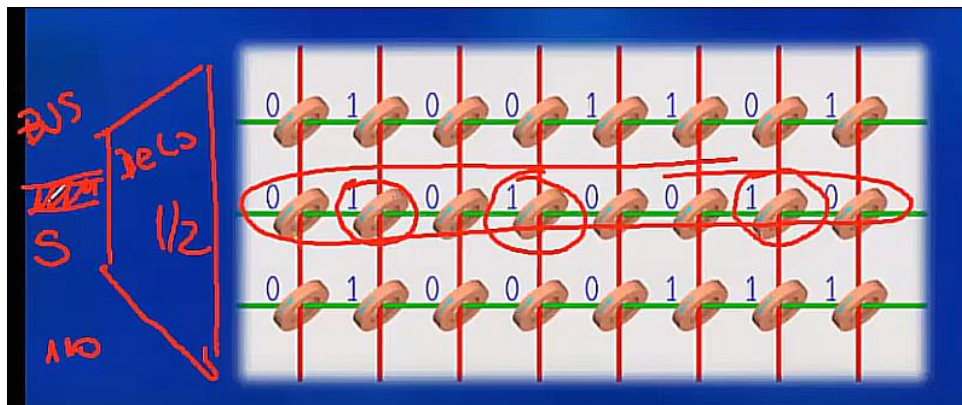
La intensidad de esas corrientes está establecida de forma que una, por sí sola, no es capaz de hacer que los anillos que atraviesa se pongan a "1"; pero si dos juntas. Los anillos que reciben el efecto combinado de la corriente en el hilo de palabra con la del hilo de bit se pondrán a "1".

Se envía una corriente V- inversa de magnetización por el **Hilo de palabra**, entonces se generaría una **Cte Inducida** en el **Hilo de bit** en caso que el valor era 1, o nada si el valor era 0.

Y después se hace un proceso de restauración, que es enviar el mismo valor de V+ por el **Hilo de Palabra** entonces se restauraría en los **núcleos** con el valor que tenía antes, hubiese sido 0 o 1.

"Esta técnica se conoce como transferencia con previa puesta a cero. Para la escritura hay que hacer una previa puesta a cero porque solo se magnetizan los 1 con corrientes coincidentes, no se pueden escribir un cero individual. Esto es porque escribir los ceros llevaría al proceso a ser mucho más complicado, trabajando con corrientes negativas y generando resistencias inversas. Por esto, se colocan todos a cero y se graban solo los unos."

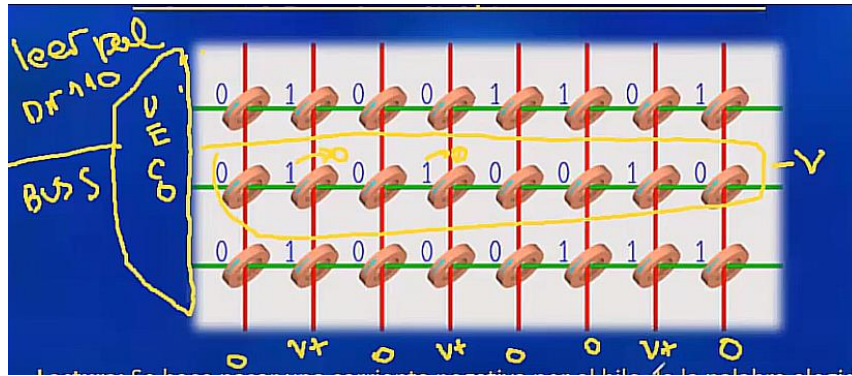
Ejemplo:



Lectura 2D

Lectura: Se hace pasar una corriente negativa por el hilo de la palabra elegida, que lleve a todos los anillos al estado "0" (lectura destructiva).

Los que tienen un "0" siguen igual, y los que tienen un "1" pasan a "0" induciendo una corriente apreciable en su hilo de bit. Pero como esta corriente es menor que la necesaria para forzar una conmutación, no afecta al estado de los demás anillos atravesados por el mismo hilo de bit.

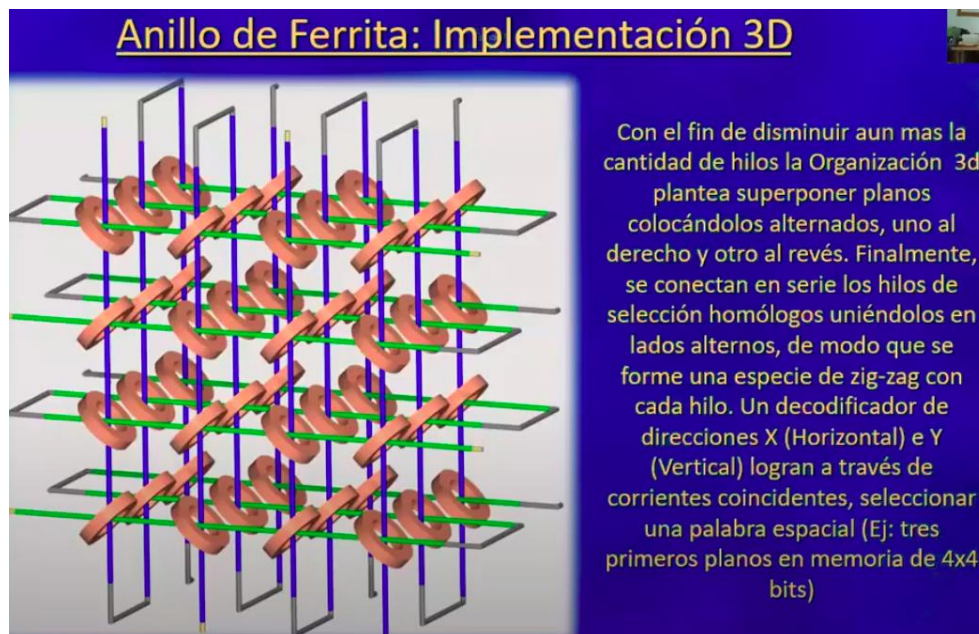


Luego, se regeneran los valores

Nota: El color verde en la imagen es hilo de palabra. El rojo es hilo de bit

“Por el hilo de palabra mandas la corriente para ponerlo a 0; aquellos que se resisten generan una corriente inducida sobre el hilo de bit. Esa corriente inducida sobre el hilo de bit no es lo suficientemente alta como para afectar a los compañeros que están sobre el mismo hilo porque no alcanza para hacerlos conmutar, es una pequeña variación de tensión que indica que hay un 1”.

Implementación 3D



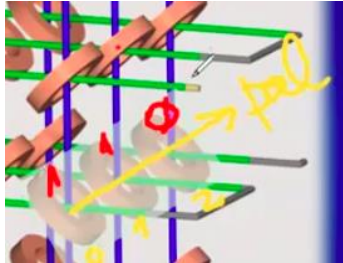
Ejemplo:

- Capacidad de palabras: Cantidad de filas x cantidad de columnas
- Tienes 4 columnas
- Tienes 4 filas
- Entonces tienes $4 \times 4 = 16$ palabras de 3 bits

Vamos a necesitar 2 decodificadores:

- 1 decodificador de fila - Todos los anillos de una columna
- 1 decodificador de columna - Solo una columna

La longitud de la palabra está dada por la cantidad de planos, quiere decir que si tenes “capas”, cada “capa/plano” representa el plano del bit, si quieres una palabra de 8 bits, vas a necesitar 8 planos.



No están paralelos, para no inducir carga magnéticas entre anillos, están unidos con los polos opuestos

“La palabra está constituida por todos los anillos de la misma posición fila columna y con el mismo ángulo de inclinación. Es necesario que los anillos se encuentren con un ángulo de inclinación porque primero, para enhebrarlos es necesario tenerlos así conmutado y segundo, es necesario para que no haya una corriente inducida entre un plano y otro. Si estuviesen paralelos, podría una palabra inducir a otra en el momento de su conmutación. Entonces, se colocaban a 45 grados opuestos cada plano de manera tal que cada palabra no induzca sobre otra palabra, no hay inducción o disminuye porque no están en el mismo ángulo.”

Ventajas 2D y 3D

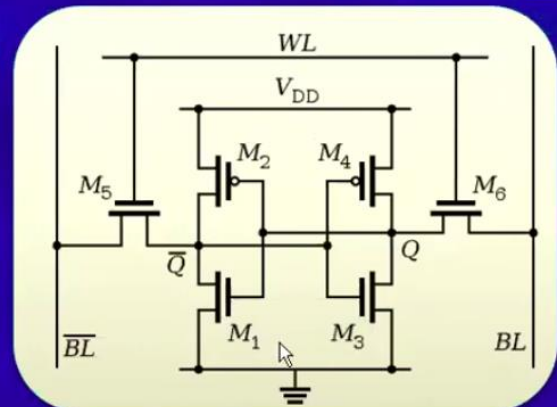
- 2D es muy fácil de hacer.
- 2D crece solo con en el mismo plano.
- 3D optimiza mejor el espacio, puedes crecer verticalmente, es más modular. Manejo espacial de la palabra mucho más optimizado.

DIFERENCIA MEMORIAS ESTÁTICAS Y DINÁMICAS

https://youtu.be/9Ptzl_juHbA?t=5113

ESTÁTICAS

- Estas memorias son de acceso aleatorio, las posiciones en la memoria pueden ser escritas o leídas en cualquier orden, independientemente de cual fuera la última posición de memoria accedida.
- Cada bit en una SRAM se almacena en cuatro transistores, que forman un biestable.
- Este circuito biestable tiene dos estados estables, utilizados para almacenar (representar) un 0 o un 1.
- Se utilizan otros dos transistores adicionales para controlar el acceso al biestable durante las operaciones de lectura y escritura.
- Una SRAM típica utilizará seis MOSFET para almacenar cada bit.



- El acceso a la celda es controlado por un bus de control (WL en la figura), que controla los dos transistores de acceso M5 y M6, quienes controlan si la celda debe ser conectada a los buses BL y BL. Ambos son utilizados para transmitir datos tanto para las operaciones de lectura como las de escritura, y aunque no es estrictamente necesario disponer de ambos buses, se suelen implementar para mejorar los márgenes de ruido.

MOSFET - Transistor de efecto de campo metal-óxido-semiconductor

Son los **registros** (primero en la jerarquía de memorias)

Los transistores están cruzados, en una estructura tipo Flip-Flop lo que permite almacenar el 1 o el 0.

M1, M2, M3 y M4 son el conjunto de semiconductores, que se encargan a nivel de circuito biestable, mantener un 1 o un 0, según lo que se quiera almacenar. Por eso están cruzados en el medio.

M5 y M6 son los encargados del **acceso** son los que se encargan de manejar las operaciones sean de lectura o escritura.

Dos bloques son los que controlan, el hilo WL que gobierna la **Base** de los 2 semiconductores de Lectura y Escritura.

VENTAJAS

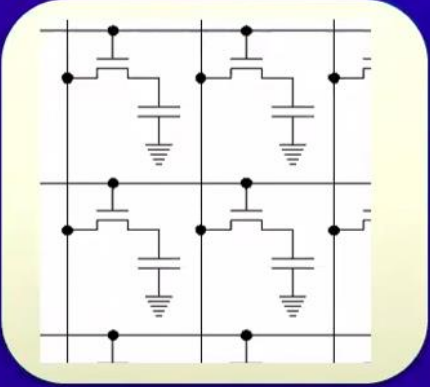
- Altísima velocidad de acceso - Están en la cúspide de la pirámide, son dedicadas para memorias de control de la **Unidad de Control**.
- **6 veces más veloces que la RAM**

DESVENTAJAS

- Consumen mucha corriente eléctrica, requiere alimentación constante. Ante la ausencia pierden toda la información.
- Son de costos muy elevados

DINÁMICAS

- La memoria dinámica fue desarrollada en los laboratorios de IBM pasando por un proceso evolutivo que la llevó de usar 6 transistores a sólo un condensador y un transistor. La invención de esta última la hizo Robert Dennard quien obtuvo una patente norteamericana en 1968 por una memoria fabricada con un solo transistor de efecto de campo y un condensador.
- Las columnas son precargadas a un voltaje igual a la mitad del voltaje de 1 lógico. Esto es posible ya que las líneas se comportan como grandes condensadores, dada su longitud tienen un valor más alto que la de los condensadores en las celdas.
- Una fila es energizada por medio del decodificador de filas que recibe la dirección de la palabra. Esto hace que los transistores conectados a una fila conduzcan y permitiendo la conexión eléctrica entre las líneas de columna y una fila de condensadores.
- Aquella fila cuya media carga indique la posición de la palabra, cargará solo los capacitores cuya columna tengan la media carga restante necesaria para hacerlo.



Son las RAM, Jerarquía en la nivel 2.

Tienen un capacitor, 1 solo transistor por punto de memoria gobierna la lectura-grabación de este punto de memoria.

El capacitor es el punto de memoria como tal.

Cuando se empieza a descargar el capacitor, el transistor cierra el circuito con la línea de carga, para que el capacitor vuelva a cargarse.

Donde hay un 1 en la columna y en la fila, permite que el transistor deje pasar la corriente y que se cargue el condensador.

VENTAJAS

- Mucho más económica, solo 1 transistor y 1 capacitor para 1 punto de memoria.
- No necesita estar constantemente conectado a una fuente de energía, sino que el **capacitor con sus propiedades físicas puede mantener una carga eléctrica.**

Y el transistor lo que hace es justamente detectar cuando el capacitor no cuenta con determinado valor, y habilita el “refresco” de la memoria, para volver a dejar pasar corriente para que el Capacitor se cargue. **Requieren refresco eléctrico para cuando van perdiendo su capacidad.** (Tasa de refresco de la RAM pué)

DESVENTAJAS

- Existe la latencia de carga para los Capacitores, para conseguir el valor interpretado como 1. Mientras que un transistor es mucho más discreto, permite el “pasa o no pasa” la corriente,
- Son mucho más lentas que las estáticas, porque el condensador tarda más tiempo en cargarse a comparación de los transistores de las estáticas.

ESTÁTICAS	DINÁMICAS
Son los registros, 1ra posición en jerarquía de memoria	Son las RAM
Utiliza 4 MOSFETS para mantener el valor del bit, 2 Semiconductores para determinar la operación de entrada o salida.	1 transistor (llave de lectura o escritura) y 1 condensador (es el punto de memoria)
VENTAJAS	
- Alta velocidad de acceso - Son dedicadas para memorias de control de la unidad de control - 6 veces más veloces que la RAM	- Son más económicas debido a la menor cantidad de elementos -
DESVENTAJAS	
- Costo muy alto - Requiere alimentación constante	- Más lentas que las estáticas debido a la latencia de carga de los capacitores

SRAM frente a DRAM:

- ★ Ambas tienen en común que son volátiles, ante pérdida de potencia eléctrica hay pérdida de datos.
- ★ Una celda DRAM es más simple que una SRAM y en consecuencia más pequeña. Por tanto, las DRAM son más densas (celdas más pequeñas = más celdas por unidad de superficie), permitiendo almacenar más datos que una SRAM de tamaño similar, y son más económicas que estas últimas.
- ★ Una DRAM requiere de circuitería adicional para realizar el refresco, aunque en memorias grandes, el coste fijo de la circuitería de refresco se ve compensado por el menor coste de las celdas DRAM. Por su parte, la SRAM consume menos energía al no requerir operaciones de refresco constantes.
- ★ Las SRAM son más rápidas que las DRAM.
- ★ Las DRAM son mucho más baratas, pero también más lentas
- ★ Debido a todo lo anterior, las SRAM se usan para memorias cachés, y las DRAM para memoria principal.

Organizaciones de memoria basadas en semiconductores (Biestables)

https://www.youtube.com/watch?v=tTJMzhr_5Cg&list=PLgTKn0MhANLTPffShcV9I2INbaxlPZvLI&index=15

MEMORIA 2D

Es una arquitectura de Memoria Volátil cuyo punto de memoria está representado por componentes de tipo flip-flop o biestables.

Necesitamos de un mecanismo de **selección** para determinar a qué punto de memoria deseamos realizarle la operación, en este caso denotado por la conexión S.

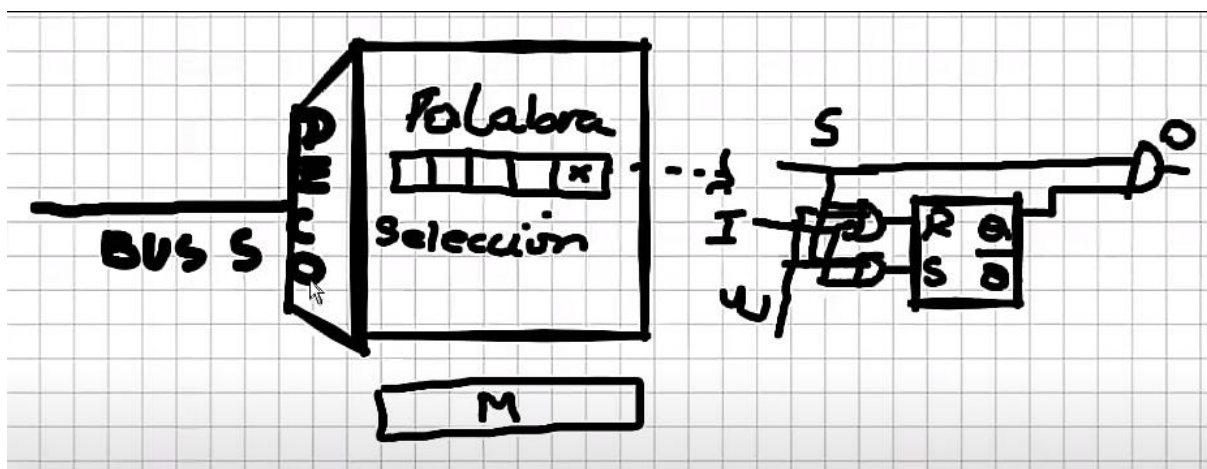
El punto de memoria 2D está compuesto por: un flip-flop de tipo RS, tres compuertas AND y una compuerta NOT. Posee tres entradas:

- S (selección): tiene una doble función, seleccionar el punto y también producir la lectura de su contenido a través de la salida O (output).
- I (input - entrada): permite el ingreso del bit que se va a almacenar.
- W (write - escritura): indica el momento en que se grabará la información de I en el flipflop.

La lectura del flip-flop saldrá por O (output) siempre que S (selección) = 1.

LECTURA

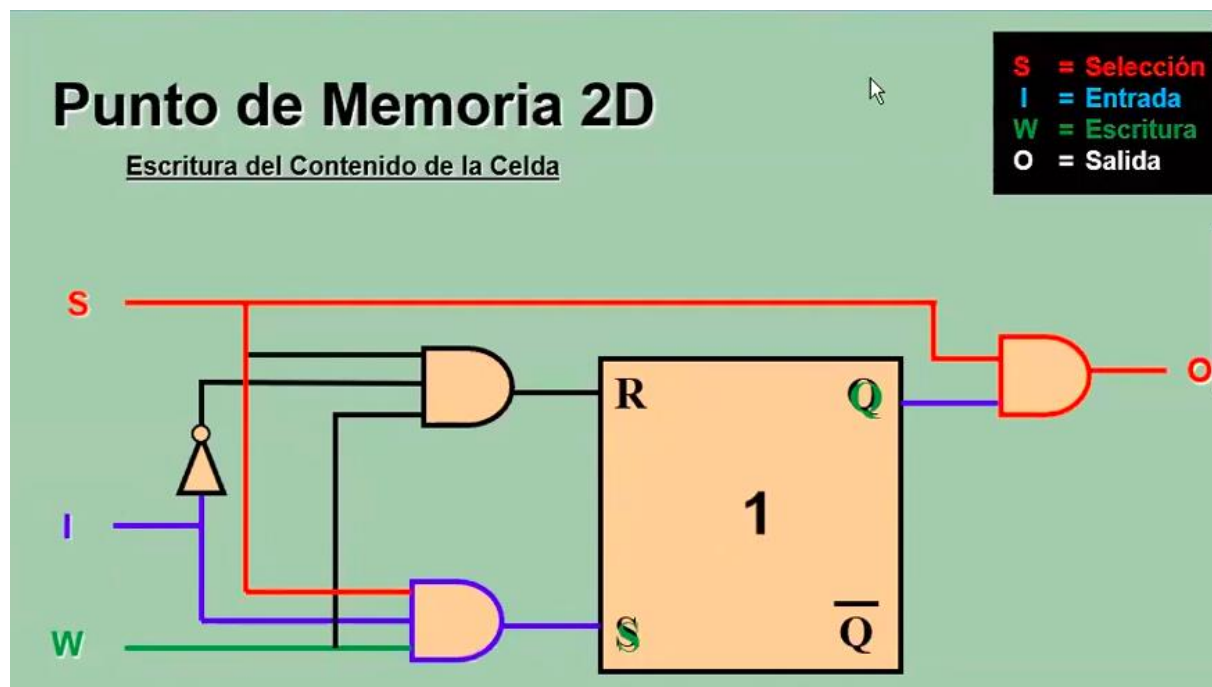
Hacer Selección, produce de manera espontánea la lectura de la información en el punto, la cual es transmitida por la compuerta AND en O. Enviar un 1 por la entrada S del biestable que se desea leer.



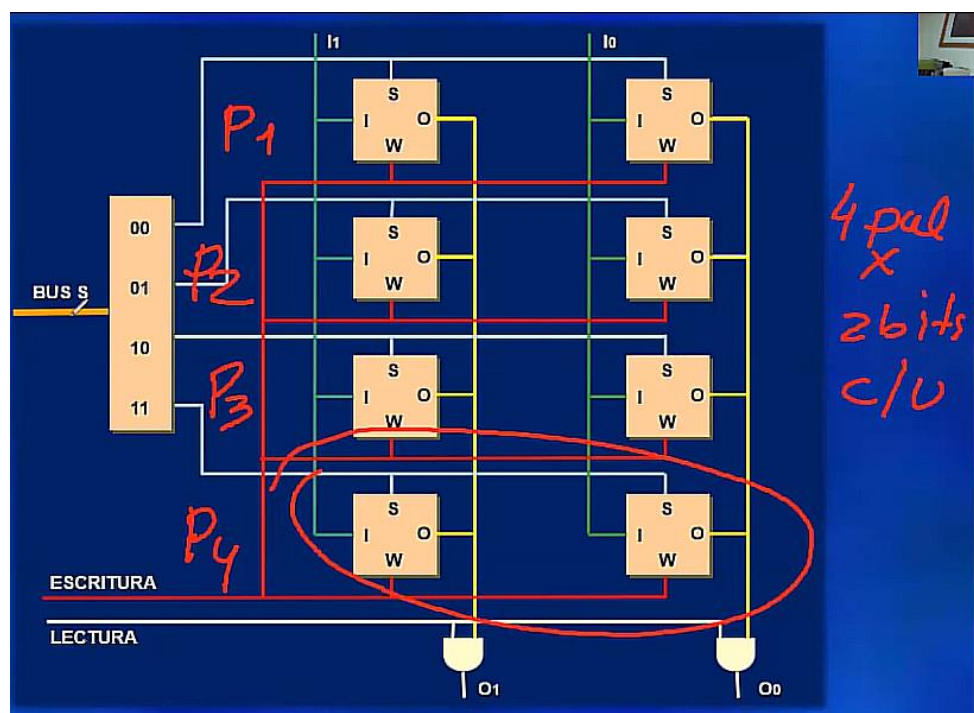
Tiene que hacer selección a toda la palabra transversalmente, uno por uno.

La operación de lectura consiste en enviar un 1 por el hilo de selección que se desea leer y un 1 por el hilo de habilitación de lectura

ESCRITURA



Para escribir una determinada configuración binaria en una palabra de memoria se envía un 1 para seleccionar (Entrada S, por medio del decodificador, hilo blanco) la fila correspondiente, la información por los hilos de bit (I1, I2, I3) y un 1 por el hilo de Escritura (W) que alcanza a todos los biestables.



Únicamente los biestables sensibilizados por el hilo de palabra almacenarán la información.

RIESGO

Como siempre que activemos S, se produce una lectura. Se tiene el riesgo de hacer que una escritura genere una lectura espontánea.

Por eso, para evitar que se produzca una lectura, se coloca una periferia combinacional (un registro combinacional extra) para prevenir que suceda eso. En el circuito global, se verá esto de cómo prevenirlo.

Combinación

Seleccionado + Input (Entry del circuito) + Write - Para hacer la escritura

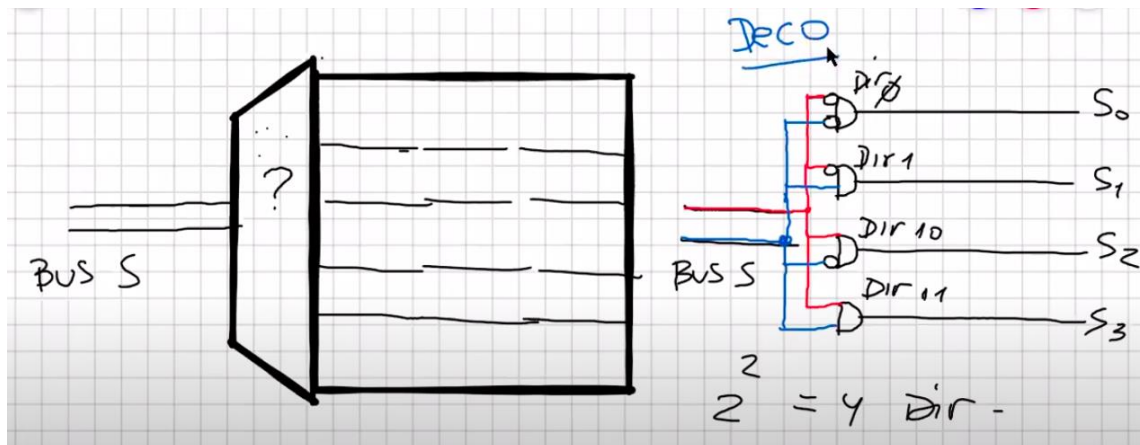
EJEMPLO de 4 palabras de 2 bits

¿Cómo se codea un DECODIFICADOR?:

Dos hilos en el bus S

4 compuertas del tipo AND

4 palabras



Bus S, tiene 2 hilos.

Cada compuerta AND está relacionada a la cantidad de palabras que podrá direccionar.

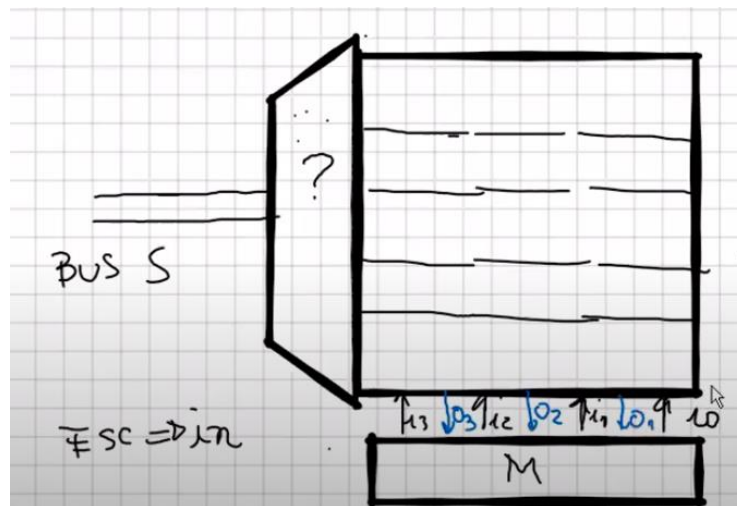
El rojo es denominado el **hilo de menor peso**.

Un dato importante a tener en cuenta, es que por cada hilo del BUS S, la cantidad de compuertas AND del decodificador crece exponencialmente.

Ejemplo

8 hilos

$2^8 = 256$ compuertas AND en el Decodificador

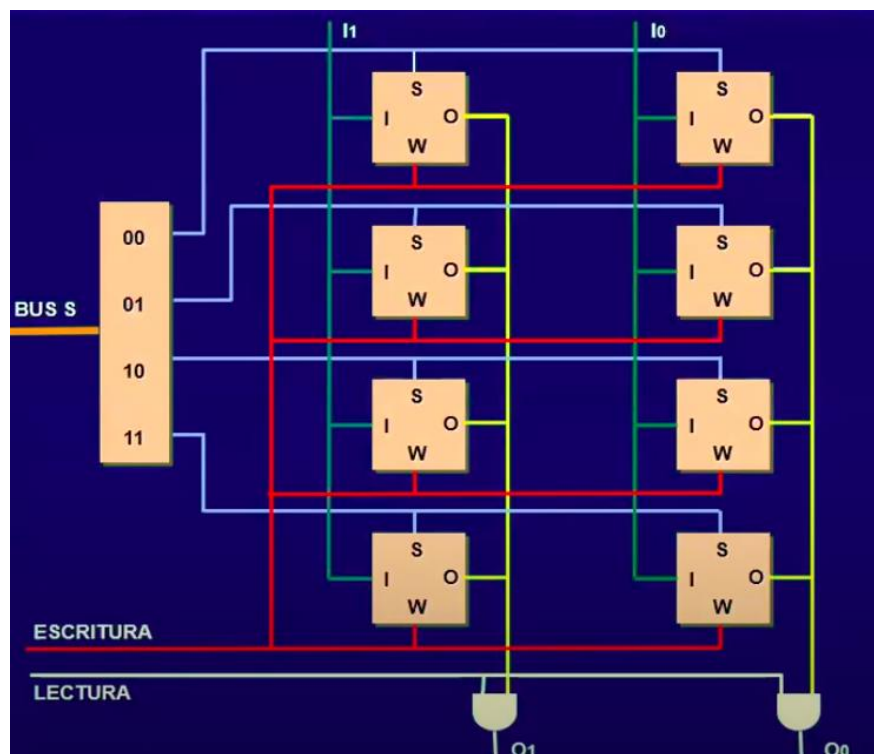


M es registro Palabra Memoria

Lectura implica recibir los $O(n)$. Es decir, la cantidad de bits de Output que van hacia M, de arriba hacia abajo.

Escritura implica cargar los $I(n)$ Es decir, la cantidad de bits de Input que vienen desde M, de abajo hacia arriba.

Para eso están los **2 compuertas AND (en este ejemplo) de abajo** para justamente, si NO tiene habilitado la señal de LECTURA, entonces NO se realice la lectura espontánea en el proceso de ESCRITURA.



VENTAJAS

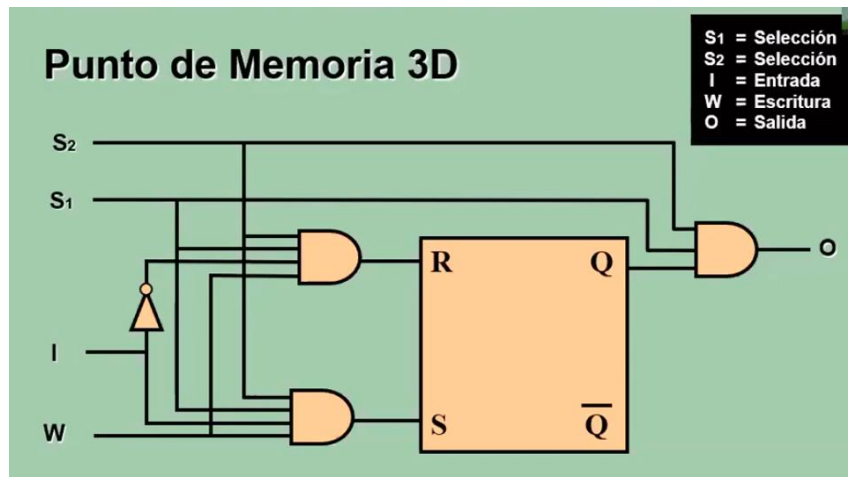
- Es muy simple de implementar, tiene un punto de memoria sencillo

- Es económica en cantidad de elementos

DESVENTAJAS

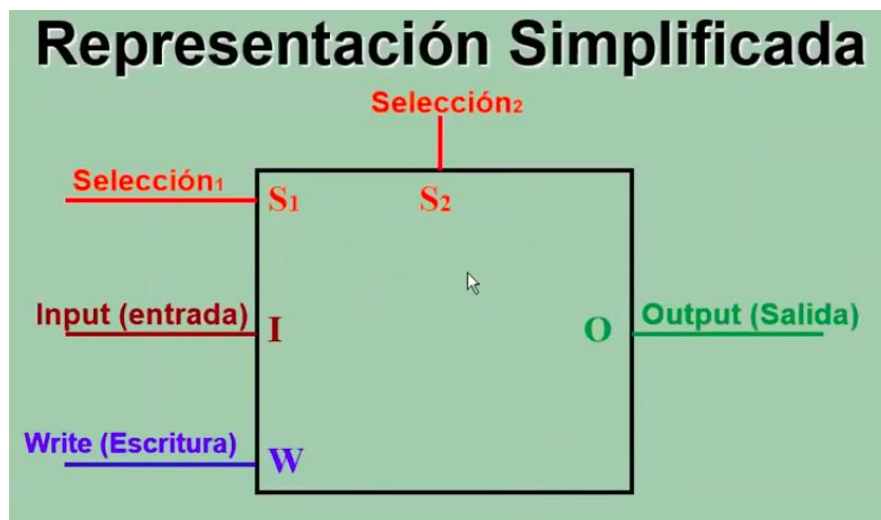
- El decodificador se pone costoso, a medida que va a creciendo la cantidad de palabras, exponencialmente aumenta
- Al ser 2D requiere mucho más espacio físico, y que los componentes no sean afectados por los efectos físicos

MEMORIA 3D



“Agrega **una entrada de selección más**, es decir, una selección del tipo fila-columnas. Un decodificador para filas y otro para columnas. Se soluciona el problema de la memoria 2D que el decodificador era excesivamente grande y largo, ahora tenemos dos entonces dividimos el problema.”

“Ahora las compuertas AND tienen 4 entradas y se eleva el costo.”



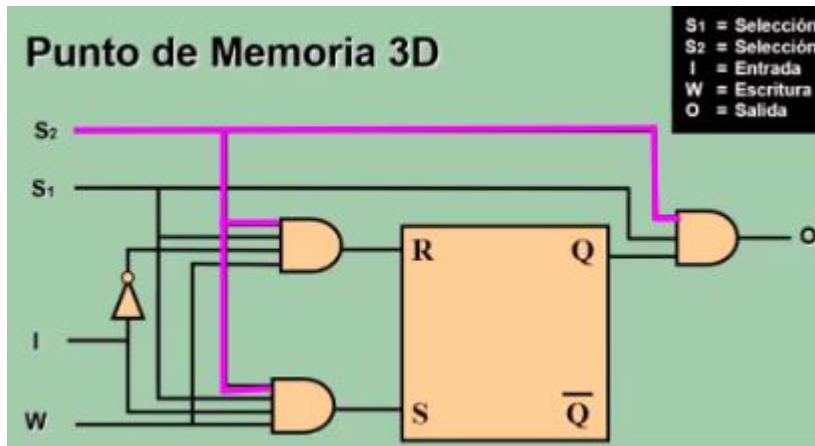
Tiene una **línea adicional de selección S2**, implica que ahora tienes 2 hilos S, es decir, 4 hilos (porque son 2 hilos por S).

- Ahora las compuertas AND necesitan de 4.

- La longitud de la palabra implica CANTIDAD DE PLANOS en el espacio.
- Cada PLANO almacena una posición de la palabra.

Ejemplo de 1 punto de memoria con RS

Ahora tiene 11 entradas, porque van de la mano con la cantidad de compuertas. 3 AND, 4 + 4 + 3.



Ejemplos de planos

Si quiero 4 palabras, de 3 bits cada una, tengo que tener 3 planos.

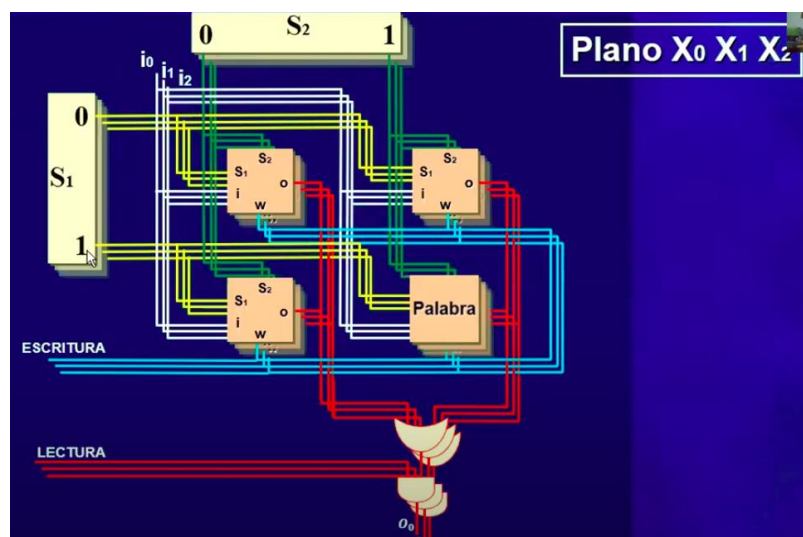
Memorias de 64 bit, se necesitan 64 planos.

Entonces para eso ahora aparece el concepto de **DECODIFICADOR DE FILA Y COLUMNA**. Si aumenta la cantidad de bits de palabra? ¿Cambia la cantidad de DECO? No. Porque avanza en profundidad, por lo que la cantidad de bits del deco no aumenta.

S1 - Deco filas - MENOR PESO

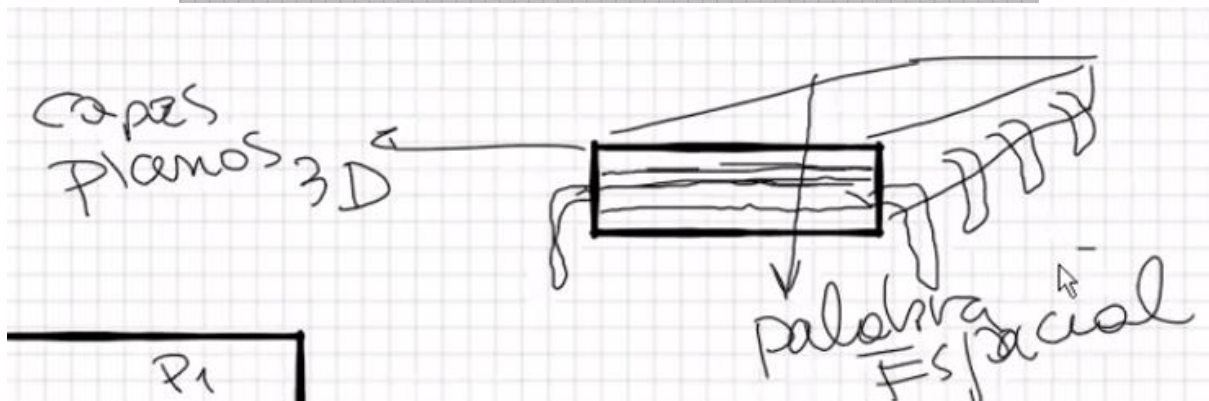
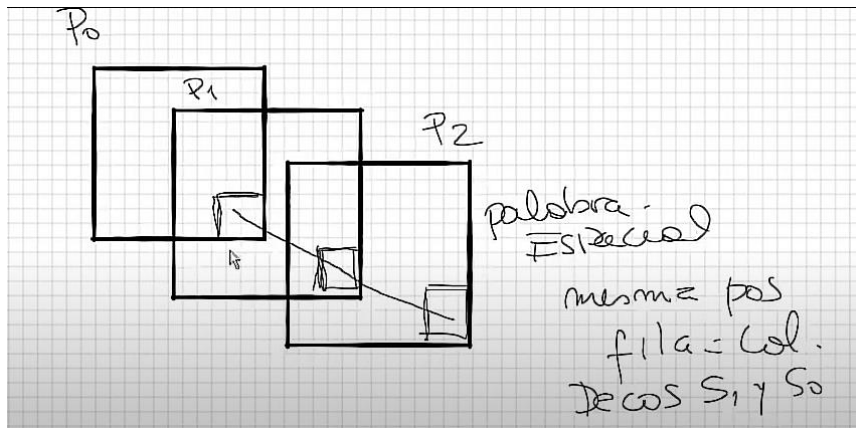
S2 - Deco columnas - MAYOR PESO

Ejemplo para 10 - 1 es S2 y 0 es S1

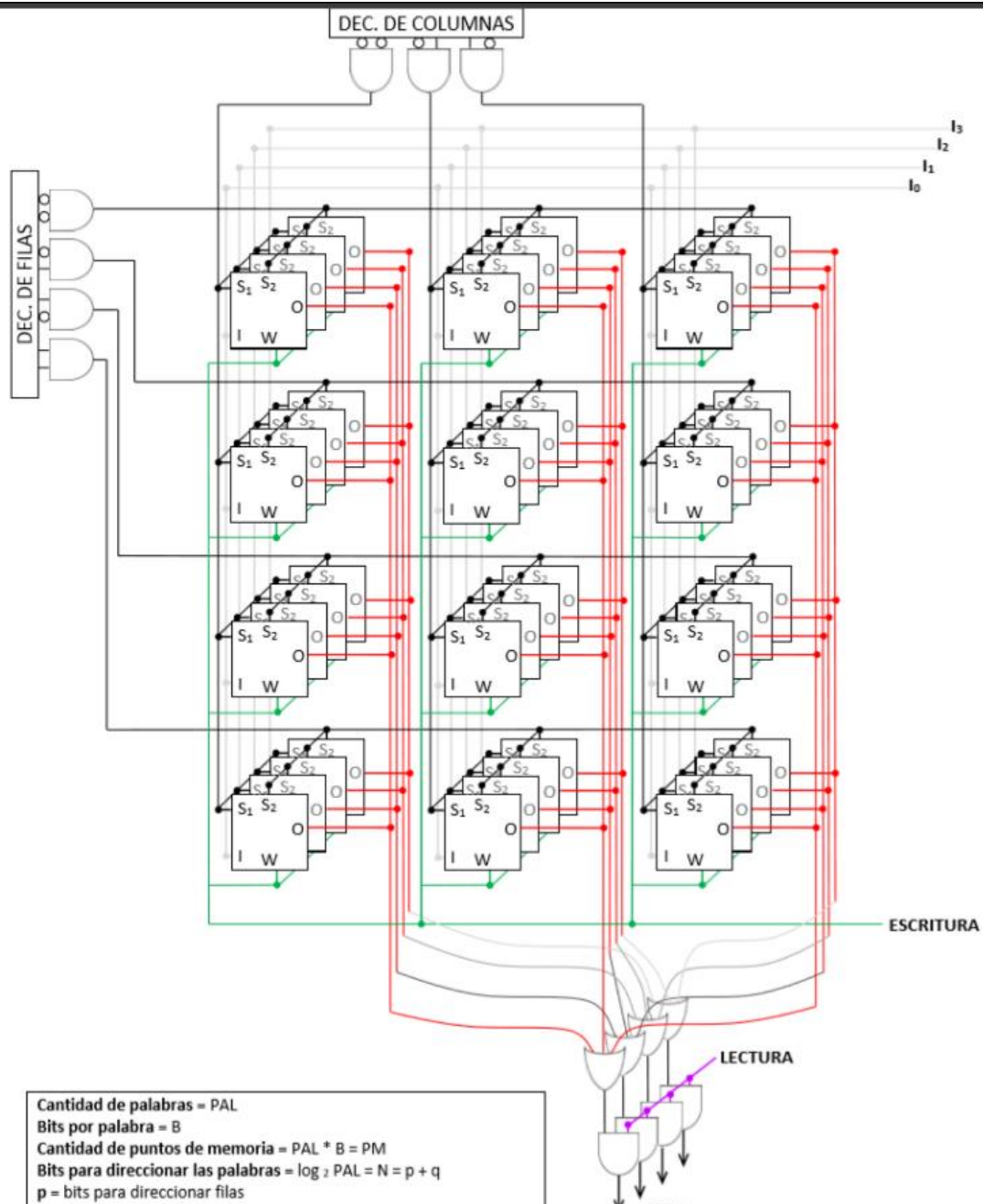


La **Palabra Espacial** puede leerse de arriba pa abajo.

Memoria de 3 bits $\rightarrow$ 3 planos



Ejemplo Memoria de 12 palabras de 4 bits



VENTAJAS

- Al estar dividido en 2 el Decodificador, van a ser mucho menos complejo que el 2D, se reduce la cantidad de compuertas AND, no crece en forma exponencial
- Ocupa menos espacio, al trabajar por capas, se reduce en tamaños

DESVENTAJAS

- La desventaja es que aumenta el costo aumenta debido a que por cada punto de memoria tenemos 3 entradas más que la memoria 2D, entonces al multiplicar por todos los puntos de memoria que se posee en la memoria aumenta notablemente el costo
 - En el punto de memoria, ahora tiene 11 entradas a diferencia de las 8 que tenía la 2D. Se produce un incremento del costo del punto de memoria.

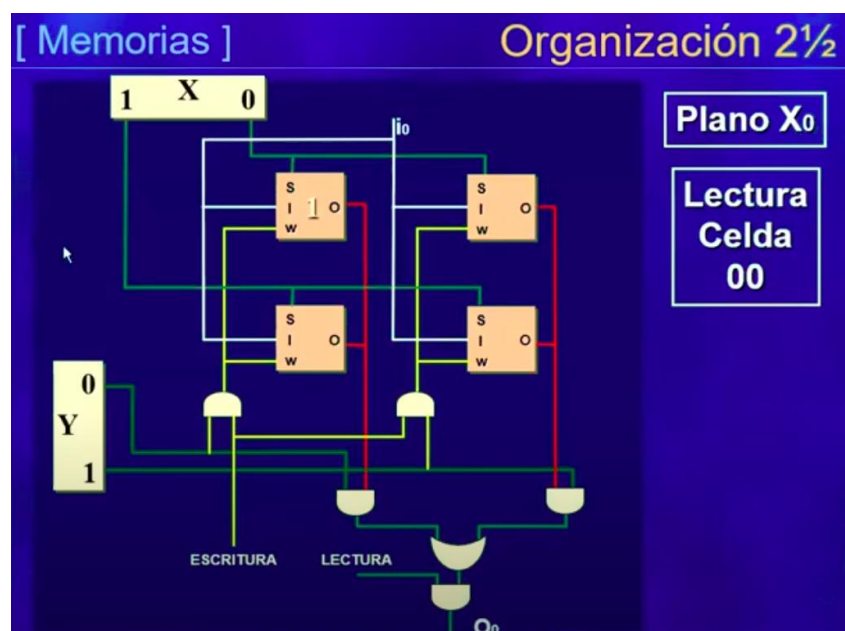
MEMORIA 2 Y 1/2 D

Link del video:

https://youtu.be/tTJMzhr_5Cg?t=4825

Es una organización 3D construida a partir de puntos de memoria 2D, aprovecha punto de memoria con única línea de selección, de la arquitectura 2D sumado a la estructura de múltiples capas/niveles/planos saca de la 3D

El esquema de selección usa un control en las entradas LEER y ESCRIBIR para así lograr la organización 3D. Es también una selección por corrientes coincidentes, con la única diferencia respecto a la selección 3D que la habilitación para escritura no se sitúa a nivel de los biestables, sino al de las líneas de selección de columna



Decodificador de Filas - Sigue seleccionando la fila a través de la entrada de selección

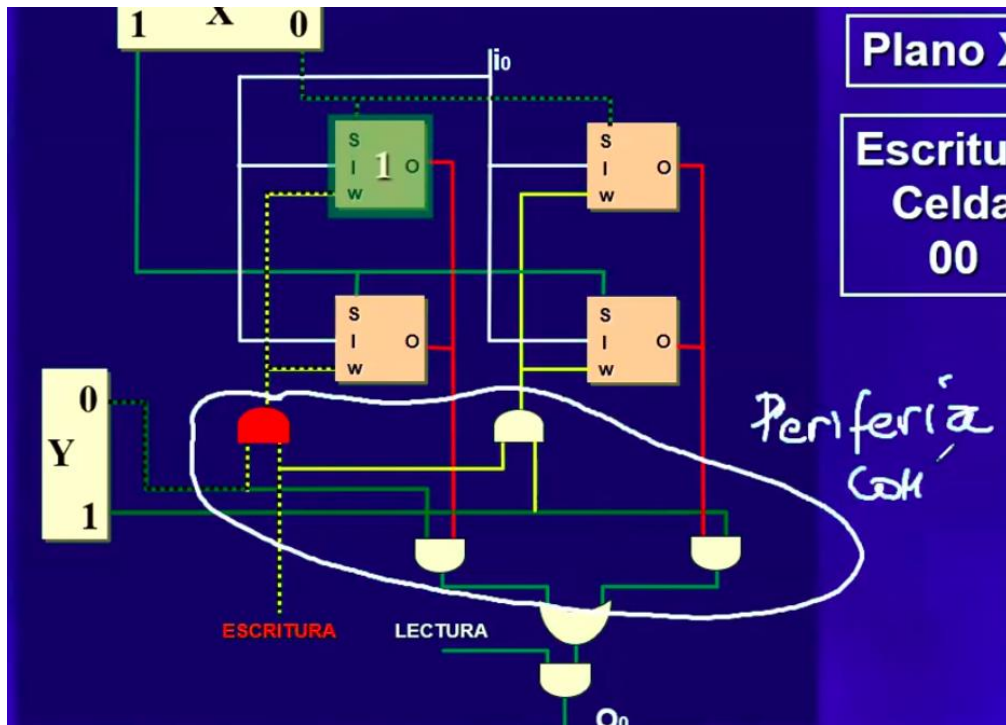
Decodificador de columnas - Al NO existir una entrada S2, la columna se la construye de manera combinacional, con una periferia combinacional.

Agrega **2 compuertas AND** por cada **columna** del PLANO , 1 para LECTURA y 1 para ESCRITURA.

Cuando está en modo LECTURA, selecciona la columna que va a LEER. Cuando está en modo ESCRITURA selecciona la columna que se va a ESCRIBIR.

ESCRITURA - Las AND de 1 solo plano, son para todos los planos, lo único que se hace es desviar el cableado.

LECTURA - Las AND se necesitan 1 por cada columna



Tener en cuenta para la #cantidad de compuertas.

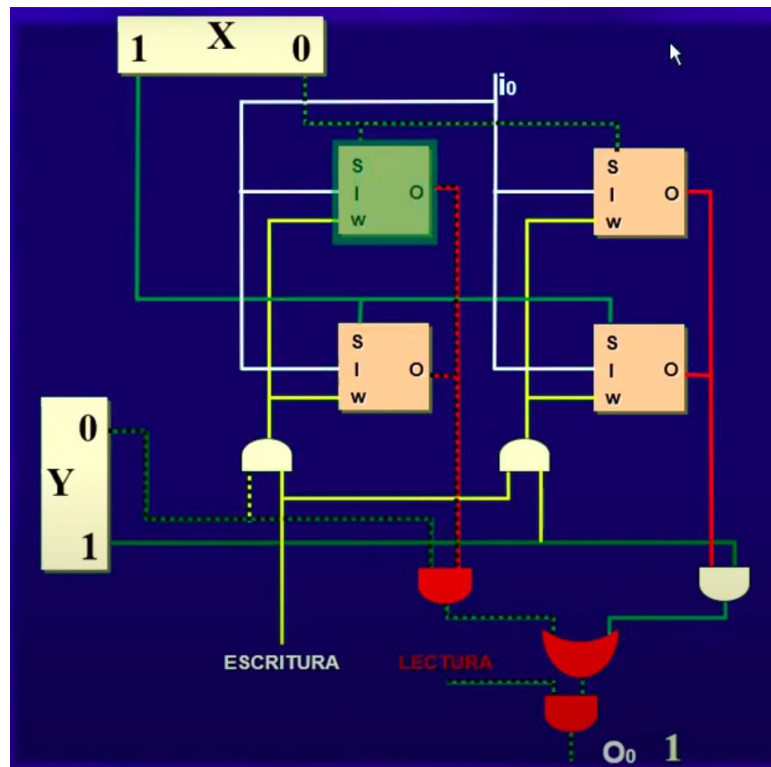


Entonces la Dir 0 en Columna, lo que va a tener es **1 AND** conectada al hilo de **ESCRITURA** conectada a todos los W de los diferentes puntos de memoria.

También continúa y se **conecta a 1 AND** con el hilo de **LECTURA** conectado a todos los O de los diferentes puntos de memoria.

LECTURA

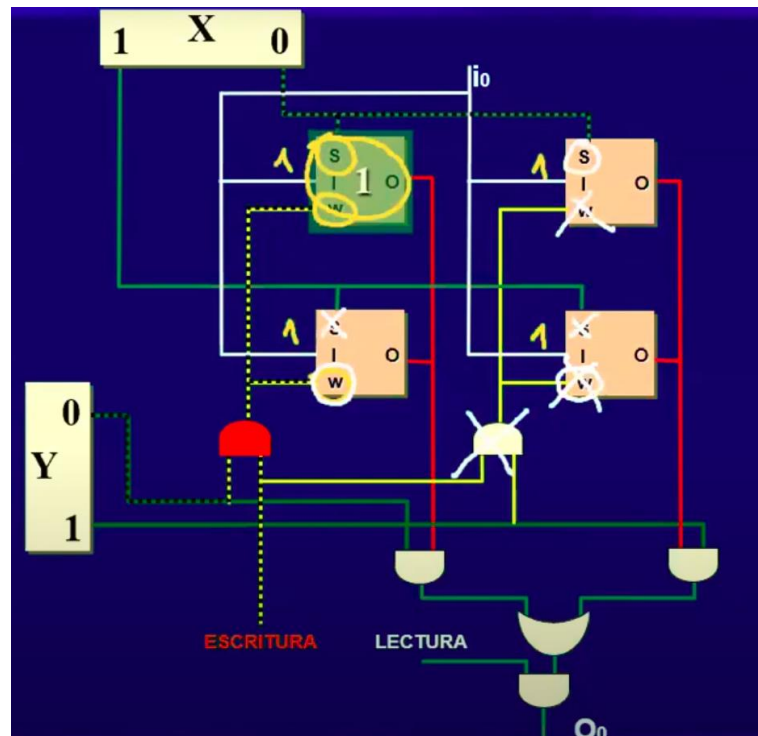
Al hacer el $S(X)$ es decir de la Fila, ejemplo **Lectura Celda 00** ya hacemos la lectura instantánea, pero también se realiza la lectura en la Fila 0, Columna 1, pero no está activada la lectura para esa columna, por lo que el resultado no continúa hacia afuera.



ESCRITURA

Queremos escribir en **Celda 00**, este i_0 está presente en todos los i de las puntos de memoria, pero solamente se cumple que:

- S está habilitada para $S(X) = 0$
- W está habilitada para $S(Y) = 0$



VENTAJAS

- Economía de fabricar los puntos de memoria
- Simplicidad de los puntos de memoria de la 2D
- Arquitectura de distintos planos de la 3D

DESVENTAJAS

- La periferia combinacional implica una complejidad distinta a las anteriores
- esta periferia es cara

https://www.youtube.com/watch?v=Xg8W9cMag_U&list=PLgTKn0MhANLTPffShcV9I2INbaxIPZvLI&index=15

MÉTODOS DE DIRECCIONAMIENTO

El direccionamiento de memoria es una técnica que permite que la arquitectura pueda acceder a una palabra de memoria central a través de una estrategia que permita de forma específica acceder a esa palabra de memoria en la lectura.

Las diferentes técnicas de direccionamiento equivalen generalmente **a una transformación de la parte de dirección de la instrucción**; en la dirección que se transferirá finalmente al registro de selección en memoria para obtener la información deseada. Llamaremos a esta última dirección efectiva.

La máquina necesita tener cableadas, estos métodos de direccionamiento a nivel de hardware.

Desde los años 90 se necesitaban tener circuitos que permitan disponer de diferentes métodos de direccionamientos.

Acá estamos hablando a nivel de **microprogramación y lógica-digital**.

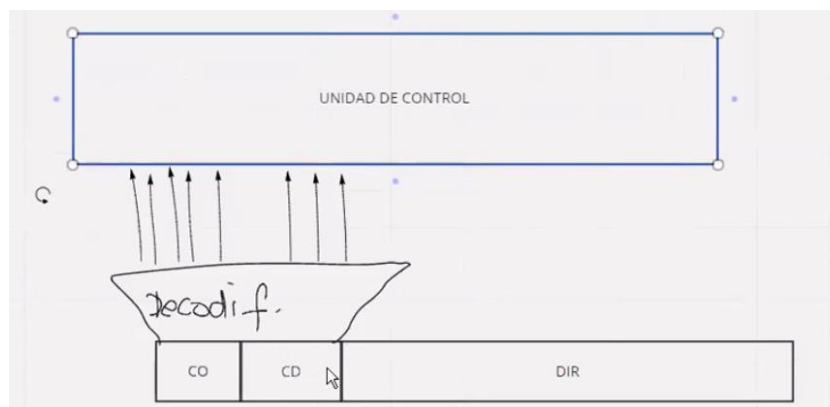
Es decir, los direccionamientos, son estrategias. Tienen que estar disponibles y cableadas para que las capas superiores (ejemplo la de aplicación) puedan aprovechar dichas prestaciones.

Los métodos de direccionamiento se refieren sobre la **SELECCIÓN** de la palabra.

Después depende de CO, de la operación, que puede ser **LECTURA** o **ESCRITURA**.

El que decide qué estrategia es el **COMPILADOR**.

La instrucción pasa previamente por un Decodificador



Hay que codificar todos las técnicas de direccionamiento dentro de la **UNIDAD DE CONTROL**.

El tipo de procesamiento que debe sufrir el contenido de la zona de dirección viene especificado (DIR), o por el código de operación (CO) cuando éste impone un tipo determinado, o por la configuración binaria de una parte de la instrucción, que contiene lo que convendremos en llamar las condiciones de direccionamiento (CD)

La instrucción en un calculador de una dirección consta de tres partes:

CO	CD	D
Código de Operación	Condiciones de Direccionamiento	Zona de Dirección de Memoria

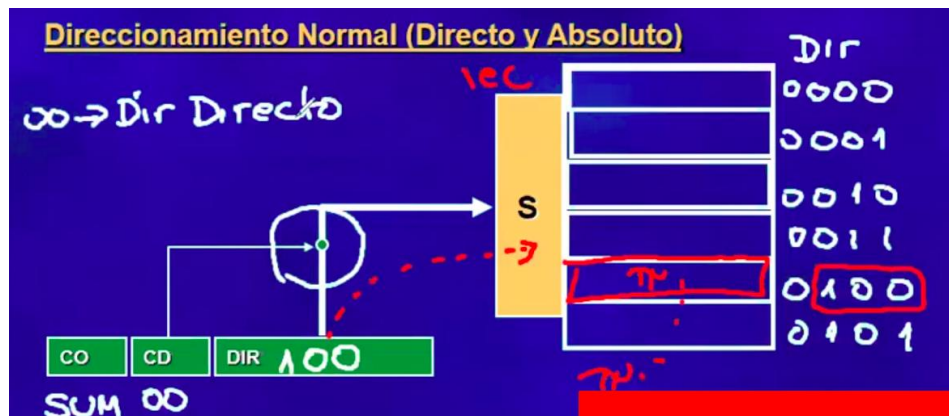
1. Direccionamiento Normal (Directo y Absoluto)

Supongamos que en **CD** está el **00** que significa = **Directo y absoluto**.

Consiste en pasarla de directa y absoluta al "Registro Selección de Memoria" sin ningún tipo de procesamiento adicional.



La posición de la palabra es coincidente al valor en DIR de la instrucción en curso. La zona de dirección de la instrucción contiene la dirección efectiva del operando. El direccionamiento normal exige un ciclo de memoria para obtener el operando.



Da soporte a técnicas más elaboradas que están en capas subyacentes (por ej. lenguaje de alto nivel).

2. Direccionamiento Inmediato

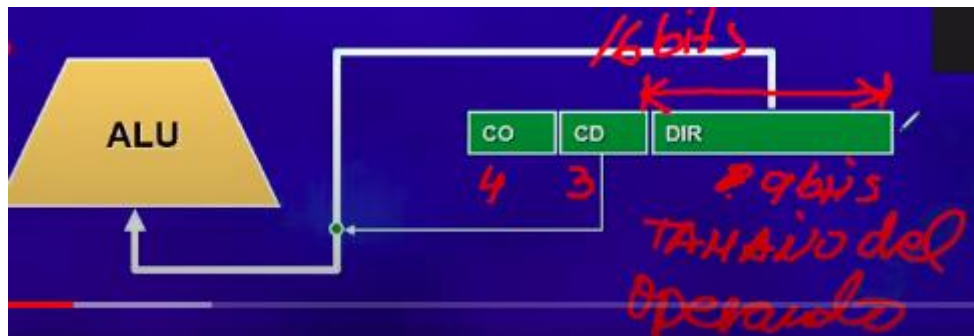


Suponemos que **01 en CD significa Direccionamiento Inmediato**.

Es inmediato cuando obliga al contenido del campo DIR a no ir a S, sino ir a la ALU a operar ese contenido como si fuese un operando. El campo DIR podrá contener el Operando mismo, que será calculado de manera directa contra la ALU.

NO va hacia el Registro de Selección de Memoria (S) sino que va directo por el Bus M al registro M y de ahí a la ALU.

No es propiamente un direccionamiento, puesto que la parte de dirección de la instrucción contiene ya el **operando** que se quiere procesar. Este tipo de direccionamiento no consume ciclo de memoria.



Ventaja

- Velocidad y performance. La memoria central atiende a procesos concurrentes de distintos tipos, libera bastante a la memoria central, porque no hay que buscar el operando en Memoria porque en DIR ya está el Operando. Evitamos ciclos de memoria.

Desventaja

- No se aprovecha el tamaño completo de la Palabra de memoria. Dependemos solo del tamaño de DIR. Entonces del ejemplo de arriba, si toda la palabra de 16 bit, CO y CD utilizan bits, y queda solo el tamaño utilizable de 9 bits. El tamaño del número está restringido a la longitud del campo de direcciones que, en la mayoría de los repertorios de instrucciones, es pequeño comparado con la longitud de la palabra

3. Direcccionamiento Indirecto

https://youtu.be/Xg8W9cMag_U?t=2025

Consiste en una técnica que permite acceder a una palabra de memoria de manera indirecta, es decir la **dirección del operando no es la dirección efectiva**, si no que la esta hace referencia a una palabra de memoria que contenga la dirección del operando; 1ro busca la dirección, 2da encuentra el operando (tiene dos ciclos)

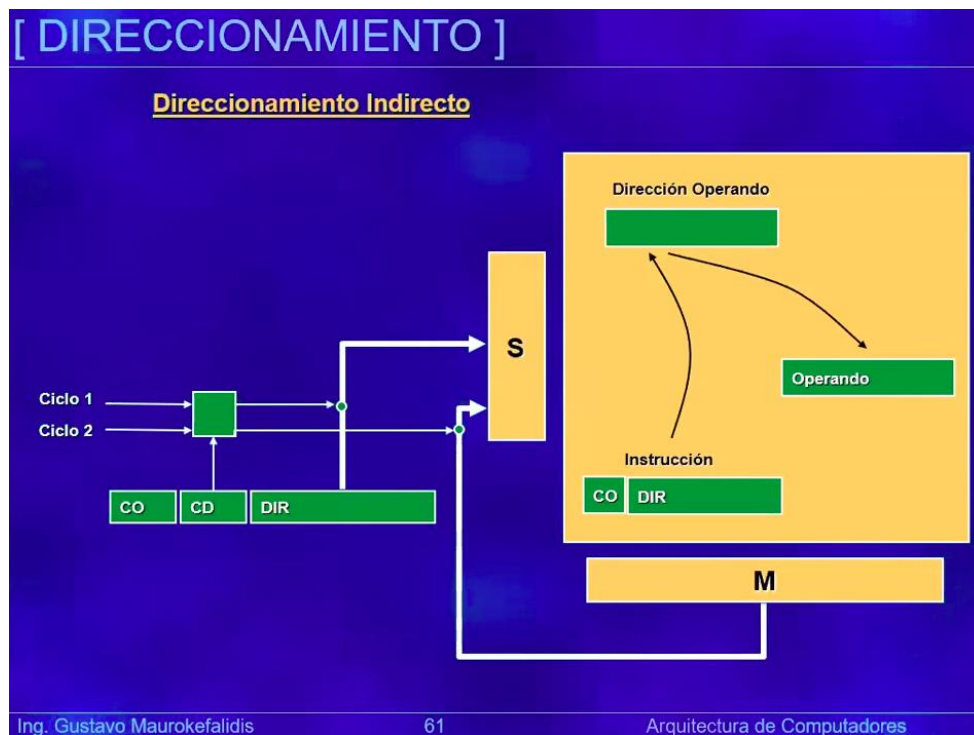
Direccionamiento INDIRECTO

La zona de Dirección del registro de Instrucción **NO** contiene la dirección efectiva del operando, sino la dirección de la palabra de memoria que contiene la dirección efectiva que se quiere procesar. Por tal motivo la búsqueda y selección de un operando en este modo, requiere de dos ciclos de memoria:

- **Ciclo 1:** Búsqueda y lectura de la palabra de memoria con la dirección del operando (indirección)
- **Ciclo 2:** Búsqueda y lectura del operando basada en la dirección leída en el ciclo 1

De esta manera es posible **organizar la memoria como una estructura de datos** gobernada por las palabras que integran los accesos al primer ciclo, y que dirigen la forma y el orden de acceso al banco de operandos del segundo ciclo.

Uno de los usos prominentes de este método es la construcción de **tablas de salto**. Una tabla de salto es un arreglo cuyos elementos son direcciones. Suponga que una de n secuencias de acciones se debe realizar dependiendo del valor de una variable entera j (sentencia case en algunos lenguajes de programación de alto nivel). Luego, al colocar las direcciones de inicio de n secuencias de instrucción en un arreglo de longitud n y ejecutar un salto indirecto al j -ésimo elemento del arreglo, logra la tarea deseada.



Consume 2 ciclos de memoria

- 1er ciclo

Buscar la dirección de la palabra entera que es la "dirección del operando".
Estoy accediendo al puntero.

- 2do ciclo

Buscar con la "dirección del operando" obtenida anteriormente, el operando en si. Estoy accediendo al dato del puntero.

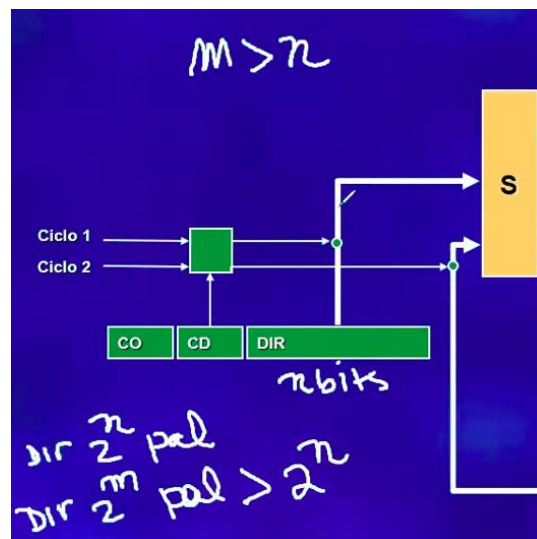
Para lo cual, vemos que el Registro M **está conectado a través de un Bus** al Registro S.
Utilizado en el ciclo 2.

Con esta técnica ya estamos haciendo no un recorrido físico, sino un recorrido virtual (porque **estamos utilizando punteros**, y no dependemos pura y exclusivamente del tamaño de DIR).

DATO

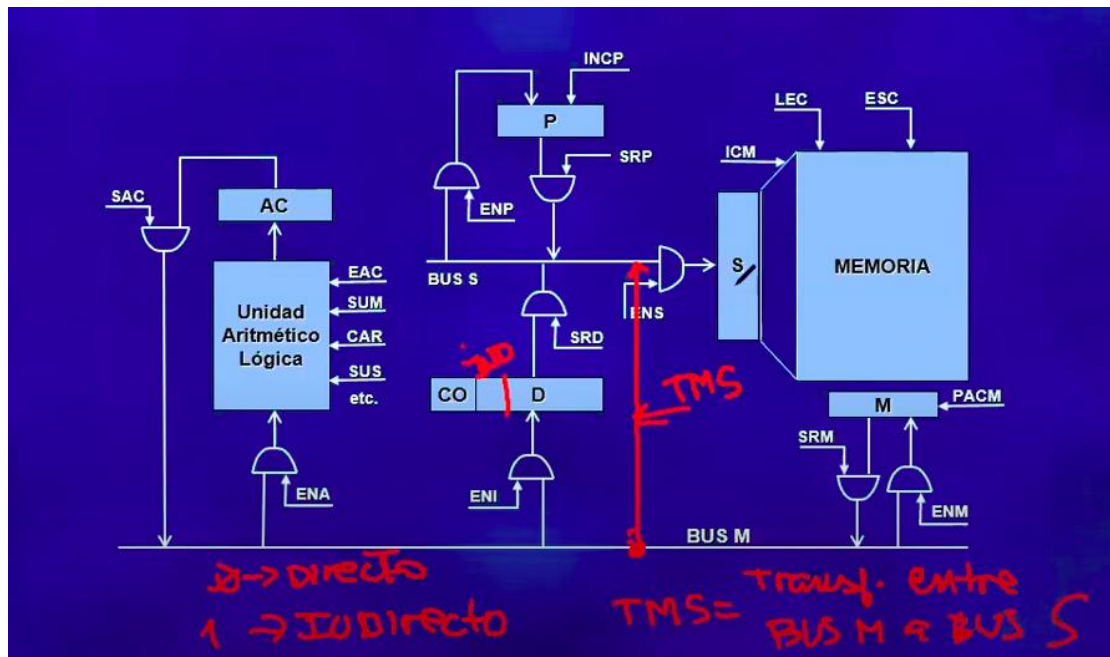
ABACUS ocupa el direccionamiento:

- Directo
- Indirecto



DIR tiene una capacidad de **n bits**, y se podría direccionar hasta 2^n bits si SOLO se utilizara DIR. PERO, como ahora en el 2do ciclo, la PALABRA de tamaño **m bits** es utilizada como una DIR. Entonces permite direccionar **2^m bits** como "DIR" palabras mucho más distantes.

Bus que conecta Bus M con Bus S para permitir el Direccionamiento Indirecto.



TMS → de nivel, transferencia de nivel

TMS, el puente se activa solo para el 2do Ciclo, es decir para la transferencia entre Bus M a Bus S.

VENTAJAS

- Entonces permite direccionar 2^m bits como "DIR" palabras mucho más distantes.
- Brinda a lenguajes de altos nivel, la capacidad de manejar estructuras de datos (punteros, son los que se utilizan en el 1er Ciclo)

Desventaja

- Utiliza 2 ciclos de memoria para buscar un Operando.

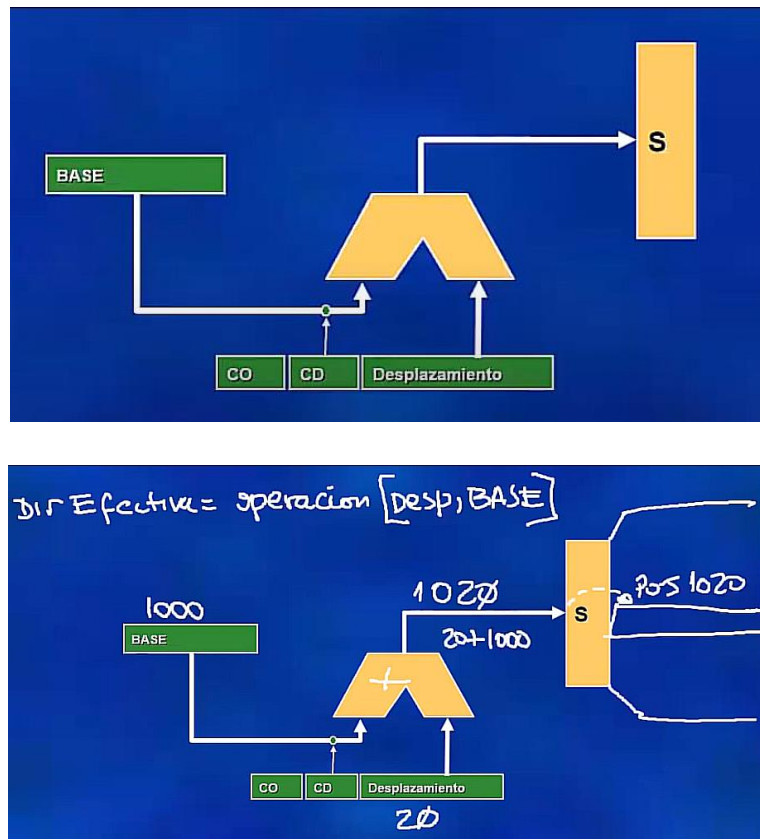
4. Direccionamiento por Base y Desplazamiento

Plantea que la dirección efectiva surge de la operación de registro base y del contenido de DIR (entre desplazamiento y base).

Un registro de la máquina, llamado **registro de base**, contiene la dirección de referencia (primera dirección de un programa o de una zona de datos, por ejemplo).

A la información que alberga la parte de dirección se le llama desplazamiento. La dirección es la suma de la base y del desplazamiento.

Algunos calculadores admiten varios registros de base. La **instrucción debe especificar** entonces la dirección de registro de base escogido. "La dirección efectiva de la palabra surge como la suma o resta de un valor del registro base y el campo D." Permite acceder a campos inalcanzables por el campo DIR



Ventajas

- Establecido un valor base, lo único que hacemos es cargar el desplazamiento en el campo DIR

Desventajas

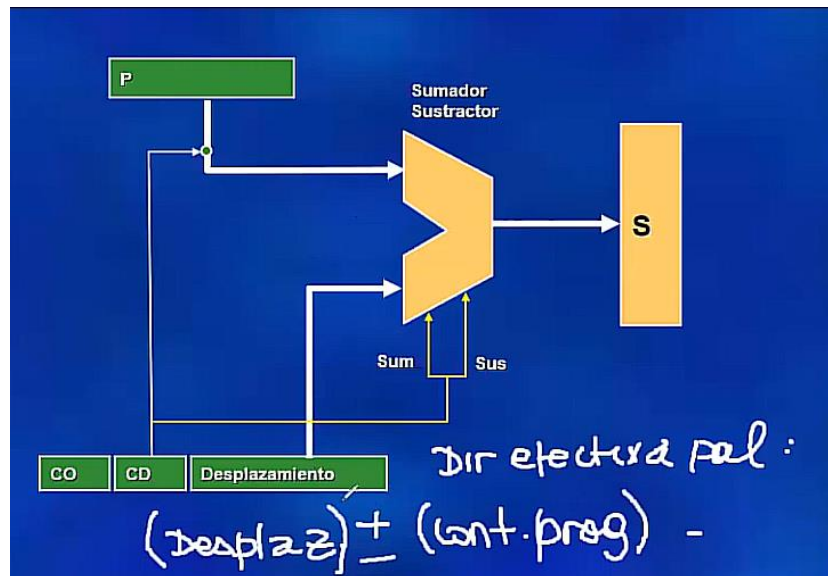
- Consume tiempo hacer el cálculo de adición/sustracción de DIR con el registro BASE

5. Direccionamiento por Referencia al Programa

Link del video: https://youtu.be/Xg8W9cMag_U?t=3133

Plantea que la dirección efectiva de la palabra de memoria surge de sumar o restar el campo DIR (desplazamiento) con el contador de programa. Esta técnica permite, basándose en el valor del cont de programa, tener un valor fijo de desplazamiento, entonces es muy utilizada para hacer barridos de palabras consecutivas.

$(Despl) \pm (\text{cant de prop}) = \text{Dirección efectiva de la palabra.}$



El contenido del contador de programa sirve de dirección de referencia. Con este sistema es posible generalmente direccionar dos zonas de memoria a un lado y a otro de la instrucción en curso (adiciona o resta de la dirección) con el contenido del contador.

Sirve para hacer barridos de múltiples palabras, por ejemplo, desde una posición inicial, el contador puede ir incrementando hasta 20 y va recorriendo todas esas palabras.

El Bus S, claramente es mayor que el tamaño del Registro DIR solo. Ya que es Tamaño de S = Tamaño de P + Tamaño de DIR

6.Direccinamiento por Página (o por Yuxtaposición)

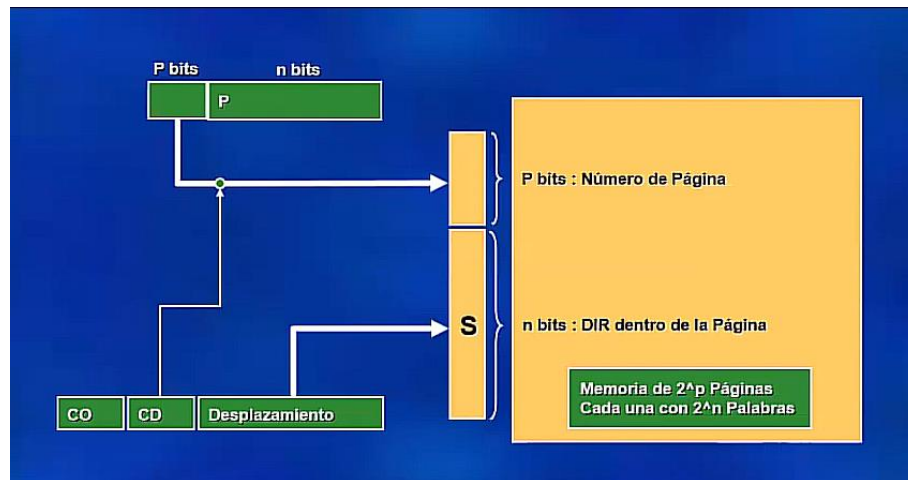
Concatena el contenido de los bits de mayor peso del contador de programa con el valor DIR.

El contador de programa mantiene fijos los de mayor peso y acelera los de menor $\Rightarrow$ la memoria se organiza en 2^p , tal que p es la cantidad de bits de mayor peso del contador de programa.

2^p = cantidad de páginas (manera virtual de ver agrupamiento de palabras en una unidad de información).

Desplazamiento indica posición de la palabra dentro de una página.

Es fundamental porque permite la multiprogramación, el procesador puede hacer procesos concurrentes, hay distintos procesos que corren en distintas páginas.



Se considera a la memoria dividida **en zonas de 2 a la n palabras, llamadas páginas**. En general la parte de dirección de la instrucción contiene n bits, por lo que no capacita a la máquina para direccionar más palabras que las que tiene una página. La dirección efectiva se obtendrá por yuxtaposición del número de página (o dirección de página), supuesto conocido y de la parte de dirección de la instrucción, que suministra la dirección dentro de la página.

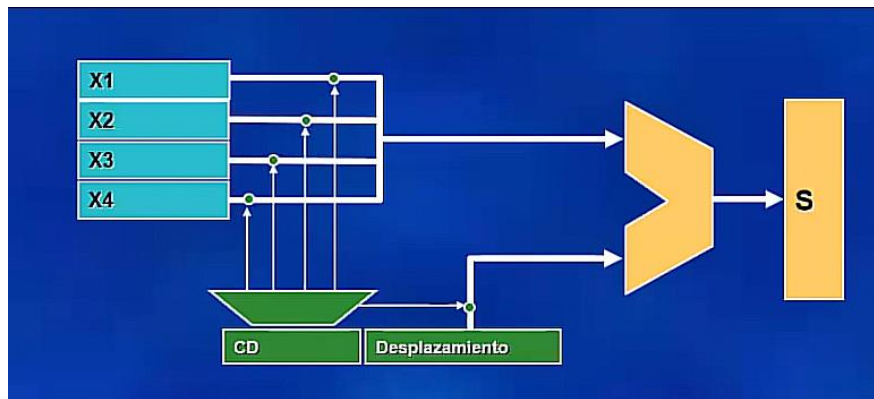
7. Direccionamiento Indexado

Es uno de los más usados. Consiste en muchos registros bases modificables por el programa. La dirección efectiva surge de **operar del contenido del índice con el desplazamiento**. En un programa se utilizan algoritmos que involucran índices, por ejemplo, para sumar datos en posiciones contiguas de memoria.

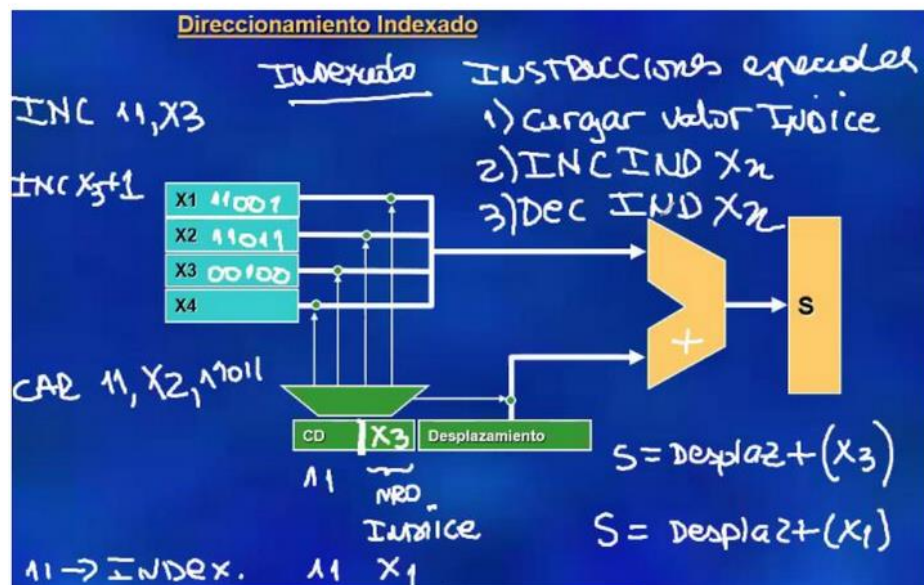
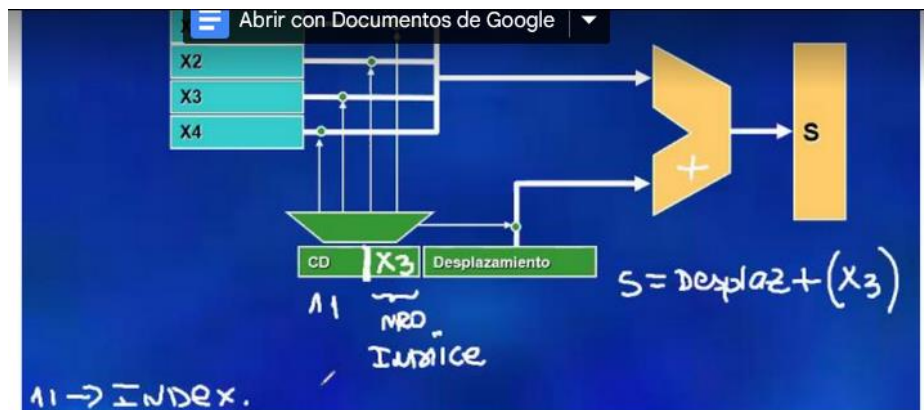
Hay instrucciones especiales:

- **cargar** el valor del índice
- **incrementar** el índice
- **decrementar** el índice
- **comparar**

El set de instrucciones debe proveer una instrucción de carga del registro índice que actúa como puntero. Es común que una CPU contenga más de un registro índice, por lo que esto se debe reflejar en el código de la instrucción.



Se obtiene la dirección efectiva sumando a la parte de dirección de la instrucción el contenido de un registro de la unidad central llamado registro índice; a menudo se llama índice al contenido. El registro de índice admite un cierto número de operaciones, como carga, lectura, incremento o decremento en 1, comparación.



APLICACIÓN

Es el apropiado para acceder a arreglos y tablas.

Este modo de direccionamiento es **especialmente útil para el direccionamiento de vectores y matrices en bucles** ya que, si se quieren direccionar elementos consecutivos

del vector o matriz, basta mantener en el desplazamiento la dirección del primer elemento e ir incrementando el registro índice. También sirve para acceder de forma relativa a elementos de vectores cercanos a uno dado, para ello, se carga la dirección del elemento de referencia en el registro índice y después se accede mediante direccionamiento indexado, con el desplazamiento adecuado, al anterior, al siguiente, etc.

UNIDAD DE CONTROL

Concepto

Esta unidad se encarga de extraer y analizar las instrucciones de la memoria central, luego establece las conexiones eléctricas correspondientes dentro de la ALU y extrae los datos de la MC implicados por la instrucción. Por último, ordena a la ALU el tratamiento de los datos extraídos. Eventualmente, suele almacenar el resultado en la memoria central.

Otra definición

Es el órgano encargado de gobernar la ruta de datos como respuesta a la ejecución de una instrucción. El gobierno de la ruta de datos se logra manejando las compuertas de gobierno a través de señales impulsionales o de nivel según corresponda.

La UC integra tres componentes lógicos:

- a) **Registro de Instrucción:** es quien indica al secuenciador (a través de un proceso de decodificación) cuál de los cronogramas deberá ejecutar en coincidencia con la ejecución en curso. La condición de direccionamiento también está vinculada al secuenciador.
- b) **Secuenciador:** órgano encargado de generar señales de gobierno que permitan gobernar la ruta de datos en coincidencia a la instrucción en curso.
- c) **Estado de máquina:** son un conjunto de biestables y registros conectados al secuenciador con el objeto de informar estados de máquina relevantes a la generación de los cronogramas.

La UC necesita dos registros para operar:

1. **Contador de programa:** contiene la dirección de la próxima instrucción por ejecutar. Su nombre se debe a que, a excepción de las rupturas de secuencia, este registro va aumentando su contenido en una unidad para pasar a la siguiente.
2. **Registro de instrucción:** contiene la instrucción extraída de la memoria. El registro de instrucción de abacus tiene 2 partes:
 - a. Una para el código de operación (**CO**), que define el tipo de instrucción a ejecutar (suma, multiplicación, salto, etc)
 - b. Otra para la dirección del operando (**DIR**)

Secuenciador: encargado de analizar el código de operación de la instrucción y en base a ello distribuir las microórdenes al conjunto de las unidades del ordenador (memoria, alu) para hacer ejecutar las distintas fases de la instrucción. La UC y la ALU forman un todo en la mayoría de los ordenadores denominado CPU

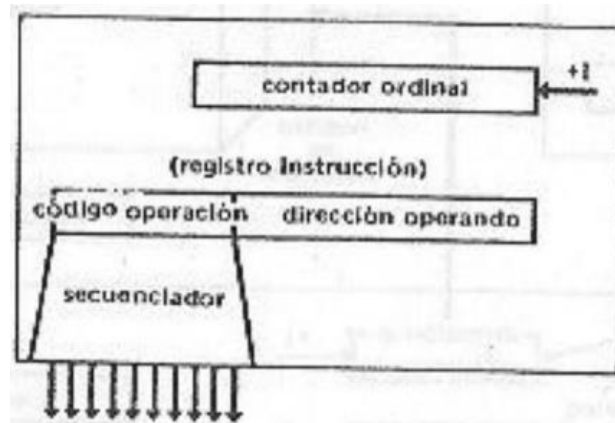


Fig. 5. Unidad de control

INSTRUCCIÓN

Las instrucciones del programa que la máquina deberá ejecutar. La Unidad de Control (Unidad de Instrucciones o Unidad de Gobierno) tratará las instrucciones.

FAMILIA IBO → Instrucciones con Búsqueda de Operando. Por ejemplo la suma, resta, or, and, xor.

Desarrollo de una instrucción de procesamiento

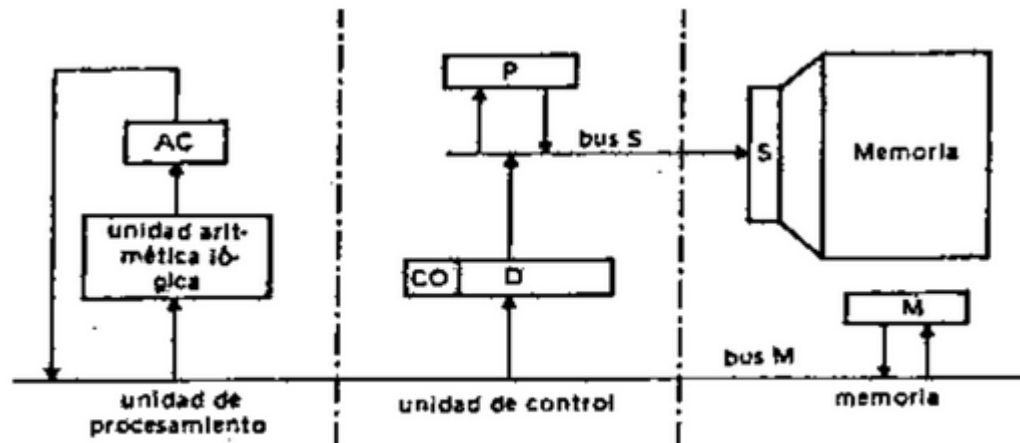
El desarrollo de una instrucción de procesamiento en un computador de una dirección, como abacus, puede descomponerse en tres fases:

1. Búsqueda y análisis de la instrucción
2. Esta fase tiene dos posibilidades y en cada instrucción se selecciona solo una (excluyentes):
 - a. Búsqueda y procesamiento del Operando
 - b. Almacenamiento del resultado
3. Preparación para la próxima instrucción

Descripción de la Unidad Central de Abacus

Características Generales:

- Instrucción de una sola dirección.
- Transferencias en paralelo.
- Máquina Síncrona (lo que implica que todos los elementos de la máquina están sincronizados con un mismo reloj/clock).
- El reloj producirá un batido cada θ segundos.
- El ciclo de base de la memoria será de 2θ .
- La longitud de la palabra será de n bits.
- La longitud del Código de Operación será de p bits.
- La longitud de la palabra es suficiente para que los m bits ($p + n$ bits) de la parte de dirección permitan direccionar toda la memoria.



Esquema simplificado de Abacus

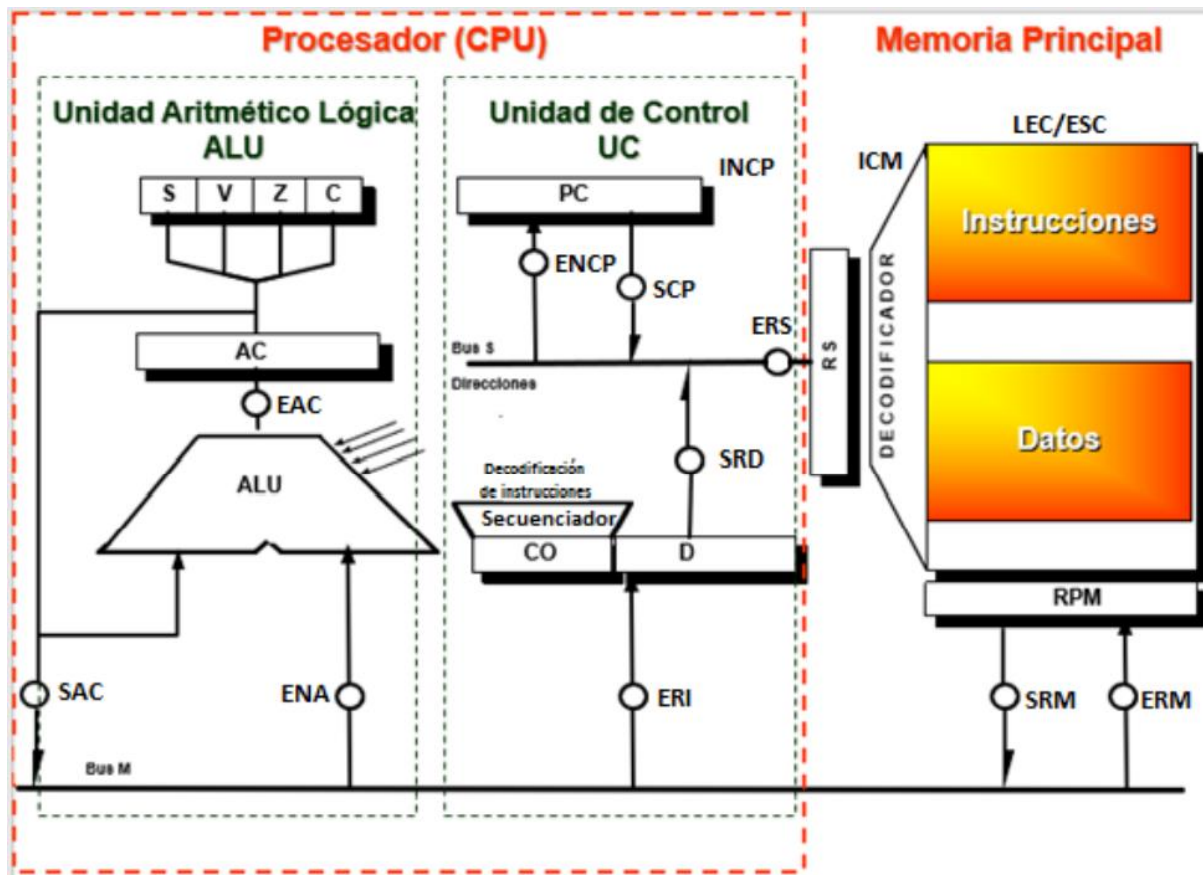
ABACUS

Abacus es una máquina organizada en torno a dos buses, que son el bus de memoria (bus M) que transfiere las palabras leídas de, o escritas en memoria y el bus de selección (bus S) que transfiere las direcciones.

S	Registro de selección de memoria
P	Contador ordinal de programa
M	Registro de intercambio o de palabra
I	Registro de instrucción con dos partes: CO y D
CO	Código de operación
D	Dirección
AC	Acumulador

RUTA DE DATOS DE ABACUS

ESQUEMA DE VON NEUMANN GENERAL



CONCEPTO DE RUTA DE DATOS

RUTA DE DATOS

Conjunto de Órganos para Memorizar, Transferir, o Procesar información proveniente de la memoria central en forma de instrucciones, direcciones, y operandos.

Cada Órgano cuenta con Registros de transferencia que le permiten comunicarse con el resto, a través de los Buses definidos en la Arquitectura.

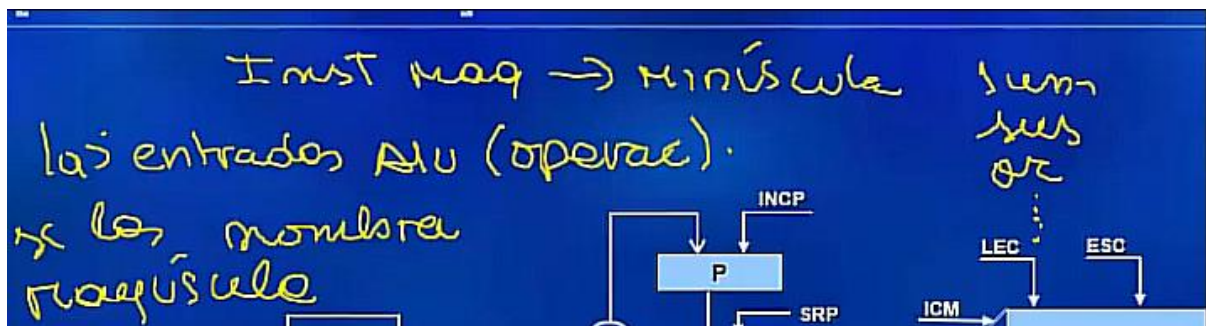
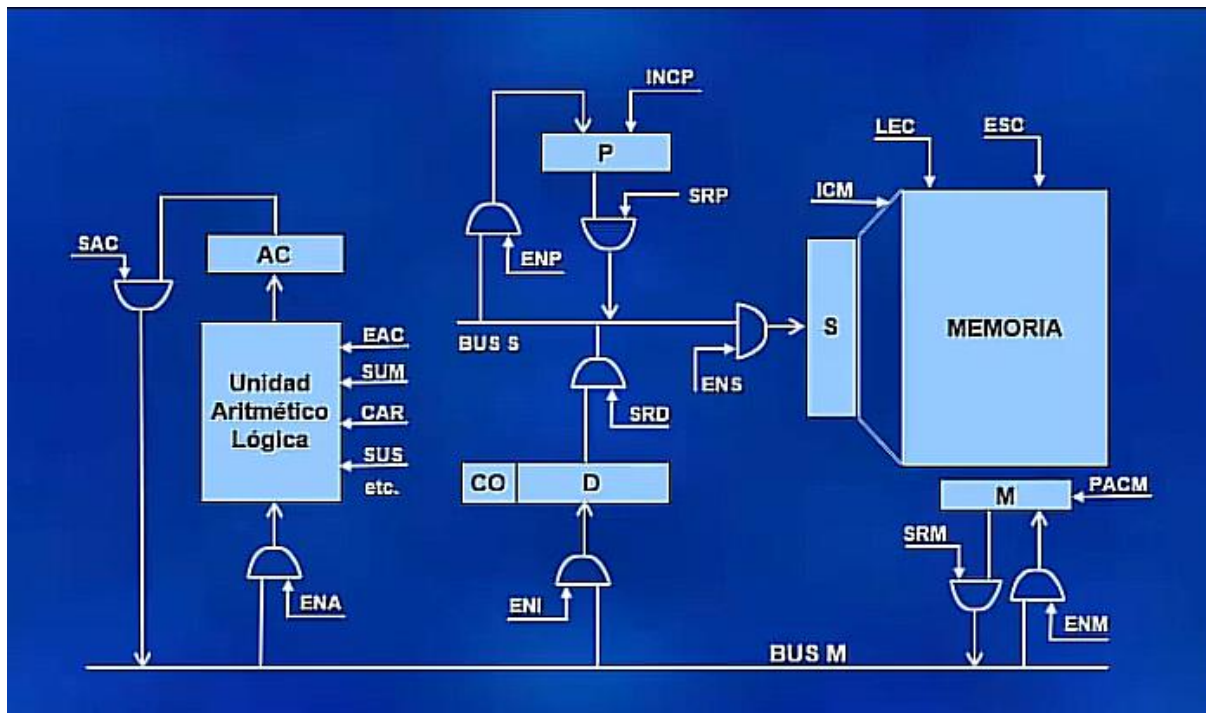
La generación de las señales de gobierno que coordinen el movimiento de instrucciones, direcciones, y operandos, esta a cargo del un Organo Central conocido como Unidad de Control.

DIMENSIÓN DE LA RUTA DE DATOS

Es el número de dígitos transmitidos en PARALELO por la ruta de datos. Esta puede variar a lo largo de una misma ruta. Puede ser fija en lo que refiere al **tamaño del registro de Instrucción y de Operando** y un número más restringido para las direcciones de los operandos.

En Arquitecturas más complejas la dimensión puede ser variable, de uno a más octetos para determinados órganos que requieren procesar información aritmética de simple/doble precisión por ejemplo.

Señales de gobierno implicadas para las diferentes operaciones en ABACUS

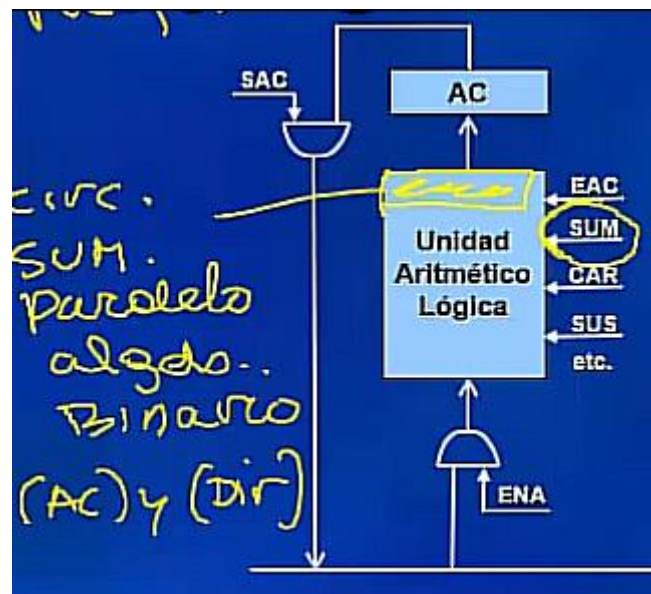
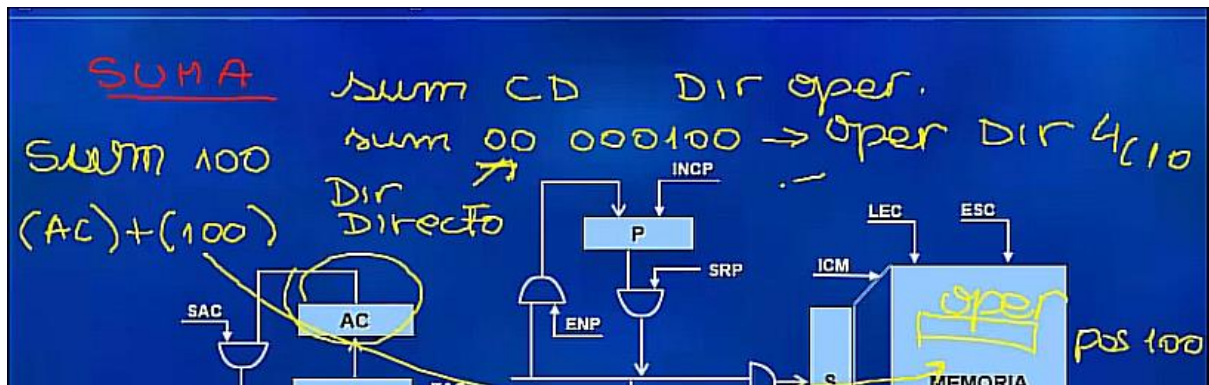


SUMA EN ABACUS COMPLETO

<https://www.youtube.com/watch?v=2kVqrubzng&list=PLgTKn0MhANLTPffShcV9I2INbaxlPZvLI&index=17>

La suma en abacus tiene 4 fases

1. Búsqueda instrucción
2. Búsqueda operando
3. Ejecución de la suma
4. Preparación de la próxima instrucción

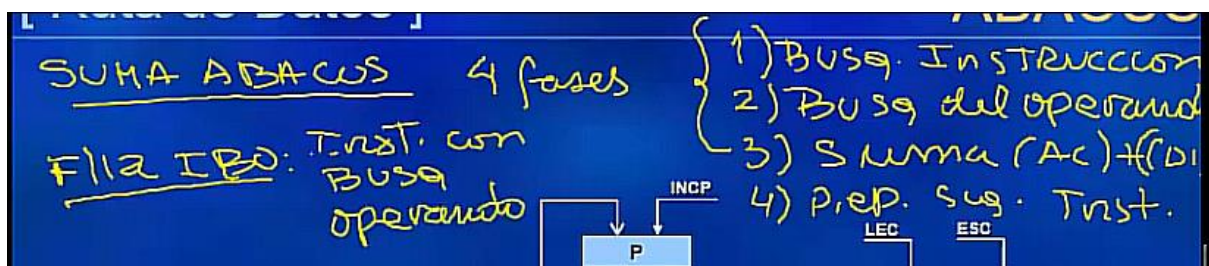


Las operaciones de la ALU van en mayúscula

Sumar el contenido del operando que se encuentra en la dirección de memoria dada por la DO, con el contenido del acumulador.

SUM

DIRECCION



Fase 1

Suponiendo que la dirección de la instrucción por ejecutar se halle en el contador ordinal, es preciso:

1. Transferir el contenido del Contador Ordinal al Registro de Selección de Memoria (RSM): $(P) \rightarrow S$
2. Lanzar un ciclo de lectura (que implica un un ciclo de memoria), de tal modo que sea transferido el contenido de la célula de memoria direccionada por el registro S al registro de palabra. $((S)) \rightarrow M$.

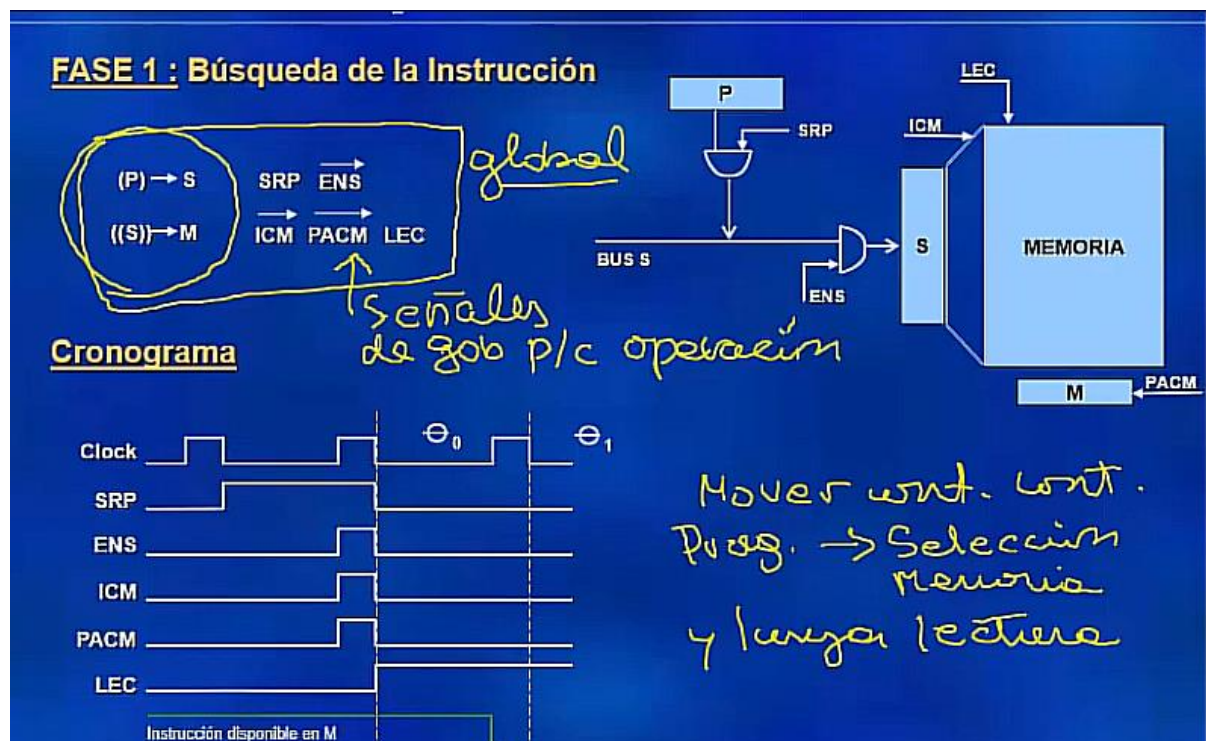
Análisis del Proceso:

$(P) \rightarrow S$.

Durante el último θ de la instrucción precedente, la señal SRP mantiene abierta la puerta de salida del registro P, por lo que la información de P está estabilizada en el bus S bajo la forma de niveles lógicos. Bastará con validar esta información al primer batido de reloj de la instrucción por medio de la señal de muestreo (de impulso) ENS para introducirla en el registro S.

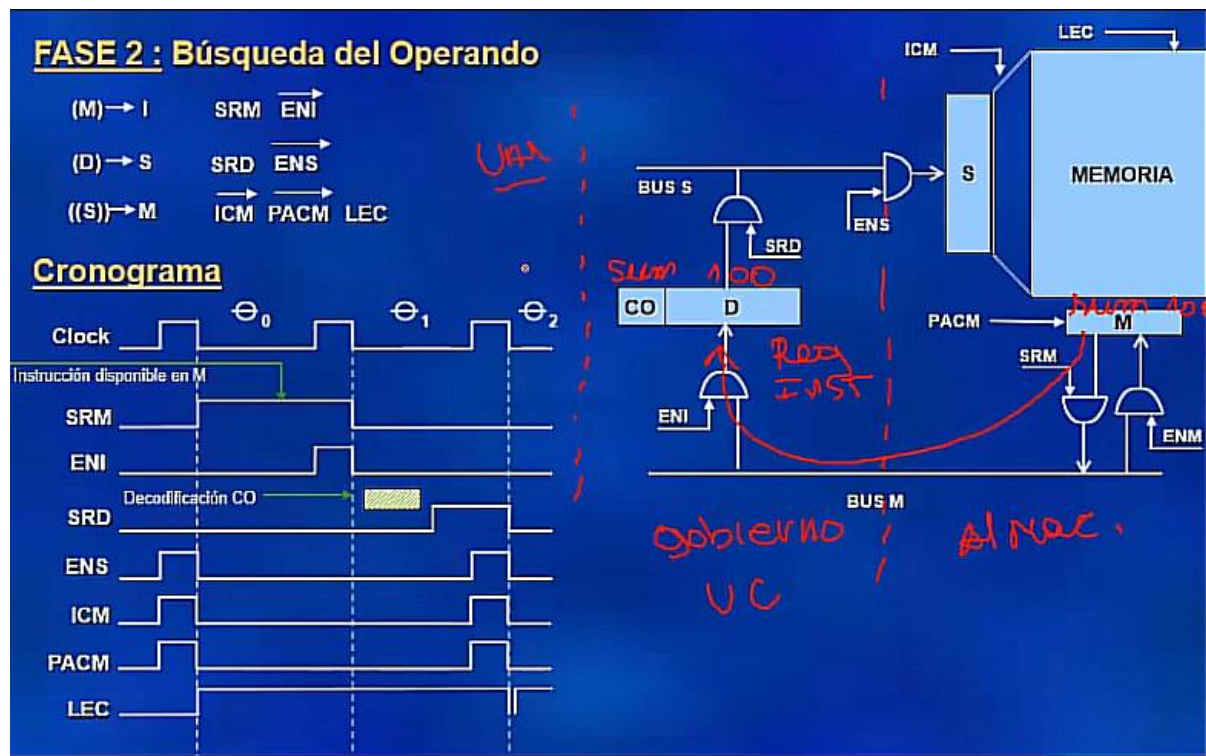
$((S)) \rightarrow M$.

Será necesario lanzar un ciclo de memoria mediante la señal impulsional ICM (inicio de ciclo de memoria), indicar que se trata de una lectura, manteniendo por ejemplo la señal LEC durante todo el intervalo del ciclo (y en cualquier caso hasta el inicio de la regeneración) y, si fuera preciso, poner a cero el registro M antes de la lectura de los núcleos (PACM)

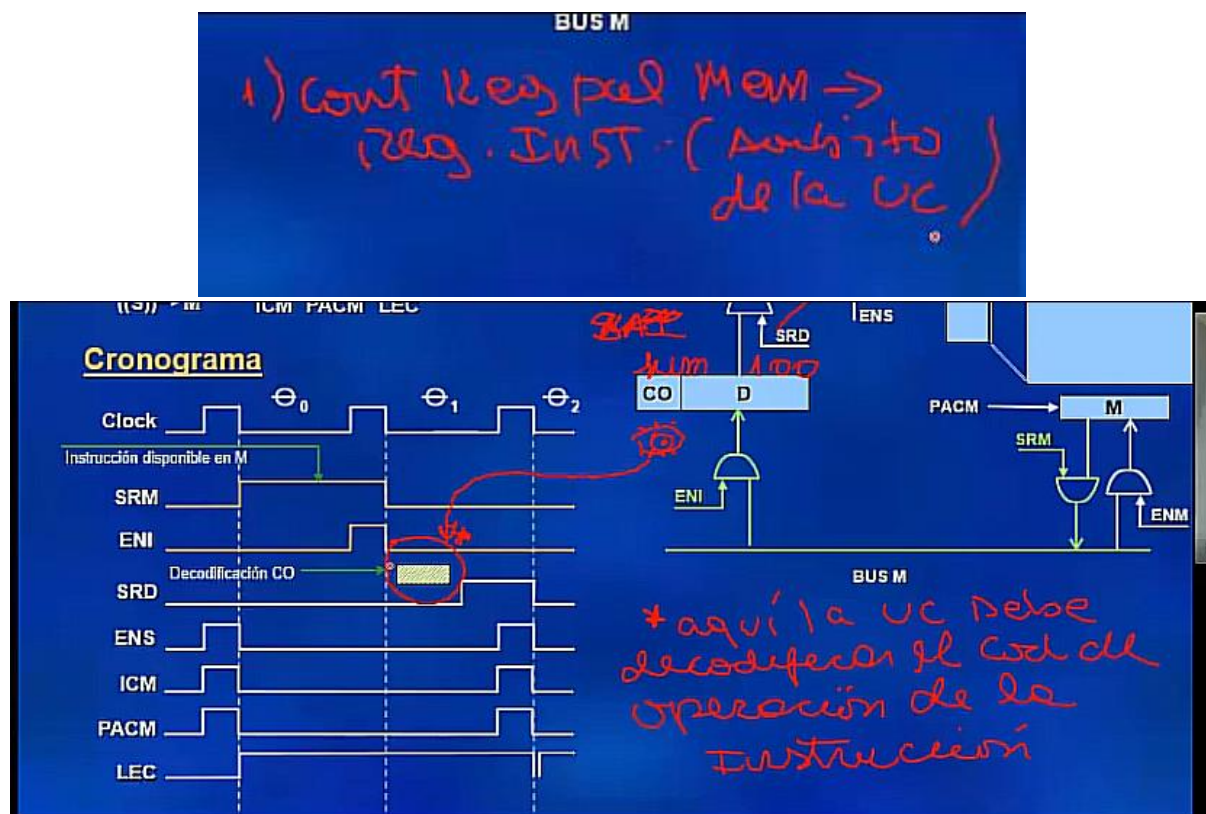


“En la fase 1 logramos que la instrucción ya está en el registro de palabra de memoria.”

Fase 2



Debe enviarse la instrucción contenida en el registro M al registro I para proceder a su decodificación: $(M) \rightarrow I$, la parte de dirección D de la instrucción, que especifica la dirección del operando ha de ser transferida al registro S: $(D) \rightarrow S$, de manera que, tras un semiciclo de lectura, el operando se encuentre en el registro M: $((S)) \rightarrow M$.



Análisis paso a paso del proceso:

(M) → I.

La instrucción se encuentra en el registro M antes del fin del primer θ del ciclo, se abre la puerta de salida del registro M al bus M durante este primer θ , para que, así mantenida la información en el bus M, pueda forzarse su introducción en I gracias a la señal de muestreo ENI, sincronizada con el segundo batido del reloj del ciclo de memoria.

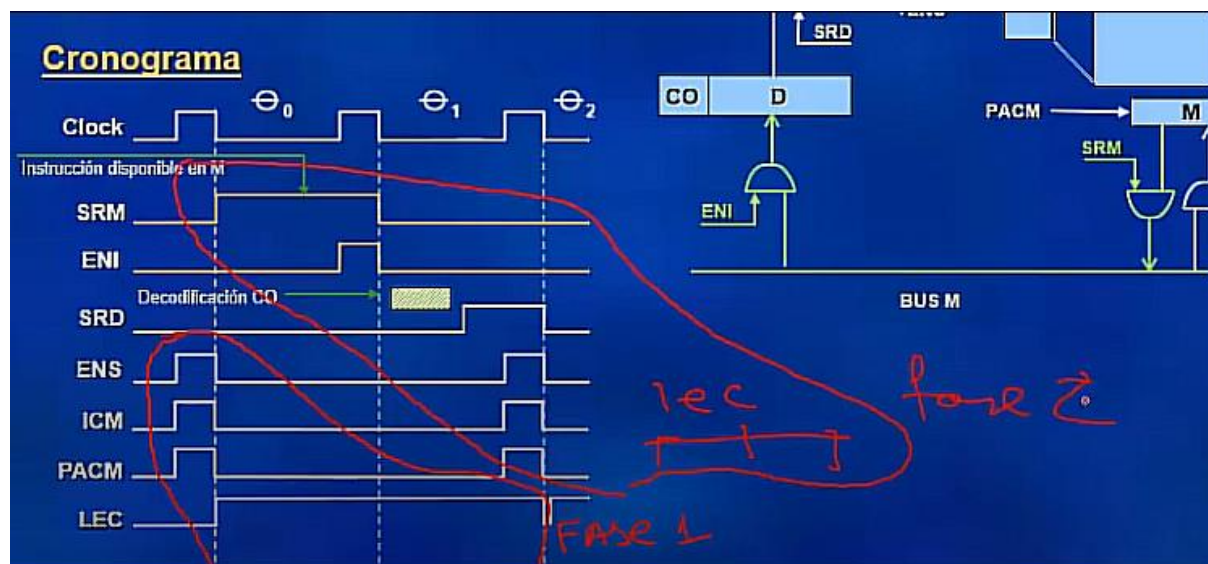
Aclaración: Se pasa el ámbito de la Unidad de Control.

(D) → S.

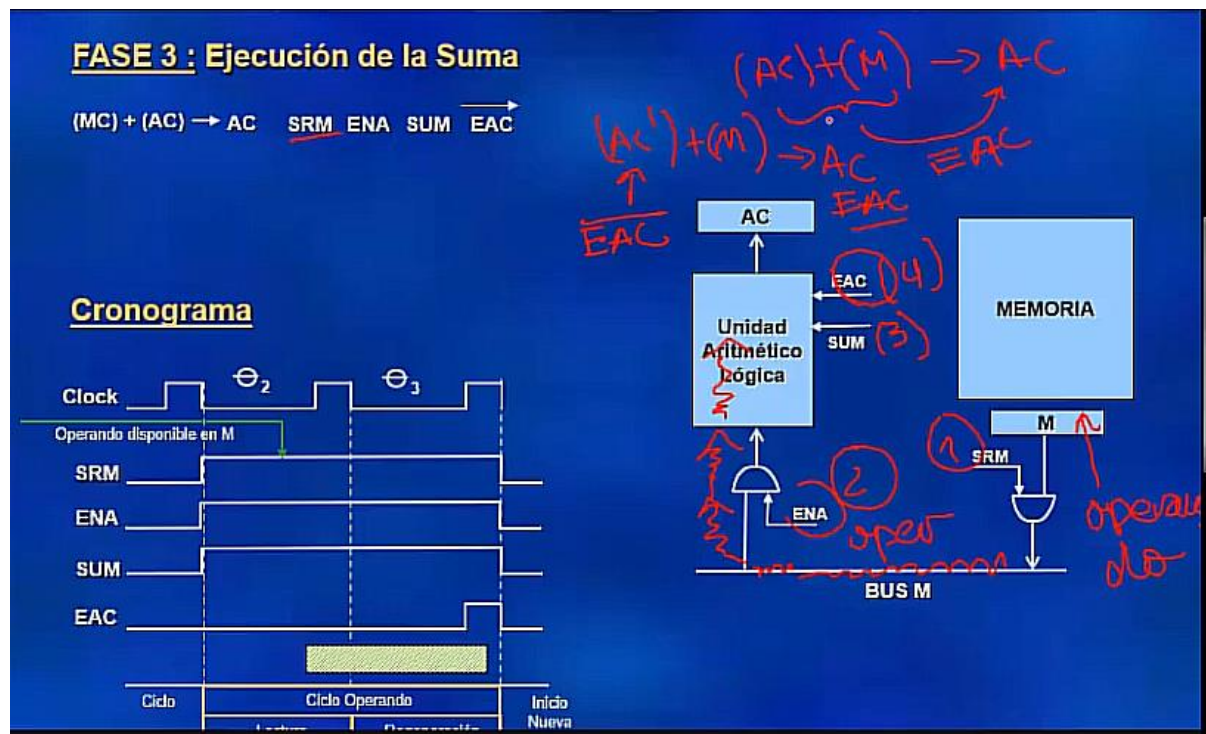
El registro I se divide en dos partes: una parte de control, con el CO e informaciones complementarias, y una parte de dirección. La decodificación de la primera parte indica que esta dirección es la del operando que debe ahora localizarse en la memoria. Por consiguiente, es preciso enviar la parte de dirección del registro I al registro S abriendo la puerta de nivel (señal SRD) para mantener la información en el bus S. Al siguiente batido se abrirá la puerta impulsional de entrada en S (señal ENS).

((S)) → M

Se borrará el registro M (señal PACM) simultáneamente con la introducción en S y se lanzará un ciclo de memoria por medio de un impulso ICM, indicando que se trata de una lectura gracias al mantenimiento durante todo el nuevo ciclo del nivel LEC.

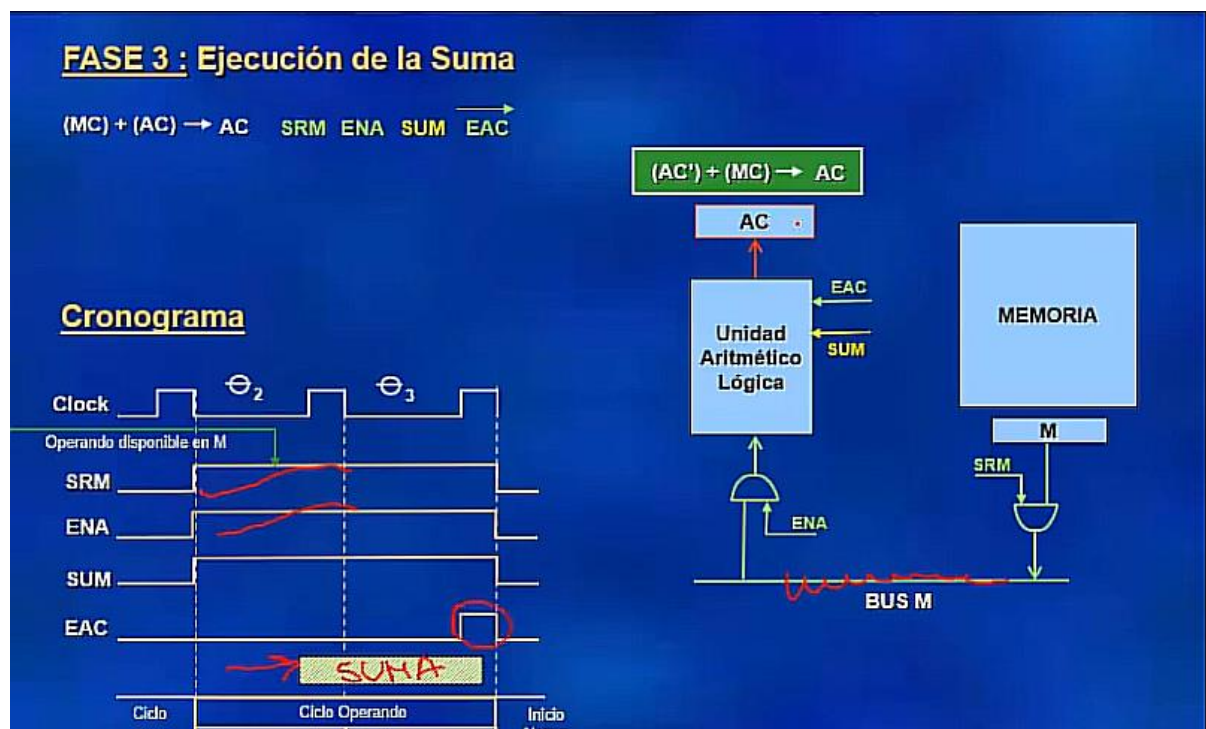


Fase 3



Se trata de sumar el contenido de M con el contenido de AC: $(M) + (AC) \rightarrow AC$.

Los niveles lógicos correspondientes a los registros M y AC durante todo el tiempo de trabajo del sumador, consagrado esencialmente a la propagación del arrastre. Se consigue por las señales SRM, ENA, SUM. Terminada la suma, una señal EAC introduce el resultado en el AC.

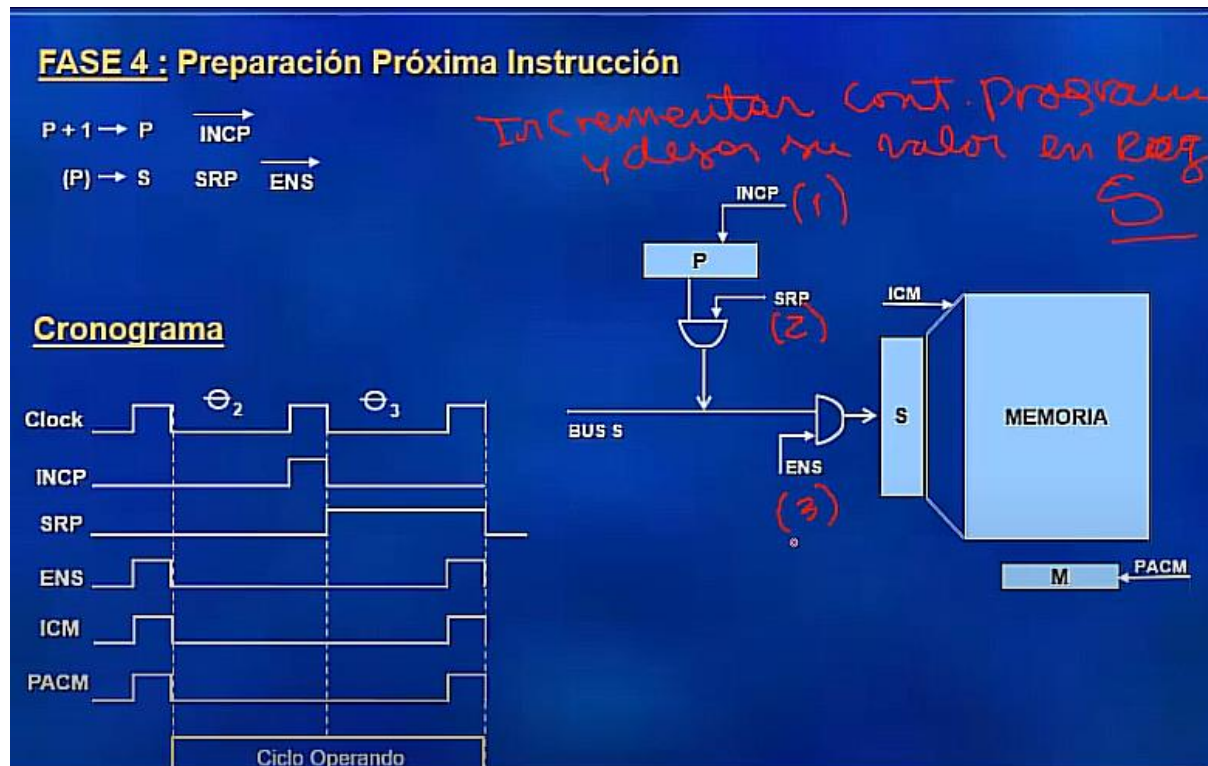


Aclaración:

En θ_2 no podemos suponer que Bus M está lleno con el operando para realizar la operación, por ello es que se consideran 2 batidos.

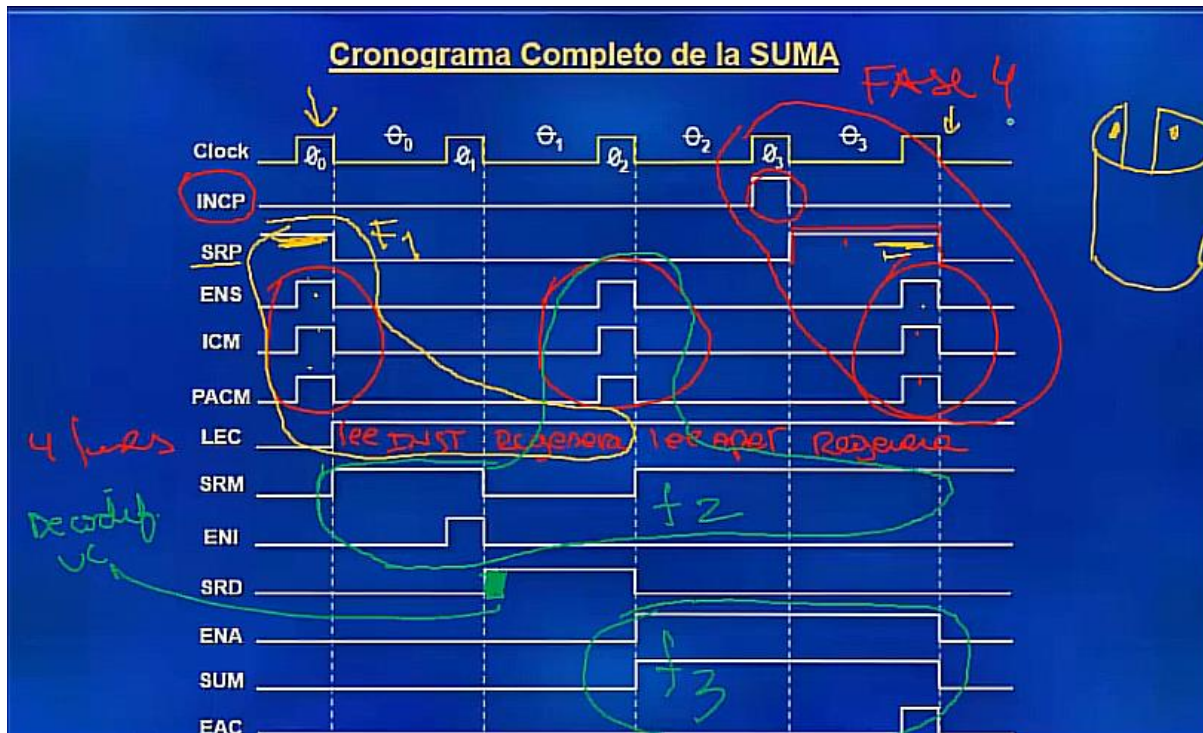
Fase 4

Esta fase se dedica esencialmente a incrementar el contador de programa en 1: $(P) + 1 \rightarrow P$, enviándole un impulso de cuenta INCP para que al final de la instrucción contenga la dirección de la próxima instrucción. Esta última reanuda nuevamente el proceso: $(P) \rightarrow S$, ciclo de búsqueda de la instrucción, etc



Resumen: preparación de la siguiente instrucción, incremento del contador de programa y deja su valor en el registro S.

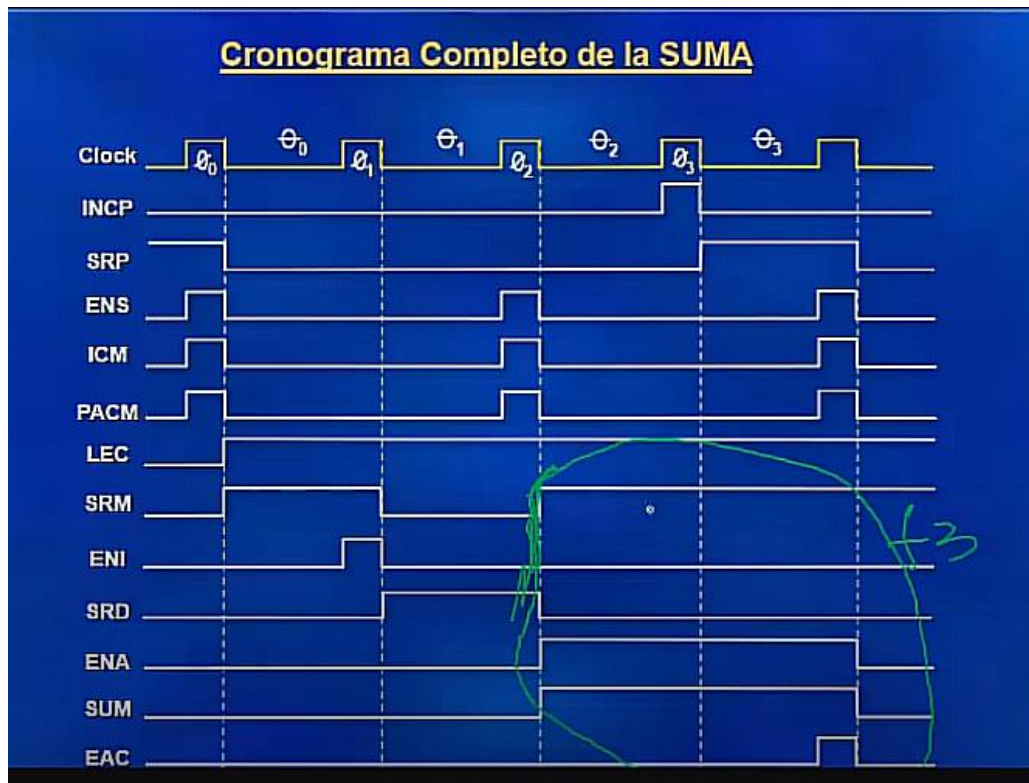
CRONOGRAMA COMPLETO

**PREGUNTA:**

¿Cuánto tiempo dura la lectura en toda la OPERACIÓN DE SUMA?

- Dura 4 batidos, 2 para leer la instrucción y 2 para leer el operando

Recordar que la lectura es destructiva, por lo que requiere un segundo batido para regenerar el contenido.



Señales de gobierno

(P) → S	SRP, ENS
((S)) → M	ICM, PACM, LEC
(M) → I	SRM, ENI
(D) → S	SRD, ENS
((S)) → M	ICM, PACM, LEC
(M) + (AC) → AC	SRM, ENA, SUM, EAC
(P) + 1 → P	INCP
(P) → S	SRP, ENS

Cronograma Completo de SUMA

<https://youtu.be/2kVgrubzng?t=1007>

RUTA DE DATOS ABACUS

<https://www.youtube.com/watch?v=2kVgrubzng&list=PLgTKn0MhANLTPffShcV9I2INbaxlPZvLI&index=18>

Almacenamiento en abacus

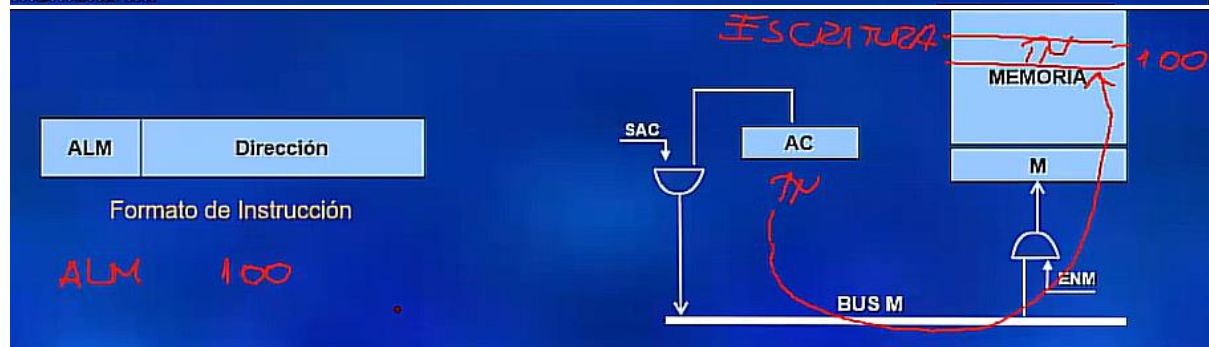
Link al video: <https://youtu.be/2kVgrubzng?t=1048>

Almacenamiento en ABACUS

Almacena el contenido del registro Acumulador en la palabra de memoria designada por la dirección del registro de instrucción.

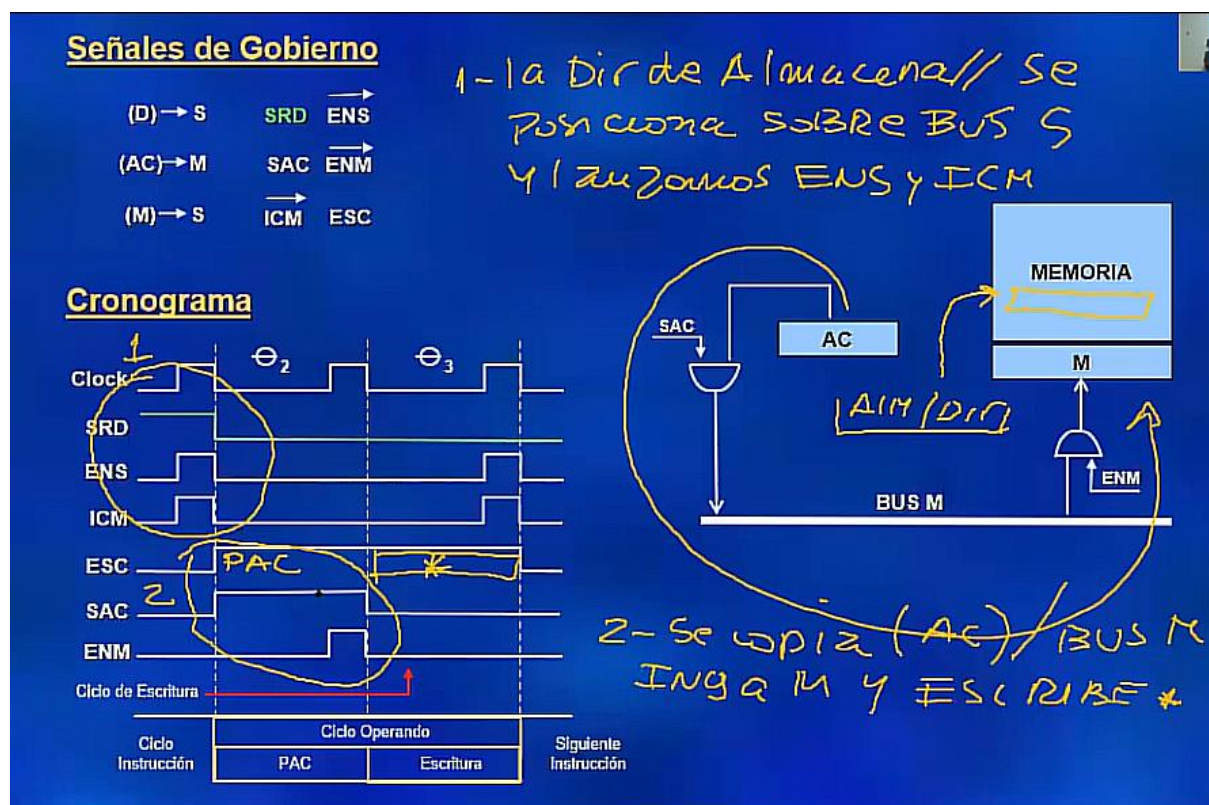
El primer ciclo es idéntico al de la suma (Búsqueda de la Instrucción) mientras que el segundo ciclo se dedica a seleccionar la palabra designada por la dirección de la Instrucción, para luego escribir en ella el valor de AC.

En tal sentido el segundo ciclo dedica su tarea a estabilizar la información contenida en AC sobre el bus de datos M (señal de gobierno de nivel SAC), para que luego sea almacenado en la palabra de memoria seleccionada por el campo Dirección, a través del registro palabra de memoria M.



ACLARACIÓN DEL CRONOGRAMA DE ABAJO

Aca en el cronograma no se agrega Tita0 ni Tita1, porque esas corresponden al Ciclo de búsqueda de la instrucción de la suma.



La **franja amarilla con asterisco** es donde realmente se ejecuta la Escritura, donde dice PAC es la puesta a 0.

PREGUNTA

¿Por qué lanzamos la escritura antes de sacar el valor del acumulador y entrarlo a M?

Para poner a cero la memoria, así para cuando termine de acercarse la palabra, se ejecuta SAC (Salida de acumulador) y con el dato ya sensibilizado en el Bus M, proceder a realizar la escritura en memoria.



SEÑALES DE CONTROL

SRD - Salida de registro de dirección y tiene que sensibilizarse sobre el Bus de Direcciones

ENS - Entrar a registro selección de memoria

ICM - Lanzar ciclo de memoria

ESC - Lanzar la escritura, por eso la escritura ya se lanza **ANTES** y consume 2 ciclos/batidos.

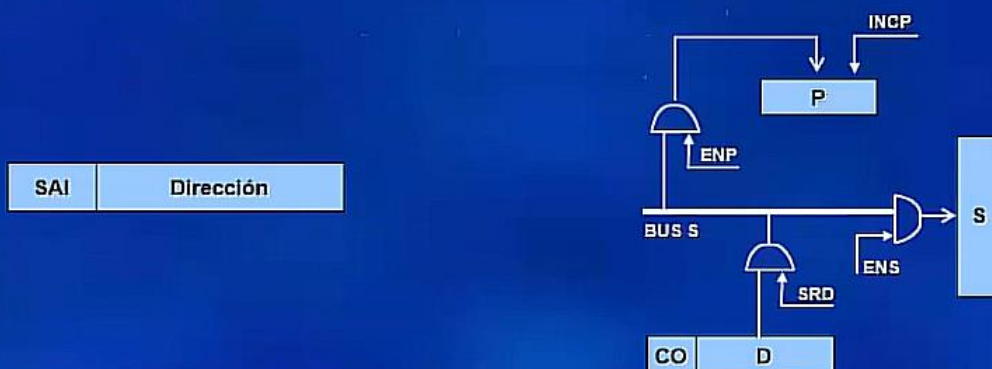
SAC - Tiene que habilitarse y sensibilizarse en el primer batido, porque en ese primer batido la palabra de memoria seleccionada se está “acerando” y recién desde el segundo ciclo, se puede escribir **Marcado con rojo en la imagen del Ciclo de Escritura**

ENM - Entrada al registro M, pero se ejecuta recién antes del 2do ciclo

Salto incondicional

Salto Incondicional en ABACUS

La información de la parte de la Dirección del registro de Instrucción, sobrescribe el valor que lleva el registro Contador de Programa, de manera tal que la siguiente instrucción sea la indicada por la instrucción en curso.



Consiste en que mueve el contenido de D a S y sobre-escribe el valor de P para que no quede desactualizado. La información de la parte de la Dirección del registro de instrucción, sobre-escribe el valor que lleva el registro Contador de Programa, de manera tal que la siguiente instrucción sea la indicada por la instrucción en curso.

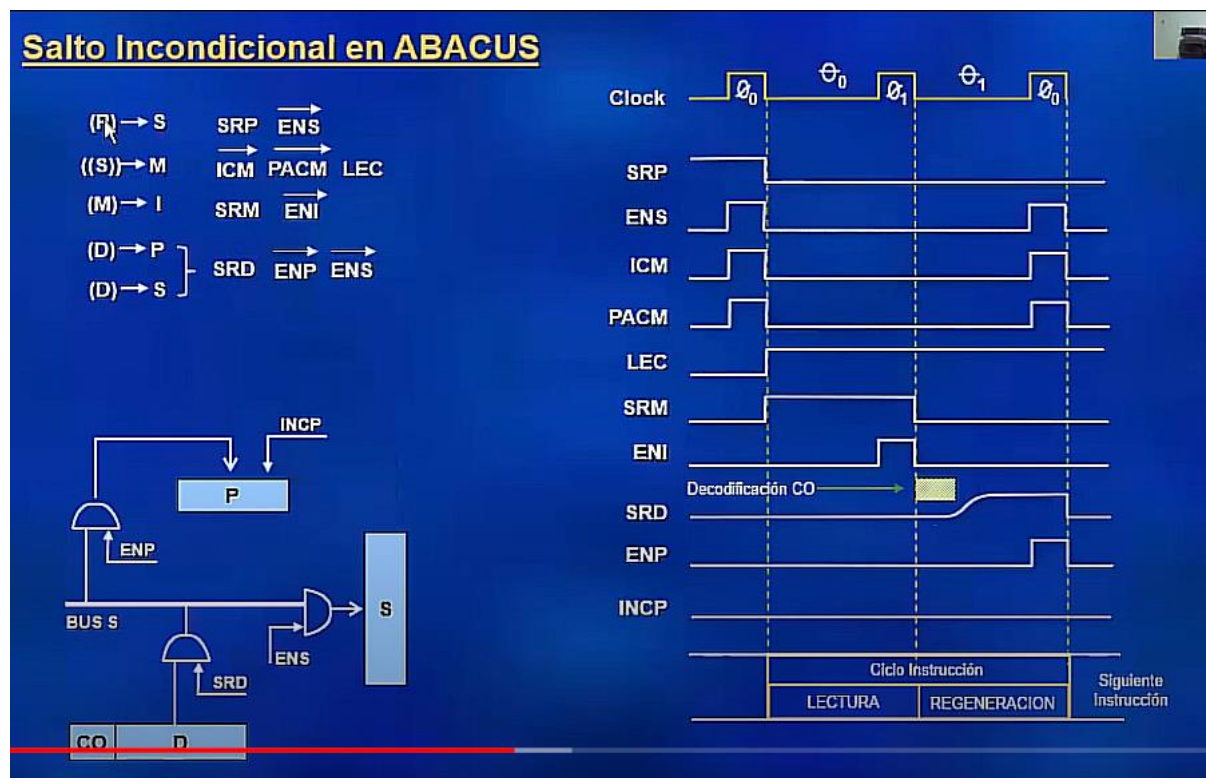
La información de la parte de dirección de la instrucción debe ser transferida simultáneamente a S para buscar la próxima instrucción y a P para actualizar el contador: $(D) \rightarrow S, (D) \rightarrow P$.

Las primeras tres es la búsqueda de la instrucción $((P) \rightarrow S, ((S)) \rightarrow M$ y $(M) \rightarrow I$ y la parte que resta está dedicada a alterar la secuencia del Programa basando el contenido de P en el valor del contenido de D $\Rightarrow$ El contenido de D va a P y a S. No se incrementa P porque ya tiene el valor de SAI

Ejemplo

Si venía instrucción 1,2,3,4,5 y realiza un salto incondicional a 20, tiene que actualizar el P, entonces la próxima instrucción sería 21 y no 6, en caso de no haber actualizado el contador de programa.

CRONOGRAMA SALTO INCONDICIONAL



RESUMEN SEÑALES GOBIERNO ABACUS

Son todas las señales de impulsos o de nivel empleadas para gobernar las distintas operaciones en el desarrollo de una instrucción. Las clasificaremos en tres categorías.

Señales de transferencia:			
ENS	Impulso	(bus S)	→ S
PACM	Impulso	0	→ M
SRM	Nivel	(M)	→ bus M
ENM	Impulso	(bus M)	→ M
ENI	Impulso	(bus M)	→ I
ENA	Nivel	(bus M)	→ UAL
SAC	Nivel	(AC) bus	→ M
ENP	Impulso	(bus S)	→ P
SRD	Nivel	(D) bus	→ S
SRP	Nivel	(P) bus	→ S
INCP	Impulso	(P) + 1	→ P

Señales de gobierno de la memoria:		
ICM	Impulso	Inicio ciclo memoria
LEC	Nivel	Ciclo de lectura
ESC	Nivel	Ciclo de escritura

Señales de gobierno de la unidad aritmético-lógica:		
CAR	Nivel	Carga acumulador
SUM	Nivel	Suma
SUS	Nivel	Sustracción
Etc	...	...
EAC	Impulso	Introducción del resultado en el AC

Registros de la ruta de datos y su interconexión

ORGANIZACIÓN DE LOS REGISTROS DE LA RUTA DE DATOS

Link de video: https://youtu.be/_2kVqrubzng?t=3011

Desde el Plano funcional:

Es como los registros dan soporte a la Arq. desde el punto de vista de su funcionamiento

NO DIRECCIONABLES	DIRECCIONABLES
Función bien definida y exclusiva. La mayoría son transparentes al programador.	Usables por el programador, para conservar determinadas operaciones en el transcurso del cálculo.
Ejemplo Registro Instrucción, Acumulador, Multiplicador cociente, Contador Programa, Registro M, Reg. Contador de desplazamientos	Se pueden diferenciar en 3 categorías: - Aritméticos: Para conservar resultados parciales - Direccionamientos: Para conservar elementos fijos de direccionamiento de partes de programas o de vectores. - Banalizados: Un registro que puede usarse indistintamente como aritmético o de direccionamiento.

Desde el plano estructural: De qué manera física están implementados los registros.

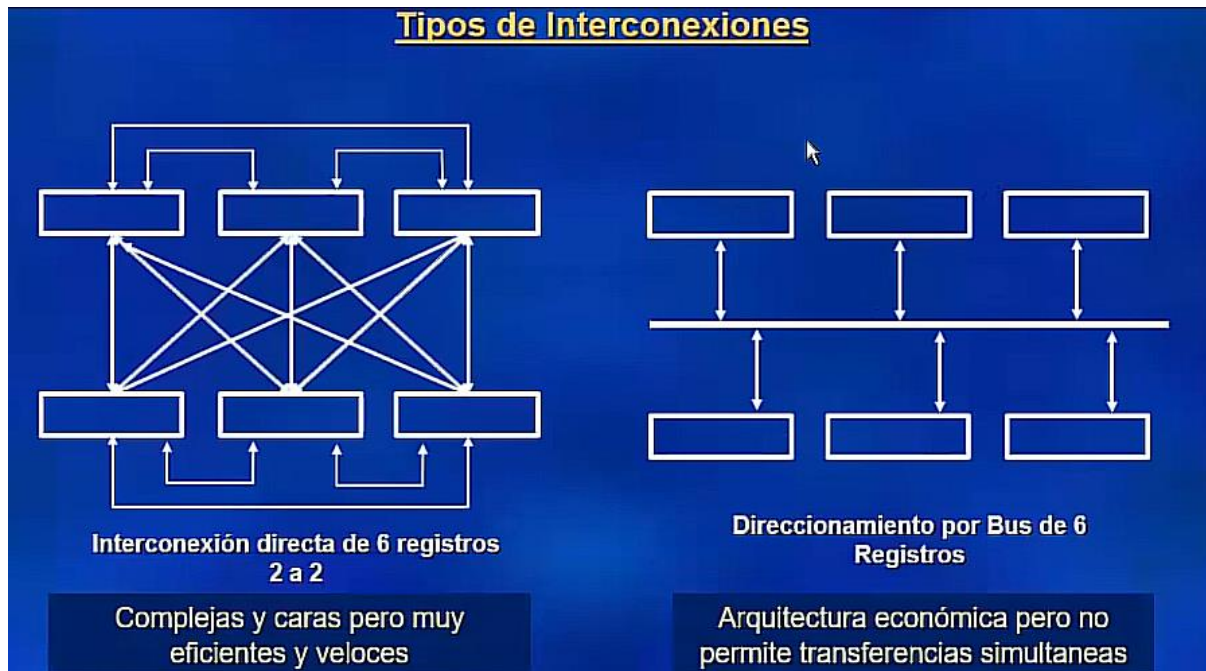
Montados como Registros Independientes	Montados de Memoria
Están instalados dentro de la arquitectura. Cableado de manera que se pueda acceder al registro de instrucción. Su dirección debe ser decodificada para seleccionarlos.	Memorias locales dedicadas a decodificar y operar sus palabras como registros independientes.
VENTAJAS	VENTAJAS
Operar de forma simultánea	Menos puntos de entrada y salida, más económicas y un solo circuito integrado Simpleza del circuito
DESVENTAJAS	DESVENTAJAS
El costo y la cantidad de puntos de E/S encarecen su costo de implementación Complejidad de conseguir llevarlos en el HARDWARE	Imposibilidad de operar con varios simultáneamente

RESUMEN



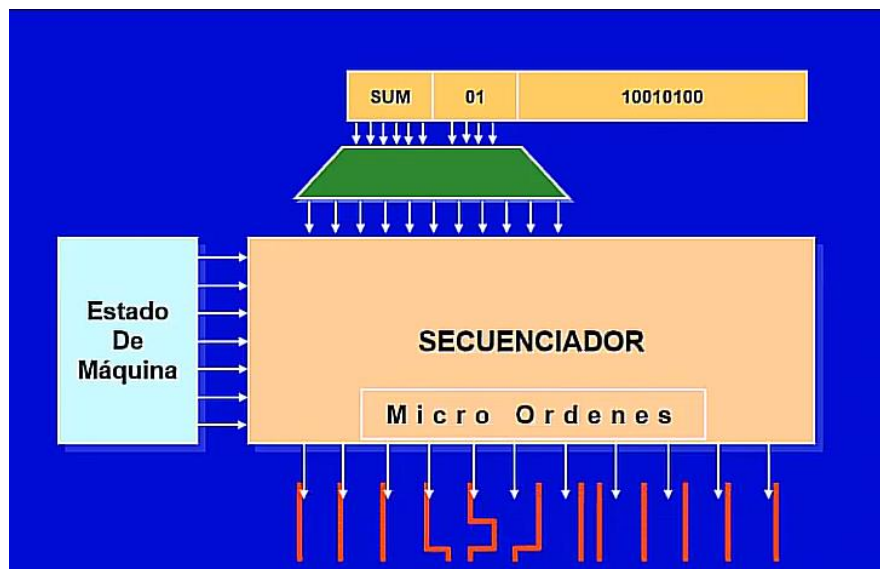
Tipos de interconexiones

Las interconexiones entre registros pueden ser directas o a través de buses. La interconexión por bus tiene, frente a las conexiones directas, la ventaja de ser barata, pero no permite transferencias simultáneas (bus ocupado/bloqueado).



DIRECTA / CABLEADA	POR BUS
Es costosa	Es barata
Es versátil, logra paralelismo en escritura y lectura	Si el bus está ocupado no permite hacer la transferencia. No permite la simultaneidad
Veloz	Lento

EL SECUENCIADOR



Es el cerebro mismo de la computadora.

Es el órgano del computador encargado de generar las microordenes necesarias para ejecutar una instrucción del lenguaje de máquina, distribuyéndolas de manera racional y eficiente a lo largo de la ruta de datos de nuestra arquitectura.

Encargado de gobernar el tratamiento de las Instrucciones de máquina a través de la generación de Microordenes a lo largo de la ruta de datos (señales impulsionales y de nivel), con el objeto de manejar transferencias de información entre los distintos órganos involucrados en la función o ejecución de la instrucción en curso.

2 condiciones en las que no debe caer:

SI NO ES RACIONAL	SI NO ES EFICIENTE
El exceso de microordenes genera alto consumo de ancho de banda de los buses	El exceso de microordenes para una misma acción consume muchos ciclos de máquina de forma innecesaria

Conceptos a analizar:

- Entorno del secuenciador
- Ecuaciones lógicas de un computador
- Modelo de Wilkes y técnicas de microprogramación

El secuenciamiento de las instrucciones



Para el caso de ABACUS, nos remontamos a la UNIDAD 1, y el SECUENCIADOR estaría entre MICROPROGRAMACIÓN Y LÓGICA DIGITAL.

Como abacus no tiene la capacidad de microprogramación, por ello decimos que es secuenciador el que cumple ese rol.

Diseño y construcción del órgano secuenciador

“El Dios interno” - Musica de Ameno ☐

Ejemplo para la suma



Dada un código de operación válido, el secuenciador activa uno de sus circuitos dedicados a generar Microordenes según cronogramas (de la instrucción en Curso).

Modelos de secuenciadores

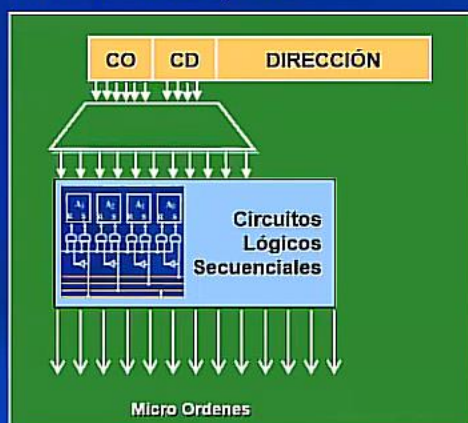
Ambos tienen el mismo objetivo → generar microordenes

CABLEADO	MICROPROGRAMADO
Es el modelo antiguo	El modelo más actual
Más rápido que Microprogramado	Posee una memoria de control. Son memorias de nivel 0, y se pueden reprogramar con el fin de mejorar o corregir
Muy caro, hay que desarrollar muchos circuitos para generar microordenes	Permite ahorrar costos al ser posible la reprogramación, por eso se utiliza más en la actualidad
Mucho más grande tamaño	Más pequeño
Consume más energía, porque tiene muchos circuitos	Consume menos energía, porque no tiene la misma cantidad de circuitos
Como programador tenes que conocer la arquitectura a fondo	Permiten que el programador se aleje del Hardware (te permite abstraerte de la arquitectura)

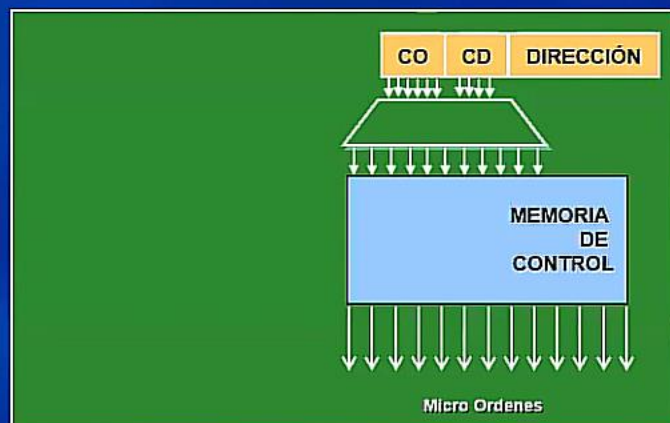
Modelo Cableado y Microprogramado

Ambos modelos tienen por objeto generar el conjunto de microordenes (señales impulsionales y de nivel) y distribuirlas a lo largo de la ruta de datos, con el fin de gobernar los distintos órganos y componentes y realizar la instrucción en curso, basándose en el conjunto de señales de gobierno diseñadas oportunamente en el cronograma de cada operación.

La diferencia se basa en que el modelo cableado posee un conjunto de circuitos lógicos encargados de generar dichas microordenes por cada operación, mientras que el modelo micro programado posee una memoria especial de control que almacena para cada operación, el conjunto de microordenes de gobiernos en el formato ceros y unos



Modelo Cableado



Modelo Microprogramado

Secuenciador cableado

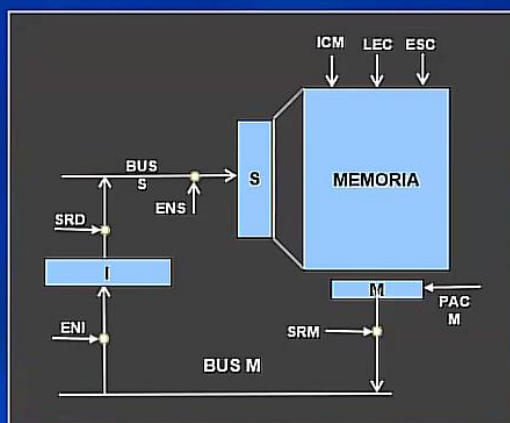
El secuenciamiento de las microórdenes en el tiempo, puede llevarse a cabo de 2 maneras distintas:

- **Síncrono**
- **Asíncrono**

SECUENCIADOR SÍNCRONO

Secuenciador Cableado (Síncrono)

No es posible en el marco de esta materia profundizar sobre la problemática y los aspectos detallados de este modelo. Por tal motivo nos limitaremos a un acercamiento intuitivo al problema tomando de referencia y como ejemplo el Secuenciamiento en la arquitectura ABACUS de Meinadier.



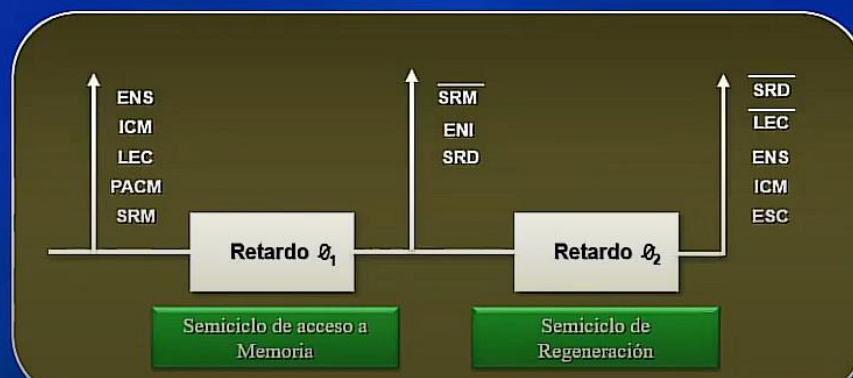
Principio del Secuenciamiento:

El objetivo del secuenciador es generar Microórdenes distanciadas en el tiempo por retardos apropiados. Una cadena de retardos permite, en teoría realizar esta función.

Tomemos como ejemplo el inicio de la instrucción ALM en Abacus de Meinadier

Secuenciador Cableado (Síncrono)

Realizados en forma de Circuito Secuencial electrónico
Su objetivo es generar microórdenes distanciadas en el tiempo por retardos adecuados. Una cadena de retardos permite en teoría, realizar esta función. **Las señales de NIVEL permanecen activas hasta su anulación.**
Por ejemplo, para la Instrucción ALM, la cadena de retardos SÍNCRONO sería:



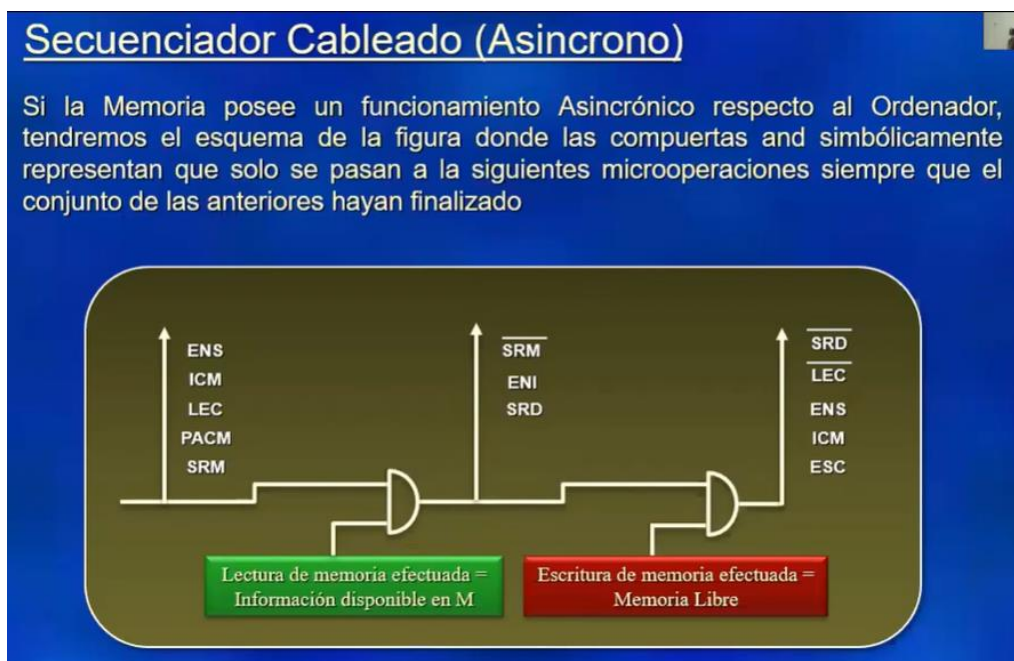
El secuenciador **conoce los tiempos de respuesta de los diferentes órganos gobernados**. Entonces hace intervenir los retardos apropiados entre las diferentes órdenes sucesivas de manera que tengan tiempo de ejecutarse las operaciones. Las microórdenes van generalmente sincronizadas con los impulsos de reloj. Así, si una operación necesita un

poco más de dos batidos de reloj, el secuenciador esperará al tercer batido posterior al lanzamiento de la operación para enviar la siguiente microórden

El esquema secuenciador cableado síncrono plantea la generación de microórdenes espaciada en el tiempo a través de retardos previamente definidos.

- Son retardos que hacen que para pasar de una columna del cronograma a otra tiene que pasar el espacio de tiempo de retardo previamente establecido.
- Son de valores fijos.
- Al ser cableados, son circuitos secuenciales por lo tanto no hay memoria de control.
- Las señales de nivel, por ejemplo SRM, dura el retardo $\square 1$ porque luego se le niega porque tiene que dejar de funcionar en esa fase. Así funciona este esquema. Cuando se quiere desactivar una de señal de nivel se debe complementar la señal.
- Las señales en este esquema siempre están separadas un espacio de tiempo fijo, por eso se dice sincrónico.

SECUENCIADOR ASÍNCRONO



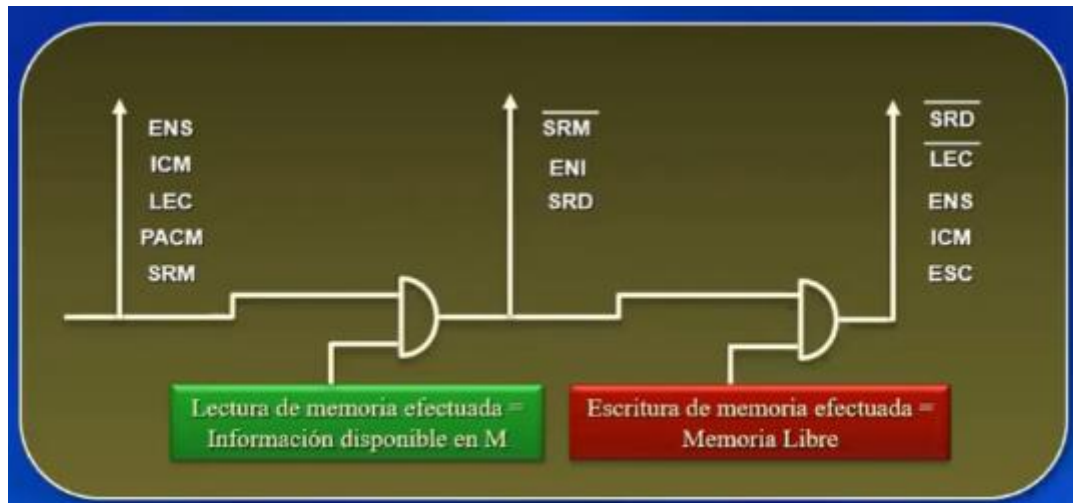
El secuenciador recibe de los diferentes órganos del calculador señales indicando que han terminado las operaciones ordenadas y que se liberan; aquel puede eventualmente comprobar su estado.

- No lanzará una nueva operación más que después de haber advertidos de que las operaciones procedentes han sido ejecutadas completamente y de haberse asegurado que el órgano que debe realizarla está libre

En el modelo asincrónico las compuertas de gobierno dejan pasar de una fase a otra cuando todas las señales de gobierno de una misma fase se cumplieron, **entonces los retardos que hay entre un grupo y otro de señales son variables y depende de que todo el grupo de señales se haya cumplido**. Es decir, en el modelo asincrónico se pasa a la siguiente fase solo cuando las señales del anterior se hubieran cumplido.

En realidad para la maquina estar controlado si todas las microinstrucciones se cumplieron es muy complejo.

Un ejemplo, los tiempos no son fijos, están definidos por el tiempo que le lleva a la memoria RAM completar una escritura o una lectura. Las AND de la siguiente figura, representan simbólicamente que SÓLO se pasa al siguiente grupo de microoperaciones, siempre que el **conjunto anterior haya finalizado**.

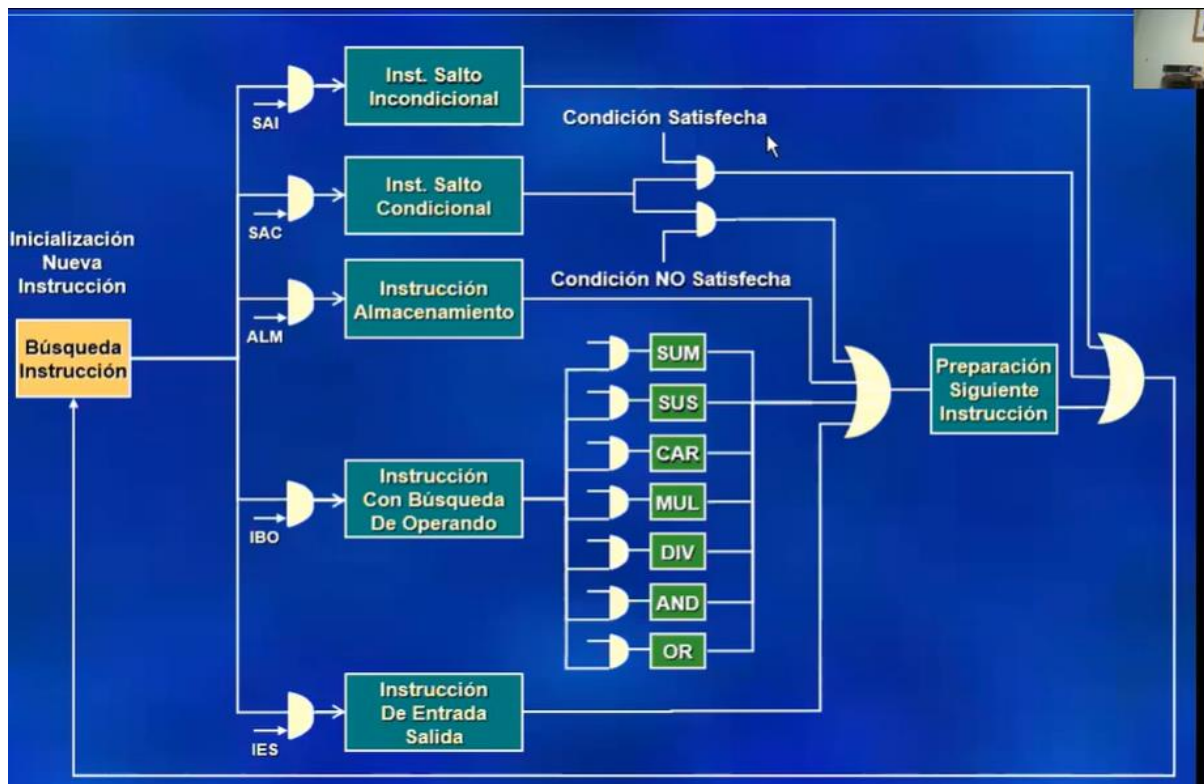


NOTA

¿Qué hace la instrucción stop en Abacus?

Con la instrucción **STOP** el secuenciador pone a 0 el biestable **detención/marcha**. El clock sigue funcionando pero no deja pasar al distribuidor de fases para que este haga andar al secuenciador. Si el distribuidor de fases se detiene el secuenciador se para donde estaba, el cronograma queda frenado. El que hace andar el secuenciador es el distribuidor de fases.

Tratamiento de instrucciones



Preparación de la siguiente instrucción



PSI

- Inc (P) - Incrementar el contador de programa
- SRP - Salida del registro P, contador de programa, estabilización en el Bus S
- ENS - Entrada de registro S, registro de selección

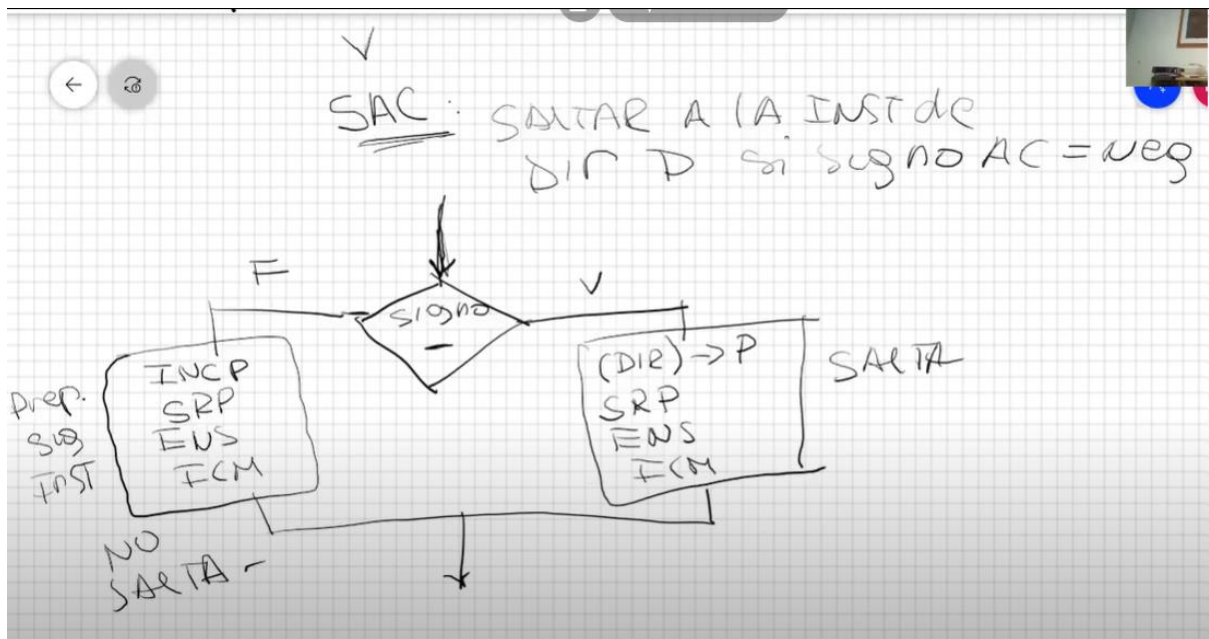
Salto incondicional

En el salto incondicional, no pasa al proceso de Preparación de la siguiente instrucción. No incrementar el contador de programa.



Salto condicional

Evalúa la condición

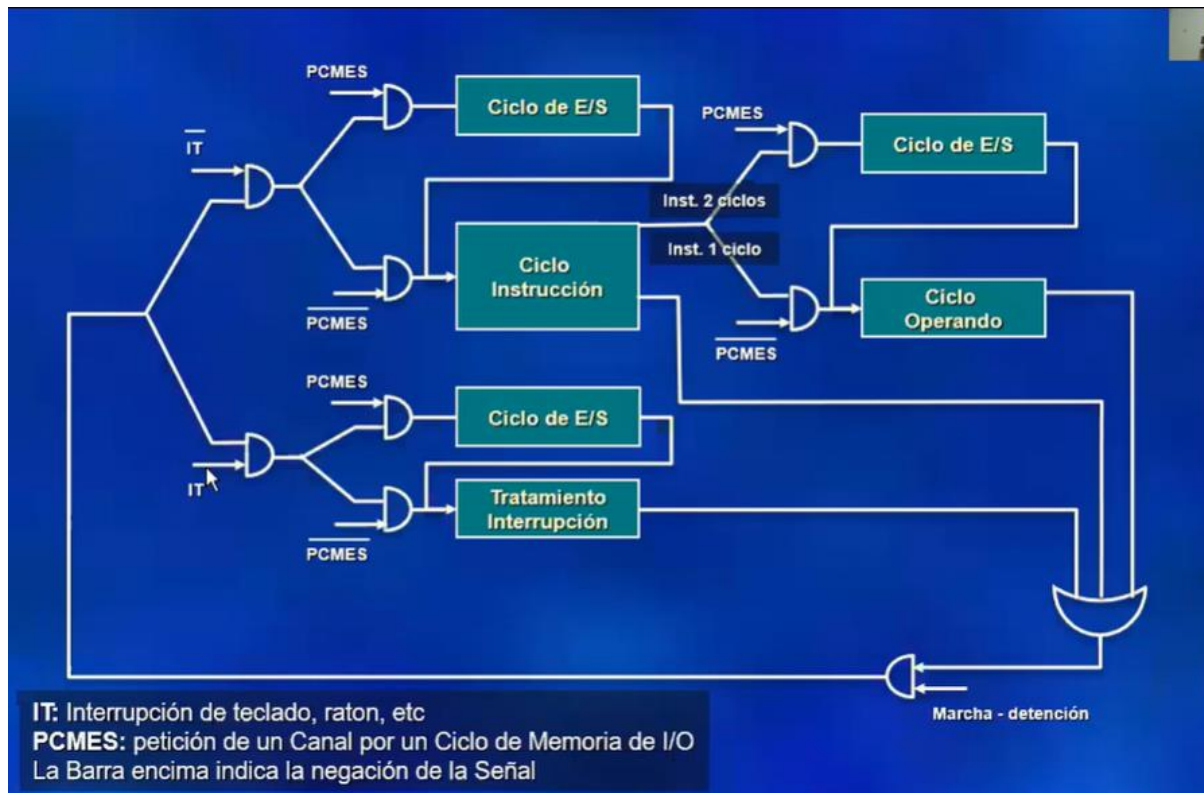


Cumple condición → salta

- (DIR) → P : Mueve el contenido de DIR (dirección) a P
- PSI

No cumple condición → sigue por PSI (Preparación de la siguiente instrucción)

Tratamiento de los indicadores de estado



Toda máquina atraviesa por una fase de búsqueda de instrucción, posible búsqueda de operando, y la ejecución de la operación. **Pero en el medio puede ocurrir dos tipos de interrupciones:**

- Una **interrupción** que se denomina **IT**, que consiste en un teclado, mouse, táctil, pidiendo que la CPU le de tratamiento prioritario. En realidad la interrupción es que el programa en curso detenga su ciclo normal y desarrolle un pequeño programa que de solución al periférico que lo interrumpió.
- Una **petición de un canal por un ciclo de memoria de I/O**: es un pedido del canal a la CPU para realizar una escritura o lectura entre la memoria y un periférico que lo requiere. Debido a que los periféricos solo dialogan con el canal, es este quien pide el ciclo de memoria.

NOTA

Va a tener más prioridad SIEMPRE petición de I/O que una IT de periférico (mouse, teclado, etc).

DISTRIBUIDOR DE FASES

DEFINICIÓN

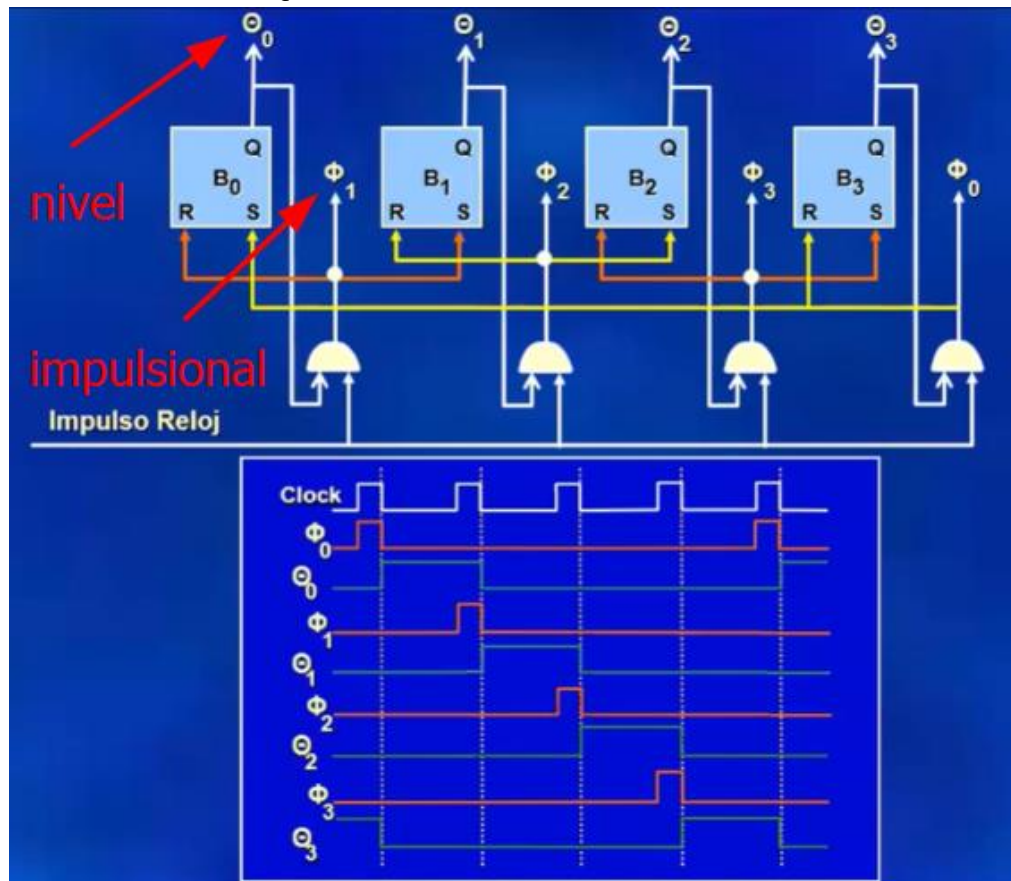
El distribuidor de fases proporciona al secuenciador de muestra impulsional o de nivel para cada una de las fases de todas las requeridas de cada instrucción de máquina, de manera que el secuenciador, que sabe cómo distribuirlas, tenga estas señales para cada fase y a partir de allí la distribuya según indique el cronograma. Alimenta al secuenciador.

Su función es producir **señales de impulso o de nivel** (acordes a las diferentes fases del ciclo de memoria o de la instrucción) a partir de los impulsos periódicos del reloj.

El distribuidor de fases debe tener tantas fases como para representar a todos los cronogramas. Tiene que abastecer al cronograma con mayor cantidad de fases.

EJEMPLO: Si existen cronogramas ocupan 4 fases y uno solo ocupa 32 fases entonces el distribuidor de fases debe tener 32 fases.

ESQUEMA DE DISTRIBUIDOR DE FASES

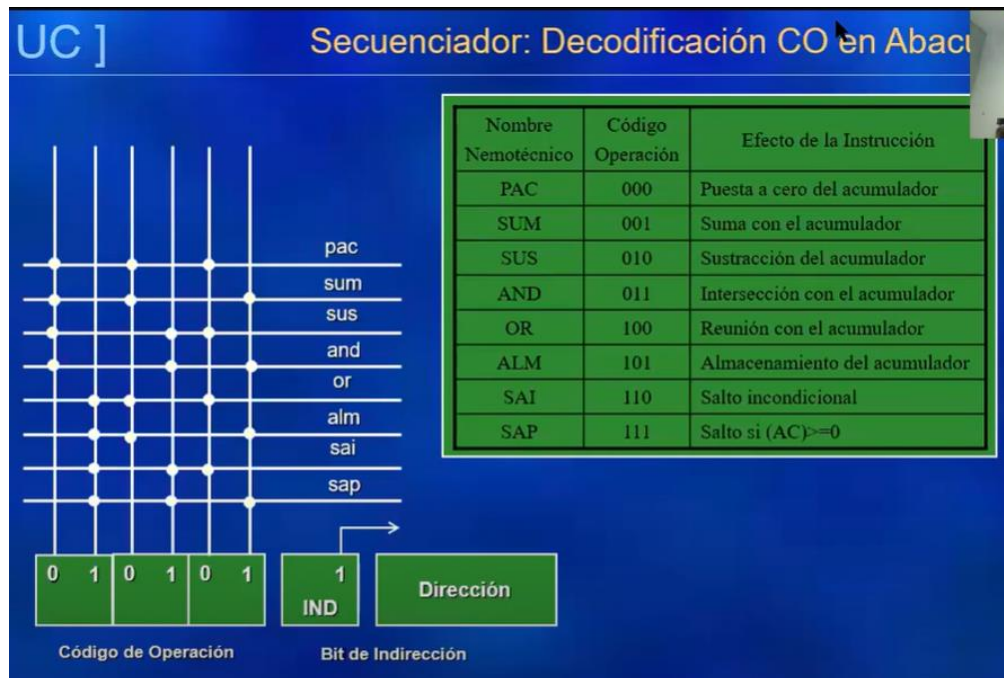
**fi - nivel**

teta - impulsional

Decodificación de la instrucción

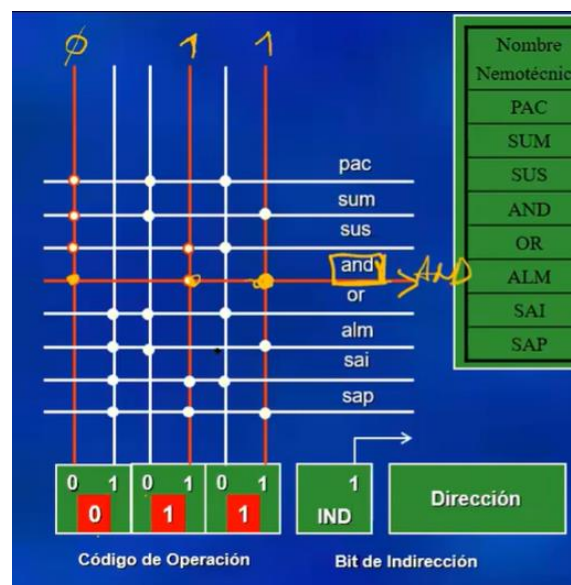
Es un circuito capaz de seleccionar uno y solo un cronograma en función al valor binario del campo CO. Los hilos intersectados equivalen a un 1 contrariamente es un 0.

Puede utilizarse una matriz de diodos o transistores



Los puntos blancos en las intersecciones, son las “soldaduras” de los hilos del código de operación con el hilo de señal de operación.

Ejemplo:



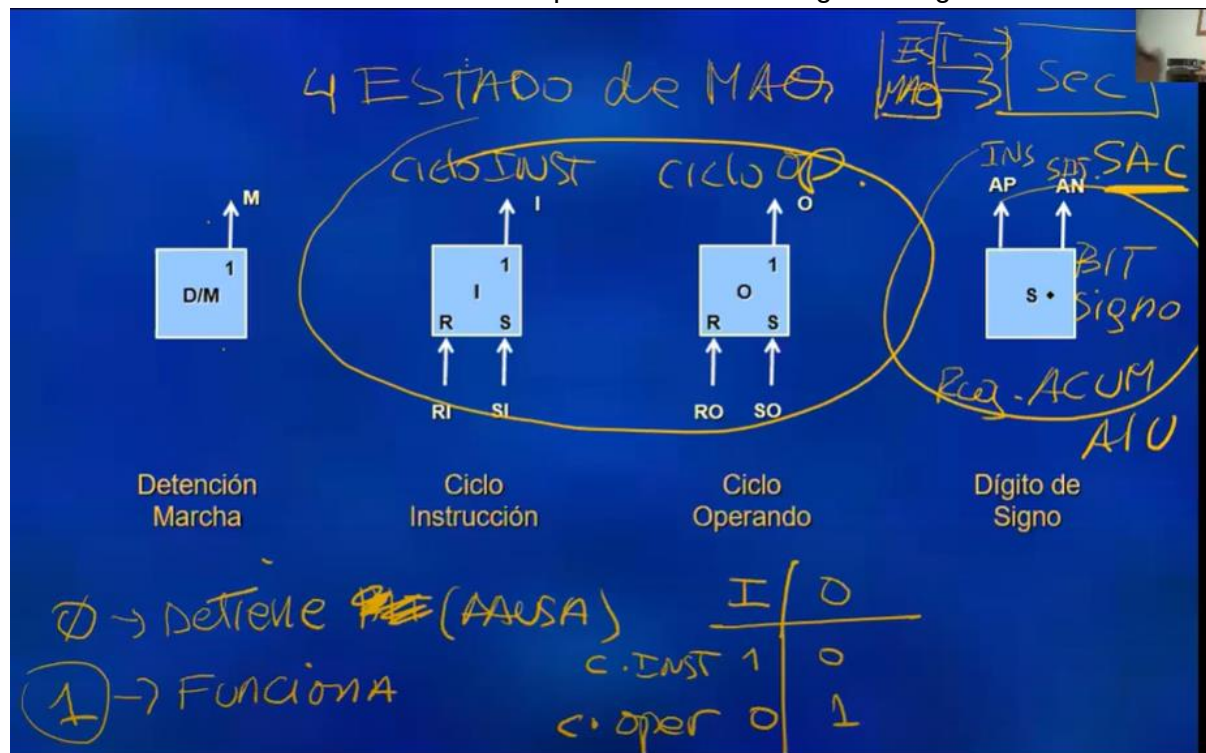
Biestable de estado

Se define al estado del computador por el contenido de un cierto número de biestables, llamados biestables de estado

Existen dos clases:

1. Los primeros memorizan lo que hemos denominado el estado de la máquina. Son posicionados por las instrucciones y los resultados de operaciones, o también por 88 acontecimientos exteriores, petición de ciclo de memoria, interrupción, etc. Sus salidas figuran entre las informaciones de entrada al secuenciador.
2. Los segundos constituyen parte integrante del secuenciador. Es éste quien los posiciona y quien utiliza su salida. Esencialmente sirven de marcas temporales. En cierto modo, los biestables del distribuidor de fases podrían incluirse dentro de esta categoría

Abacus consta de los cuatro biestables representados en la siguiente figura:



- **Detención/Marcha:** Si está en 0 pausa a la máquina. Si está en 1 continúa con la ejecución. Detiene la generación del distribuidor de fases, por lo que también hay una pausa en la generación de las microordenes
- **Ciclo Instrucción:** Si está en 1 indica que está en ciclo de instrucción
- **Ciclo Operando:** Si está en 1 indica que está en ciclo de operando
- **Dígito de Signo:** Es un biestable del acumulador y sirve para determinar si es salto o no. El biestable de signo es el signo del acumulador y determina en la instrucción SAC si es satisfecha o no la condición.

RECORDAR

¿Por qué se necesitan dos biestables, para ciclo de instrucción y operando?

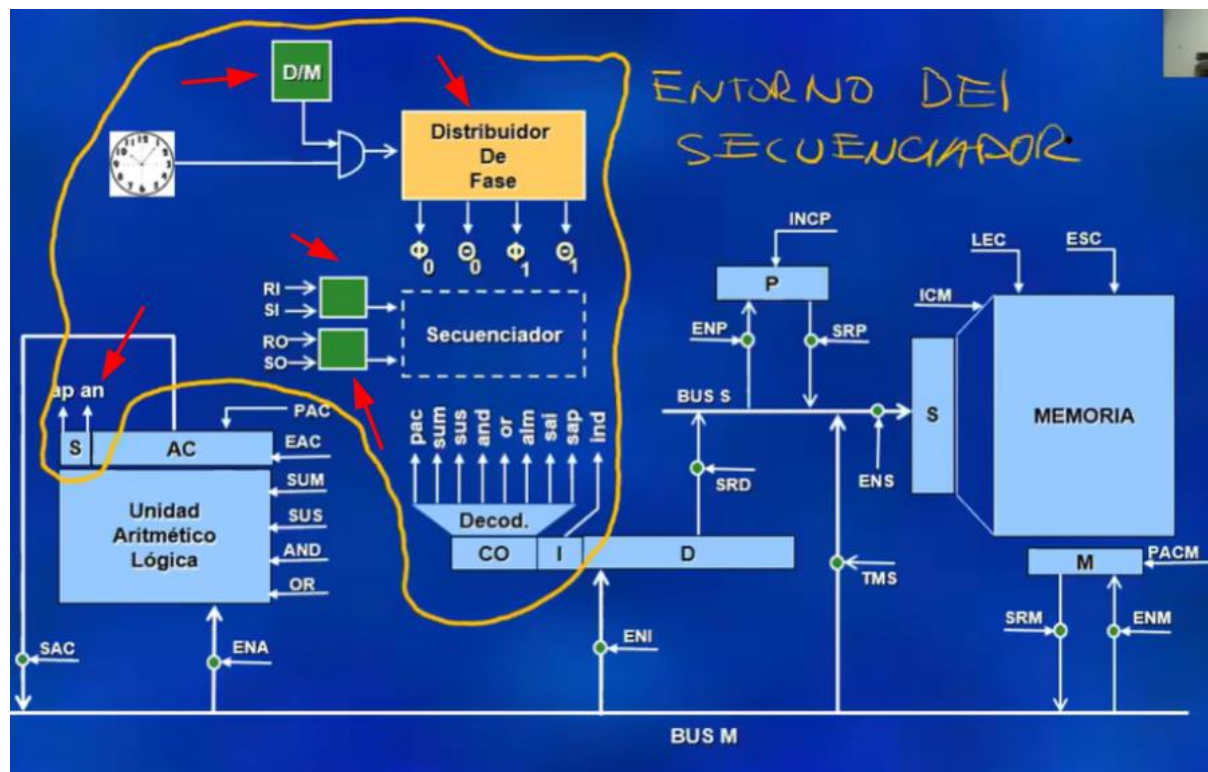
Se utilizan 2 biestables, el I (Ciclo de Instrucción) y O (Ciclo de Operando), pero no se utiliza sólo 1 porque en caso de estar 0 0, indica modo de funcionamiento Indirecto.

Asumimos que es el direccionamiento Indirecto, donde DIR no es el operando sino la dirección de la próxima dirección

LINK DE VIDEO: <https://youtu.be/AIWH0O9a1q?t=6492>

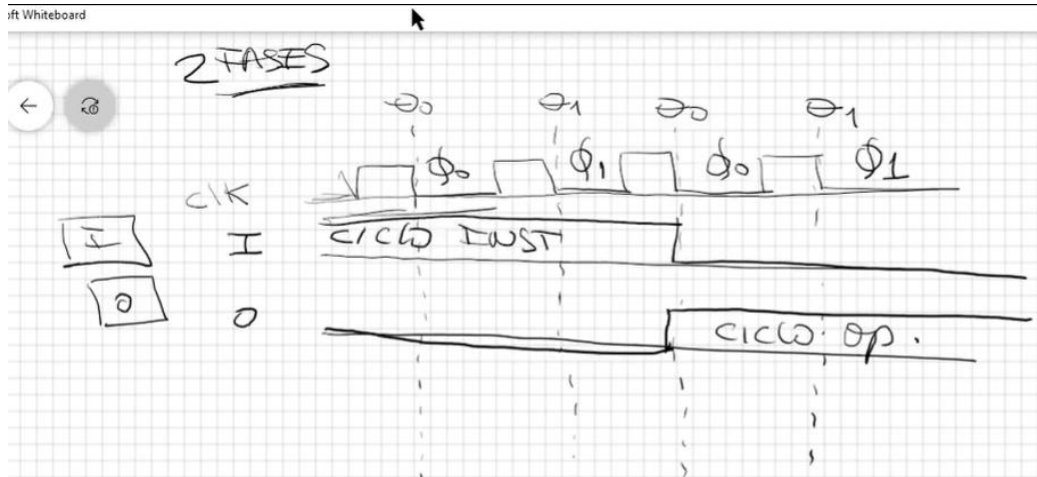
Esquema general de Abacus

El entorno del secuenciador es lo que está en amarillo:



Secuenciador de 2 FASES

Se utilizan 2 señales para que sea óptimo, y utilizar la mínima cantidad de componentes.



Resumen esquema secuenciador

LINK DE VIDEO: <https://youtu.be/AIWH0O9a1q?t=7046>

Para que la **unidad de control** realice su tarea debe tener entradas que le permitan determinar el estado del sistema y salidas que le permitan **controlar el comportamiento del mismo**.

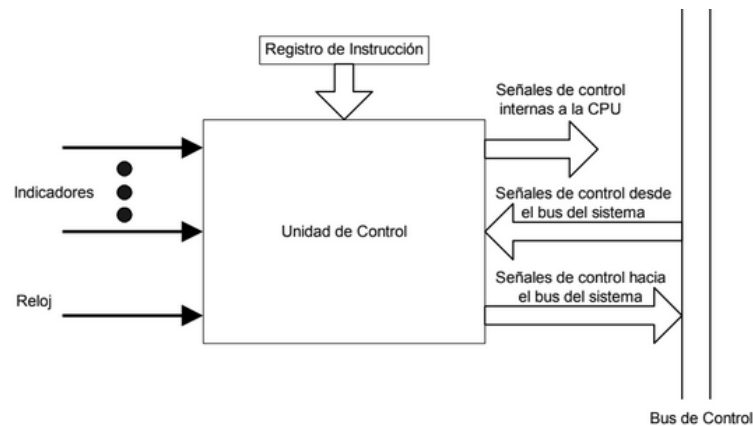
Las **entradas al secuenciador** serán:

- Las señales del distribuidor de fases.
- Señal de los biestables que indican el subciclo que se está ejecutando (I-subciclo de búsqueda de la instrucción, O-subciclo de búsqueda del operando)
- Señal del biestable S que indica el signo del acumulador
- Código de operación y condiciones de direccionamiento del reg. de instrucción.
- Señales de control de bus de control, incluye interrupciones, de reconocimiento, etc. (no se contempla en el análisis a fin de facilitar el estudio del funcionamiento)

Las **salidas del secuenciador** serán:

- Señales de control que gobiernan la transferencia de datos, funciones específicas de la ALU, control de memoria y control de los módulos de entrada/salida.

Resumiendo, se podría decir que el objetivo del secuenciador es generar las señales de control o micro-órdenes para gobernar el funcionamiento del computador. En la siguiente figura se observa un esquema general de lo descrito anteriormente.



Modelo de la Unidad de Control

Trazado de Cronogramas

Cronograma

Representación que nos permite conocer cómo se comportará el secuenciador ante una determinada instrucción.

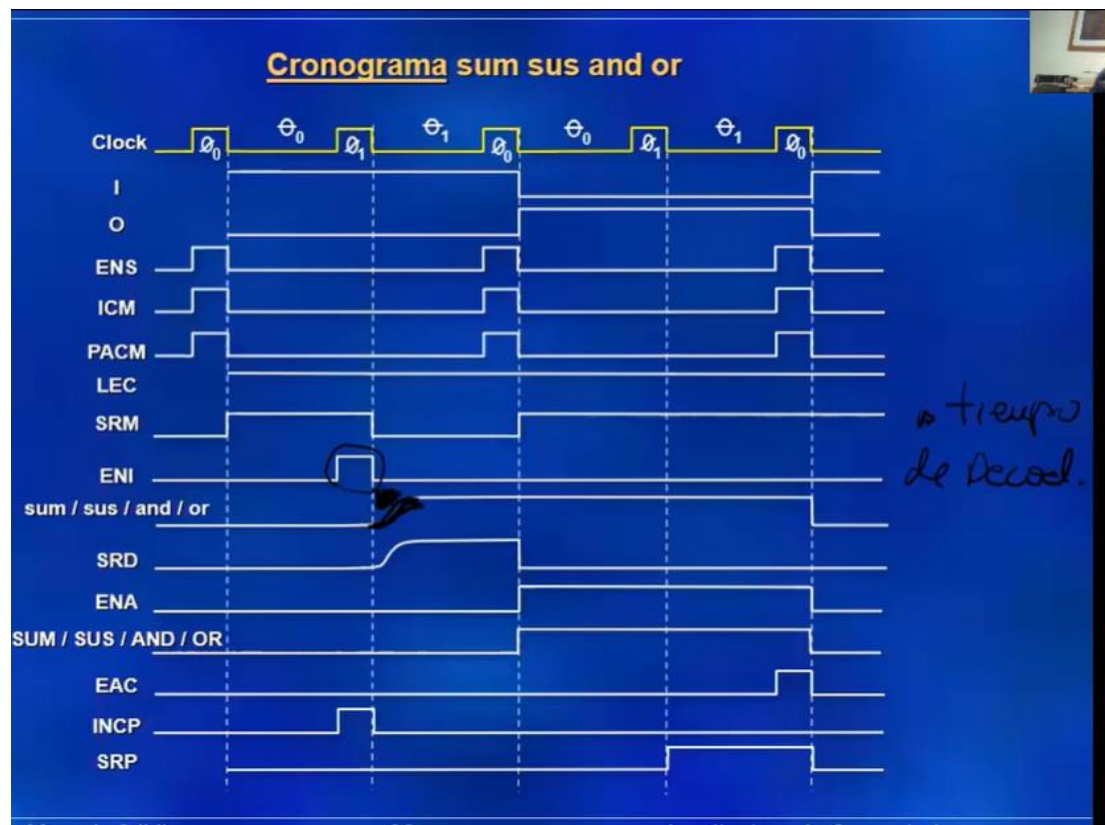
Abacus está provisto por:

- Un distribuidor de fases de dos fases.
- Decodificador de instrucción (pac, sum, sus, and, or, alm, sai, sap).
- De los 4 biestables de estados (figura anterior). Se supondrá que el funcionamiento del reloj está condicionado por la señal de detención-marcha, y que la puesta en marcha inicializa al distribuidor de fase.

Agrega I y O, los cuales indican cuál es la fase.

NOTA

Entonces acá tenes 2 fases, 1 fase por tita. Entonces cada ciclo va a implicar la cantidad de fases según la arquitectura, si tenes 6 fases, cada ciclo va a tener desde tita0 a tita5.



SRD señal de nivel más tardía porque hay una decodificación del código de operación.

Minúscula - Salida del decodificador

Mayúscula - Entrada a la ALU (señal)

Cronograma de PAC (Puesta cero acumulador)

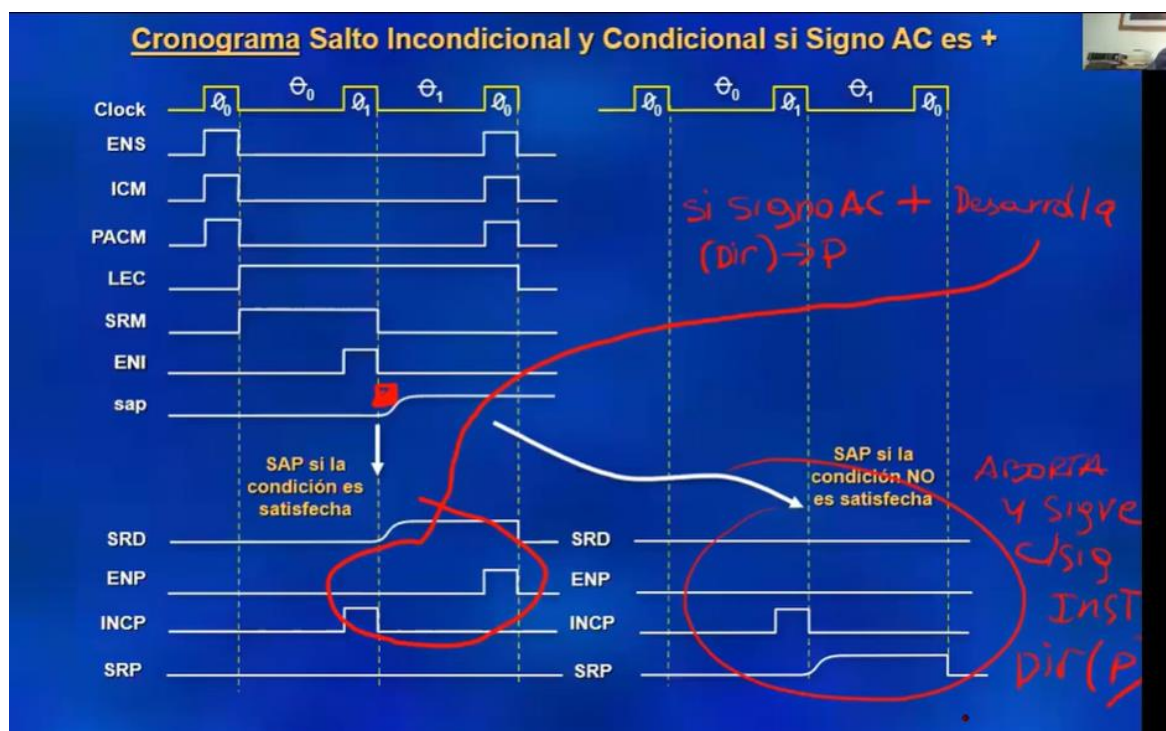


Cronograma saltos

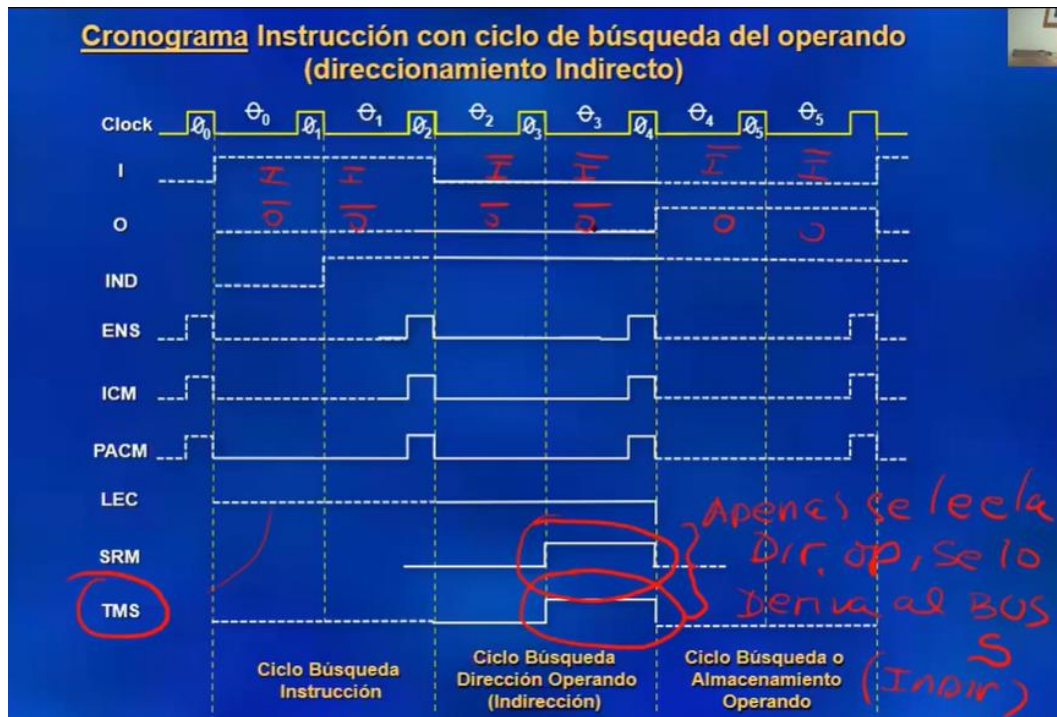
Lo que ya vimos pero tiene salida el decodificador.

Acá se grafican ambas condiciones, en caso de que el **Salto condicional condición exitosa**.

En caso de que el **Salto condicional condición fallida**.



Cronograma Direccionamiento indirecto



Apenas se lee la Dirección del operando, se transfiere de Bus M a Bus S.



Ecuaciones lógicas

DEFINICIÓN: Conjunto de expresiones lógicas del ordenador relacionadas con las informaciones de entrada y de estado del secuenciador.

Tomando una a una cada microorden, se debe buscar todas las eventualidades en que debe ser activada y de ellas deducir la expresión lógicas en relación con las informaciones de Entrada y de Estado del Secuenciador. El conjunto de estas expresiones constituye las ECUACIONES LÓGICAS DEL COMPUTADOR

$ENS = \phi_0 \quad ICM = \phi_0 \quad PACM = \phi_0$

Deben activarse siempre en ϕ_0 , ϕ_2 , y ϕ_4 , pero debido a que ABACUS trabaja con un Distribuidor de Fase de dos Fases, se deduce que deben estar activos en el instante ϕ_0 de TODOS los cronogramas, tanto para la fase de Instrucción como de Operando

$ENI = \phi_1 \cdot I \quad INCP = \phi_1 \cdot I \equiv \phi_3$

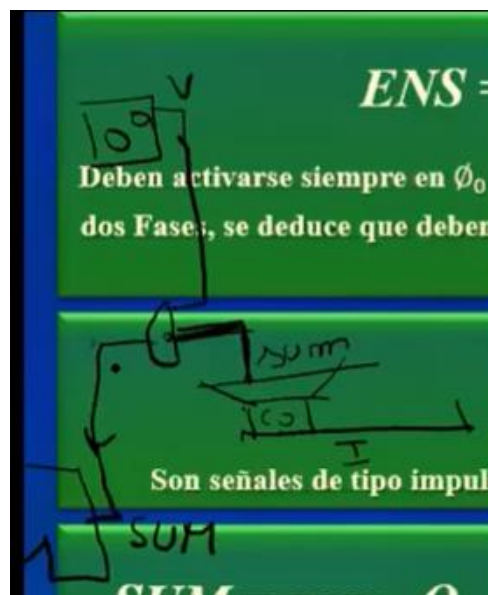
Son señales de tipo impulsional que solo deben generarse en el ciclo de Instrucción, para la etapa θ_1

$SUM = sum \cdot O \quad SUS = sus \cdot O \quad AND = and \cdot O \quad OR = or \cdot O$

Son señales de Nivel que deben sostenerse a lo largo de $\theta_0 + \theta_1$ y para cada operación en particular, dentro del ciclo del Operando. La expresión $\theta_0 + \theta_1$ se elimina ya que la operación dura las dos fases completas

Ejemplo SUM

SUM es la señal de entrada a la ALU, y su ecuación es **sum (señal de habilitación)** y el Ciclo operando



Lectura

Esto significa que LEC es **TRUE** cuando:

- Ciclo de instrucción - **Búsqueda de instrucción**
- Ciclo de operando SOLO cuando NO es ALM (porque ahí no hay lectura)
- Cuando NO Ciclo operando y NO Ciclo Instrucción porque es direccionamiento INDIRECTO.

$$LEC = I + O \cdot \overline{alm} + (\overline{O} \cdot \bar{I})$$

La señal de lectura es de nivel y debe durar todo el espacio de tiempo entre ciclo de Instrucción y de Operando (menos para almacenamiento) o bien fuera de los ciclos de Operando e Instrucción

$$LEC = (\overline{Inst}) + (\overline{oper} \cdot \overline{alm}) + (\overline{O} \cdot \bar{I})$$

alm ?

2 ciclos IND

TEC

1 ciclo INST

grabar (AC) → M

INST IND oper

IND

Salida del registro M (palabra memoria) al bus M

Explicación en criollo:

- En la búsqueda de instrucción
- En el ciclo de operando pero cuando NO es ALM
- En el ciclo de Direccionamiento Indirecto, en la búsqueda de la palabra que contiene la dirección de la palabra efectiva. Es decir, **NO para TMS**. Y por eso, es **tita1**.

$$SRM = \theta_0 \cdot I + O \cdot \overline{alm} + IND \cdot \theta_1 \cdot (\bar{I} \cdot \bar{O})$$

La salida del Registro M de Nivel, debe ser activada :

- 1) En la búsqueda de instrucción para θ_0 , es decir $(\theta_0 \cdot I)$
- 2) En el ciclo de operando para *sum sus, or, and* y durante el tiempo que le lleve a la ALU realizarlo $O \cdot (\theta_0 + \theta_1) \cdot (sum + sus + or + and)$. Debido a que $(\theta_0 + \theta_1)$ cubre todo el espacio de tiempo que dura O, es inútil expresarlo. A demas debido a que *sum sus, or, y and* requieren de un ciclo completo de Operando (con excepción de *alm*), podemos expresarlo de manera más simple como $O \cdot \overline{alm}$
- 3) En el caso del Direccionamiento Indirecto, en el ciclo de búsqueda de la DIRECCIÓN y durante todo el ciclo θ_1
- 4) Para Direccionamiento Indirecto, es necesario contemplar la habilitación de la señal IND en θ_1 dentro del "Ciclo De Almacenamiento Del Operando (fuera del ciclo de instrucción y el de operando)

Son señales de tipo **de nivel** que solo deben generarse en el ciclo de Instrucción, para la etapa θ_1

Set Instrucción

Fi0 - Set I - Comienzo del ciclo de instrucción, por eso va al comienzo de todo

$$SI = \phi_0 \cdot O$$

El propio secuenciador deberá tener la tarea de de posicionar a los Biestables I y O , por tal motivo I debe ser puesto a Uno al final del ciclo de operando

Reset Instrucción

Reset I - I debe ser puesto a cero, para aquellas operaciones que requieren ciclo de operando. Para las operaciones de 4 fases.

No se puede hacer como esas para salto incondicional. No PAC, SAP, SAI.

$$RI = \phi_0 \cdot I \cdot (sum + sus + or + and + alm)$$

I debe ser puesto a Cero al final del ciclo de aquellas instrucciones que requieren ciclo de operando, es decir para ciclo de instrucción dentro del espacio ϕ_0 , para la suma, resta, and, or y alm, el biestable I deberá ser Reseteado ya que se encuentra en la fase de resolución del Operando

Set O

Cuando es con operando, y sin direccionamiento indirecto, por eso el !IND

Cuando estoy finalizando el TMS (búsqueda de la dirección efectiva), tengo que setear el valor de O, porque hasta ahora tengo la dirección efectiva, pero tengo que poner en True el O.

$$SO = \overline{IND} \cdot \phi_0 \cdot I \cdot (sum + sus + and + or + alm) + IND \cdot \phi_0 \cdot \bar{I} \cdot \bar{O}$$

O debe ser activado ya sea despues d eun ciclo de instrucción, cuando es con operando y sin direccionamiento Indirecto, o bien al final del ciclo de búsqueda de la dirección efectiva, en el caso de direccionamiento Indirecto y por fuera del del instrucción y operando.

Reset O

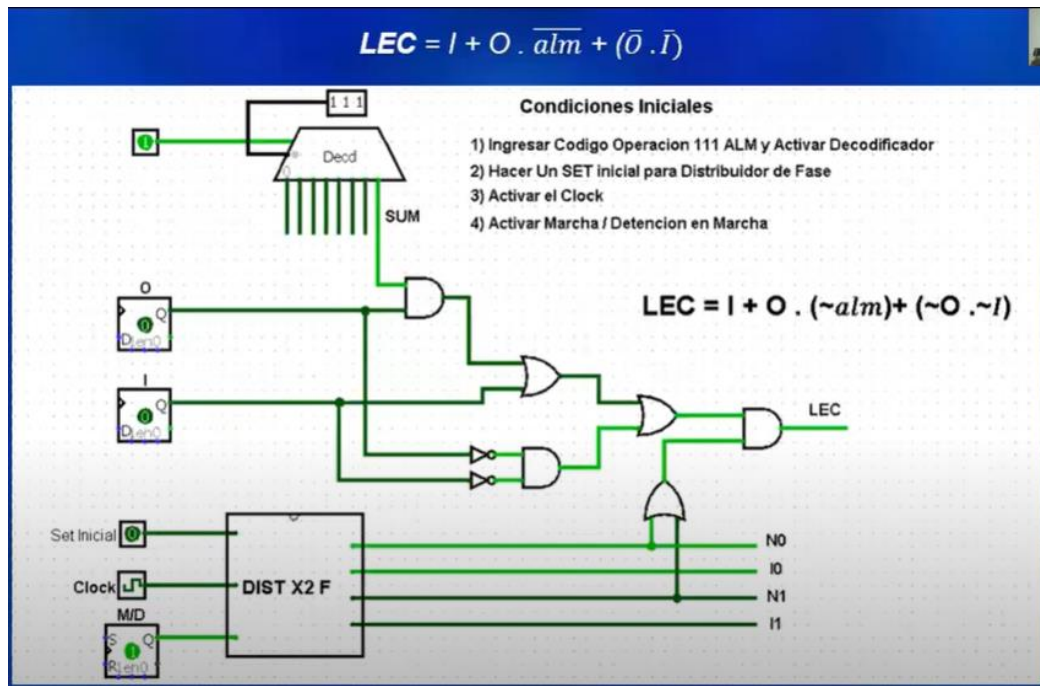
Al finalizar el ciclo de operando

$$RO = \phi_0 \cdot O$$

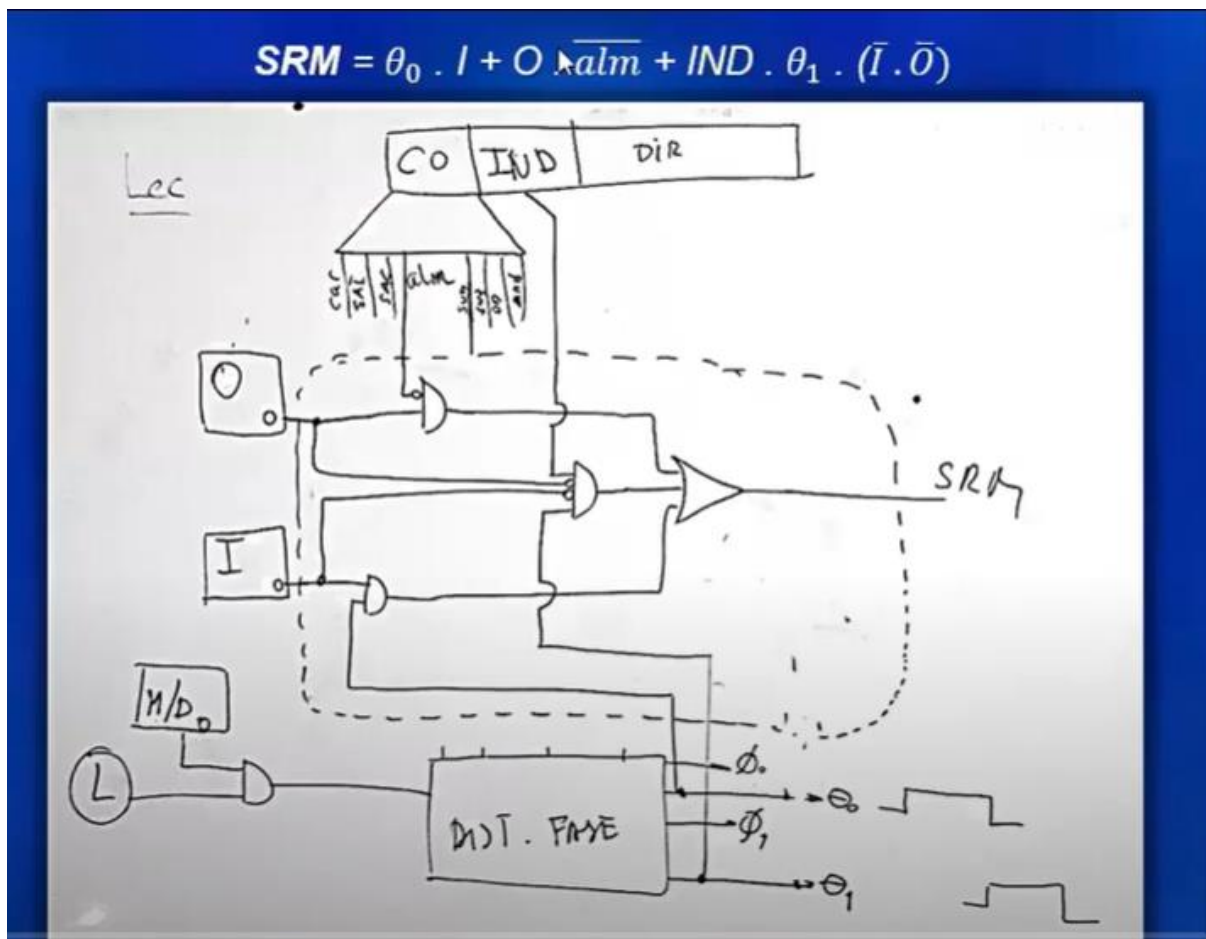
O debe ser puesto a cero al final del Ciclo de Operando

$ENS = \phi_0$ $ICM = \phi_0$ $PACM = \phi_0$
 $LEC = I + O \cdot \overline{alm} + (\bar{O} \cdot \bar{I})$
 $ESC = O \cdot alm$
 $SRM = \theta_0 \cdot I + O \cdot \overline{alm} + IND \cdot \theta_1 \cdot (\bar{I} \cdot \bar{O})$
 $TMS = IND \cdot \bar{I} \cdot \bar{O} \cdot \theta_1$
 $ENI = \phi_1 \cdot I$
 $INCP = I \cdot \phi_1$
 $SRD = I \cdot \theta_1 \cdot (sum + sus + and + or + alm + sap \cdot AP + sai)$
 $SRP = \theta_1 \cdot I \cdot (pac + sac \cdot AN) + \theta_1 \cdot O$
 $ENP = \phi_0 \cdot (sai + sap \cdot AP)$
 $SUM = sum \cdot O$ $SUS = sus \cdot O$ $AND = and \cdot O$ $OR = or \cdot O$
 $ENA = O \cdot \overline{alm}$
 $EAC = O \cdot \phi_0 \cdot \overline{alm}$
 $PAC = pac \cdot \phi_0$
 $SAC = \theta_0 \cdot O \cdot alm$
 $ENM = \phi_1 \cdot O \cdot alm$
 $SI = \phi_0 \cdot O$
 $RI = \phi_0 \cdot I \cdot (sum + sus + or + and + alm)$
 $SO = \overline{IND} \cdot \phi_0 \cdot I \cdot (sum + sus + and + or + alm) + IND \cdot \phi_0 \cdot \bar{I} \cdot \bar{O}$
 $RO = \phi_0 \cdot O$

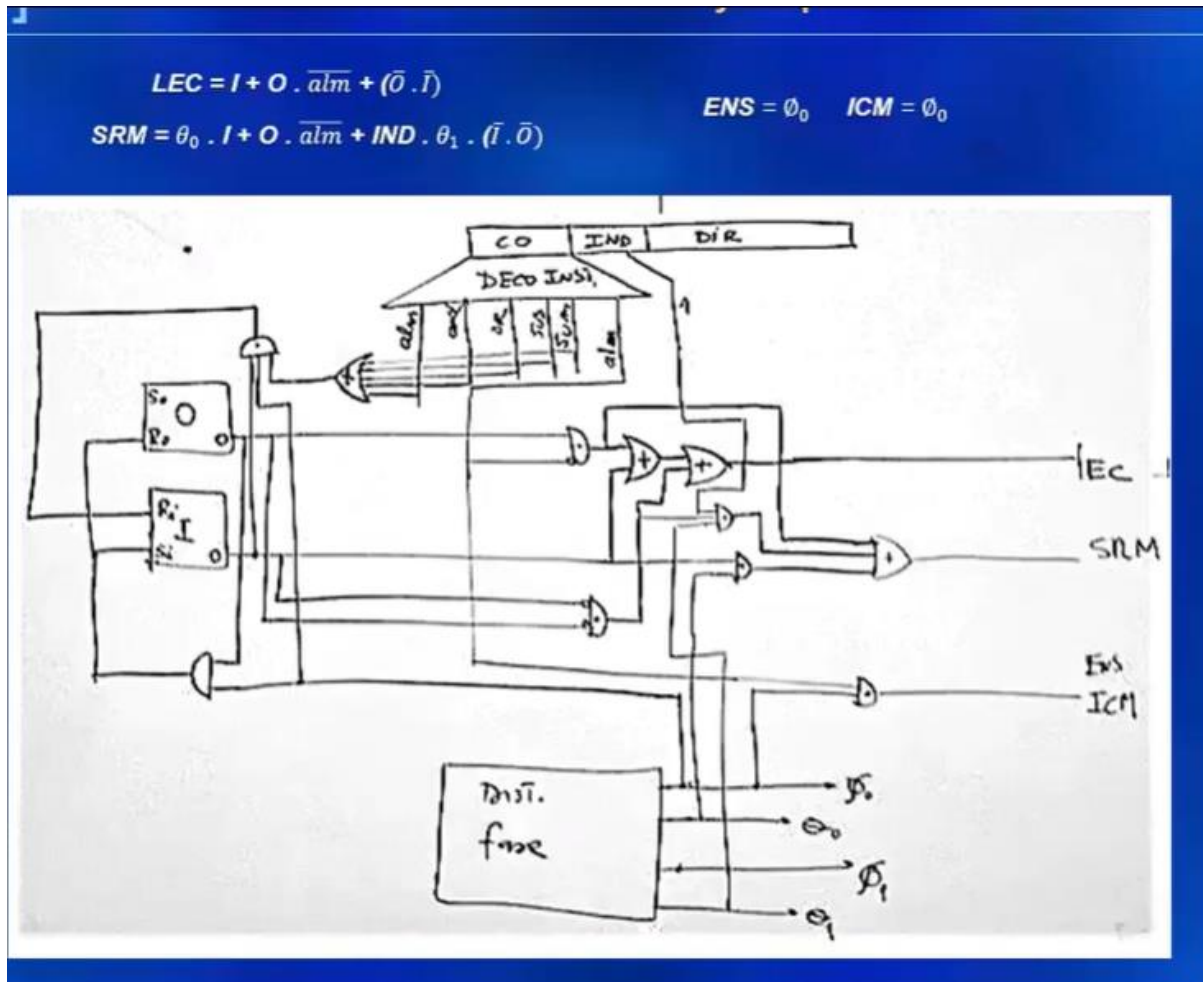
Ejemplo lectura en Logism



Ejemplo SRM



Otro ejemplo con ambos:



NOTA

¿Qué pasa con TMS? ¿Y sus cronogramas? ¿Y qué pasa en las fases que se agregan con las señales I y O? TMS se ocupa para cualquier instrucción en una modalidad que puede elegir el programador cuando quiera hacerlo por direccionamiento indirecto. Los cronogramas son idénticos a las operaciones pero agrega dos fases más para resolver la búsqueda del operando que no está en DIR 95 sino que está en M. En estas dos fases las señales I y O se encuentran muertas (0) porque no hay ni ciclo de instrucción ni ciclo de operando.

SECUENCIADOR MICROPROGRAMADO

La microprogramación consiste en reemplazar el secuenciador cableado por un secuenciador programado.

A cada instrucción del calculador (secuenciador) le corresponde un microprograma que se encuentra almacenada en una memoria especializada, llamada **memoria de control o memoria de gobierno**. El desarrollo de este microprograma genera las microórdenes que gobiernan la ejecución de la instrucción. Se llama micro-instrucciones a las instrucciones del microprograma. La ejecución de una micro-instrucción genera una o varias microórdenes. En los ordenadores microprogramados existe la posibilidad de modificar fácilmente el conjunto de instrucciones, puesto que cambiar o añadir un microprograma es mucho más sencillo que alterar el cableado

DEFINICIÓN MODELO DE WILKES

El secuenciador multiprogramado a diferencia del cableado consiste una memoria de control no volátil, que es mucho más veloz que la central (que la RAM) la cual posee dos zonas:

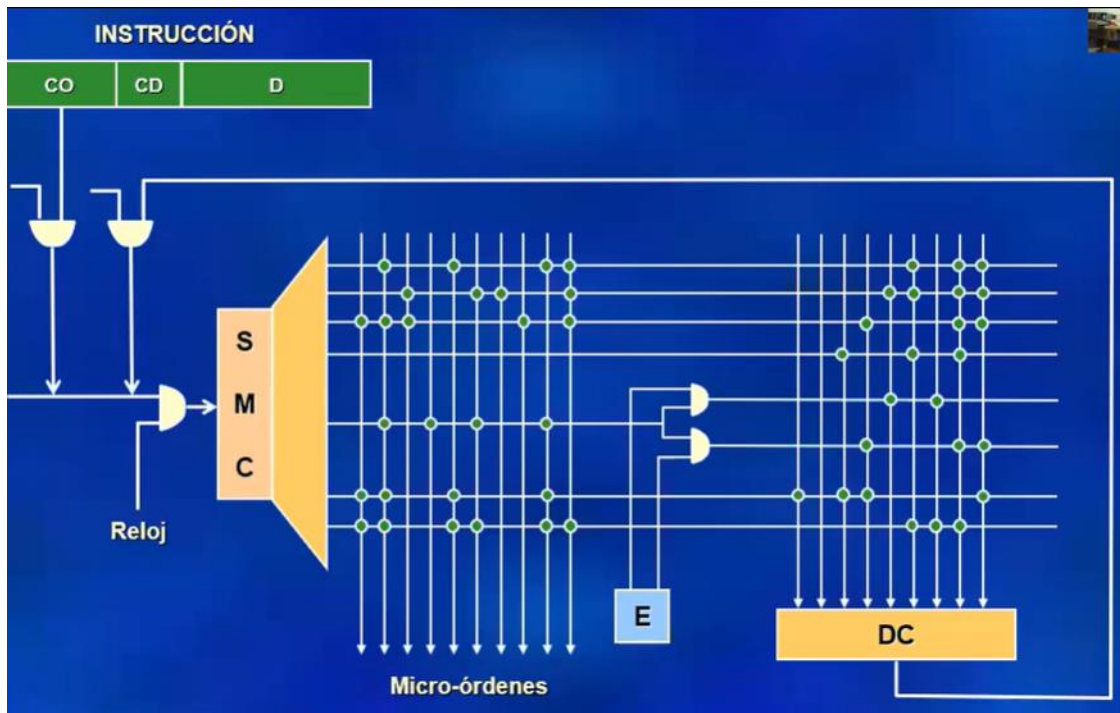
- a la izquierda se almacenan las microinstrucciones
- a la derecha la dirección de la próxima microinstrucción.

La dirección de la primer microinstrucción (es lo mismo que palabra de control) es la que proviene del código de operación (CO), luego cada microinstrucción estará relacionada con una palabra que contenga la dirección de la próxima a ejecutar; terminando el proceso cuando la dirección de la próxima instrucción dirija a una vacía.

Al no poder hacer saltos, introduce el **Biestable de Estado (E)**.

Podrá existir la posibilidad de bifurcación incrementando un biestable de estado (E) con dos compuertas AND que relaciona excepcionalmente a una microinstrucción con dos direcciones distintas de próximas microinstrucciones basado en el valor del biestable de estado. Por ejemplo, SAP, si se cumple $E=1$ indicando una dirección de salto distinta a la de $E=0$ (condición no cumplida).

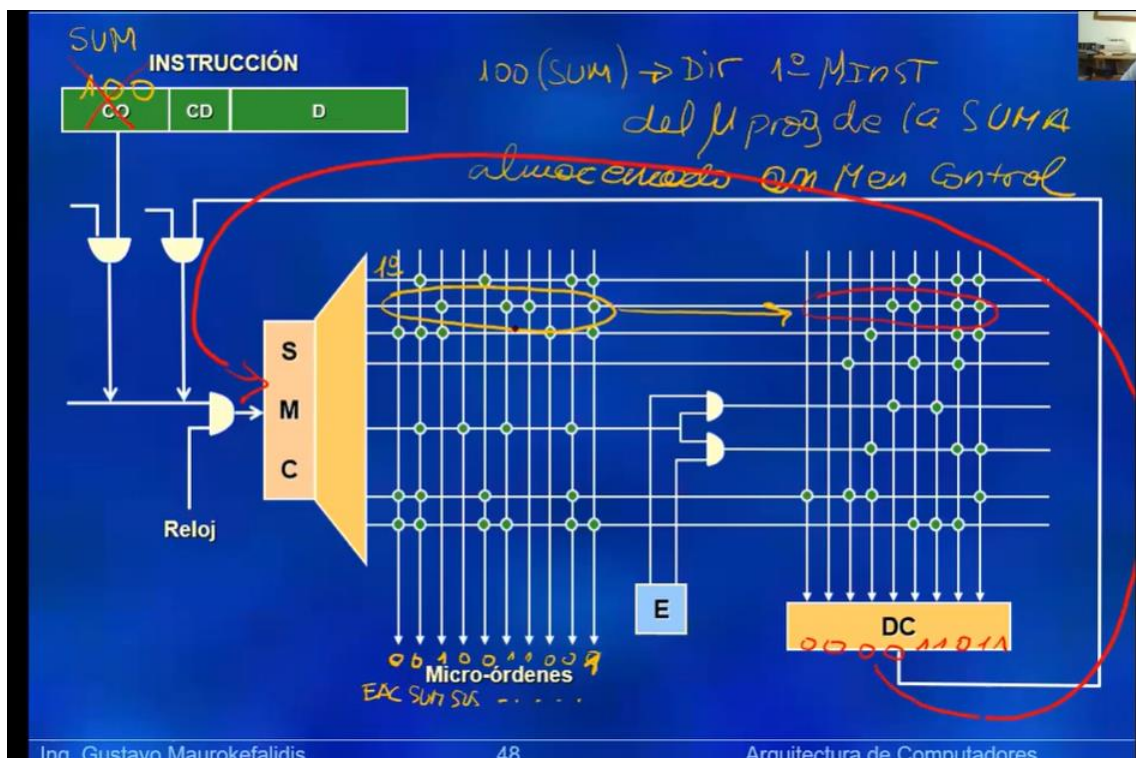
Cada **hilo** está asociado a una **señal de gobierno**.



Funcionamiento: Aclaración de términos utilizados:

- Operación = Instrucción. Por ejemplo SUM
- Microinstrucción = palabra de control. Por ejemplo: búsqueda del operando
- Microórden: señales de gobierno. ENS, SRM, etc.

Ejemplo:



La operación del salto está **cableada** y relacionada con el Biestable E, para tomar la decisión de continuar a la próxima instrucción o realizar el salto.

VENTAJAS

- Optimización de componentes
- Flexibilidad, puede probar una nueva configuración y aplicar a toda una línea de producción.
- Actualización de BIOS

DESVENTAJAS

- Longitud de la palabra depende de la cantidad de microórdenes
- Si la ruta de datos es una Abacus 32 bits, anda bien
- Tarda más porque hay acceso de memoria tiene que ser de una relación de 6 - 10 con la Memoria RAM, porque en caso de no optimizar podría la memoria de control ralentizar toda la arquitectura.

Microprogramación	Cableado
Programación es muy flexible	Rígida
Diseño reducido	Tamaño grande
Bajo consumo	Alto consumo
Diseño más económico	Caro
Generación de microórdenes más lentas	Generación más veloz
A mayor complejidad de la arquitectura, mayor tamaño de la palabra de control	

"Peor desventaja los microprogramados (Wilkes): A mayor complejidad de la arquitectura, mayor tamaño de la palabra de control."

PREGUNTA DE FINAL

<https://youtu.be/UAac3iJ3VRc?t=2221>

Relación de la memoria de wilkes con el tamaño de la arquitectura: Es ineficiente porque no prevé ninguna codificación, ninguna estrategia para comprimir. Que permita reducir de alguna manera el tamaño. Lo pone más complejo, tenes que pensar en la memoria de la siguiente micro-instrucción.

La memoria de control está relacionada con la cantidad de compuertas que gobiernan la ruta de datos.

Wilkes no presenta ninguna estrategia para comprimir esta información, para codificarla.

Cuanto más compuertas tenga la ruta de datos, más longitud tendrá la palabra en tamaño.

Lo que se plantea es codificar la información, para solucionar :

Codificación de las microinstrucciones

Características de la Codificación

- ☐ En la práctica las Unidades de Control Microprogramadas no se diseñan usando un formato de instrucción puro No Codificado u Horizontal.
- ☐ Se hace uso al menos, de algún grado de codificación para reducir el ancho de la memoria de control y simplificar las tareas de la microprogramación.

Características	
Microinstrucción no codificada	Microinstrucción muy codificada
Muchos Bits	Pocos Bits
Visión detallada del Hardware	Visión global del Hardware
Difícil de Programar	Fácil de Programar
Poca o ninguna lógica de control	Lógica de control compleja
Ejecución Rápida	Ejecución Lenta
Optimización de las Prestaciones	Optimización de la Programación
Tecnología	
Horizontal	Vertical
Desempaquetada	Empaquetada
"Hard"	"Soft"

Las microinstrucciones se pueden diferenciar

- Horizontal y Vertical.
- Empaquetadas y Desempaquetada
- Hard y Soft

Técnicas de codificación en MICROPROGRAMADOS

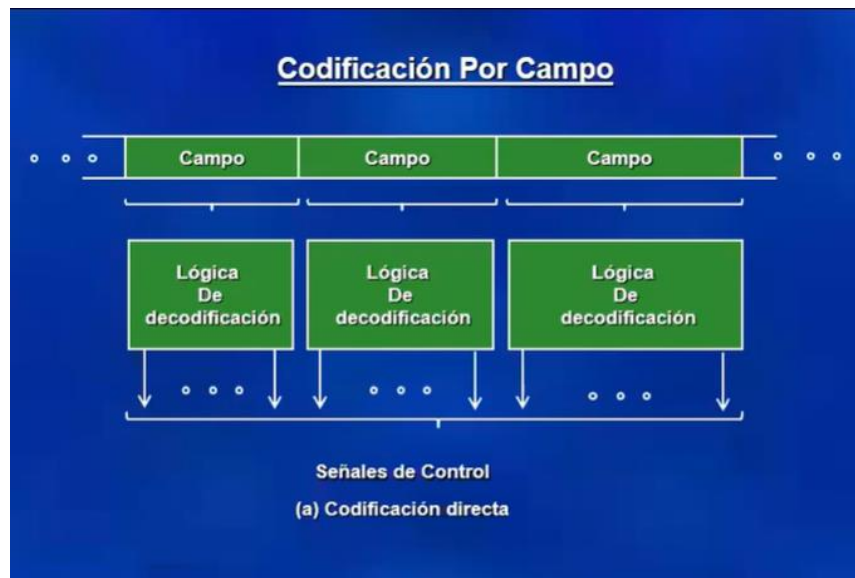
Por campo o por instrucción

Existen dos métodos extremos para pasar de la configuración binaria de la micro-instrucción a las diversas micro-órdenes correspondientes.

El primer método consiste en asociar un bit de la microinstrucción a cada micro-orden (mayor longitud de la micro-instrucción y difícil programación, permite ejecutar varias micro-órdenes simultáneas).

En el otro extremo, el segundo método consiste en codificar el conjunto de las posibles micro-órdenes con el número mínimo de bits (menor longitud de microinstrucción y mayor decodificación, se necesitaría una micro-instrucción por cada micro-orden).

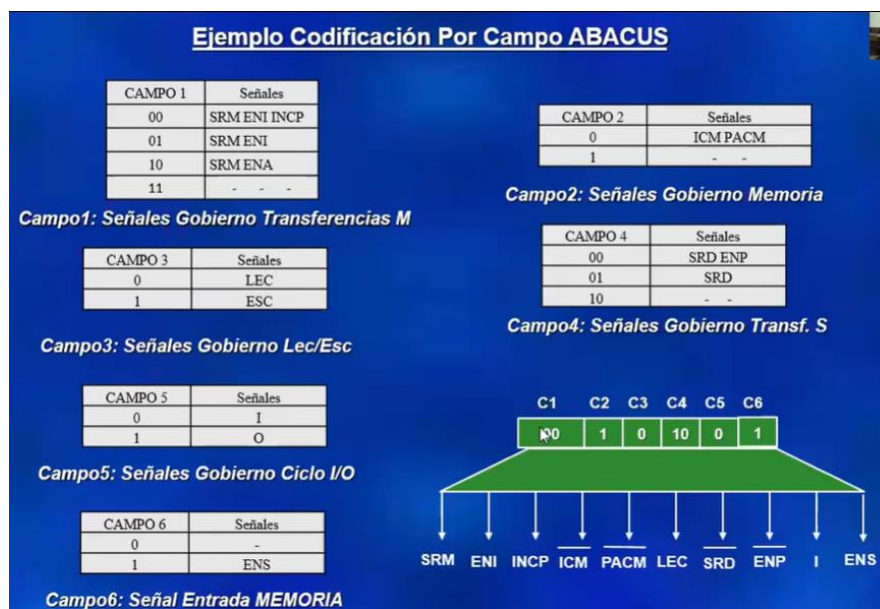
CODIFICACIÓN POR CAMPO



Se dividen las distintas **micro-órdenes en grupos independientes** de modo tal que se den micro-órdenes simultáneas en un batido de reloj.

Cada **grupo**, además, corresponde a un determinado **tipo de función**: apertura de puertas de diversos registros a un mismo bus, gobierno de una unidad funcional, gobierno de la memoria local, etc.

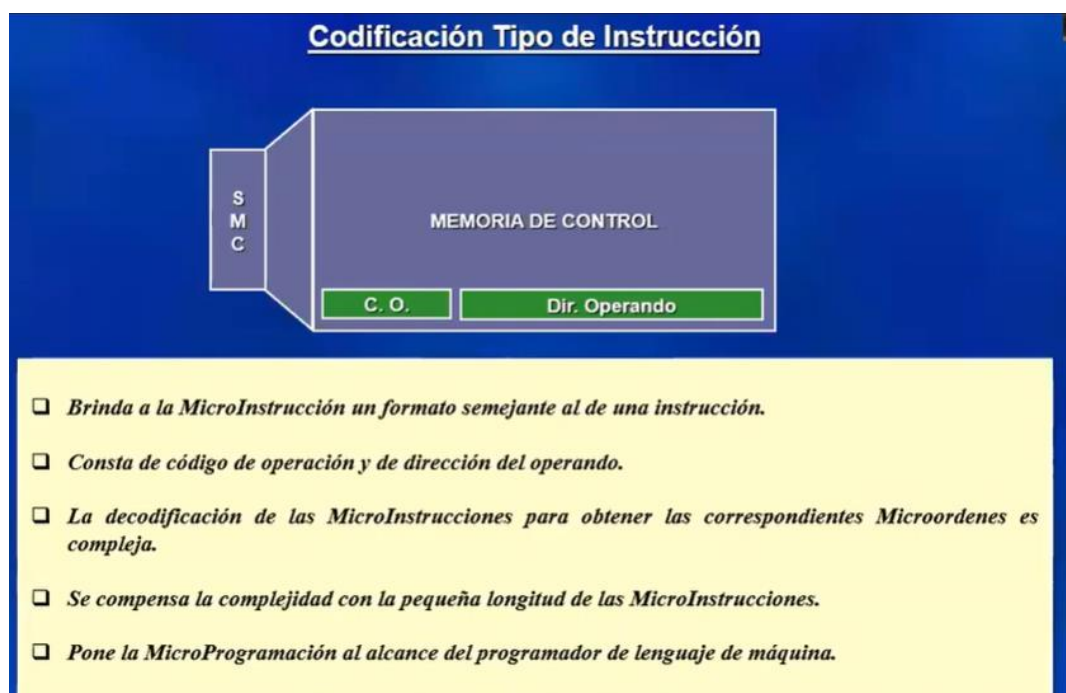
Ejemplo:



Características:

- ☐ Cuando la MicroInstrucción se ejecuta, cada campo se decodifica y genera señales de control.
- ☐ Con N Campos se definen N acciones simultaneas.
- ☐ Cada acción se traduce en la activación de una o mas señales de control.
- ☐ Se debe cuidar que cada señal de control no pueda ser activada por mas de un campo (no siempre se puede).
- ☐ Debe ser posible activar cada señal de control al menos por un campo
- ☐ Un campo individual podrá constar de L bits o sea 2^L códigos posibles donde cada uno deba codificar un patrón distinto de señales de control

CODIFICACIÓN POR TIPO DE INSTRUCCIÓN



El CO (Codigo operación) NO es SUM, RES, etc sino que es un Programa encargado de generar microordenes.

Tiene que haber un microprocesador, para poder decodificar justamente el Código de Operación al ser complejo.

VENTAJA VS POR CAMPO

- Menor el tamaño de la memoria de control
- Permite al programador NO conocer la arquitectura

DESVENTAJAS VS POR CAMPO

- Es mucho más lenta, porque hay un nivel de interpretación
- Es muy compleja en la decodificación

- Es más cara

TIPOS DE DIRECCIONAMIENTO DE LAS MICROINSTRUCCIONES

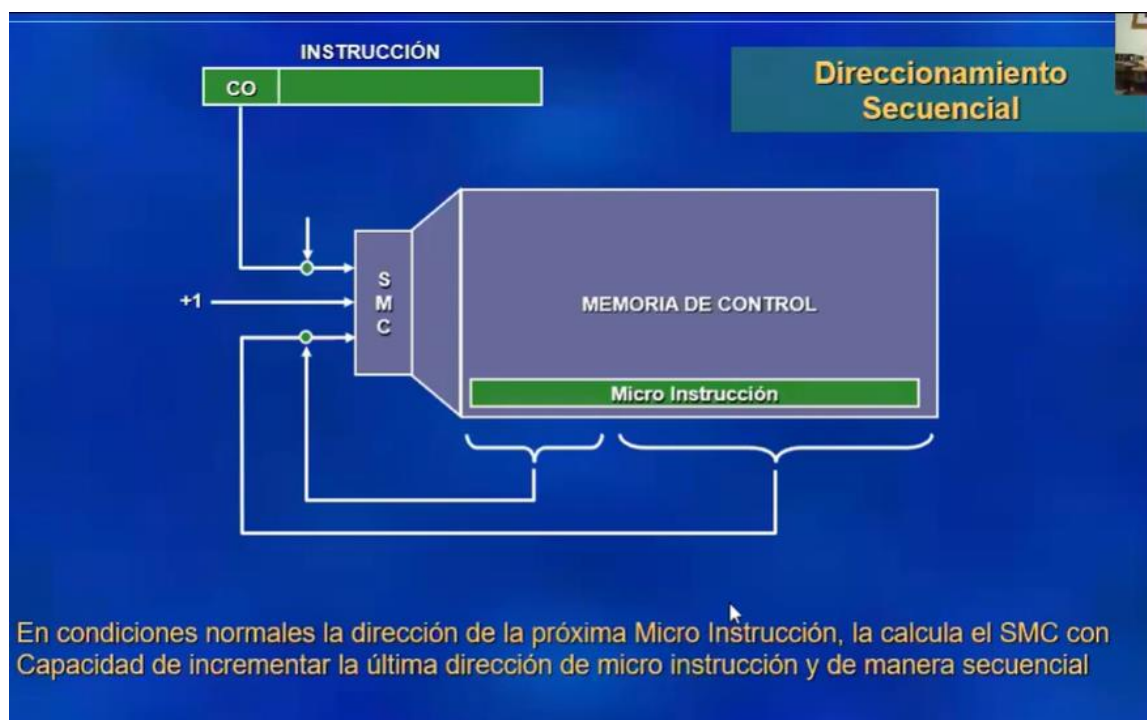
Son las técnicas de búsqueda de instrucción para microprogramados.

SMC - Tiene capacidad de incrementar la dirección de la última instrucción y de manera secuencial

Existen dos tipos:

- **Secuencial**
- **Explícitos**

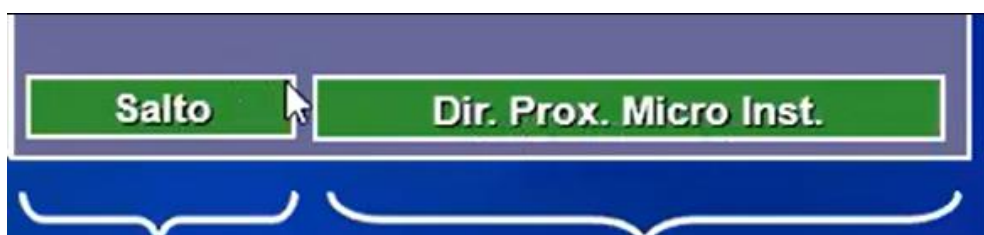
DIRECCIONAMIENTO SECUENCIAL



Continúa a la siguiente instrucción, pero el SMC es el encargado de sumar +1 de incrementar la dirección para ir a la siguiente instrucción.

Por eso la 1er Instrucción es la que te dicta donde va a arrancar la 1er instrucción.

Para el caso de un **salto**, se sacrifica 1 bit de la Microinstrucción como indicador de V o F y el resto de la palabra de control para indicar la dirección de la próxima instrucción.



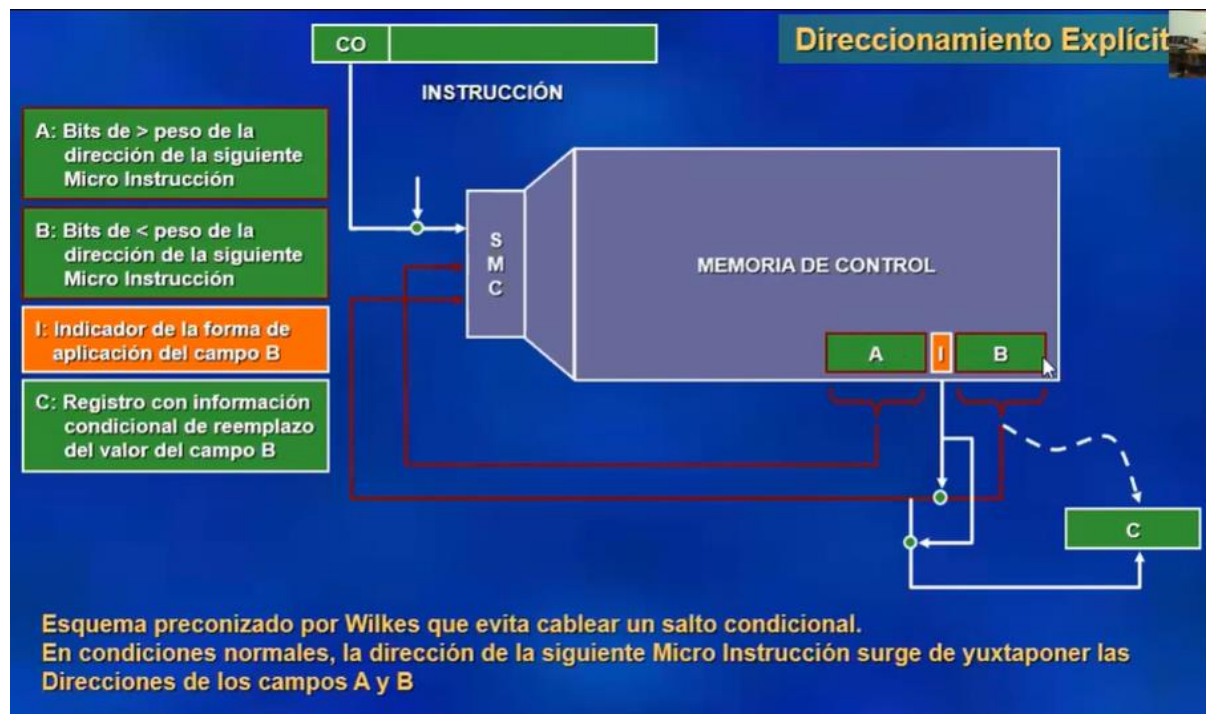
VENTAJAS DIR SECUENCIAL

- Simple de programar
- No alarga el tamaño de la microinstrucción

DESVENTAJAS DIR SECUENCIAL

- Para una 8086 que tienen muchos saltos, el esquema no es nada óptimo porque malgasta una instrucción para el salto por lo tanto no es bueno.
- perdes un ciclo de memoria para indicar un salto

DIRECCIONAMIENTO EXPLÍCITO



En un salto condicional, SI $I=0$ el campo B podrá contener los bits de menor peso de la dirección de salto si esta fuese conocida. Si $I=1$ B contiene la dirección de un registro C que almacena los bits de menor peso de la dirección de salto.

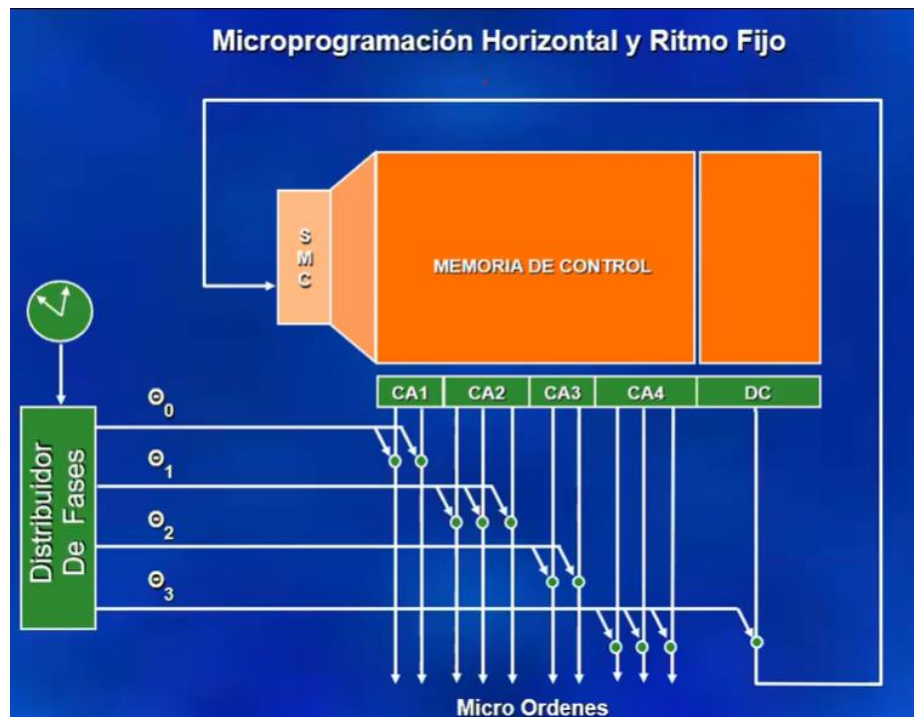
En un salto condicional con condición NO cumplida, SI $I = 0$ es el campo B, A se concatena con B, para dar resultante la siguiente dirección.

SI $I = 1$, salto condicional con condición cumplida, A se concatena con C, y para ir a la dirección del salto a donde querés ir.

Acompasamiento

Es con qué ritmo se gobierna la salida de las micrordenes

MICROPROGRAMACIÓN HORIZONTAL Y RITMO FIJO



Gobernada por un distribuidor de fases, es codificación por campo.

El distribuidor de fases es el que va gobernando la salida, la cual debe ser decodificada.

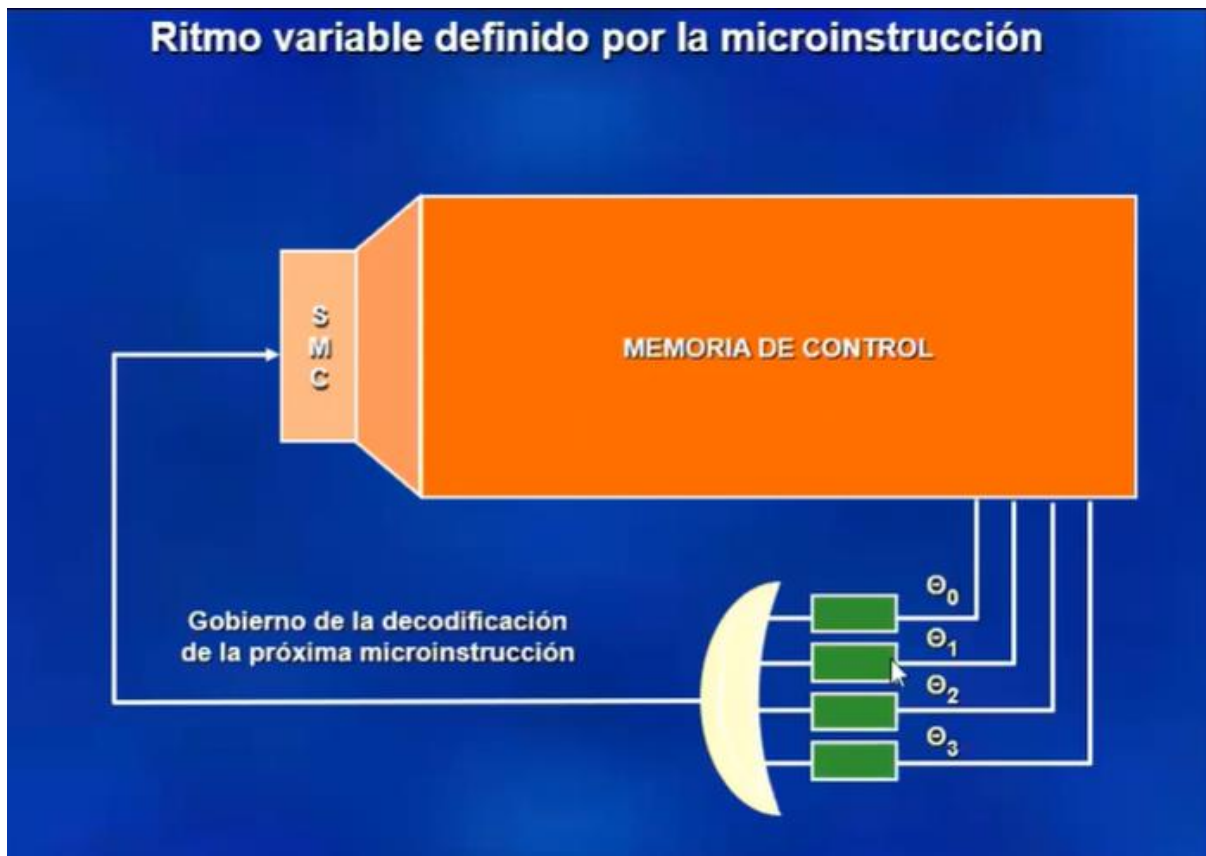
La última microorden de la próxima dirección.

Tendrá tantas fases como campos tenga.

Ejemplo: Tiene 4 campos, por lo tanto tiene 4 fases. Tamaño total de campos de la palabra - 1.

La última fase se encarga de ejecutar la dirección de la siguiente instrucción

MICROPROGRAMACIÓN RITMO VARIABLE



Ambos métodos de dirección no se pueden comparar porque son de naturaleza distinta.

CISC y RISC

LINK

<https://www.youtube.com/watch?v=e10fMEpXHbk&list=PLgTKn0MhANLTPffShcV9I2INbaxlPZvLI&index=23>



CISC - Computador con Set de Instrucciones Complejas

Computador con Set de Instrucciones Complejas CISC (Complex instruction set computing)

- ☐ La arquitectura CISC se caracteriza por tener un repertorio de instrucciones complejas
- ☐ Muchas instrucciones potentes para lograr una implementación mas directa con Lenguajes de Alto Nivel.
- ☐ La ejecución de estas instrucciones puede ser bastante complejas
- ☐ Considera que la compilación de programas de alto nivel requiere muchas menos instrucciones de máquina y con tiempos mas cortos.
- ☐ En la práctica lograr elevadas velocidades de ejecución se torna difícil, requiere chips con superficie elevada de silicio, y el repertorio de instrucciones dificultan la generación de códigos óptimos por parte de los Compiladores.

Notas

- Acá era necesario CISC para poder manejar ventanas por ejemplo. Porque se deberían manejar intersecciones de ventanas, procesos en Background, etc
- *Una máquina sin assembler es como un auto sin volante*
- CISC tiene un montón de registros de propósito general.
- Programar CISC es muchísimo más sencillo para programar que un RISC. Las instrucciones están muy cercanas a los lenguajes de alto nivel.
- Monopoliza la CPU en la ejecución de cada instrucción.
- Necesita que la RAM esté fuera
- Ejemplo, la SUMA consume en todo su proceso a la LECTURA
- Hay una cabecera, dentro de la instrucción, para poder adaptar por ejemplo, el formato de la instrucción, por eso los procesadores actuales, pueden ejecutar una instrucción de una instrucción de 16bits por ejemplo, no así en su compilación. Pero esto tiene la contra, de que el compilador tiene que poder interpretar este cambio, y tiene un consumo de procesamiento.

Criterios para definir una Arquitectura CISC

- ☐ Cantidad de instrucciones en lenguaje máquina (tan grande como sea posible)
- ☐ Cantidad de modos de direccionamiento (tan grande como sea posible).
- ☐ Cantidad de formatos de instrucción (tan grande como sea posible).
- ☐ Muchas instrucciones requieren más de un ciclo para su ejecución.
- ☐ Varios tipos de instrucciones tienen acceso a memoria.
- ☐ Existencia de registros de propósito específico.
- ☐ Control microprogramado.
- ☐ Instrucciones de máquina de un relativo alto-nivel (cercano al alto nivel de las sentencias de los lenguajes de alto nivel).
- ☐ Las instrucciones compuestas son decodificadas internamente y ejecutadas con una serie de microinstrucciones almacenadas en una ROM interna.
- ☐ La microprogramación es una característica importante y esencial de casi todas las arquitecturas CISC. significa que cada instrucción de máquina es interpretada por un microprograma localizado en una memoria.

PREGUNTA DE FINAL

Control microprogramado - por qué motivo una arq CISC utiliza esto? Porque tiene muchas instrucciones → necesita mucha más lógica. Se utilizarían demasiados cables si fuera cableada.

RISC - Reduced instruction set computing

Computador con Set de Instrucciones Reducidas RISC

(Reduced instruction set computing)

- ☐ La arquitectura RISC se caracteriza por tener un repertorio de instrucciones simples.
- ☐ Se necesitan muchas más instrucciones para realizar la misma tarea que una instrucción CISC.
- ☐ A pesar de la cantidad necesaria, son muy convenientes para una ejecución en un cauce Segmentado, logrando velocidades elevadas de ejecución.
- ☐ Su repertorio es utilizado convenientemente por compiladores optimizadores.
- ☐ Ventaja relacionada con la tecnología de fabricación VLSI (Very Long Scale Integration) ya que necesitan menos área para control y procesamiento de secuencia de instrucciones (mayor espacio para registros y cache).
- ☐ Reducción de acceso a datos e instrucciones por fuera del chip logran mejores prestaciones.
- ☐ Unidad de Control Cableada debido a la sencillez de la Arquitectura, e implementación de operaciones por hardware.

Criterios para definir una Arquitectura RISC

- ☐ Analizar el objeto sobre el cual se desarrollan las aplicaciones a fin de determinar cuáles son las operaciones más frecuentes.
- ☐ Optimizar los caminos que deben recorrer los datos para ejecutar las operaciones o instrucciones (anteriores) tan rápido como sea posible.
- ☐ Incluir otras instrucciones solo si forman parte de los caminos optimizados previamente y si son de relativa frecuencia, y si su inclusión no hace más lenta la ejecución de las instrucciones más frecuentes.
- ☐ Aplicar una estrategia similar a los otros recursos del procesador. Incluir un recurso solo si éste está justificado por su frecuencia de uso, y su inclusión no ententece a otros recursos más utilizados.
- ☐ Tratar de trasladar lo más que se pueda la complejidad en tiempos de ejecución al momento de compilación, recargando el software de compilación y liberando al hardware de ejecución.

Características comunes en una Arquitectura RISC

- ☐ Muchas de las instrucciones se ejecutan en un solo ciclo de máquina.
- ☐ Conjunto de instrucciones de cargar/almacenar (load/store). Es decir, se accede a memoria exclusivamente mediante las instrucciones load y store, el resto de las instrucciones realizan sus operaciones entre registros.
- ☐ Decodificación de instrucciones hardwired (en oposición a la técnica de microprograma).
- ☐ Existen relativamente pocas instrucciones y modos de direccionamiento
- ☐ Todas las instrucciones deberían tener la misma longitud para facilitar la tarea de decodificación y homogeneizar los tiempos de carga de las mismas.

Características comunes en una Arquitectura RISC

- ☐ La complejidad se ha desplazado hacia los compiladores optimizados.
- ☐ Existe un alto grado de pipeline en los caminos de los datos para obtener mucha concurrencia.
- ☐ Gran cantidad de registros (windowed o no-windowed).
- ☐ Muchos niveles de jerarquías de memoria.
- ☐ Conjunto de instrucciones diseñado para determinada clase de aplicaciones.
- ☐ Soporte de lenguajes de alto nivel (Esto último debido a que como hay pocas instrucciones sobraría lugar en el chip para colocar elementos que ayuden a lenguajes de alto nivel, como manejo de listas, stacks, etc.).

NOTAS DE RISC

- Repertorio de instrucciones Simple y reducido
- Muchas instrucciones de bajo nivel
- **Mejora la segmentación, paralelismo o pipeline**, por lo que no monopoliza la CPU, aprovechando el ciclo para realizar múltiples acciones: ej. de SUMA y puede el mismo ciclo buscar la siguiente instrucción
- Desechaba el concepto de la microprogramación, y se mandaba por el camino del cableado (al tener pocas instrucciones)
- Puede lograr una compilación mucho más óptima, debido a que existe más granularidad en el set instrucciones. Su repertorio es usado por los compiladores optimizadores (analiza el código y ve la posibilidad de reducirlo)
- RAM integrada, necesita estar cerca de la RAM, por el intercambio constante
- Es muy importante poder realizar operaciones entre registros, para tratar de evitar tratar de ir a la RAM.
- Muy pocas cantidad de opciones de direccionamiento
- Hay que estudiar bien el negocio, para determinar qué operaciones van a ser más frecuentes y tratar de reducir
- Frecuencia de utilización es muy importante
- No quiero perder tiempo a nivel de Hardware, recarga al compilador con el peso del procesamiento, para justamente no impactar en la ejecución en el Hardware
- Registros de longitud fija, hace que la decodificación sea mucho más efectivo

Comparación

CISC	RISC
Énfasis en el Hardware (Velocidad)	Énfasis en el Software (Sencillez y rapidez en el Pipeline)
Incluye instrucciones complejas multi-reloj	Instrucciones con reloj simple
Instrucciones complejas (Más de 1000)	Instrucciones sencillas (Menos de 100)
Instrucciones que trabajan de memoria a memoria	Instrucciones que trabajan de registro a registro
Carga y almacenamiento incorporados en la misma instrucción	La carga y el almacenamiento son instrucciones independientes
El tamaño del código es pequeño	El tamaño del código es muy grande
Muchos ciclos de reloj por segundo	Pocos ciclos de reloj por segundo
Pocos registros para almacenar los resultados	Muchos registros para almacenar resultados
Alto costo de producción	Bajo costo de producción

CISC

- Palabra memoria a palabra memoria
- Mucha disipación del calor por el silicio

RISC

- Registro a registro
- Carga y almacenamiento a parte, para no monopolizar la CPU
- Muchos registros para almacenar los resultados, para minimizar ir a la RAM
- Unidad de control cableada es lo caro y complejo

Característica	CISC	RISC
Memoria de control	Sí	No
Set de instrucciones	Grande	Reducido
Ciclos de reloj por instrucción	No existe un número definido	Uno o dos
Accesos a MP mediante microinstrucciones	Sí	No, utiliza registros auxiliares. (excepto LOAD – STORE)
PIPE LINE	Ineficiente	Eficiente
Compiladores	Simple	Complejos
Permite modularidad y el crecimiento del set de instrucciones	Sí	No
Formato de Instrucciones	Variable	Fijo
Implementación de paralelismo en la ejecución de las instrucciones	Compleja	Simple
Velocidad de procesamiento	Reducida por los accesos a memoria de control	Optimizada por el paralelismo de ejecución
Predicción de bifurcaciones	No	Sí

Conclusiones

- ☐ En la actualidad se han visto las ventajas de ambas arquitecturas y por tal motivo la convergencia RISC-CISC es cada vez más vista en los nuevos procesadores.
 - ☐ Los Procesadores CISC incluyen nuevas técnicas de Pipeline para acelerar el número de instrucciones por segundo.
 - ☐ Los Procesadores RISC incluye cada vez más instrucciones para acelerar aún más los programas en hardware.
 - ☐ Actualmente la mayoría de los sistemas CISC de alto rendimiento implementan un sistema que convierte dichas instrucciones complejas en varias instrucciones simples del tipo RISC, llamadas generalmente microinstrucciones. Algunos ejemplos de ellos son: Motorola 68000, Zilog Z80 y toda la familia Intel x86 usada en la mayoría de las computadoras personales.
-
- ☐ Actualmente CISC va de la mano con procesadores de la familia Intel x86o AMD x86-64
 - ☐ RISC actualmente ha evolucionado a la arquitectura ARM (Advanced RISC Machine)
 - ☐ CISC debido a su complejidad, es altamente usado en ordenadores de escritorio
 - ☐ ARM debido a su bajo nivel de componentes, sencillez de su set de instrucciones, y bajo consumo, es ideal en dispositivos móviles
 - ☐ APPLE COMPUTERS esta trabajando para implementar ARM es su nueva línea de computadoras, abandonando a la familia de procesadores intel

LENGUAJE DE MÁQUINA

Link del video tio hostias: <https://youtu.be/e10fMEpXHbk?t=3810>

- **Concepto**
- **Relación con el Computador y la Memoria**
- **Conjunto de Instrucciones**
- **Programa Ilustrativo**

CONCEPTO

Lenguaje de máquina es el sistema de códigos directamente interpretable por un circuito microprogramado, como el microprocesador de un ordenador o el microcontrolador de un autómata.

Este lenguaje está compuesto por un conjunto de instrucciones que determinan acciones a ser tomadas por la máquina.

Un programa en Lenguaje de Máquina consiste en una cadena de estas instrucciones (mas datos y operandos).

El lenguaje de máquina es específico de cada máquina o arquitectura, aunque el conjunto de instrucciones disponibles pueda ser similar entre ellas.

Definición del profe

El lenguaje de maquina es un sistema de códigos definidos para una arquitectura (interpretable por ella) que permite a través de una instrucción de maquina debidamente codificada, solicitar a los circuitos lógicos intervinientes el desarrollo de un microprograma, entendiendo a este último como el conjunto de señales de gobierno impulsionales y de nivel, necesarias para ejecutar a la instrucción en curso.

Elementos informativos de una instrucción

- A. *La naturaleza de la Operación por realizar*
- B. *Datos y Operandos implicados en la Instrucción*
- C. *Lugar donde deba almacenarse el resultado*
- D. *La dirección de la Próxima Instrucción por ejecutar*

Son los 4 elementos que componen a una instrucción.

Clasificación de las arquitecturas

Arquitectura de cuatro direcciones :



Arquitectura de tres direcciones :



Arquitectura de dos direcciones :



Arquitectura de una direccion :



4 Dirección:

- DPI - Dirección de la próxima instrucción, no hay un contador de programa, se lleva a través de instrucciones

3 direcciones:

- DR - Direccion del resultado, por ejemplo una SUMA d1 con d2 se almacena en dr
- Instrucciones de memoria a memoria, no de memoria a registro

2 direcciones:

- Típico caso de ALU con dos entradas
- Ejemplo SUMA, contenido de d1 con contenido de D2, vaya a D1. Porque a no darle direccion del resultado, entonces se sobreentiende que vaya a D1.

1 dirección: (EL ABACUS PE)

- Abacus
- Ejemplo del AC, se pierde el valor del AC', porque se sobrescribe con el resultado de la operación con el contenido de D2.

Conjunto de instrucciones de un microprocesador

- Desde una decena hasta un centenar de Instrucciones de máquina dependiendo de la arquitectura
 - Suma, complementación, borrado, almacenamiento, y desplazamiento son suficientes para realizar todas las Operaciones.
 - cablear o Microprogramar operaciones incrementa la productividad, pero aumenta costo de fabricación y producción
- Dependiendo de la arquitectura, RISC o CISC va a variar la cantidad de instrucciones
- Saltos condicionales
 - Esto de cableado o microprogramado, es lo que se habló en la unidad anterior

Nivel de microprogramación según la arquitectura

Cuadro referencial:

	Calculadores de Despacho	Computadores de Gestión	Mini ordenadores	Grandes Ordenadores
Suma/Sustracción punto fijo Binaria				
Multiplicación División Fija Binaria				
Aritmética Fija Decimal				
Aritmética Flotante Binaria				
Aritmética Flotante Decimal				
Raíz Cuadrada Decimal				
Funciones Clásicas Log, Sen, Exp, etc.				
Funciones Especiales				

Leyenda

 Siempre Cableado o Microprogramado
  Eventualmente Cableado o Microprogramado
  Per Programa

Tipos de instrucciones - Formato (Convención)

	<i>Símbolo</i>	<i>Significado</i>
Memoria	M	Dirección palabra de Memoria
	M+1	Dirección de la siguiente palabra de memoria
	(M)	Contenido de la palabra de dirección M
	(M,M+1)	Contenido de las dos palabras consecutivas de memoria de dirección M y M+1
Registro	R	Dirección de un Registro
	R+1	Dirección del registro siguiente
	(R)	Contenido del registro de dirección R
	(R,R+1)	Contenido de los dos registros consecutivos de dirección R y R+1
	→	Situado entre dos miembros, indica que la información almacenada en el primer miembro es almacenada en el elemento M o R del segundo miembro

Instrucciones de Transferencia				
Carga	CAR	R	M	$(M) \rightarrow R$
Carga Complem.	CARC	R	M	$\text{Comp}(M) \rightarrow R$
Almacenamiento	ALM	R	M	$(R) \rightarrow M$
Transferencia	TRA	R_i	R_j	$(R_i) \rightarrow R_j$
Intercambio	INT	R_i	R_j	$(R_i) \rightarrow R_j \text{ y } (R_j) \rightarrow R_i$
Longitud Simple Instrucciones Aritméticas en Punto Fijo Binaria				
Suma	SUM	R	M	$(R) + (M) \rightarrow R$
Sustracción	SUS	R	M	$(R) - (M) \rightarrow R$
Multipliación	MUL	R	M	$(R) \times (M) \rightarrow R, R+1$
División	DIV	R	M	$(R, R+1) / (M) \rightarrow R$

Longitud Doble		Instrucciones Aritméticas en Punto Fijo Binaria		
Suma	SUM2	R	M	$(R, R+1) + (M, M+1) \rightarrow R, R+1$
Sustracción	SUS2	R	M	$(R, R+1) - (M, M+1) \rightarrow R, R+1$

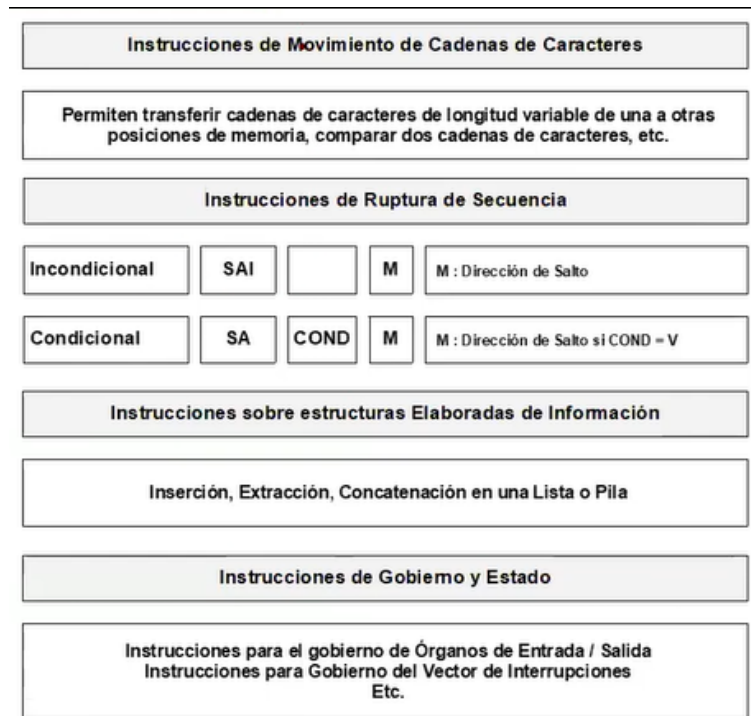
Longitud Simple		Instrucciones Aritméticas en Punto Flotante		
Suma	SUMF1	R	M	$(R) + (M) \rightarrow R$
Normalización	NORM1	R		Normaliza contenido Reg. R

Instrucciones Lógicas				
Complementación	COM	R		$\text{Comp}(R) \rightarrow R$
Producto Lógico	AND	R	M	$(R) \text{ AND } (M) \rightarrow R$
Suma Lógica	OR	R	M	$(R) \text{ OR } (M) \rightarrow R$

Instrucciones de Desplazamiento			
DESD	R	n	Desplazamiento Derecha de R, n Posiciones
DESI	R	n	Desplazamiento Izquierda de R, n Posiciones

Instrucciones Aritméticas Decimales Fijas				
Suma (paralela)	DSUM	R	M	R : N° de caracteres o palabras 2° Operando M : Dirección último carácter o palabra
Suma (serie)	DSUM	M1	M2	M1 y M2 : Direcciones de Fin de Operando

Instrucciones de Conversión	
Permiten convertir valores numéricos de un formato a otro, por ejemplo de Punto Fijo Binario a Decimal	

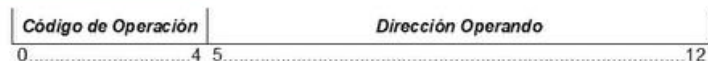


Ejemplo en formato abacus (pueden tomar algo así en la práctica)

<https://youtu.be/e10fMEpXHbk?t=5212>

Dado el siguiente formato de registro de instrucción y el set de instrucciones correspondiente, diseñar un programa en lenguaje de máquina que almacene el contenido de la dirección A en la dirección C, si (A)>(B); contrariamente lo almacene en la dirección B.

• Formato de la Instrucción :



• Inicialización de Memoria :

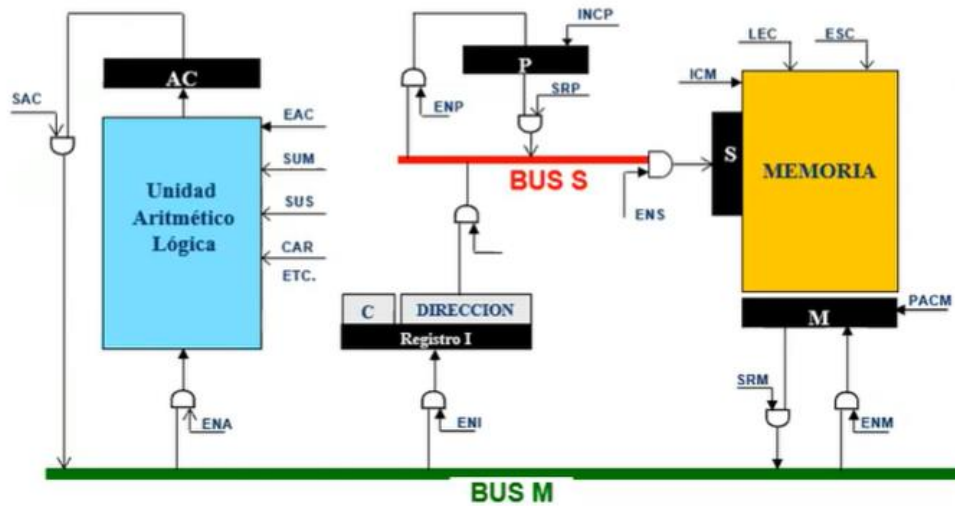
A = 0110 1100 = 6C	B = 0110 1110 = 6E	C = 0111 1010 = 7A
--------------------	--------------------	--------------------

• Set de Instrucciones :

Código	Descripción
00000	Fin
00001	Cargar AC
00011	Incrementar (AC)
00100	Decrementar (AC)
00101	Almacenar (AC)
00110	Salto si (AC) = 0
00111	Salto si (AC) < 0
01000	Salto si (AC) > 0

Código	Descripción
01001	Salto Incondicional
01010	Sumar al (AC)
01011	Restar al (AC)
01100	Multiplicar al (AC)
01101	Dividir con el (AC)
01110	XOR con (AC)
01111	Complementar (AC)
10000	Poner a cero AC

Dirección Memoria	Código Operación	Dirección	Comentarios
00000001	00001	0110 1100	$AC \leftarrow (A)$
00000010	01011	0110 1110	$AC \leftarrow (Ac) - (B)$
00000011	01000	0000 0111	Salto a Dir 7(h) si $(AC) > 0$
00000100	00001	0110 1100	$AC \leftarrow (A)$
00000101	00101	0110 1110	$B \leftarrow (Ac)$
00000110	00000	Xxxx xxxx	Fin
00000111	00001	0110 1100	$AC \leftarrow (A)$
00001000	00101	0111 1010	$C \leftarrow (Ac)$
00001001	01001	0000 0110	Salto incondicional a dir. 6(h)

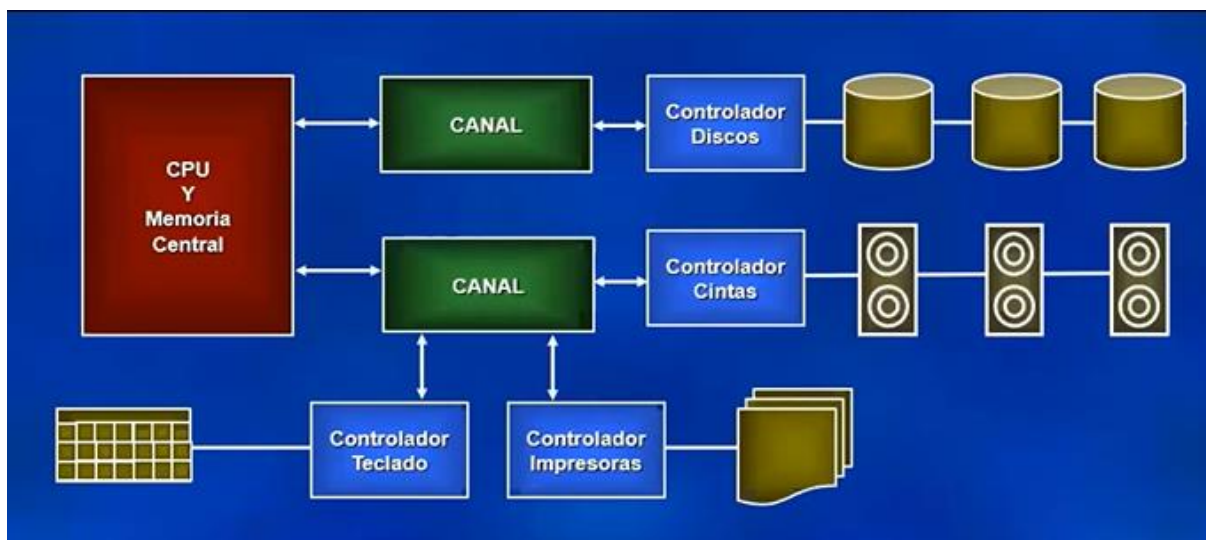


Canales

Link del video:

https://www.youtube.com/watch?v=7_WCPcLicQs&list=PLgTKn0MhANLTPffShcV9I2INbaxlPZvLI&index=23

Esquema general



CONCEPTO DE CANAL

Órgano encargado de establecer un vínculo entre la unidad central de proceso y los (conjunto de dispositivos externos) **periféricos** a través de sus controladores.

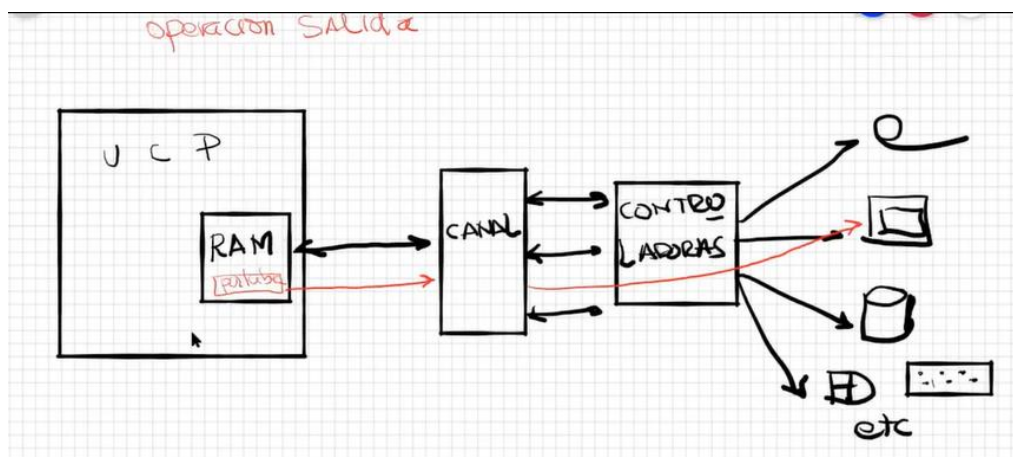
El CANAL está conectado a un CONTROLADOR, no directamente al dispositivo externo.
El canal gestiona las transferencias de información entre la RAM y los periféricos

Vincularse con la memoria tratando de que esta no quede atrapada por el canal, y a través de diferentes técnicas, proveerles las palabras para enviárselas al periférico correspondiente.

Tratando de impactar lo menos posible en el CPU.

CONTROLADOR

Un controlador maneja una familia de periféricos



Operación de salida

Implica que una palabra sea capturada por el canal y este establece la conexión con la controladora correspondiente.

- **SELECCIÓN DEL PERIFÉRICO**
 - Pasa por un proceso de prueba de estado
 - Gobierno de las operaciones de entrada salida



Controladoras (es la que está en la mother)

Se encarga de entender pedidos del canal, por otro lado gobierna los periféricos (presentarla en el formato que necesita el periférico).

Ejemplo: Controladora de Pantallas, tiene que poder manejar HDMI, VGA, DisplayPort, etc
Entiende y dialoga con el HW del periférico
Dialoga con el CANAL,

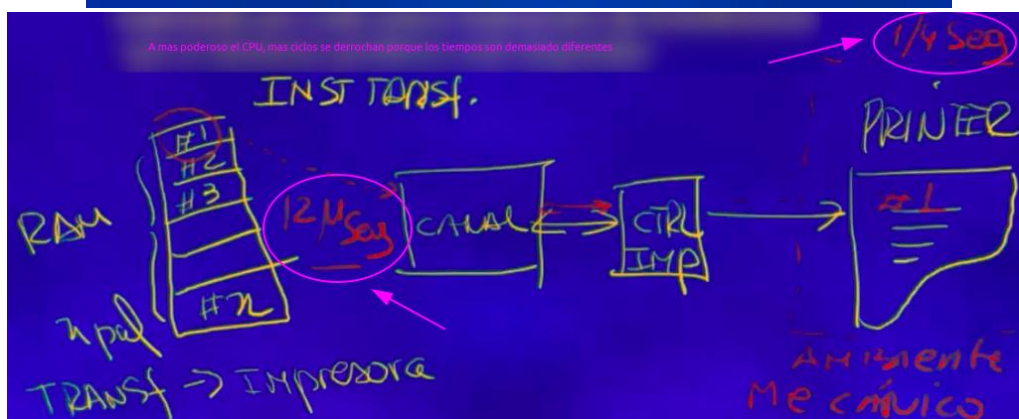
El periférico, por ejemplo Mouse, mete una interrupción y consume un ciclo de CPU correspondiente para realizar la acción que se está solicitando.

Modos - Simultaneidad de los procesos de E/S

Técnica para posibilitar al usuario final no vea degradada la performance, cuando se realizan las acciones de E/S por parte de los periféricos.

1. Modo bloqueado → técnica muy rudimentaria

El computador ejecuta el programa de entrada – salida esperando por cada nueva instrucción de transferencia que el elemento periférico esté disponible



PRINTER es un ambiente mecánico, a la impresora le puede llevar mucho tiempo imprimir la primera palabra #1

La **CPU** hasta que la impresora termine de imprimir esa palabra, queda **BLOQUEADA**. La CPU malgastaba tiempo, esperando hasta que la impresora terminara su tarea.

Malgasta ciclos y derrocha tiempo. Se bloquea el procesador debido a la asimetría entre los tiempos de la UC y los periféricos.

Queda a la espera de una nueva transferencia.

2. Modo por prueba de estado

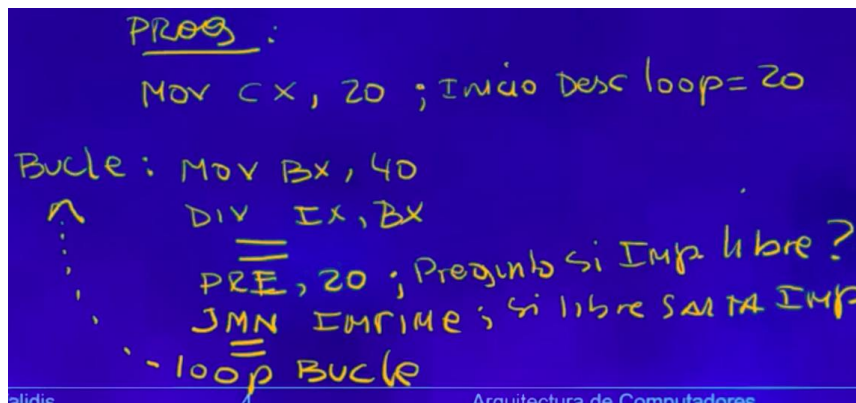
Influir en la arquitectura una instrucción PRE (Prueba de estado)

**Inclusión de puntos de prueba en el programa con el objeto
De consultar el estado del periférico antes de realizar la
Transferencia elemental**

En algún punto del programa, pregunta si el periférico “está libre”, si estaba disponible, ejecutaba la rutina de impresión, sino **SE SALTEABA** dicha rutina.

Con el objeto de introducir una cierta simultaneidad, parece lógico no ordenar la operación de transferencia de una palabra hasta que la unidad periférica este presta a ejecutarla completamente. Para ello, se incluye en el programa, con una periodicidad adecuada, puntos de prueba para conocer si la unidad periférica estaba presta a realizar la operación. Esta técnica, muy enojosa para el programador, no puede hacer frente más que a necesidades muy particulares.

“El programador tiene la facultad de preguntar por el estado del periférico antes de usarlo. Se trata de una instrucción de prueba de estado del periférico que permite al programador la posibilidad de preguntar por el estado del periférico antes de transmitir. Es como un salto condicional pero por estado del periférico libre. El problema de este modo es que la actitud del programa depende del tiempo de respuesta del periférico.”



```

PROG:
  MOV CX, 20 ; Inicio Desc loop=20

Bucle: MOV BX, 40
      DIV IX, BX
      PRE, 20 ; Pregunto si Imp libre?
      JMN IMPRIME ; si libre SALTA IMP
      -loop Bucle
  
```

Acá se tiene el Bucle de 20 veces, por la primer instrucción.

VENTAJA

Se logra evitar el modo bloqueado.

DESVENTAJA

- Tira del lado del programador la racionalización del uso de la CPU.
- Problema de que los periféricos iban mejorando en los tiempos de respuestas y los programas quedaban obsoletos.

Transferencia elemental

Transferencia de palabra/caracter al periférico

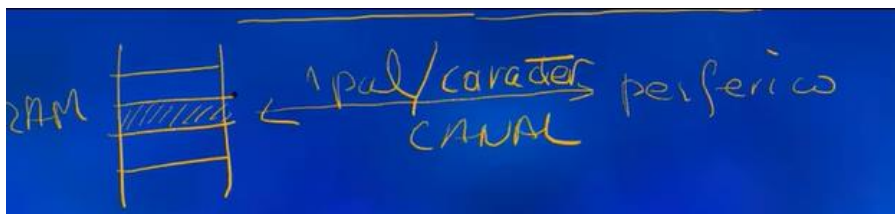
3. Modo por interrupción de programa

**Son los periféricos los que interrumpen para avisar al computador que se encuentran disponibles.
La interrupción detiene momentáneamente el programa en curso
Para permitir que el programa de transferencia sea ejecutado con prioridad**

Devuelve el registro (AX) con el ID del dispositivo que interrumpió. Ej. 20 es de teclado
Más óptimo. Es el periférico el que interrumpe el programa

En este procedimiento, al calculador se le descarga de la vigilancia periódica de los periféricos y son estos los que lo interrumpen para avisarle que están disponibles. La interrupción permite detener momentáneamente el programa en curso para permitir que el programa de transferencia sea ejecutado con prioridad. **El modo por interrupción permite cierta simultaneidad entre cálculos y entradas – salidas, retrasando al programa de cálculo una cantidad de tiempo igual a la suma de los tiempos de ejecución del programa de interrupción.**

“No tiene el problema del anterior, que depende de respuesta del periférico porque es el periférico quien interrumpe al programa y el que le da ritmo a la rutina de transferencia.”



Ahora el canal con este modo escucha.

4. Modo automático, por suspensión de programa o MODO CANAL

Son unidades capaces de tomar a su cargo la tarea de realizar La transferencia de todo un bloque de información

Si el canal necesita realizar una transferencia, ya no genera Una interrupción, sinó una solicitud de servicio para que se le Conceda un ciclo de memoria

La instrucción provee al canal de la siguiente info:
A-Número y emplazamiento de información a transferir
B-dirección del periférico
C-condición de finalización de la transferencia

La transferencia se realiza al ritmo del periférico
El canal roba ciclos al procesador
El canal avisa la finalización

El canal internamente maneja las transferencias al exterior.

Una instrucción única de entrada salida, que le da el **sentido**.

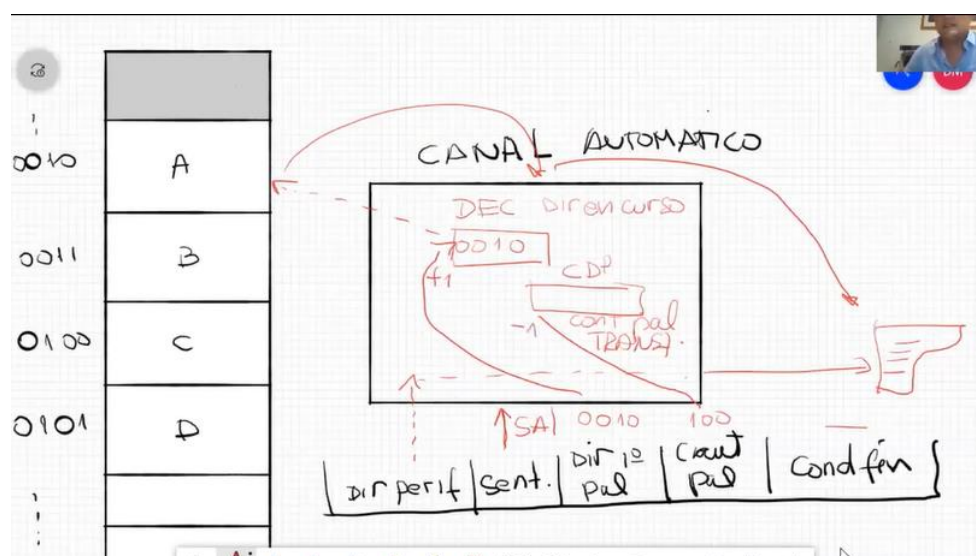
En este procedimiento se crean **unidades automáticas de intercambio o canales**, capaces de tomar completamente a su cargo la transferencia de todo un bloque de informaciones. “**Permite transferir una o varias palabras contiguas desde o hacia la memoria.**”

Cuando el canal necesita intercambiar una información con la memoria, no produce ya una interrupción de programa, sino simplemente una solicitud de servicio para que le sea concedido un ciclo de memoria, con objeto de realizar dicho intercambio. Por consiguiente, la simultaneidad entre programa y entrada/salida es casi total. **La realización de una operación de entrada/salida se desarrolla de la forma siguiente:**

- I. Es el programa quien inicializa la operación: las instrucciones proveen al canal de todas las informaciones necesarias a la operación: número y emplazamiento de los datos por transferir, dirección de la unidad de entrada/salida, condiciones bajo las cuales debe terminarse la transferencia, etc.; una instrucción especial lanza la transferencia y el programa puede continuar independientemente.
- II. La transferencia se efectúa al ritmo del equipo periférico bajo control del canal, que roba ciclos al calculador.
- III. El canal avisa que la transferencia ha acabado generalmente mediante una interrupción. El programa de interrupción podrá preguntar entonces al canal en que condiciones se ha efectuado aquella.

Se distinguen **dos tipos de canales automáticos**:

- los canales simples o selectores que no pueden gestionar más de una transferencia a la vez
- los canales multiplexados en el tiempo, capaces de gestionar varias transferencias simultáneamente.



Canal pasa a tener una cierta lógica, y establecer dir de primer palabra y cantidad de palabras a transferir.

Condición de fin, disco fallo, falta papel, etc

ORDEN DE SERVICIO O INSTRUCCIÓN CANAL formas de llamar esa instrucción

El canal avanza al ritmo del periférico.

La memoria ahora tiene 2 muelles, por parte del programa en curso y por parte de la entrada/salida.

5. Modo de encadenamiento automático de las transferencias

Qué pasa si la memoria pasa a estar no con las palabras contiguas, sino con las palabras dispersas por la memoria (memoria altamente fragmentada).

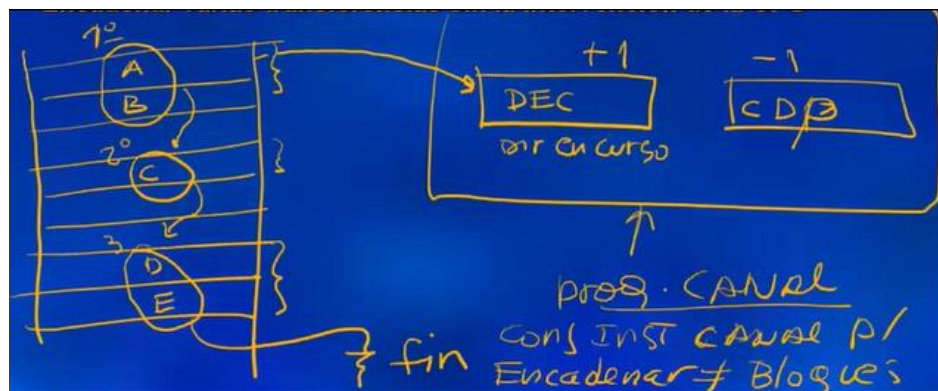
Es una versión mejorada de la del modo automático.

La CPU genera en memoria un programa CANAL, permitiendo Encadenar varias transferencias sin la intervención de la CPU

Trabaja con:

PROGRAMA CANAL

Conjunto de instrucciones de canal (conjunto de órdenes de servicio) para encadenar los bloques. No necesitan estar contiguas las palabras de memoria que recibe/busca



Cantidad de instrucciones tantas como cantidad de bloques fragmentados existan.

En el modo precedente, el ordenador está obligado a inicializar cada transferencia cargando las informaciones necesarias en los registros del canal. A fin de aliviar al ordenador de esta tarea, se concibe el canal como una unidad poseedora de su propio programa.

La unidad central generará en memoria central un programa de control de entrada/salida o programa de canal definiendo las diferentes transferencias sucesivas a realizar por el canal. Para lanzar este programa de canal, la unidad central proporcionará su dirección al canal que se encargará de su ejecución completa. El programa de control de entrada/salida permite encadenar varias transferencias sin intervención de la unidad central.

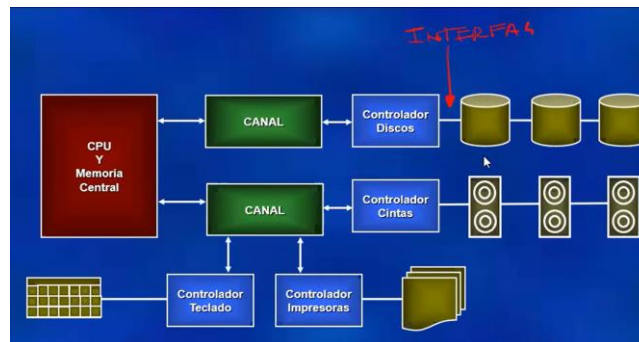
Se distinguen dos clases de encadenamiento de bloques: el encadenamiento de datos, que después de transferir datos desde (o hacia) una tabla en memoria, permite pasar automáticamente a otra tabla; el encadenamiento de funciones, que permite hacer ejecutar sucesivamente varias transferencias sobre varios registros diferentes en un mismo órgano periférico.

“Es una mejora del modo anterior, porque aparte de lograr el automatismo de la transferencia también permite lograr que la información este dispersa en bloques, es decir, que no sea contigua, que es lo que sucede normalmente en la memoria.”

Link del video:

<https://www.youtube.com/watch?v=nM3GGI9sHlk&list=PLgTKn0MhANLTPffShcV9I2INbaxlPZvLI&index=24>

Interfaz



Establece la manera en que las controladoras se van a comunicar con los distintos periféricos de la misma familia de controladoras. Establece un **protocolo de comunicación entre la controladora y sus periféricos (de la misma familia)**.

Desde el punto de vista físico establece voltajes y tensiones con los que debe trabajar para garantizar la funcionalidad

Se denomina interfaz a las especificaciones de conexión entre un canal y cualquier tipo de unidad periférica. Es la caja negra que permite adaptar las señales del periférico a las especificaciones de la interfaz ordenador.

“Todo canal se comunica con una controladora de periférico, por eso se diseñó la interfaz. La interfaz es un circuito lógico que permite iniciar, sincronizar y transferir información de un canal hacia una controladora.”

La **interfaz**, está formada físicamente por un cierto número de conectores, salidos del canal, en los cuales pueden ir a enchufarse. Es necesario precisar además las especificaciones de formato y de tiempo de las señales dadas o esperadas por el canal.

Básicamente, la interfaz se compone de dos tipos de hilos:

1. Hilos de información, que llevan en paralelo la información por transmitir, en forma de niveles;
2. Hilos de gobierno (indica el sentido, si es de entrada o de salida) y sincronización, que capacitan para el diálogo de las dos unidades interconectadas.

Hilos de gobierno

E/S Gobierno: Es muy importante, define el sentido.

Sincronización

- **DEM:** De demanda de transferencia, bit que solicita el periférico, cuando esté libre que le conteste
- **ADEM:** Aceptación de demanda, es cuando ya la conexión fue establecida.

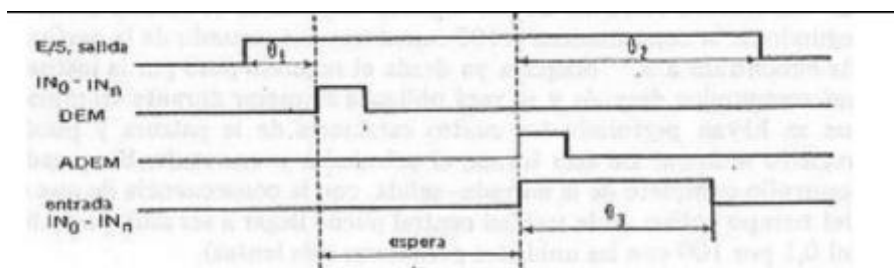
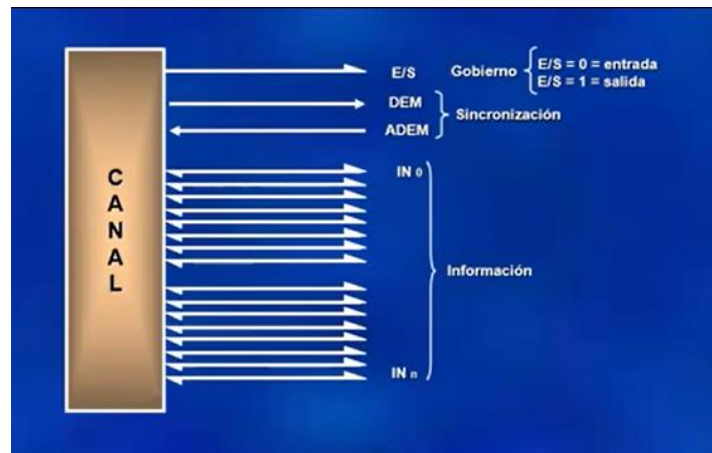


Fig. 4. Ejemplo de cronograma asociado a la interfase de la figura anterior¹

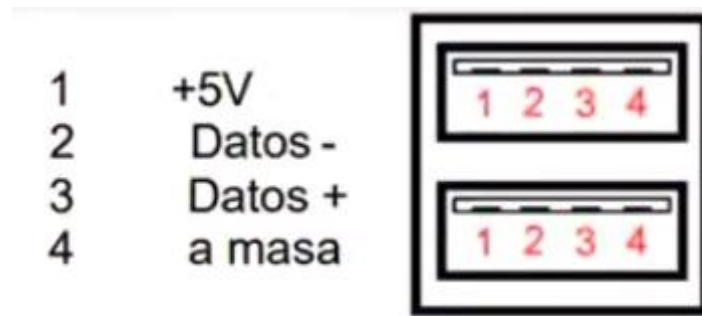
Descripción de una salida de información:

- La unidad de intercambio posicionará la señal de nivel E/S para indicar que se trata de una salida de información, posicionará la información por salir bajo la forma de niveles en las líneas de información IN0 a INn y, por último, enviará un impulso DEM de demanda de servicio.
- Después permanecerá en espera.
- El equipo exterior reconocerá la demanda DEM, probará el hilo de gobierno E/S para saber si la operación consiste en una entrada o en una salida de información. Como E/S vale 1, el equipo exterior muestrearán los hilos de información IN0 a INn (apertura, mediante un impulso, de las puertas de entrada a un registro tampón). Dicho impulso podrá ser también enviado al canal como señal de acuerdo, que indica que la demanda de salida ha sido reconocida y muestreada la información (ADEM).

Descripción de una entrada de información:

El canal pondrá E/S a 0 y enviará el impulso DEM. La solicitud de entrada será reconocida por el equipo periférico, que deberá efectuar los niveles de información IN0 a INn, y, después, enviará la señal de acuerdo ADEM. La lectura será realizada por el canal tras recepción de ADEM.

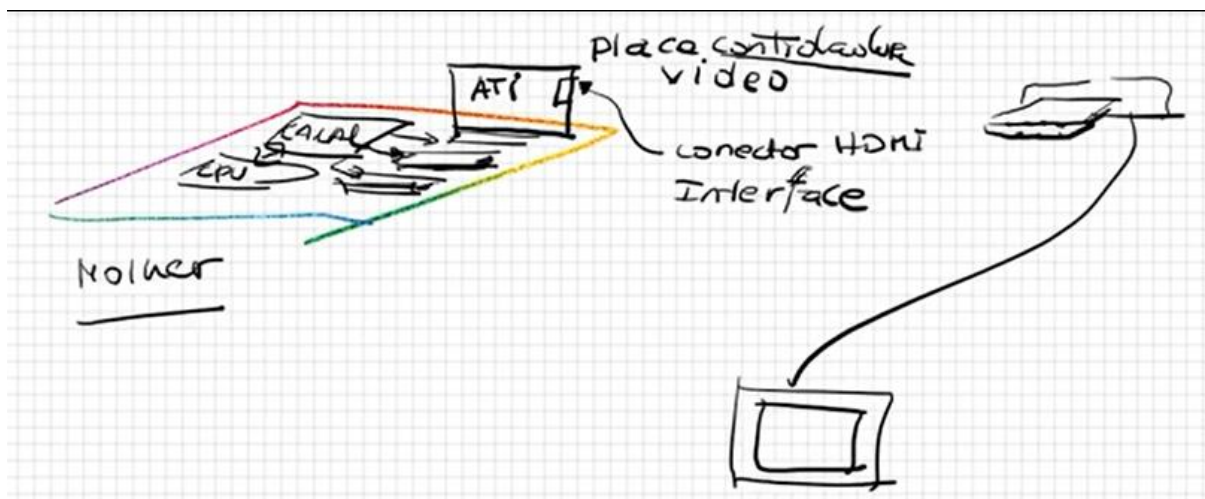
Hoy en día ya no hay casi interfaces por el uso generalizado del USB



Notas de USB

- Consta de 2 Hilos de datos
- Estandarizó las interfaces de múltiples dispositivos, permitiendo así el desuso de interfaces como PS2, ESB25, etc

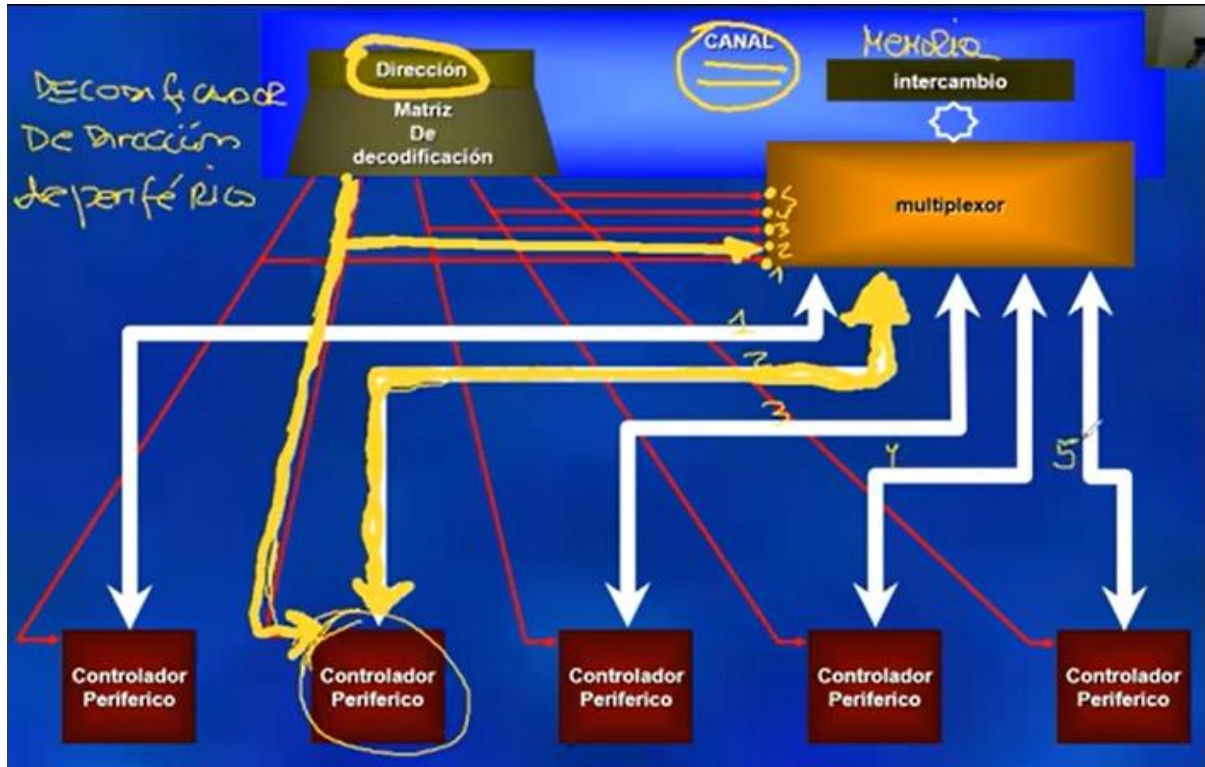
“El problema que se presenta es que el canal debe poder comunicarse con varias controladoras (disco, usb, etc.). Existen dos técnicas de comunicación entre un canal y sus controladores: la primera es concentración por multiplexación y la segunda concentración por líneas ómnibus.”



¿Cómo hace el canal para distribuir la información a través de la interfaz desde o hacia un grupo de periféricos?

Dos técnicas:

Concentración por multiplexaje



NOCIÓN DE MULTIPLAJE

El multiplexaje permite concentrar varias vías de entrada en una sola o distribuir una vía de salida en varias vías. También permiten conectar varias unidades externas a un mismo canal. A veces, se utilizará este método en los canales simples, es decir, aquellas unidades de intercambio capaces sólo de controlar una transferencia simultáneamente.

Entonces el direccionamiento de las unidades externas se **efectuará en estrella**, un hilo por unidad, después de decodificar la dirección en el canal.

El Multiplexor es como un conmutador

“La dirección del periférico es decodificada a través del canal, el cual realiza dos operaciones:

- Selecciona con una señal de nivel uno de los controladores afectados a la tarea
- Establece un canal único de comunicación entre la memoria central y el controlador seleccionado”

VENTAJAS

- Alternar rápidamente hacia/desde múltiples controladores (Ideal sería tener una RAM para cada Controlador, pero no se da eso) por parte del canal
- Da la sensación de simultaneidad debido a que se comunica con muchos controladores

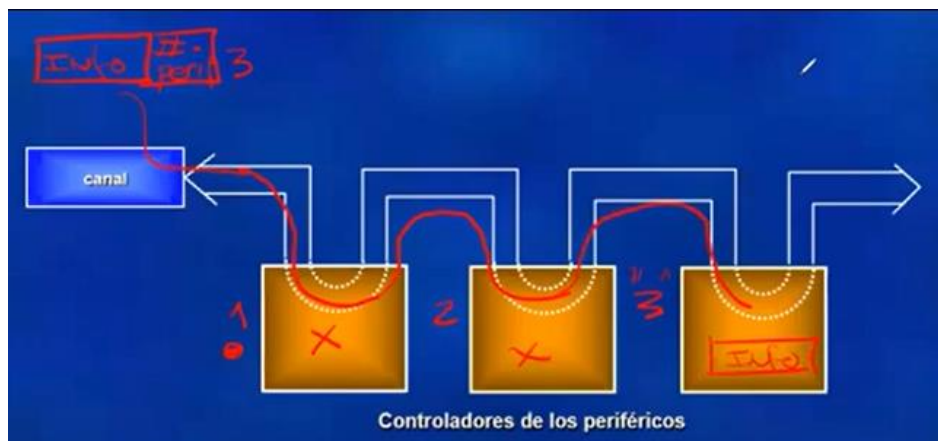
- Ya establece el vínculo de datos entre controlador y la RAM por este medio, unico recurso canal / memoria con multiples (N cantidad) controladores
- Conecta varios controladores con un solo canal



DESVENTAJAS

- Encarece la cantidad de buses, Ej. 5 controladores equivale a 5 buses
- Implica un costo en lógica combinatorial + multiplexor
- Cantidad de materiales, costos altos

Concentración por línea omnibus



La info va acompañada cuenta con una cabecera que contiene el número de periférico #1, #2, etc

Para conectar varias unidades externas en un canal, puede también emplearse una línea ómnibus que las una. Si la unidad externa no resultase selecciona se transmite la información sin alterarla.

“Solo aquel controlador que identifique la dirección transmitida por el canal es el que lo va a tomar y liberar de la circulación para que venga una nueva palabra (en el ejemplo anterior, el #3). Si la dirección transmitida no coincide con la del controlador, este deja pasar la información al siguiente y así sucesivamente. Hasta que llega al #3 y ahí SI ”

El direccionamiento puede realizarse siguiendo la técnica en estrella. Es posible, también, emplear la línea ómnibus para el direccionamiento: un hilo especial indica, entonces, que la información emitida por el canal es una dirección. La unidad externa atravesada comprueba este hilo a cada transferencia. Para el caso en que aquella detecte que se trata de una dirección, compara la dirección emitida con la suya propia. Si hubiera identidad entre ambas, la unidad externa se pone en estado de diálogo con el canal; si no sucediera así, transmite la dirección a la unidad contigua. Esta técnica de direccionamiento y de transferencias por bus la emplean, sobre todo, los canales multiplexados en el tiempo.

VENTAJAS

- Economía en cantidad de buses

DESVENTAJAS

- Tarda mucho para viajar
- Solamente puede viajar 1 sólo paquete

Técnicas de transferencia

Estrategias que permitan enviar una palabra de información desde o hacia un periférico

Llamaremos transferencia elemental a la transferencia de una información (carácter o palabra) desde una posición de memoria central a una unidad externa (o viceversa, según se trate de una entrada o salida).

“Una técnica de transferencia es una estrategia planteada por la arquitectura con el fin de controlar de forma más eficiente un proceso de entrada salida con la memoria del computador.”

La ejecución de una transferencia elemental supone que se conozca las informaciones siguientes:

- Información pidiendo el lanzamiento de la transferencia: DEM
- Indicación del sentido de la transferencia (entrada o salida): SEN
- Dirección de almacenamiento en memoria de la información transferida: DIR

Las diferentes técnicas de ejecución de una transferencia elemental dependen de la manera como sean suministradas al computador estas tres informaciones

Lista de tipos de técnicas de transferencia:

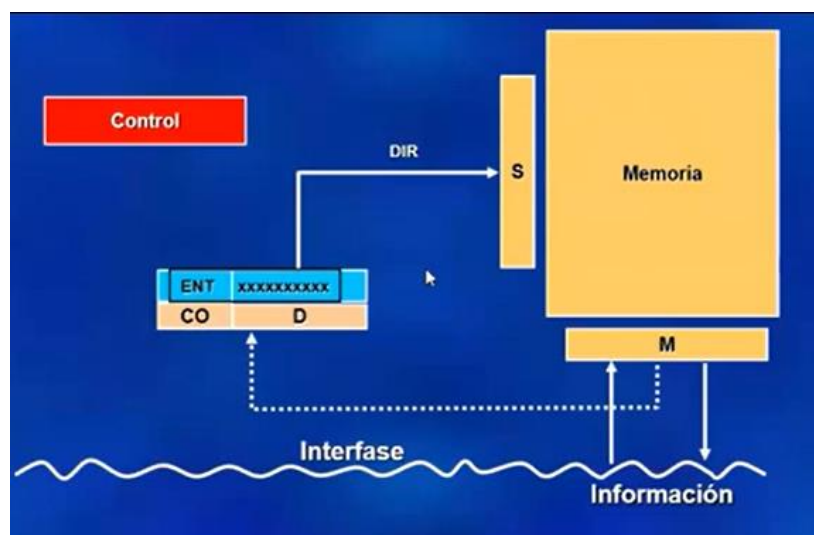
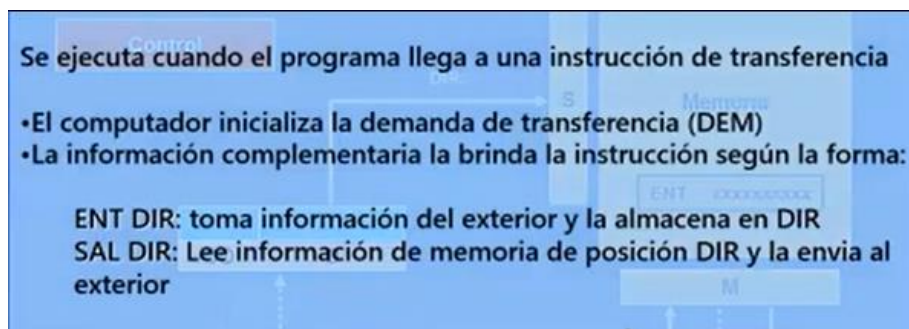
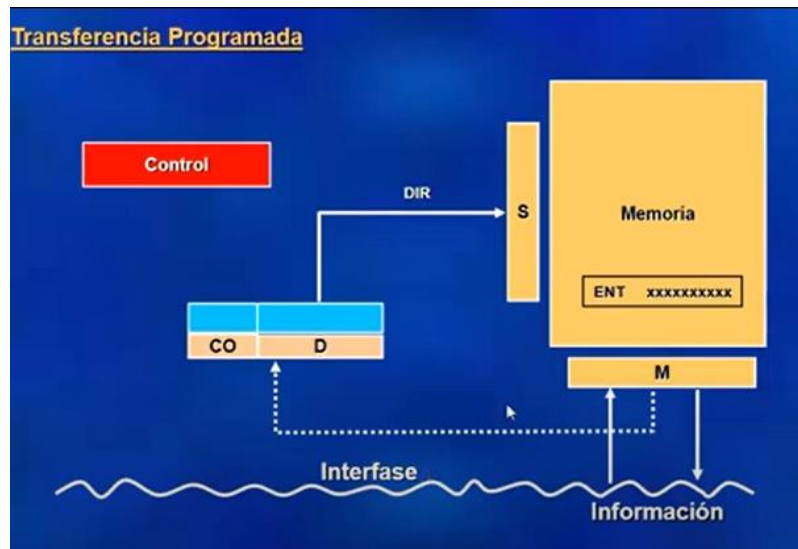
- Transferencia programada
- Transferencia por instrucción forzada
- Transferencia por robo de ciclo
- Transferencia por acceso directo a Memoria

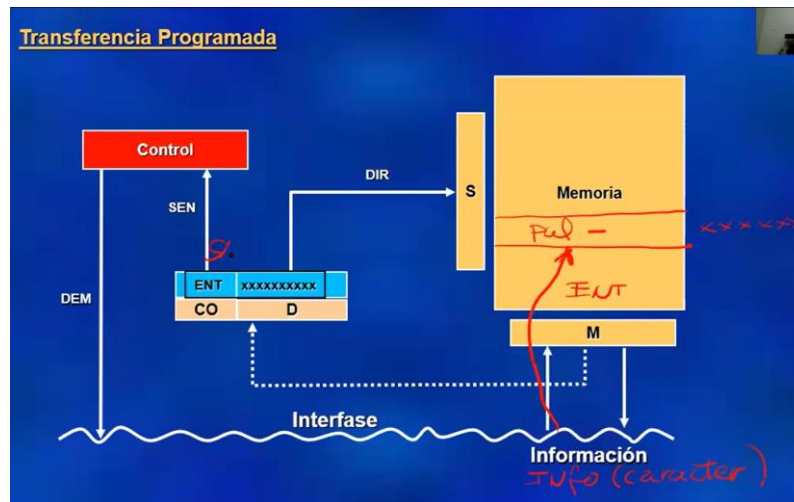
Transferencia programada

Se da con una instrucción de transferencia, la cual debe proveer el sentido y la dirección de la palabra. Habrá un órgano de control que va a coordinar estas operaciones.

¿Por qué se llama Transferencia Programada?

Se desencadena a través de una instrucción I/O, le brinda el sentido y la dirección de la palabra memoria al órgano de Control.





Alcanza y sobra con una interfaz, porque no hace falta tener más.

Se ejecuta cuando el programa llega a una instrucción de transferencia (es una instrucción dentro del set de instrucciones de la maquina). Es el ordenador quien inicializa la demanda de transferencia (DEM). Las informaciones complementarias son suministradas por la instrucción, que puede adoptar dos formas:

ENT	DIR	Entrar información y almacenar en la dirección DIR
SAL	DIR	Leer información almacenada en la dirección DIR y hacerla salir

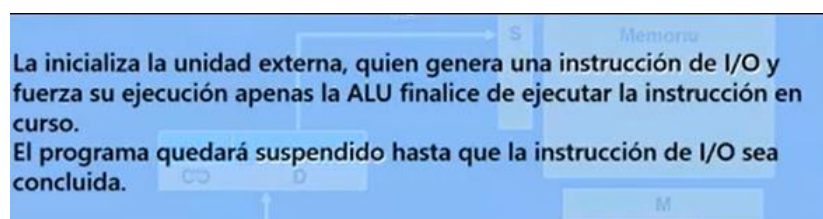
“La instrucción puede ser de entrada o salida y esto va a ser determinado dependiendo del código de operación (CO), es decir, el CO le brinda al controlador el sentido de la transferencia.

- Si es entrada activa ENM y realiza una escritura (ESC).
- Si es salida activa SRM y realiza una lectura (LEC).

Pero en ambos casos realiza las señales inicio de ciclo de memoria (ICM) y puesta a cero del registro palabra de memoria (PACM).”

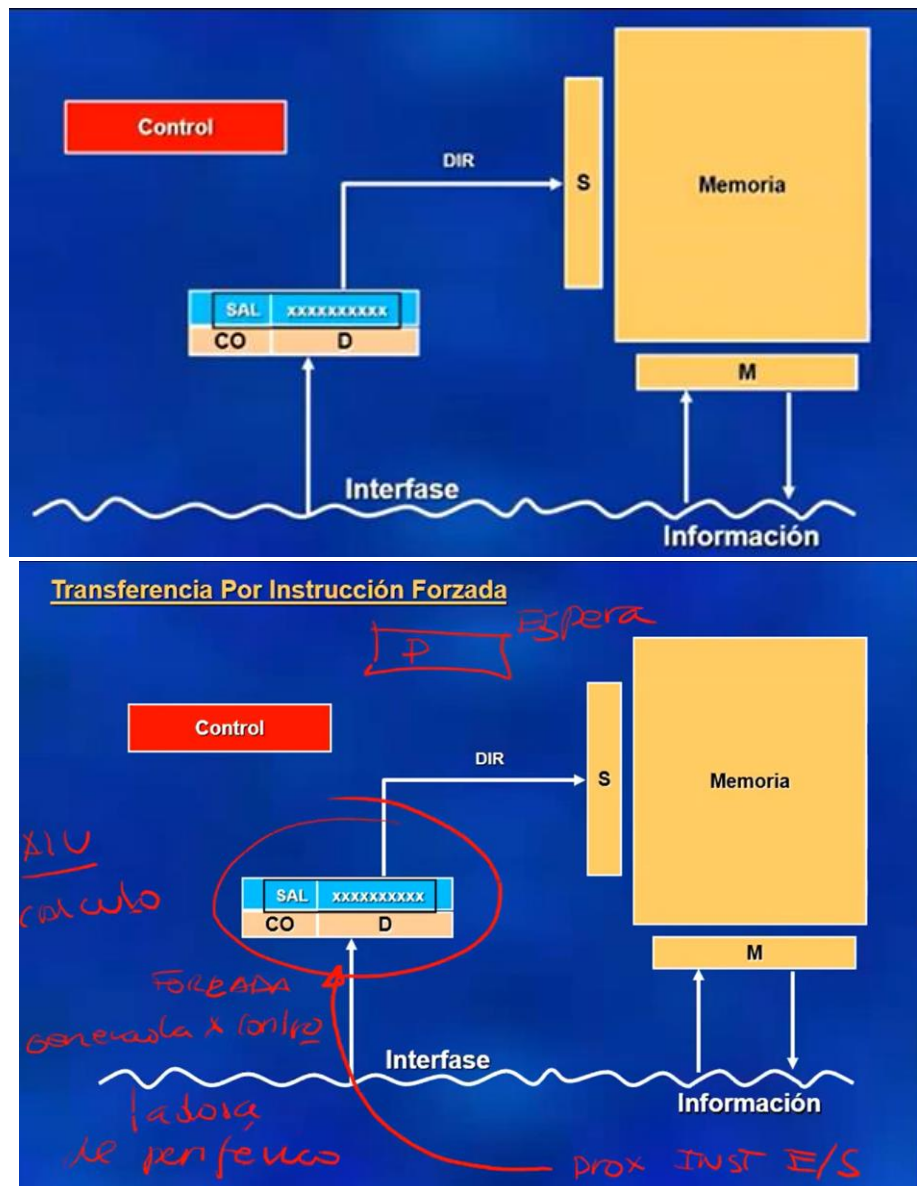
“El dispositivo control de transferencia es parte de la unidad de control. Este recibe si la instrucción es de entrada o salida y dependiendo de esto articula la entrada o salida, desde o hacia el dispositivo externo.”

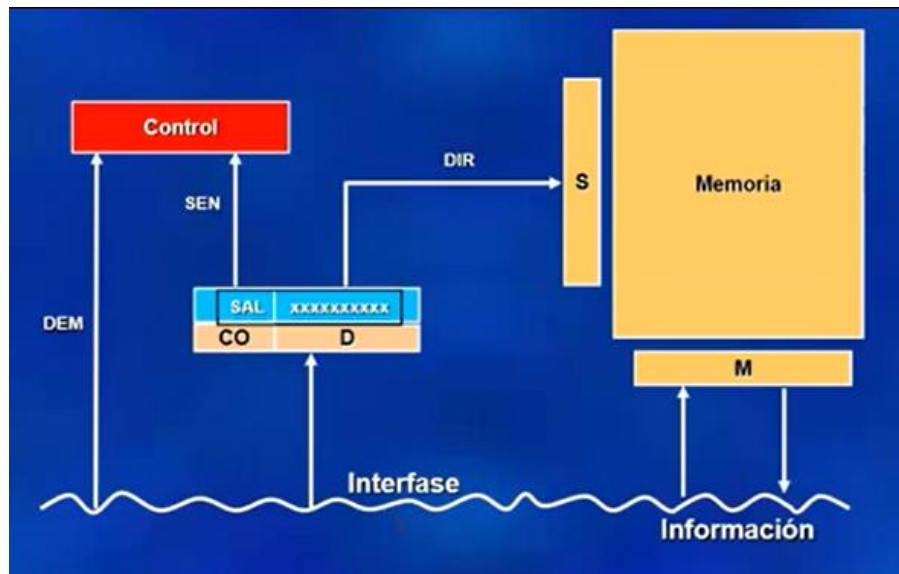
Transferencia por instrucción forzada



La transferencia por instrucción forzada es inicializada por la unidad externa, que genera la instrucción de entrada/salida y fuerza su ejecución prioritaria en cuanto que la unidad

central ha concluido su instrucción en curso. El programa queda suspendido durante el tiempo de ejecución de esta instrucción.





“Por ejemplo, el controlador del mouse, genera la instrucción cuando uno mueve el mouse.”
 “Antes que el contador de programa se incremente ingresa esta instrucción, por tanto, es la que se va a ejecutar y por eso recibe el nombre de forzada.”

EJEMPLO

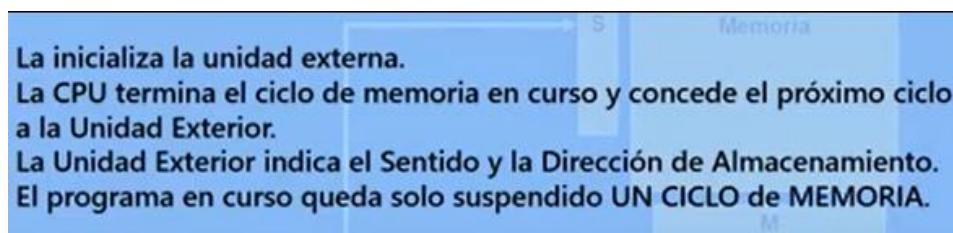
Entonces si venia Instruccion 1, Instruccion 2.. pero aparece una Instruccion A (I/O), entonces va a esperar que se termine de ejecutar la Instruccion 1, despues se ejecuta Instruccion A, y despues recien se ejecutaria Instruccion 2. Tiene prioridad la Instruccion A, pero si el programa esta en proceso de “Búsqueda de Instruccion” entonces **NO** puede ingresar la Instruccion A.

Si el programa esta en proceso de ESC, tambien puede aprovechar si el periferico necesita ESC, se aprovecha el ciclo, se sigue manteniendo la señal de ESC.

Es una programada, pero con la diferencia de que la genera el controlador del periférico, que le da el sentido y la dirección.

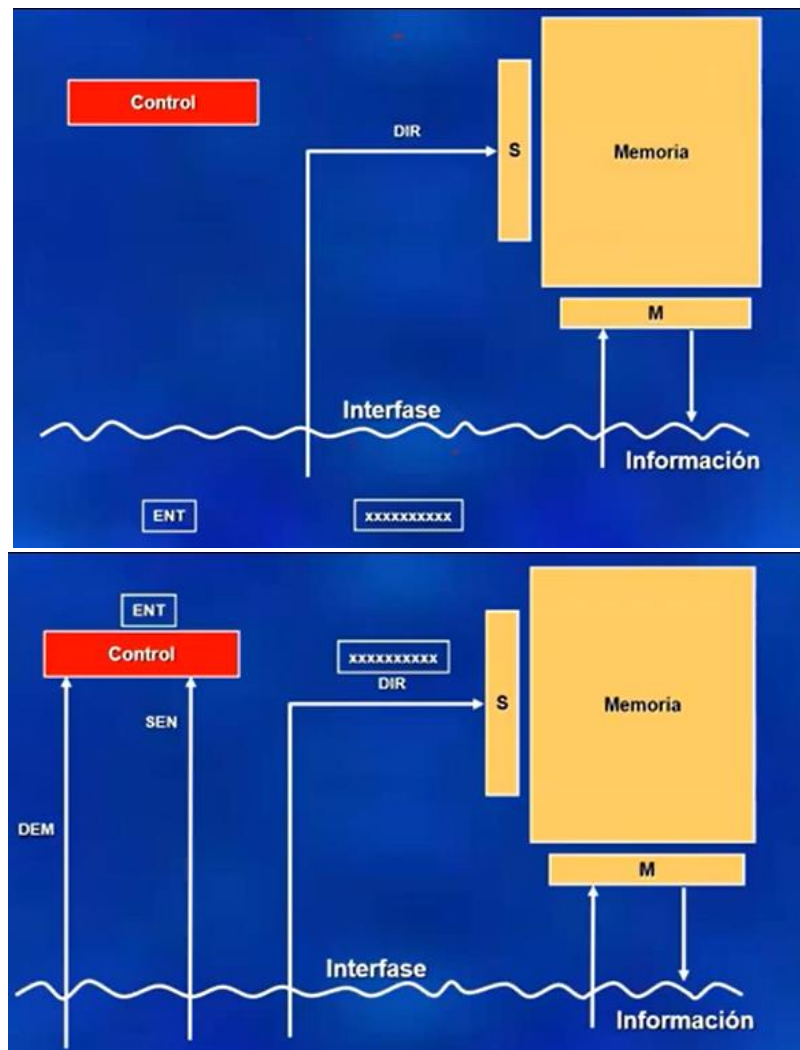
- Se mete la instrucción dentro del RI
- Si está haciendo un cálculo debe esperar

Transferencia por robo de ciclo



La inicializa también la unidad externa

La transferencia por robo de ciclo se inicializa a petición de la unidad exterior. La unidad central termina el ciclo de memoria en curso y concede el próximo ciclo a la unidad exterior. Esta indica si se trata de una lectura o de una escritura y proporciona la dirección de almacenamiento. El programa en curso queda en suspenso durante un ciclo de memoria.



DIFERENCIA CON INSTRUCCIÓN FORZADA

Se mete directamente en memoria, no pasa por decodificar una instrucción, cuando? Cuando la memoria está disponible.

Va directo a la dirección.

Ciclo, no ejecución de instrucción (sutil diferencia)

“Robo de ciclo es más óptima porque no genera una instrucción que tiene que entrar a la mezcla de instrucciones, va directamente a la lectura o grabación de memoria.”

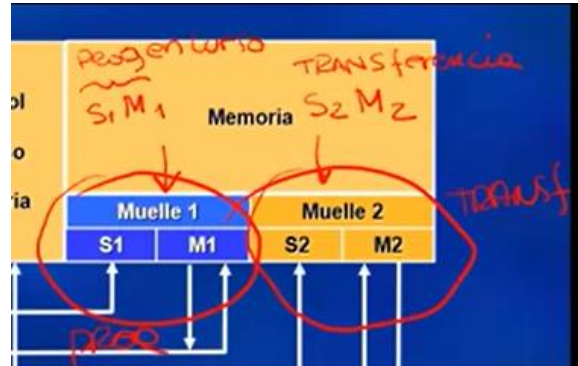
“El CO va directamente a la unidad de control de transferencia y la dirección se mete directamente al registro de selección de memoria. No pasa por el registro I. Esto quiere decir que ni bien termina la instrucción anterior lo realiza, es decir, entre una búsqueda de instrucción y otra búsqueda de instrucción, entre dos tratamientos de instrucción roba un ciclo. Es más veloz.”

“No suspende el programa en curso solamente roba un ciclo. Apenas está libre el ciclo de memoria realiza la transferencia.”

Transferencia por acceso directo a memoria (DMA)

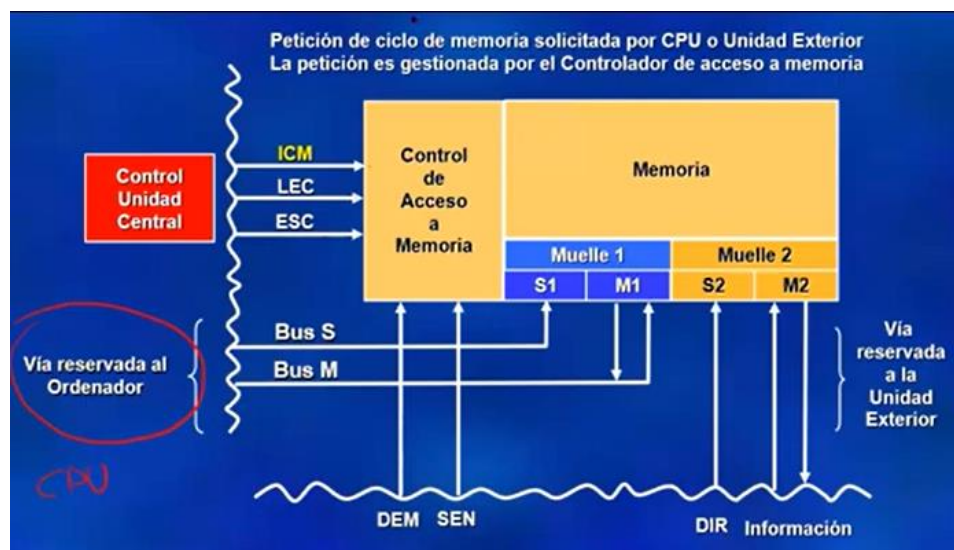
(Profe recomienda a Patricia Quiroga)

Dos muelles/vías de acceso:



Sacado del libro:

Se supone que la memoria posee dos vías de acceso (podrían llamarse muelles de acceso). Una de las vías está reservada al ordenador, la otra a la unidad exterior. Las peticiones de ciclo de memoria son dirigidas a la unidad de control de acceso a memoria, tanto si proceden de la unidad central como de la unidad exterior. Esta unidad de control gestiona las prioridades de acceso de la unidad central y de la unidad exterior. El programa en curso no resulta suspendido durante el ciclo de memoria solicitado, excepto cuando la unidad central hubiera pedido simultáneamente un ciclo de memoria. Este sistema es interesante si se dispone de varios bloques independientes de memoria.



“Una palabra de memoria puede ser accedida o bien por una dirección del registro selección 1 (S1) y sacada por el registro palabra de memoria 1 (M1). Pero la misma palabra también

puede ser seleccionada por otro acceso (S2) y sacada por M2. Se tiene dos formas de acceso a una palabra.”

“El objetivo de tener dos accesos es dedicar el muelle 1 exclusivamente para el bus S y M, es decir, el **programa en curso (CPU)** y el muelle 2 exclusivamente para la **comunicación con el exterior (periféricos)** entonces se puede lograr un cierto paralelismo.

En la misma lectura se podría acceder a una palabra distinta a otra. Esto me permite que el mismo ciclo de memoria, con suerte, pueda estar destinado para buscar un operando para el programa en curso pero también para estar leyendo una palabra de memoria.

Lo que no se puede hacer es grabar y leer, o grabas o lees, una cosa o la otra. En el mismo ciclo puedes estar leyendo una nueva instrucción o un nuevo operando y simultáneamente una palabra que no esté en conflicto con las direcciones del operando o de la instrucción. También se puede estar grabando el resultado de un operando producto del programa en curso pero también guardando información del exterior en otra zona.”

CUANDO SE SUSPENDE EL PROGRAMA?

La que gestiona estos accesos simultáneos es la unidad central de control.

Se suspende cuando la unidad central pide el ciclo simultáneamente con la de periféricos, entonces se le da prioridad a la de transferencia, por lo que primero se trata la instrucción de periféricos y después se continúa con la de CPU, en ese caso funcionaría como un **robo de ciclo**.

Este sistema es interesante si se dispone de varios bloques de memoria independiente. Si están mezclados los bloques, se puede llegar a producir una colisión.

Comparación de las técnicas

TECNICA EMPLEADA	TIEMPO MAXIMO DE ESPERA DE LA UNIDAD EXTERIOR	TIEMPO RESTADO AL PROGRAMA EN CURSO
Programada	Hasta el final de la instrucción en curso, más inicio del programa solicitado de interrupción.	Procesamiento de un programa de interrupción.
Por instrucción forzada	Duración de la instrucción más larga del repertorio.	Una instrucción generalmente ejecutada en uno o dos ciclos de memoria.
Por robo de ciclo	Un ciclo de memoria.	Un ciclo de memoria.
Por acceso directo	Un ciclo de memoria.	Nada, o un ciclo de memoria.

Enlace programado en Abacus

NO es un CANAL, es una INTERFAZ es una organización de señales de gobierno con el objeto de realizar **transferencias elementales** utilizando la técnica de transferencias **programadas** porque son de tipo de instrucción de memoria que se introduce dentro de las mezclas de programas. Coordina la interfaz para que ejecute una instrucción de entrada salida.

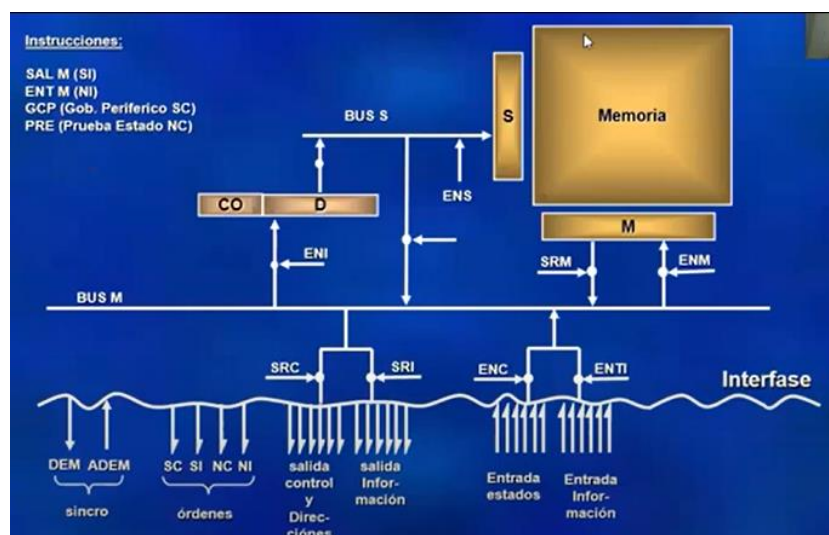
Es mejor que un Modo Bloqueado, porque justamente no estamos bloqueando el recurso hasta finalizar el proceso, por eso se ocupa el PRE.

“Es una técnica, interfase lógica para poder gobernar a un periférico ante la acción de una instrucción de entrada/salida. Es un esquema lógico que permite gobernar y sincronizar las transferencias con el exterior en el modelo abacus. No llega a ser un canal.”

No es propiamente un canal sino un medio de acceso controlado por el programa. En efecto, cada transferencia elemental está gobernada por una instrucción del programa en curso. Por el hecho de estar controlados por el programa, tales enlaces pueden utilizarse tanto en modo bloqueado como, mucho más a menudo, en modo por interrupción. Casi siempre aparecen integrados en la unidad central.

Son necesarias cuatro instrucciones de entrada/salida para realizar entradas/salidas en modo programado:

Instrucciones de gobierno de transferencias de información	SAL M	Para extraer el contenido de la dirección M hacia las líneas de salida de información de la interfase.
	ENT M	Para almacenar en la dirección M una información presente en las líneas de entrada de información a la interfase.
Instrucciones de control general de las operaciones de entrada – salida	GCP	Permite direccionar una unidad de control de periférico y especificarle la operación por ejecutar.
	PRE	Permite adquirir una información descriptiva del estado del periférico anteriormente direccionado.



“Este esquema solo permite transmitir una sola palabra desde o hacia el periférico. Pero no permite bloques, solamente una palabra.”

“El enlace programado trabaja por transferencia programada y con una transferencia por línea omnibus.”

TIPOS DE OPERACIONES 4 (CUATRO)

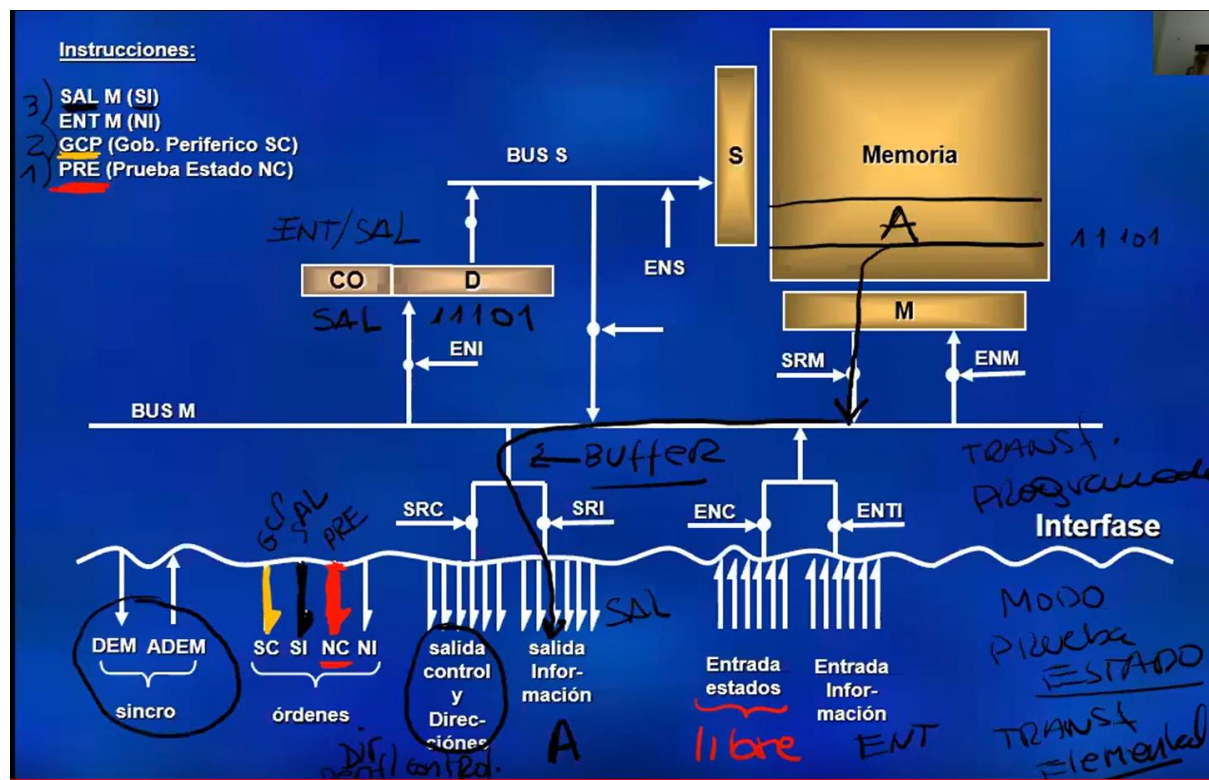
Más que un Canal es un medio de acceso controlado por el Programa
Cada Transferencia elemental la gobierna el programa en curso
El acceso a memoria utiliza modo Bloqueado y Por Interrupción de Programa

Instrucciones:

- a) SAL M y ENT M Transfiere o recibe un carácter de dirección M
- b) GBP (gobierno de Periférico) Transfiere (M) a las líneas de control y dirección
- c) PRE (Prueba de estado) Guarda en AC contenido líneas de estado

Caminito de la instrucción:

1. PRE (Prueba de estado NC) que activa NC (Orden que manda la controladora para que sepa que se está realizando una prueba de estado)
2. Si está pasa a la siguiente instrucción que es GCP (Gobierno de Periférico), tomar control del periférico para que otra instancia no lo quiera gobernar al periférico. En SC manda una señal de gobierno para que quede ligado a la espera de transferencia
3. Ahora una vez linkeado el periférico, se puede realizar una ENT/SAL



Ejemplo

Con SAL 11101 (acá está contenida el valor A) activa la línea de ordenes SI, con esta orden le estamos diciendo a la controladora que le vamos a estar llevando una palabra de memoria. Palabra sale con SRM, va al Bus y va a salir por SRI y por ahí va a salir la letra A

- En la operación de salida SAL, la información permanece memorizada en el registro M hasta ser tomada en cuenta por el órgano exterior, las puertas SRM y SRI permanecen abiertas para mantener activos los niveles lógicos de las líneas de salida de información y la señal SI (salida de niveles de información) permanece activa.
- El computador envía entonces al controlador del periférico previamente escogido, una señal impulsional DEM. Al recibirla éste, suponiendo que está disponible, comprueba los hilos de órdenes y tras haber detectado que se trata de una salida, muestrea la información y transmite al computador una señal de acuerdo ADEM para liberarle.
- En la operación de entrada ENT, el ordenador posiciona la señal de orden de entrada de información NI y realiza una demanda de servicio DEM. Luego el periférico comprueba los hilos de órdenes y reconoce una demanda de información.
- El ordenador posiciona las líneas de entrada de información activando ENTI y el periférico envía la información por las líneas de entrada de información. El ordenador muestrea el registro M mediante ENM y el periférico finaliza la operación con la señal ADEM.
- En la operación GCP se activa TSM SRC DEM SC ADEM. En la operación PRE se activan TSM SRC NI salida de control luego se producen entrada de estados ENC ENM.

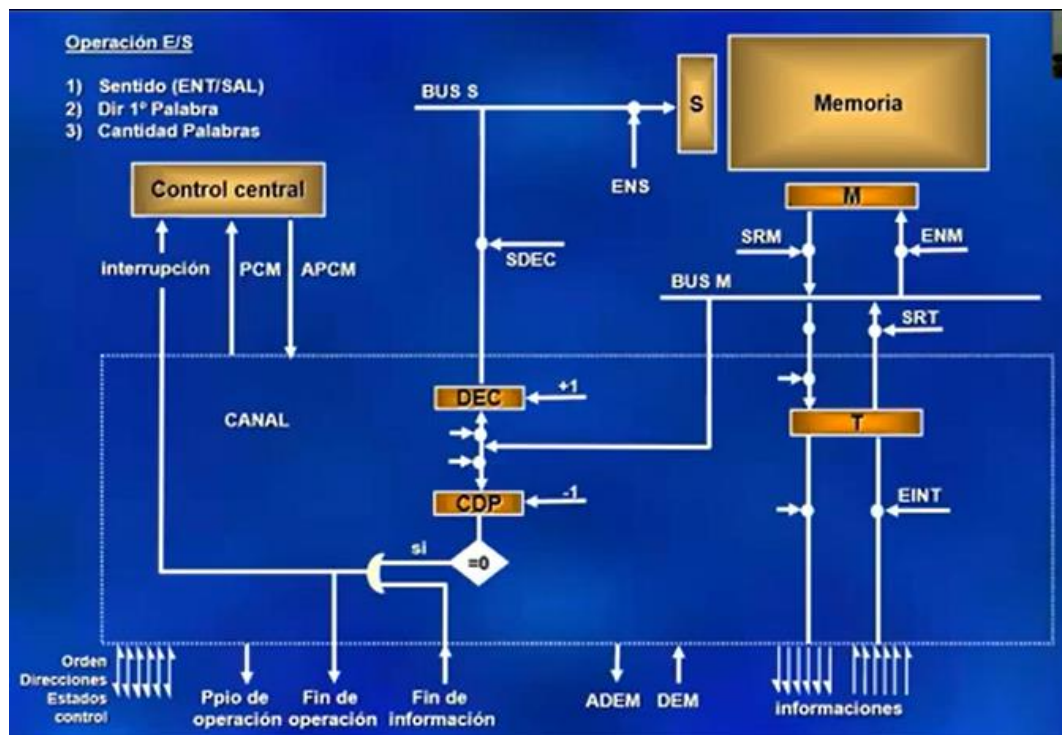
Pregunta: La información permanecería memorizada en el registro M?

Los periféricos manejan una velocidad distinta, por lo que haría falta un buffer, el periférico ya está tomado y a la espera de la ENT/SAL, entonces la memoria está ociosa, entonces queda a la espera de que M evacúe la información hacia/desde periférico (se complete la transferencia).

está faltando un buffer que libere a la memoria para que siga con la siguiente instrucción

La memoria va a estar ociosa un tiempo, hasta que M no evacúe

Canal automático para abacus



Este segundo modelo toma forma de un canal porque a través de este se puede realizar la transferencia de todo un bloque de información desde o hacia un periférico.

El canal toma el gobierno por sí mismo de la transferencias de una o más palabras contiguas en memoria

DIFERENCIA CON ENLACE PROGRAMADO

Incluye ya un conjunto de registros para tomar el control de la transferencia de la palabra. Permite una transferencia de TODO un BLOQUE, no solo de una Transferencia Elemental.

Los canales automáticos son capaces de gestionar el conjunto de una operación de entrada/salida sin intervención de la unidad central, excepto para inicializar y concluir dicha operación. Las transferencias elementales pueden ser efectuadas, bien por instrucción forzada, bien por robo de ciclo, bien por acceso directo a la memoria.

Se les llama **canales simples** cuando **no gestionan más que una operación de entrada/salida a la vez**.

Describiremos una unidad de este tipo en que las transferencias se consiguen por robo de ciclo, interesándonos exclusivamente en los problemas de intercambio de informaciones. Desde este estricto punto de vista, la **operación de entrada/salida** queda enteramente definida por:

- el **sentido de la transferencia** por realizar (entrada o salida)
- **zona de almacenamiento en memoria** de las informaciones por transferir. Esta zona puede delimitarse mediante la dirección de su primera palabra (DIR) y su número de palabras (CDP)

La unidad central es quien **inicializa la operación por programa**.

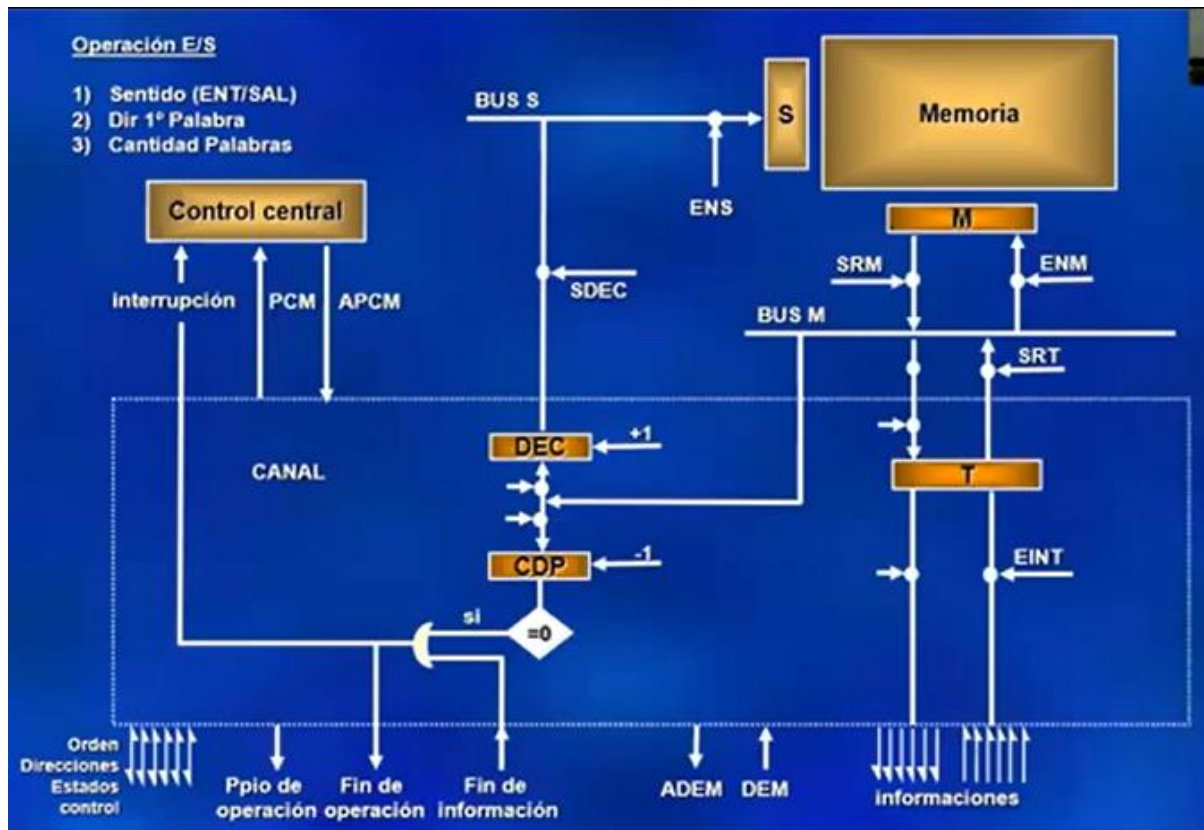
1. Selecciona el periférico (indicando eventualmente el sentido de la transferencia por efectuar) y carga las informaciones que definen a la zona de almacenamiento en memoria en dos registros del canal, a los que llamaremos DEC (dirección en curso) para la dirección de la zona y CDP (cuenta de palabras), para el número de palabras.
2. Una vez efectuada la inicialización, la operación de entrada/salida se desarrolla ya al ritmo de la unidad periférica. Es a **petición de ésta** que el **canal ejecuta una transferencia elemental**, utilizando el contenido del registro DEC como dirección de almacenamiento en memoria, y después actualiza la dirección en curso, sumando 1 al registro DEC, y la cuenta de palabras, restando 1 al registro CDP.
3. La **transferencia termina** cuando el canal detecta que el registro contador de palabras llega a cero o cuando el controlador de periférico envía una señal de fin de información. En este momento **genera el canal una interrupción** para indicar a la unidad central que ha concluido la operación de entrada/salida.

Tal modo de funcionamiento, soportado por dos registros, uno para la dirección en curso y otro para la cuenta de palabras, es designado generalmente como **modo canal**. A la correspondiente unidad de intercambio se le llama canal, por excelencia.

EJEMPLO CON SAL (SALIDA)

Para realizar una operación de salida SAL el sentido (salida) va a la unidad de control para que esta sepa cómo desarrollar la instrucción.

1. La dirección de la primer palabra se carga en DEC y la cantidad de palabras en CDP.
2. Luego se activa la señal "inicio de operación", se manda el sentido por "orden direcciones" y también se activa la señal "DEM" para empezar a transmitir. Después, se lee la palabra de memoria que está en la dirección de DEC (por medio de activar las señales SDEC y ENS) y pasa al registro M.
3. Activando las señales SRM y ENT (entrada al registro tampón) pasa la palabra de memoria que se encontraba en el registro M al registro T.
4. En esta etapa, la computadora puede seguir con el programa en curso e incluso ejecutando varios ciclos de memoria o distintas instrucciones. Leyó la primer palabra y la almacenó en el registro tampón. En este momento, ya es responsabilidad del canal ver como transmite la palabra del registro T al medio exterior (al ritmo del periférico).
5. Cuando el periférico se encuentre libre se produce la señal SRT con una aceptación de demanda ADEM y se transmite la palabra del registro T al periférico.
6. Además, cuando se produce esta transferencia, se incrementa la dirección de DEC y se decrementa CDP produciéndose así, una lectura de la nueva dirección que se encuentra en DEC y dejará almacenada una nueva palabra en el registro T. Es importante aclarar que mientras el periférico no esté libre la palabra quedará en el registro T todo el tiempo necesario hasta que este se libere, pero con la gran ventaja de que la computadora puede estar ejecutando sus instrucciones normalmente, es decir, queda bloqueado el canal pero no la CPU.
7. Toda vez que se produce un incremento de la direccion de DEC y un decremento de CDP se produce una señal de PCM (petición de ciclo de memoria).



- **Registro T - Registro tampón** : liberamos a la arquitectura de tener que bloquear al computador a la espera de que la memoria procese la información que se está por transferir o la cual está por recibir. Es el registro T quien trata de minimizar el efecto del robo del ciclo. Permite amortiguar la asimetría de velocidad entre Canal y Memoria.
- **Registro DEC**: dirección en curso. Lleva la dirección de cada una de las palabras que deben ser transferidas desde o hacia el periférico
- **Registro CDP**: cantidad de palabras es un registro con un descontador que va descontando cada vez que una transferencia de la lista es realizada de manera exitosa. Va a definir la finalización de las transferencias.

Diagrama con resumen de las señales



ACPM - Aceptación de petición de ciclo de memoria

Si la cantidad de CDP = 0, le manda una Interrupción hacia la Unidad Central porque ej. no hay papel en la impresora. O sino, se le manda el Fin de Operación al Controlador del periférico por finalización de la información.

VENTAJAS

- **El canal utiliza robo de ciclo** - Es el registro T quien trata de minimizar/amortigua el robo de ciclo, ya que T apenas se concede el ciclo retiene la información y la memoria ya queda disponible para la próxima instrucción.
- La operación de entrada/salida tiene que proveer el sentido (entrada o salida), la dirección de la primer palabra y la cantidad de palabras. No necesita proveer condición para finalizar porque finaliza cuando termina la cantidad de palabras a transferir (CDP=0) o cuando el periférico por una excepción manda fin de transmisión (la señal fin de información).
- Toma el control del canal
- Transferencias se hace por conjunto de caracteres → bloques

DESVENTAJAS

- Es caro de implementar.
- Este modelo requiere que la información esté contigua en memoria.

PREGUNTA DE FINAL

¿Cuál es la diferencia entre el enlace programado y el canal automático de abacus?

La primer gran diferencia es que uno es un **enlace programado** ya es el programa quien gobierna, en cambio, en **programa canal automático** el que gobierna es el CANAL el que toma el control ahora. Hay una instrucción en curso, y una petición del canal, ambas

compiten por un ciclo de memoria. Y acá gana el del Canal, porque va derecho al Bus S, le primera el ciclo.

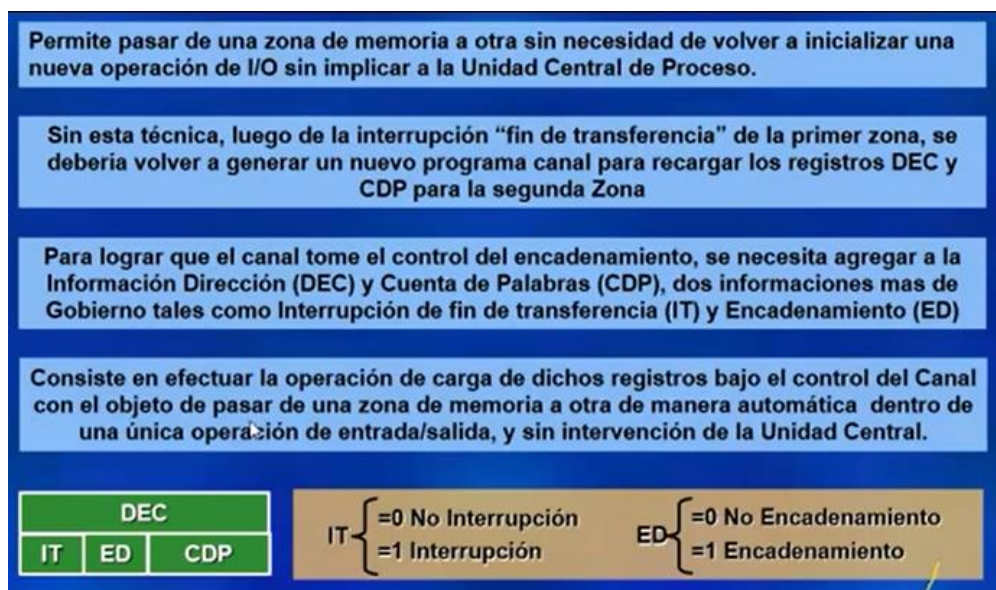
El que está beneficiado acá es el programa en curso, nos ahorramos los señales de PRE (Prueba de Estado), acá tampoco hay que esperar a que el periférico termine, se lo entrega todo al canal para que resuelva todo.

Encadenamiento automático

Es una técnica que viene a ayudar al canal automático para que este pueda transferir información que **NO necesariamente esté contigua en memoria**.

Los bloques van a destinar la ult palabra en actualizar, en darle información al canal para que este se actualice y no tenga que ocupar un ciclo de memoria ni un servicio nuevo para reinicializar la dirección del nuevo bloque.

De esta manera se evita que el haya que entre Bloque y Bloque no inicializar SEN (sentido), DEC, CDP a cada rato.



NOTAS

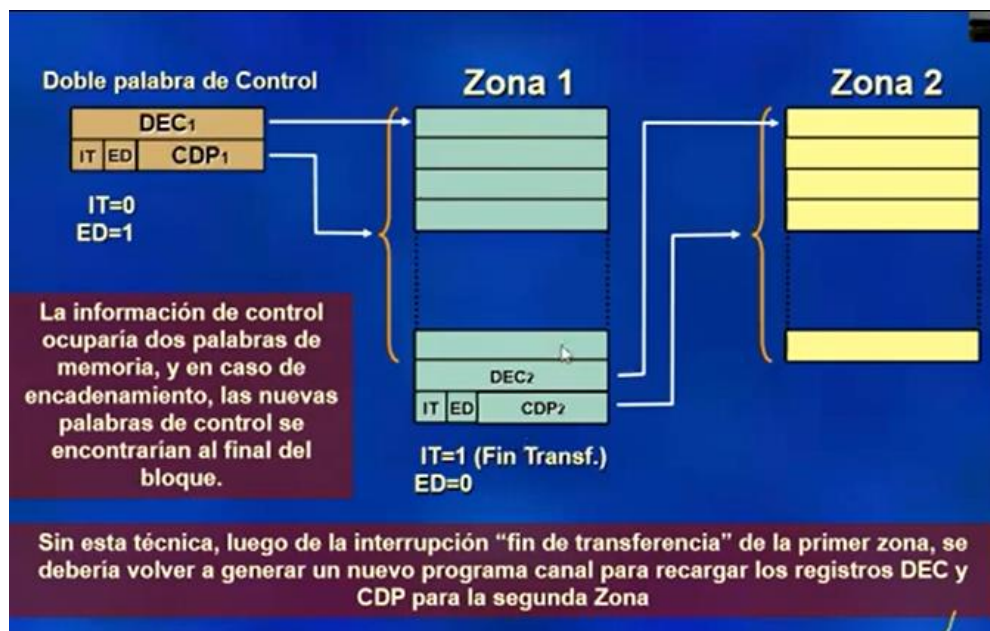
Se necesita 1 palabra de doble longitud (pensando esquema IBM de los años 80, doble palabra), porque en la primera parte constamos la información de IT y ED.

- **IT:**
 - 0 no hay interrupción, hay todavía bloques
 - 1 interrumpi, no hay, hasta aca llegamos no hay mas información
- **ED:**
 - 0 no tomes en cuenta DEC y CDP, seguí nomás con los valores que tenes
 - 1 hay encadenamiento, tomá en cuenta DEC y CDP

IT y ED son mutuamente EXCLUYENTES, si se tiene IT (fin de información) no se puede tener ED (encadenamiento) y viceversa.

En el canal automático, para pasar de una zona de memoria a otra, la unidad central debía desarrollar, después de la interrupción de fin de transferencia correspondiente a la primera zona, un programa cuyo objetivo sería **recargar los registros DEC y CDP con las características de la segunda zona** y relanzar la operación de entrada o de salida.

El **encadenamiento de datos** consiste en efectuar esta operación de carga de dichos registros bajo control del canal, de tal suerte que se pase automáticamente de una zona a otra dentro de una operación de entrada – salida, **sin implicar a la unidad central**. Para conseguirlo se agregan al **FINAL de la zona en curso**, las **nuevas palabras de control y direcciones características de la zona siguiente**, en el caso de que hubiera encadenamiento, y suponiendo, además que las informaciones de control caben en dos palabras de máquina.



Es de la anteúltimo bloque Zona 1 donde se ensoquetan 2 palabras DEC2 y IT, ED, CDP2 que hacen referencia al ÚLTIMO BLOQUE que sería Zona 2.

La operación de entrada/salida definida por las zonas de la figura se descompone según el siguiente esquema:

Nivel de la UC	Nivel del Canal
. Cargar la doble palabra de control 1 en los registros de canal . Iniciar la transferencia	. Transferir la primera zona . Cargar las palabras de control 2, puesto que el encadenamiento está pedido en 1 . Transferir la segunda zona . Generar una interrupción pedida en 2

VENTAJAS

- Mejora la performance de este modelo de canal que tiene a su cargo el control de las transferencias.
- Ahorrar 2 palabras en las arquitecturas que manejaba IBM en su tiempo, era valioso.

Noción de programa canal

IBM 360, lo que quería hacer era trabajar con distintas zonas de memoria de manera flexible y lo más económicamente posible, en los mainframes.

Una unidad de canal, que ahora es programable, que trabaja con la noción del programa. Ese programa va a estar en la zona de memoria, tenemos 1 sola de memoria. Y el canal no está pensado para tener su propia memoria.

Entonces ese programa canal, va a tener el objetivo, se va a ejecutar con los mismos derechos que cualquier instrucción de Programa, de manera de poder darle información necesaria al canal para hacer encadenamiento de datos y tareas mucho más complejas.

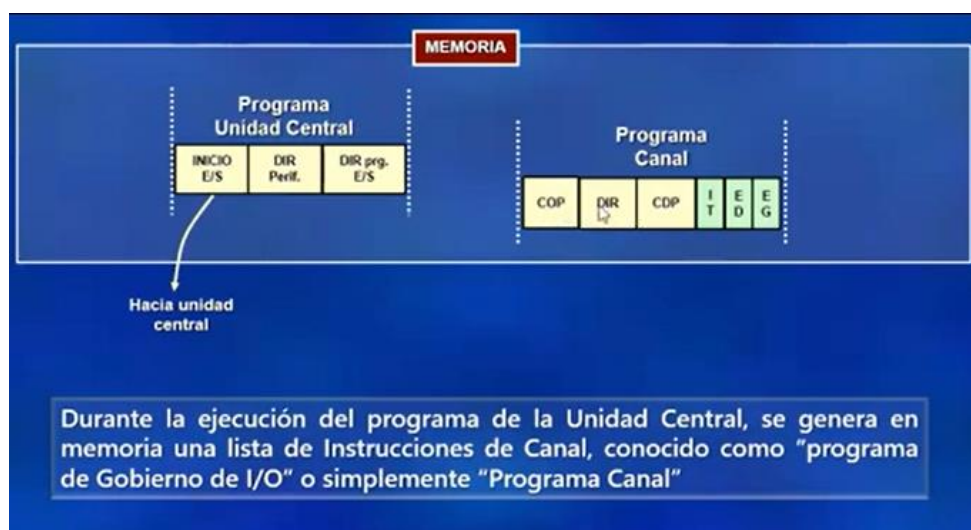
Es una unidad programable capaz de leer, codificar, y ejecutar un programa de gobierno de I/O registrado en memoria central.

El programa de Gobierno se ejecutará con los mismos derechos con los que la Unidad Central lee, decodifica, y ejecuta un programa de procesamiento.

Convenciones:

- La Unidad Central lee, decodifica, y ejecuta Instrucciones
- El Canal lee, decodifica, y ejecuta instrucciones de Canal
- El controlador de Periférico recibe, decodifica, y ejecuta códigos de operación de periféricos

Programa canal



Lista de instrucciones de canal

- COP - Código de operación
- DIR - Dirección de la 1er palabra
- CDP - Cantidad de palabras

- IT - Interrupción de fin de transferencia
- ED - Hay o no encadenamiento
- EG - Encadenamiento de gobierno, tiene que encadenar programas y encadenar datos, hay un doble encadenamiento. Entonces tiene que encadenar 2 conceptos ahora

El Canal busca en memoria la 1º instrucción de Canal que consta de:

COP: Código Operación de Periférico

DIR: Dirección de la 1º Palabra

CDP: Cuenta de Palabras

IT/ED/EG: Indicadores Varios (Interrupción, Encad. Datos, Encad. De Gobierno)

Así como esta instrucción de la unidad central inicializa la transferencia, en este caso solamente le va a dar.

- INICIO E/S - Es una instrucción
- DIR PERIF - Dirección del periférico
- DIR PRG E/S - Dirección del programa memoria que va a manejar esa transferencia, que va a efectuar los encadenamientos

Durante la ejecución de la instrucción de la Unidad Central, se genera un Programa de Entrada/Salida, es el **Programa Canal**.

Doble encadenamiento

El Canal tiene que tener ahora un **Decodificador de dirección** de periféricos.

Entonces el programa canal, puede desarrollarse sin la UC.

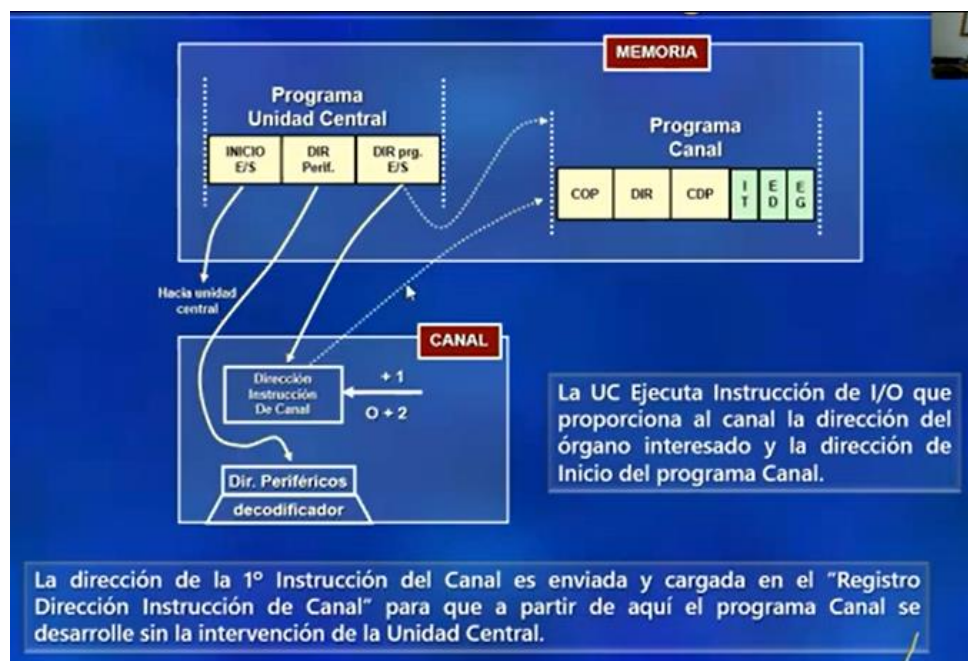
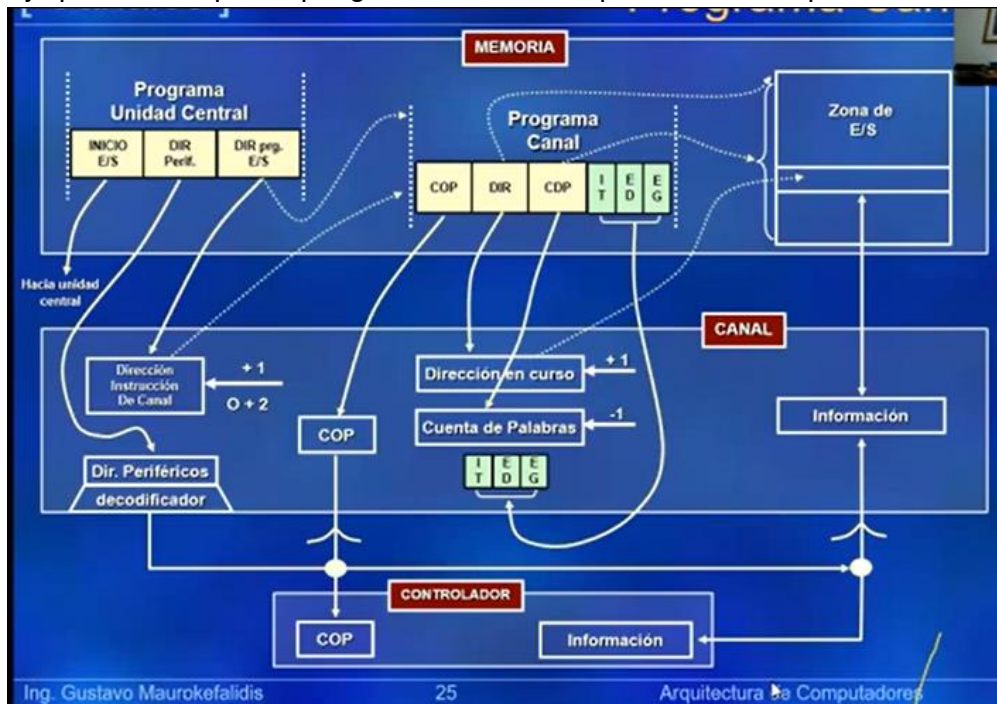


Diagrama expandido

EG sirve para encadenar también instrucciones de canal.

Se agrega COP, el programa canal tiene un **Código de operación que es una instrucción interpretable por el Controlador**. Acá ya estamos hablando con el controlador.

Ej. que Grabe, que se ponga en estado de espera, etc eso le puede decir al Controlador.



Las demandas de transferencias elementales de información emanan del controlador de periférico quien trabaja a su propio ritmo.

Por cada transferencia de información, se actualizan dirección en curso y cuenta de palabra.

Se actualizan DIR y CDP.

La transferencia termina cuando cuenta de palabras=0 o por señal "fin de transferencia" del controlador de periférico.

El canal controla entonces los tres indicadores:

Si ED=0 canal pasa a siguiente instrucción de canal ignorando COP (Cód.Op. Periférico)

Si EG=0 canal pasa a siguiente instrucción de canal usando el nuevo COP

Si IT= V Se genera interrupción al final de una transferencia

El canal de los calculadores evolucionados es una unidad programable, capaz de leer, de codificar y ejecutar un programa de gobierno de entrada/salida o programa de canal, registrado en memoria central, con los mismos derechos que la unidad central lee, decodifica y ejecuta un programa de procesamiento.

"El programa canal está compuesto de tres grandes órganos: la memoria, el canal y el controlador. La memoria se encuentra dividida por 3 partes: programa unidad central, programa de canal y zona de E/S. Dentro de la zona de programa unidad central están las distintas operaciones sum, res, etc, cuando llega una operación de transferencia, a diferencia del canal automático (indicaba dirección de inicio y cantidad de palabras), en el programa canal la instrucción de transferencia de la unidad central de proceso provee una

indicación de inicio de transferencia, la dirección del periférico y la dirección de la primer línea del programa canal que se va a encargar de transferir. La dirección del periférico va a ir a un decodificador (dentro del canal) que habilitará, por medio del controlador, el periférico deseado.

La dirección del programa de E/S irá a un registro del canal llamado dirección de instrucción de canal que puede incrementarse para poder acceder a cada una de las líneas del programa canal que se está ejecutando

En el programa canal cada palabra tiene:

- Código de Operación de Periférico (COP): que le manda al periférico. Una orden específica.
- Dirección. Será la dirección transmitida a dirección en curso.
- Cantidad de palabras a transmitir
- 3 banderas de control: Si hay o no interrupción, si hay o no encadenamiento de datos y si hay o no encadenamiento de gobierno (el programa también puede estar fragmentado)

Además, sigue teniendo dirección en curso y cuenta de palabras y están referidas a la zona de memoria de E/S, donde están los bloques de memoria que se van a transferir. El registro "información" dentro de canal es el tampón, que es el encargado de mantener la información hasta que el controlador le permita transmitir la información.

Una transferencia elemental es una transferencia de un dato de la memoria hacia el periférico o del periférico hacia la memoria. La transferencia elemental va a estar controlada por el controlador del periférico; es el quien marca la necesidad de leer o grabar una palabra de memoria. Ese gobierno es parte del controlador. Por cada transferencia de información se actualiza la dirección en curso y la cuenta de palabras."

Encadenamiento de Gobierno (EG)

"Cuando hay encadenamiento de gobierno significa que las palabras del programa canal dejaron de ser contiguas y tiene que ir a actualizar el registro dirección de instrucción de canal para continuar con el programa de gobierno. En ese momento, suspende la transferencia de datos momentáneamente porque tiene que pasar a actualizar la dirección del programa canal a una nueva dirección. Para hacer esto, una palabra completa del programa canal va a estar destinada a actualizar la nueva dirección en el registro dirección de instrucción de canal. Esto implica que la información que va a venir a continuación en el programa canal se va a perder, no va ser de transferencia, sino que va a ser de actualización de la dirección de instrucción de canal. En el caso que haya encadenamiento de gobierno hay que volver a replantear cuál es la dirección de la nueva instrucción de canal (la dirección de la palabra a ejecutar del programa canal). Esa dirección es la palabra que se va a obtener cuando el contador de dirección de instrucción de canal haga un +1 en el programa canal, y cuando aumente +2 va a ser la dirección de la palabra en el programa canal a partir de la cual tiene que tomar el encadenamiento. En otras palabras, la dirección de instrucción de canal se tiene que incrementar en 2 cuando hay encadenamiento de gobierno: tendrá que hacer una transferencia y después la búsqueda de la próxima instrucción de programa canal."

Funcionamiento

La operación de una entrada/salida consiste en:

1. Durante la ejecución del programa de la unidad central, se genera en memoria una lista de instrucciones de canal conocido como programa de gobierno de E/S, o también llamado programa canal.
2. La unidad central ejecuta una instrucción de entrada/salida que proporciona al canal la dirección del órgano periférico interesado y la de la primera instrucción del programa canal. Si el órgano periférico designado está libre, se le pone en actividad (si está ocupado la unidad central es avisada de que el programa de canal no debe arrancar). La dirección de la primera instrucción de canal es enviada y almacenada en el contador de instrucciones del canal. A partir de este instante, el programa de canal se desarrollará sin intervención de la unidad central.
3. El canal va a buscar en memoria la primera instrucción de canal, que consta de los siguientes elementos:

C. O. Periférica	Dirección 1ª palabra	Cuenta de palabras	Indicadores
------------------	----------------------	--------------------	-------------

El canal transmite al controlador del periférico el código de operación que le es destinado. En este momento, la operación de entrada/salida queda inicializada y un proceso idéntico se desarrolla cada vez que el canal pide una nueva instrucción de canal

4. Las demandas de transferencia elemental de información, una vez inicializada la operación de entrada /salida, emanan del controlador de periférico, que trabaja a su propio ritmo. Al recibir una de estas demandas, el canal efectúa la transferencia pedida utilizando y actualizando las informaciones dirección en curso (que sustituye a dirección de la primera palabra) y cuenta de palabras.
5. Una transferencia se terminará cuando sea nulo el contenido del contador de palabras o bien cuando se reciba una señal de fin de transferencia procedente del controlador de periférico. El canal comprobará entonces los tres indicadores para saber lo que ha de hacer:
 - a. Encadenamiento de datos: si este indicador está puesto, el canal pasará a la siguiente instrucción de canal, ignorando el nuevo código de operación periférica.
 - b. Encadenamiento de gobierno: si el indicador está puesto, el canal pasará a la siguiente instrucción de canal, utilizando el nuevo COP.
 - c. Interrupción: se generará una interrupción al final de la transferencia, si el correspondiente indicador está activado

El programa de canal está controlado por un cierto número de instrucciones de entrada/salida, entre las cuales he aquí las más importantes:

Instrucciones de E/S	
Inicio de E/S	Permite inicializar una operación de E/S, señalando el periférico y la dirección de la primera instrucción de canal.
Detención de E/S	Permite detener una operación de E/S, señalando el periférico y la dirección de la primera instrucción de canal.
Prueba de E/S	Se emplea para obtener informaciones acerca del estado actual de una operación de E/S, sin alterar el funcionamiento de esta.
Prueba de órgano periférico	Permite obtener informaciones, más precisas que con la instrucción de prueba de E/S, acerca del estado de un órgano periférico; tales informaciones son suministradas por el controlador del periférico.
Identificación de una interrupción	Esta instrucción permite, tras recibirse una interrupción emitida por el canal, identificar al órgano provocador de la interrupción y conocer la causa de ella. La información de estado devuelta por el canal comprende, por una parte, la dirección del periférico causante de la interrupción, por otra parte, indicadores concernientes a la operación que ha dado lugar a la interrupción. Por ejemplo, desbordamiento de datos, contador de palabras nulo, final de transferencia, longitud incorrecta, etc.

Dispositivos de Entrada Salida - PERIFÉRICOS

Se denominan **periféricos** tanto a las unidades o dispositivos a través de los cuales el ordenador se comunica con el mundo exterior, como a los sistemas que almacenan o archivan la información, sirviendo de memoria auxiliar de la memoria principal. Se entenderá por periférico a todo conjunto de dispositivos que, sin pertenecer al núcleo fundamental de la CPU-Memoria Central, permitan realizar operaciones de E/S, complementarias al proceso de datos que realiza la CPU.

Los periféricos se conectan a la CPU a través de grupos de hilos que se conoce como buses. El bus que conecta la CPU con los otros elementos del procesador se conoce como bus local o bus de la CPU. Las conexiones entre los periféricos y los controladores se realizan a través de un bus más general llamado bus del sistema. Algunos periféricos requieren un bus especializado que se adapte a su velocidad de transferencia, sus niveles de tensión, la naturaleza de sus señales de control y otros requerimientos. A estos buses se les llama bus de entrada/salida o bus de expansión.

UNIDADES DE ENTRADA

TECLADO

El teclado es básicamente un hardware periférico de entrada que provee de una serie de interruptores, llamados teclas, dispuestos en una determinada distribución, adecuada a diferentes lenguajes y modos de escritura. Estas teclas, que pueden alcanzar hasta las 108, pueden ser mecánicas o electrónicas, y su principal función es la permitirnos ingresar texto u órdenes a la computadora.

El teclado es un periférico de entrada inspirado en una máquina de escribir que posee teclas en una disposición denominada QWERTY y que se basa en la idea de que las teclas más utilizadas (en inglés) estén lo más separadas posibles entre sí.

Funcionamiento del teclado

Los teclados poseen internamente una matriz de teclas y un circuito integrado que se encarga de supervisar esta matriz y detectar las pulsaciones. Cuando se detecta una pulsación, o la liberación de una tecla, se escribe su código en un buffer interno del teclado y a continuación se sigue un proceso hasta que su código queda almacenado en memoria, y de forma básica es el siguiente:

1. El circuito del teclado detecta la pulsación. Genera el código de muestreo (llamado scan code) de la tecla pulsada y lo envía al puerto del teclado.
2. Se informa al procesador del suceso.
3. El procesador accede a la dirección de memoria donde se guarda el código de la tecla pulsada y la lee.
4. Verifica si hay alguna tecla de control pulsada: mayus, ctrl, alt, etc.
5. Traduce y guarda el código de la tecla pulsada hasta que el software solicite la entrada del teclado.

Cuando se suelta la tecla el circuito del teclado genera un código de tecla soltada (break code). El carácter que aparezca, se corresponde con la tecla pulsada y depende del idioma utilizado (página de códigos) en el sistema operativo. El sistema de transmisión del teclado es bidireccional, controlado por un reloj, en el que se mueven bloques de datos de 11 bits de longitud, que se corresponden con la siguiente secuencia: 1 bit de inicio, 8 bits de datos que se transmiten, 1 bit de paridad y 1 bit de parada.

Tecnología de Teclado

Desde el punto de vista del funcionamiento de las teclas en sí, existen dos tipos fundamentales de teclado: teclado de contacto y teclado capacitivo.

El teclado de contacto puede ser de dos tipos:

- Mecánico: usa pequeños interruptores individuales para cada tecla. Cuando se presiona una tecla se cierra el interruptor y permite el paso de corriente. Tiene el inconveniente de ser bastante vulnerable a la suciedad, de forma que después de un tiempo de uso es posible que alguna de sus piezas empiece a fallar; este teclado se nota porque al pulsar una tecla, suena el tic del pulsador.
- Membrana: las teclas llevan una membrana de goma que al pulsar hace que se unan dos pistas conductoras en la parte inferior y permite el paso de corriente: son similares a las que llevan las calculadoras. Estas membranas pueden ser individuales para cada tecla o bien una lámina completa con membranas que se mueven con cada tecla. Al pulsar una tecla, no suena como el mecánico.

El teclado capacitivo es de más capacidad y más caro que el de contacto, teniendo una vida de uso bastante más larga. Éste teclado está construido sobre una tarjeta de circuito impreso grabada, de tal forma que cuando se pulsa una tecla, esta hace presión sobre un condensador, el cual produce una señal eléctrica que es detectada e interpretada por el procesador de teclado.

Tipos de teclado

- Teclado Ergonómico: Se basan en el principio que dividiendo el teclado principal colocando en ángulo cada una de las mitades, los codos descansan en una posición

mucho más natural, y cambiando la curvatura del teclado y añadiendo un pequeño “reposamuñecas”, el ángulo de escritura es mucho más cómodo para el usuario. Pero tienen una desventaja, y es que hace falta acostumbrarse a una disposición de teclas muy diferente.

- Teclado Programable: El propio teclado lleva un microcontrolador interno (que se comunica con la controladora de teclado de la placa base) con un programa integrado que interpreta las señales producidas al cerrarse el circuito cuando dos terminales (tecla y circuito integrado) entran en contacto. Este programa reside en una ROM, la cual puede almacenar muchos otros datos, además del código del teclado (país) y la posición de las teclas, pero para interpretarlos, se ha de instalar un driver o controlador del dispositivo que interprete las señales.
- Teclados inalámbricos: Caracterizados por la ausencia de cable. Los primeros utilizaban infrarrojo actualmente utilizan la radiofrecuencia. Su gran virtud es que no necesitamos tener una línea visual entre el teclado y el sensor, lo que nos da una extraordinaria movilidad. El gran problema de estos dispositivos ha sido siempre el gasto de pilas. La controladora no reside en el propio teclado, sino en el receptor que se conecta al conector de teclado en el PC.

RATÓN

El **mouse** es uno de los periféricos de entrada que forman parte de un computador, a través de él se puede interactuar directamente con la computadora mediante un puntero (indicador) que se muestra en la pantalla. Básicamente es un dispositivo de hardware cuya función principal es controlar las funciones que le ordena el usuario, debido a que gracias al complemento de su puntero en la pantalla puede desplazarse a través de ella y por medio de un botón puede seleccionar cualquiera de las aplicaciones que estén contenidas en el ordenador. La función principal del ratón es transmitir los movimientos de nuestra mano sobre una superficie plana hacia el ordenador. Allí, el software denominado driver se encarga realmente de transformarlo a un movimiento del puntero por la pantalla dependiendo de varios parámetros. En el momento de activar el ratón, se asocia su posición con la del cursor en la pantalla. Si desplazamos sobre una superficie el ratón, el cursor seguirá dichos movimientos.

Existen diferentes tipos de ratones:

- Mecánicos: Son sencillos y tienen un bajo coste. Se basan en una bola de silicona que gira en la parte inferior del ratón a medida que desplazábamos éste. Dicha bola hace contacto con dos rodillos, uno perpendicular al ratón y otro transversal, de forma que uno recoge los movimientos de la bola en sentido horizontal y el otro en sentido vertical. En cada extremo de los ejes donde están situados los rodillos, existe una pequeña rueda conocida como “codificador”, que gira en torno a cada rodillo. Estas ruedas poseen en su superficie, y a modo de radios, una serie de contactos de metal, que a medida que gira la rueda toca con dos pequeñas barras fijas conectadas al circuito integrado en el ratón. Cada vez que se produce contacto entre el material conductor de la rueda y las barras, se origina una señal eléctrica. Así, el número de señales indicará la cantidad de puntos que han pasado éstas, lo que implica que, a mayor número de señales, mayor distancia habrá recorrido el ratón. Tras convertir el movimiento en señales eléctricas, se enviaban al software del ordenador por medio del cable.

- **Ratones Ópticos:** Los ratones ópticos carecen de bola y rodillos, y poseen unos fotosensores o sensores ópticos que detectan los cambios en los patrones de la superficie por la que se mueve el ratón. Estos ratones ópticos disponen de un LED que ilumina la superficie sobre la que se mueve el ratón. Una microcámara se encarga de digitalizar con un CCD una pequeña región y luego un circuito integrado compara la imagen producida con la imagen anterior para decidir hacia donde se está desplazando el ratón.
- **Ratones Inalámbricos:** en este caso el dispositivo carece de un cable que lo comunique con la computadora, sino que utiliza algún tipo de tecnología inalámbrica. Para ello requiere un receptor de la señal inalámbrica que produce, mediante baterías, el mouse. El receptor normalmente se conecta a la computadora por USB, o por PS/2. La tecnología inalámbrica usada puede ser:
 - **Radio Frecuencia (RF):** Es el tipo más común y económico de este tipo de tecnologías. Funciona enviando una señal a una frecuencia de 2.4Ghz, popular en la telefonía móvil o celular.
 - **Infrarrojo (IR):** Esta tecnología utiliza una señal de onda infrarroja como medio de transmisión de datos, popular también entre los controles o mandos remotos de televisiones.
 - **Bluetooth (BT):** Bluetooth es la tecnología más reciente como transmisión inalámbrica (estándar IEEE 802.15.1), que cuenta con cierto éxito en otros dispositivos.

UNIDADES DE SALIDA

MONITORES

El monitor es un dispositivo electrónico de salida (puede considerarse entrada/salida si es táctil) de la computadora en el que se muestran las imágenes y textos generados por medio de un adaptador gráfico o de video de ésta. El término monitor se refiere normalmente a la pantalla de vídeo, y su función principal y única es la de permitir al usuario interactuar con la computadora. El monitor es una de las partes más importantes de la computadora, ya que es el primer medio por el que el usuario conoce los resultados de su trabajo. El monitor se conecta a la tarjeta gráfica, estando dicha tarjeta integrada en la placa base, o conectada a un bus de expansión.

TIPOS DE MONITORES

- **Monitor CRT:** Un monitor CRT tiene un tubo de rayos catódicos en la parte trasera. La electricidad causa que el rayo catódico libere electrones, los cuales aceleran y golpean la pantalla. A medida que los electrones chocan con la pantalla, que está hecha con fósforo con frecuencia, la misma fluoresce para crear una imagen. La interacción entre los electrones y el fósforo genera tres colores: rojo, verde y azul. El monitor CRT utiliza estos colores en distintas combinaciones para formar todos los otros colores necesarios. Es por esto que el cable de monitor que se conecta con la computadora es a veces llamado el cable RGB. El monitor CRT utiliza información enviada desde la computadora para crear imágenes una por una, esencialmente "refrescando" la pantalla un cierto número de veces por segundo. Cuanto más alto

sea el número de refrescos que hace la pantalla por segundo, más suaves serán las imágenes en la pantalla. Una tasa de refresco baja causa el efecto titilante que se ve comúnmente en viejos monitores CRT y televisores.

Características:

- Excelente calidad de imagen (definición, contraste, luminosidad).
 - Económico.
 - Tecnología robusta.
 - Resolución de alta calidad.
 - Presenta parpadeo por el refresco de imagen
 - Alto peso y tamaño.
- Pantallas LCD: A este tipo de tecnología se le conoce por el nombre de pantalla o display LCD, sus siglas en inglés significan “Liquid Crystal Display”.

Características:

- Poco peso y tamaño.
- Buena calidad de colores.
- No contiene parpadeo.
- Poco consumo de energía.
- Poca generación de calor.
- No genera radiaciones eléctricas y magnéticas.
- Alto costo.
- Angulo limitado de visibilidad.
- Brillo limitado.
- Bajo tiempo de respuesta de píxeles.
- Contiene Mercurio.

IMPRESORAS

La impresora es un periférico del ordenador, que está conectado a la unidad central de procesamiento de una computadora, su función es hacer una copia física de aquellos documentos que han sido almacenados en un formato electrónico. Estos documentos pueden ser textos o imágenes que son impresos en una hoja o transparencia utilizando cartuchos de tintas o tecnología láser. Son diseñadas para realizar trabajos repetitivos de poco volumen. Sin embargo, las impresoras son generalmente dispositivos lentos y el gasto por página es relativamente alto. La mayoría de las impresoras, están unidas a un ordenador por medio de un cable, aunque también existen las impresoras de red, las cuales poseen una interfaz de red interna, que le permite a cualquier usuario de la red imprimir documentos.

Algunas de las características que tenemos de las impresoras son:

- Velocidad de impresión: paginas por minutos (ppm) o en caracteres por segundos (cps),
- Resolución: se refiere a la calidad de la impresión, es decir, cantidad de puntos (píxeles) que la impresora puede crear sobre el papel
- Buffer de memoria (Zona de almacenamiento temporal de datos en la impresora)
- Interfaz de conexión
- Memoria Integrada

TIPOS DE IMPRESORAS

1. Impresora de Agujas

También llamadas matriciales, utilizan una serie de agujas o pines dispuestos en forma vertical, que se disparan mediante electroimanes, impactando en una cinta entintada que se encuentra en la aguja y el papel, forma un punto sobre el papel. Así, los caracteres están formados por puntos creados por el choque de un número determinado de agujas cada vez. El número de puntos que constituye un carácter (o el número de agujas) determina la resolución, y por tanto, la calidad de impresión, siendo las más comunes las de 9, 18 y 24 agujas.



Funcionamiento

1. La computadora envía los caracteres a imprimir por el puerto de la impresora.
2. Los caracteres son almacenados en el buffer de la impresora.
3. El procesador de la impresora reconoce cada uno de los caracteres enviados gracias a una memoria ROM en la que están grabadas qué agujas son necesarias accionar para formar el carácter.
4. El cabezal se sitúa en la posición adecuada y el procesador envía una señal eléctrica a las agujas que deben accionarse. Cada aguja en su extremo está compuesta de un pequeño martillo que en presencia de tensión golpea a la aguja correspondiente gracias a un electroimán.
5. La aguja golpea sobre una cinta impregnada en tinta, provocando la transferencia de la tinta al papel. Después, la aguja vuelve a su posición original y el cabezal se encuentra listo para realizar la siguiente ráfaga.
6. El cabezal irá accionando distintas combinaciones de agujas a medida que se desplaza por la página para ir representando los distintos caracteres.
7. Una vez el cabezal ha terminado de imprimir una línea, el tambor (rodillo) que arrastra el papel girará un poco subiendo el papel para que sea impresa una nueva línea.

Ventajas:

- Pueden trabajar con papel continuo en lugar de requerir hojas individuales.
- Conforme se termina la tinta, la impresión pierde intensidad gradualmente en lugar de terminar repentinamente durante un trabajo.
- Bajo costo de mantenimiento (el costo de la cinta entintada es insignificante comparado con el tóner de una láser o los cartuchos de tinta de inyección).

- Fiables
- Conveniente para grandes volúmenes de impresión que no necesiten tanta calidad.

Desventajas:

- Baja velocidad
- Son muy ruidosas (debido al sonido de las agujas que golpean el papel)
- Calidad inferior comparada con otras del mercado

2. Impresora de Inyección a tinta:

Imprime línea a línea con la ayuda de un cabezal de impresión móvil que se desplaza (junto con los cartuchos) por todo el ancho de la hoja, de forma parecida a como lo hace la de agujas, pero a diferencia de esta usa un sistema de no impacto, por el cual el cabezal no toca en ningún momento la hoja de papel, sino que pulveriza pequeños chorros de tinta sobre el mismo. Para esto, el cabezal de impresión tiene un cierto número de pequeños depósitos llenos de tinta, cada uno con un orificio (inyector), de un grosor inferior al de un cabello, por los que se expulsa la tinta formando minúsculos puntos sobre el papel. Según el dato a imprimir se expulsará la tinta por unos orificios u otros.

Características:

- Bastante silenciosa
- Mucha mejor calidad y mayor velocidad que las matriciales
- Bajo precio
- Más rápida que las de aguja, aunque más lenta que las láser.

3. Impresora Láser:

La impresora láser se creó en la década de los 70 y desde la época hasta ahora han evolucionado para consolidarse como una de las impresoras más rápida, silenciosa y eficiente, tanto así que en pocos segundos pueden imprimirse textos, gráficos e imágenes de gran calidad. El dispositivo central que utiliza este tipo de impresión es un material fotosensible que se descarga con luz, denominado cilindro fotorreceptor. Cuando es enviado un documento a la impresora, todo lo que se desee imprimir (caracteres, números, imágenes) va a ser dibujado sobre el cilindro con un láser. La carga del láser va a ionizar al cilindro y también le va a poner una carga electroestática. Este pequeño laser se encarga de cargar electroestáticamente diminutas partes del cilindro, con lo cual se forma la imagen electrostática (no visible) de nuestro documento a imprimir sobre este cilindro fotorreceptor. Posteriormente el cilindro es bañado por un polvo muy fino de color negro, el cual posee carga contraria a la del cilindro y por lo tanto es adherido a las partes que se encuentran con una carga distinta en el cilindro. Esto se debe a la ley de cargas, la cual enuncia que cargas iguales se repelen y cargas diferentes se atraen. Las partes cargadas con igual carga repelen este polvo llamado tóner (tinta seca) con lo cual queda formada la imagen visible sobre el tambor. Hay que aclarar también, que el papel cuando ingresa al sistema se carga electroestáticamente con una carga distinta a la que tiene el cilindro. Por lo tanto, mientras el cilindro sigue girando y el pasa el papel por ahí se produce como una lluvia de polvo del cilindro al papel que dibuja sobre el papel el carácter, gráfico, etc. que se desea imprimir. Hasta este

momento, el polvo se encuentra sobre el papel con el dibujo deseado pero no está adherido, es decir, se puede perder fácilmente el dibujo formado sobre la hoja. A continuación, el tóner que se transfirió al papel es adherido a éste por medio de un par de rodillos, encargados de generar calor y presión sobre la hoja; a esta unidad se le denomina de fijado y es el paso final de la impresión láser. Para regresar al estado inicial, el tóner restante en el cilindro fotorreceptor es limpiado por medio de una lámina plástica y al mismo tiempo se incide luz sobre el cilindro para dejarlo completamente descargado para un nuevo proceso de impresión.

Características:

- Muy rápidas
- Muy silenciosas
- Altísima calidad de impresión, superior a las impresoras a tinta
- Reducción de costos: a pesar que el tóner es más costoso que los cartuchos de tinta de otras impresoras, el número de impresiones que pueden hacerse es mucho mayor, lo que se traduce en un valor por impresión mucho menor.
- Se necesita un cartucho de tóner por cada color. Así pues, habría que elegir entre una impresora capaz de imprimir en color aunque se pierda algo de resolución (inyección) o una de gran calidad (láser) pero resignarse a no imprimir en color

UNIDADES DE MEMORIA MASIVA AUXILIAR

Pueden ser como Discos Duros, Cintas magnéticas, Discos flash

Se denomina **dispositivo de almacenamiento masivo** a las unidades de disco que almacenan los bits utilizando tecnología magnética u óptica, y que permiten el almacenamiento “masivo” de bits por su bajo costo. Estas memorias deben ser **no volátiles**, que en su mayoría se pueden leer y escribir, y reciben el nombre de memorias externas, masivas o secundarias. Si bien es necesario que esas memorias estén asociada a la CPU para acceder a ellas, el soporte de bits, por ejemplo, un CD, puede guardarse en un cajón sin que se pierda la información grabada en él.

Las unidades de almacenamiento masivo corresponden a aquellos dispositivos encargados de guardar la información permanente los datos y/o programas para ser utilizados en el momento adecuado y poder ser modificados, vueltos a guardar y recuperados cuando se desee. Son por lo tanto dispositivos que guardan permanentemente la información en ausencia de alimentación, siendo mucho de ellos capaces de ser transportables, es decir, poder llevar la información a otro equipo o guardarla como copia de seguridad.

En un principio la CPU obtiene un programa y los datos necesarios desde el almacenamiento secundario. El programa se copia literalmente desde su ubicación en el almacenamiento secundario a la memoria principal (DRAM) de la computadora, lo que permite ejecutarse con mayor rapidez. Además, la CPU solo tiene acceso directo a esta última por medio del bus de direcciones. De la misma forma, cualquier dato necesitado por el programa se copia desde el almacenamiento secundario hacia la memoria y esos datos retornaran al almacenamiento secundario si se los modifica y se quiere guardar los cambios en forma permanente. Los dispositivos de E/S más utilizados para “soportar” estas

memorias son el disco rígido y los dispositivos de tecnología óptica, como la unidad de DVD.

1. Discos Magnéticos

Son los dispositivos de E/S de mayor uso en nuestros días. Los discos magnéticos pueden ser rígidos, dispuestos en unidades de cabezas fijas o móviles, en grupo de uno o más platos.

Un disco magnético es un plato circular construido con un material no magnético, llamada sustrato, cubierto por un material magnetizable. Tradicionalmente, el sustrato es aluminio o una aleación de aluminio. Recientemente, se han utilizado sustratos de cristal. Los datos se graban y después se recuperan del disco a través de una bobina, llamada cabeza, en muchos sistemas, hay dos cabezas, una de lectura y otra de escritura. La cabeza es un dispositivo relativamente pequeño, capaz de leer o escribir en una zona del plato que rota bajo ella. Esto da lugar a que los datos se organicen en un conjunto de anillos concéntricos en el plato, llamado pistas. Cada pista es del mismo ancho que la cabeza. Usualmente hay cientos de pistas por platos. Las pistas adyacentes están separadas por bandas vacías. Esto previene, o por lo menos minimiza, los errores debido a desalineamientos de la cabeza o simplemente a interferencias del campo magnético. Los datos se transfieren al y desde el disco en sectores. Normalmente hay cientos de sectores por pista, y estos pueden tener una longitud variable o fija. En la mayoría de los sistemas de hoy se utilizan sectores de longitud fija (512 bytes).

En la mayoría de los discos, la cubierta magnetizable se aplica a ambas caras del plato. Denominándose estos discos de doble superficie. Algunos discos, menos caros, son de una superficie.

Algunas unidades de discos poseen varios platos apilados verticalmente y separados por una distancia de alrededor de una pulgada. Disponen de varios brazos. Los discos de varios platos utilizan una cabeza que se mueve, con una cabeza de lectura-escritura por cada superficie del plato. El conjunto de todas las pistas que tienen la misma posición relativa en el plato se denomina cilindro.

Operación de Lectura y Escritura

Durante una operación de lectura o escritura, la cabeza permanece quieta mientras el plato rota bajo ella:

El mecanismo de escritura se basa en el hecho de que un flujo eléctrico atravesando una bobina crea un campo magnético. Se envían pulsos eléctricos a la cabeza de escritura, y se graban los patrones magnéticos en la superficie bajo ella, con patrones diferentes para corrientes positivas y negativas. La propia cabeza de escritura está hecha de un material fácilmente magnetizable y tiene forma rectangular con un agujero en un lado y varias vueltas de cable conductor a lo largo del lado opuesto. Una corriente eléctrica en el cable induce un campo magnético a lo largo del agujero que magnetiza una pequeña área del medio grabable. Si se cambia la dirección de la corriente, cambia el sentido de magnetización del medio de grabación.

El mecanismo de lectura se basa en el hecho de que un campo magnético en movimiento respecto a una bobina, induce una corriente eléctrica en la bobina. Cuando la superficie del disco pasa bajo la cabeza, en esta se genera una corriente de la misma polaridad que la de la que produjo la grabación magnética. La estructura de la cabeza de lectura es, en este caso, esencialmente la misma que la de escritura y, por tanto, se puede usar la misma cabeza para ambas operaciones.

Este mecanismo de lectura es muy antiguo. Actualmente, es necesario una cabeza de lectura separada posicionada, por conveniencia, cerca de la cabeza de escritura. La cabeza de lectura consiste en un sensor magnetoresistivo parcialmente blindado. Funciona con una resistencia eléctrica que depende de la dirección de la magnetización del medio que se mueve bajo él.

2. Discos Estado Solido (SSD)

Una memoria de estado sólido es un dispositivo de almacenamiento secundario hecho con componentes electrónicos en estado sólido pensado para utilizarse como memoria auxiliar. La unidad de estado sólido, SSD (solid-state drive) es un tipo de dispositivo de almacenamiento de datos que utiliza memoria no volátil, como la memoria flash, para almacenar datos, en lugar de los platos o discos magnéticos de las unidades de discos duros (HDD) convencionales. Los SSD almacenan los archivos en microchips con memorias flash interconectadas entre sí.

En comparación con los discos duros tradicionales, las unidades de estado sólido son menos sensibles a los golpes al no tener partes móviles, son prácticamente inaudibles, y poseen un menor tiempo de acceso y de latencia, lo que se traduce en una mejora del rendimiento exponencial en los tiempos de carga de los sistemas operativos. En contrapartida, su vida útil es muy inferior, ya que tienen un número limitado de ciclos de escritura, pudiendo producirse la pérdida absoluta de los datos de forma inesperada e irre recuperable. Las SSD hacen uso de la misma interfaz SATA que los discos duros, por lo que son fácilmente intercambiables sin tener que recurrir a adaptadores o tarjetas de expansión para compatibilizarlos con el equipo.

PRINCIPALES VENTAJAS	SSD	HDD
CAPACIDAD	En general entre 256 GB y 4 TB	En general entre 1 y 10 TB
CONSUMO	Menor consumo	Mayor consumo
COSTE	Bastante más caros	Mucho más económicos
RUIDO	Más silencioso por no tener partes móviles	Algo más ruidoso por tener partes móviles
VIBRACIONES	No vibra por no tener partes móviles	El giro de sus discos puede provocar leves vibraciones
FRAGMENTACIÓN	No tiene	Puede darse
DURABILIDAD	Sus celdas pueden reescribirse un número limitado de veces	Con partes mecánicas que pueden dañarse con movimientos
TIEMPO DE ARRANQUE DE SO	7 segundos	16 segundos
TRANSFERENCIA DE DATOS	En general, entre 200 y 550 MB/s	En general entre 50 y 150 MB/s
AFECTADO POR EL MAGNETISMO	No	El magnetismo puede eliminar datos

Arquitectura 8086

Link de video: <https://www.youtube.com/watch?v=BCvq-S622qk&list=PLqTKn0MhANLTPffShcV9I2INbaxIPZvLI&index=28>

Terminologías

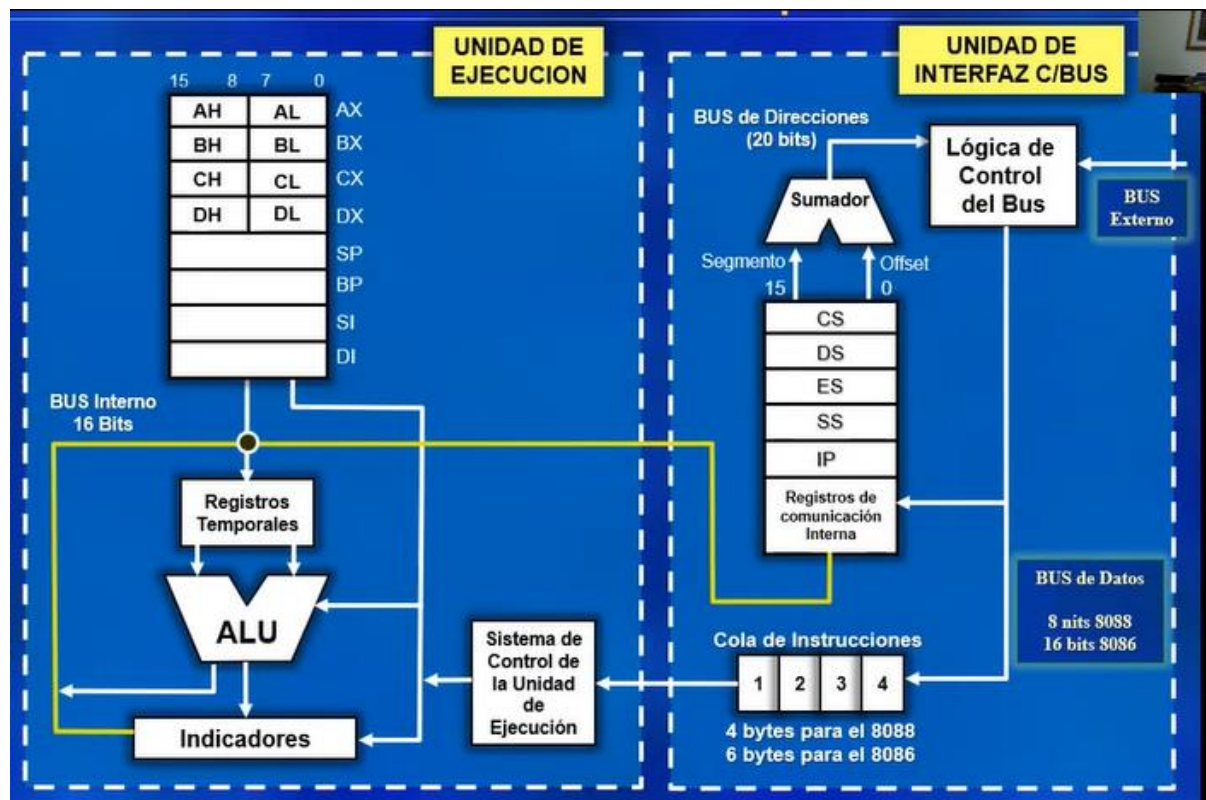
Procesador.	Modo Real.
Set de instrucción.	Modo Protegido.
Lenguaje de Máquina.	Multitareas.
Lenguaje de alto nivel.	Modo Virtual 86
Lenguaje de bajo nivel.	Enlazador (Linker).
Lenguaje Ensamblador.	Programa Fuente ó Código Fuente.
Compilador.	Modulo Objeto.
Interprete.	Modulo Cargable (Load Module ó Run Module).
Compilador Incremental.	Depurador (Debugger).
Pseudo-Código.	
Ensamblador.	
Coprocesador.	

- **Compilador** - Traduce un programa de alto nivel, primero a assembler y después a máquina
- **Intérprete** - Traduce un programa de alto nivel, de 1 línea a la vez, se iba generando el código de máquina
- **Compilador incremental** - Es un híbrido compilador + interprete
- **Ensamblador** - Traduce de un lenguaje de alto nivel a lenguaje de máquina (pasando seguramente por el assembler)
- **Coprocesador** - Es un procesador dedicado a operaciones aritméticas con punto flotante. Servía para aliviar todas las operaciones de alineamiento de mantisa, etc. 486 y Pentium en adelante - ya venían con el coprocesador incluido.
- **Modo Real** - Trabaja en un modo nativo donde no posee multitarea, virtualización, etc
- **Modo Protegido** - Puede tener acceso el procesador a una mayor cantidad de memoria, rompe la barrera de los 64kbytes
- **Modo virtual 86** - Linker, es un programa encargado de traducir de ensamblador a un .exe, para que pueda ser ejecutado dentro de un computador.
- **Modulo Objeto** - Casi a 1 paso de ser ejecutables, tienen acomodada la memoria, ya se pasó por un conjunto de procesos, para poder ser ejecutable. Son almacenados como archivos .obj

Historia de Intel y sus Procesadores 8086/88

- **Noviembre de 1971:** Lanzaron su primer microprocesador: el Intel 4004 para facilitar el diseño de una calculadora. En lugar de tener que diseñar varios circuitos integrados para cada parte de la calculadora, diseñaron uno que según un programa almacenado en memoria podía hacer distintas operaciones.
- **Abril de 1972:** Anunciaron la versión mejorada de su procesador. Se trataba del 8008, y su principal ventaja contra otros modelos, como el Intel 4004 fue poder acceder a más memoria y procesar 8 bits. La velocidad de su reloj alcanzaba los 740KHz.
- **Abril del 1974:** Lanzaron el Intel 8080, su velocidad de reloj alcanzaba los 2 Mhz, permitiendo direccionamiento de 16 bits, un bus de datos de 8 bits y acceso fácil a 64k de memoria.
- **junio de 1978 y 1979:** Fechas en las que respectivamente, hacían su aparición los microprocesadores 8086 y 8088, que pasaron a formar el denominado IBM PC, que vendió millones de unidades de ordenadores.
 - El éxito fue tal, que Intel fue nombrada por "Fortune" como uno de los mejores negocios de los años setenta.
 - De los dos procesadores, el más potente era el 8086, con un bus de 16 bits, velocidades de reloj de 5, 8 y 10 Mhz, 29000 transistores usando la tecnología de 3 micras y hasta un máximo de 1 Mega de memoria direccionable.
 - En cuanto al procesador 8088, era exactamente igual a éste, salvo la diferencia de que poseía un bus de 8 bits en lugar de 16, siendo más barato y obteniendo mejor respaldo en el mercado.
 - Cabe mencionar que el procesador 8086 es un microprocesador tan conocido y probado, que incluso en el 2002 la NASA ha estado adquiriendo microprocesadores 8086 de segunda mano.

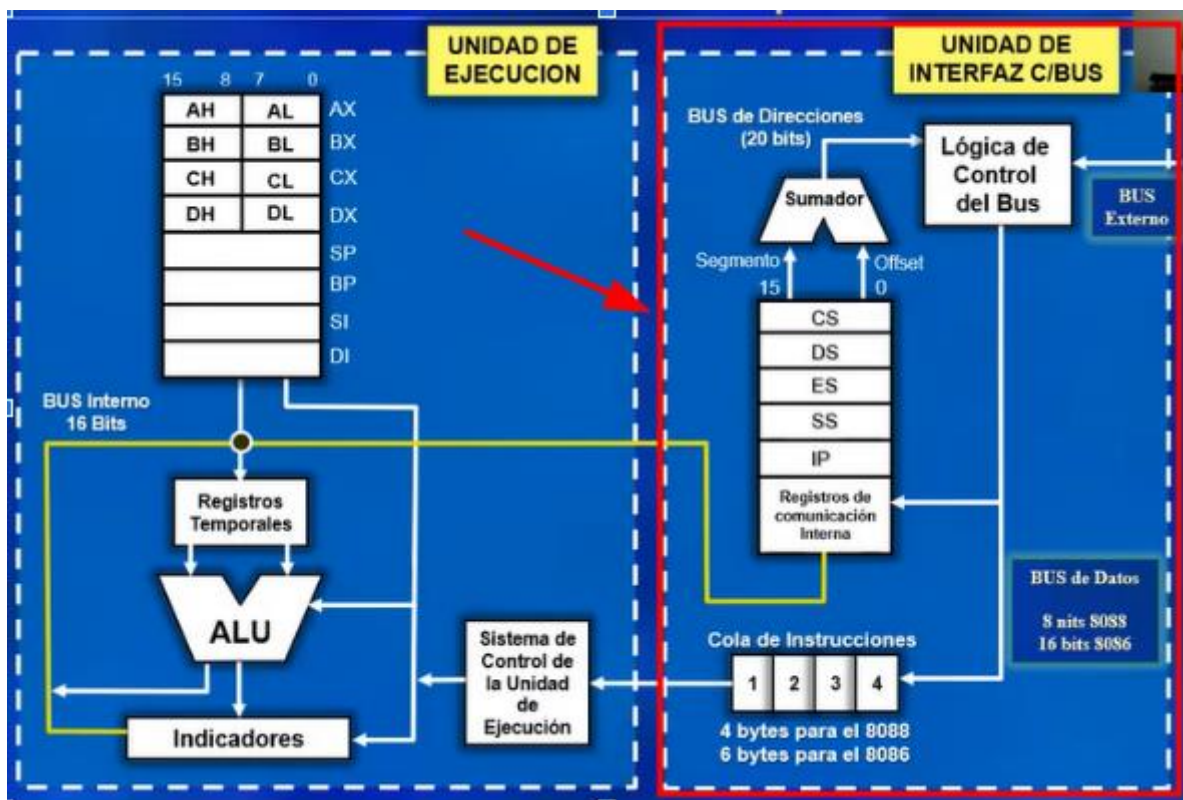
Arquitectura en modo de bloques 8086



Notas generales

Unidad de interfaz c/bus

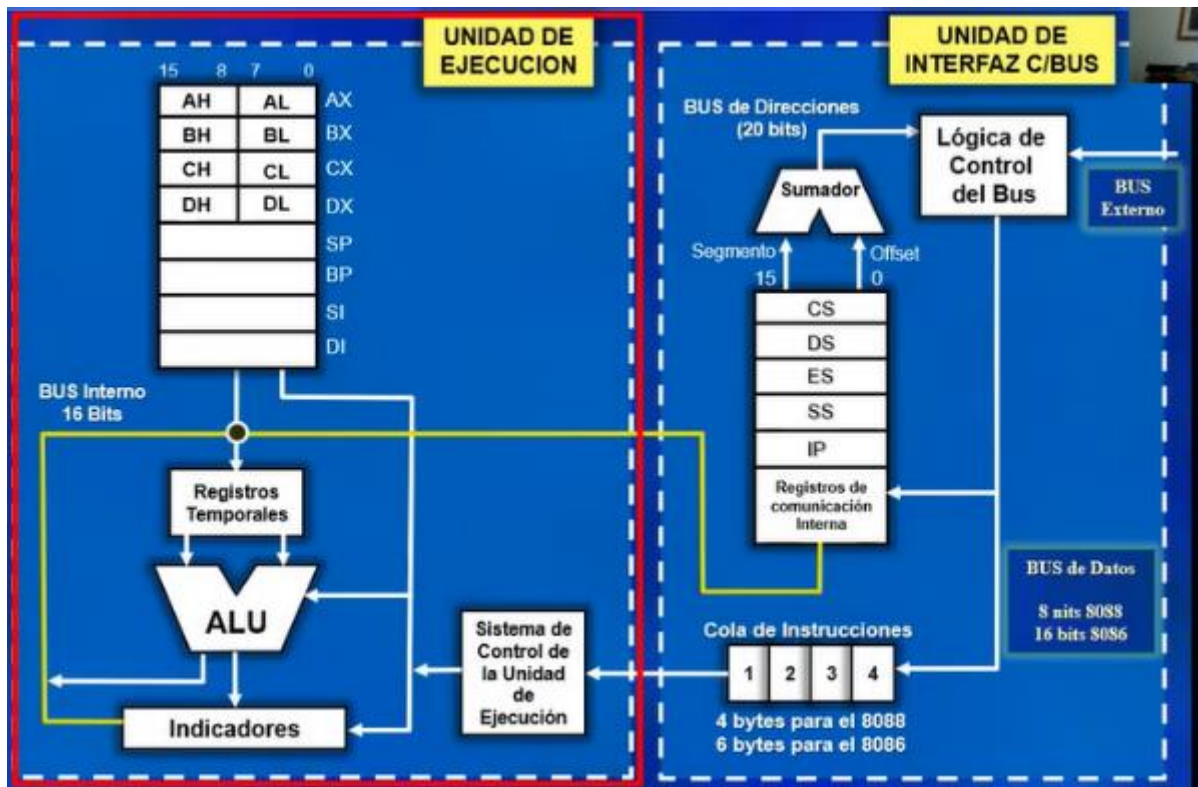
- Dedicada a la cola de instrucciones y a la comunicación con la memoria central para llevar y traer operandos dentro del ciclo de instrucción
- Lógica de control del bus: dedicado a conectarse con la memoria
- Sumador: para obtener la dirección efectiva
- Registros de Direcciones de segmentos



Conjunto de registros de direcciones de segmentos:

- doble palabra, dedicados a direccionar la ubicación de los segmentos de memoria
- SS - Segmento de pila
- Registros dedicados a la cola de instrucciones: 6 palabras para el 8086, 4 para el 88, almacena las instrucciones y las va alistando con ciclo fetch
- El ciclo Fetch se encarga de toda la búsqueda de la instrucción y el tratamiento.
- IP - Contador de programas
- El 8086, $2^{16} = 65536$, 64Kbytes es lo máximo que puede llegar a direccionar este tipo de arquitectura.

Unidad de ejecución



Flags: estado de la ALU

Ciclo fetch o de búsqueda

Encargado de la operatoria de la búsqueda de las instrucciones y el tratamiento

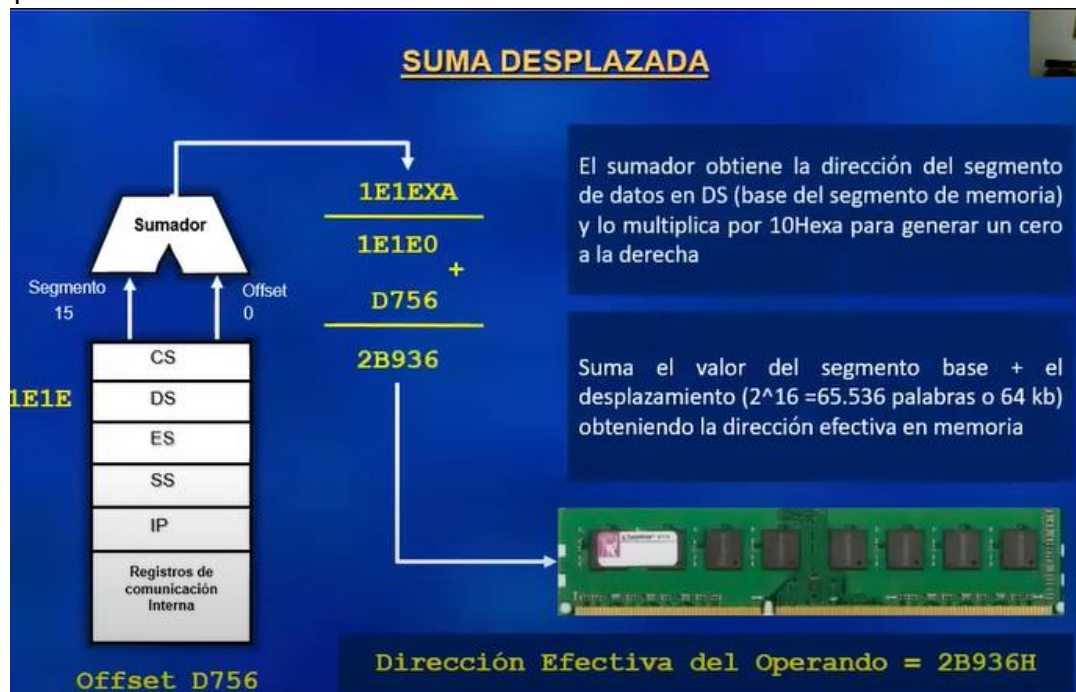
Qué es la técnica de pipeline?

Tratar de hacer operaciones simultáneamente

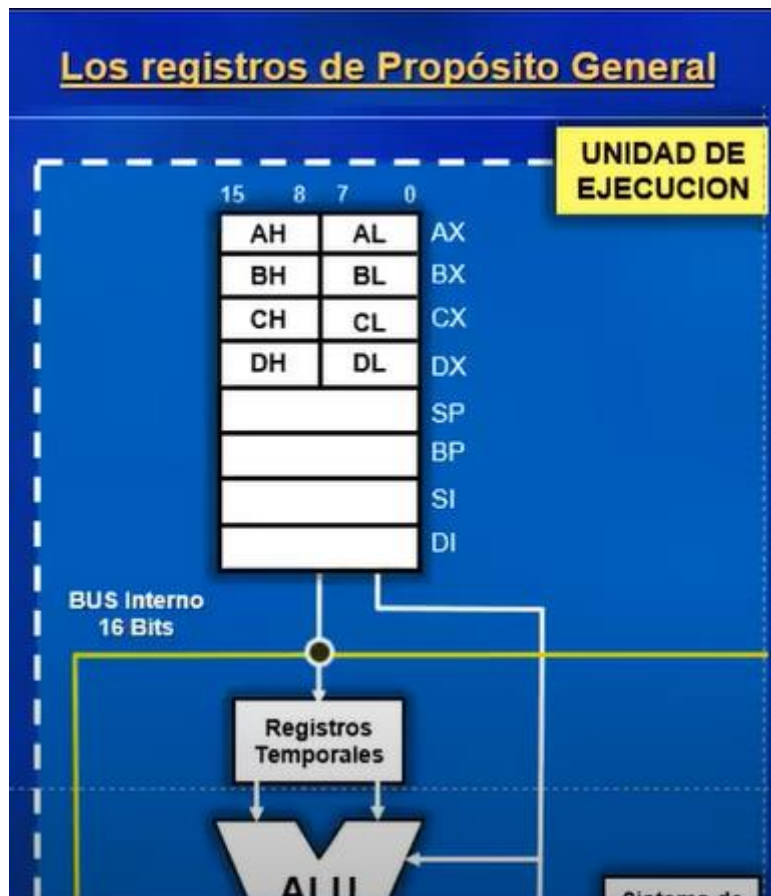
Ciclo FETCH (búsqueda) de la arquitectura 8086

- 1) La instrucción ingresa a la Lógica de Control del Bus o Controladora del Bus, encargado de comunicar al procesador con el exterior, en este caso con la Memoria RAM. Recibe todas las instrucciones del Bus externo y las envía a la Cola de Instrucciones
- 2) La cola de Instrucciones es una estructura FIFO de 6 palabras encargada de ir almacenando en orden de llegada cada instrucción de la memoria central (conocido como ciclo prefetch).
- 3) Sistema de control de la unidad de la UE se encarga de ir sacando las instrucciones de la cola, decodificarlas y ejecutarlas.
- 4) Las operaciones de la ALU generan una serie de eventos, registrados en el Registro Indicadores de eventos o bits bandera (resultado cero, negativo, acarreo, etc), que almacenan 1 si se produce el evento o cero si no se produce.
- 5) Para realizar las Operaciones Aritméticas, la ALU utiliza a los Registros de Propósito General, lo que almacenan los valores temporales que la instrucción necesita.
- 6) Si la instrucción requiere acceso a operando en memoria, pasan al entorno del Sumador, el que obtiene la dirección efectiva del operando sumando el valor del registro DS (Segmento de Datos) mas el valor de desplazamiento

Ejemplo:



Registros de propósito General



Unidad de ejecución: se encarga de decodificar las instrucciones de la cola de instrucción y realiza el tratamiento de cálculo relacionado con la unidad aritmético lógica. Vincula la instrucción con un conjunto de registros conocidos como registros de propósito general. Estos son 4 y están vinculados a la ALU.

Registro AX es el acumulador principal, utilizado para operaciones que implican entrada/salida y la mayor parte de la aritmética. Por ejemplo, las instrucciones para multiplicar, dividir y traducir suponen el uso del AX. Algunas operaciones generan código más eficiente si se refieren al AX en lugar de a los otros registros.

CH CL CX

Registro BX conocido como el registro base ya que es el único de propósito general que puede ser índice para direccionamiento indexado y emplearse para cálculos. Se utiliza para almacenar el offset de una estructura de datos (normalmente indica el número de posiciones de memoria sumadas a una dirección base para conseguir una dirección absoluta específica)

DI

Registro CX conocido como el registro contador. Usado para cálculos, pero su función principal es la de controlar la cantidad de ciclos de una estructura loop.

Registro DX conocido como el registro de datos. Algunas operaciones de entrada/salida requieren su uso, y las operaciones de multiplicación y división con cifras grandes suponen al DX y al AX trabajando juntos.

Ejemplo del uso de registros

```
ORG 100h

tabla DB 2,9,5,12,45,33,99,67,3,1
fin_tabla DB ?

mov BX, OFFSET tabla           ; Carga en BX direccion de Tabla
mov CL, OFFSET fin_tabla - OFFSET tabla ; Carga en Cl la canidad de elementos

bucle:
    mov AL, [BX]                ; [BX] contenido direccion de BX
    ;
    ;Aqui va el código
    ;
    Loop bucle

hlt
```

Ejemplo del uso de registros "Acumular valores del Arreglo en Dx"

```
ORG 100h ; Directiva requerida para 1 segmento simple programa .COM

tabla DB 2,9,5,1,4,3,7,6,3,1 ; Total=41
fin_tabla DB ?

mov BX, OFFSET tabla           ; Carga en BX direccion de Tabla
mov CL, OFFSET fin_tabla - OFFSET tabla ; Carga en Cl la canidad de elementos

bucle:

    Add Dx, tabla[Bx]

    Inc Bx

    Loop bucle

hlt
```

Registros de Direcciones de Datos (Apuntadores)



SP, BP, SI, DI son registros que tienen un funcionamiento orientado al manejo de índices, de inicio y finalización de bloques de datos

Los registros de Direcciones de Datos (Apuntadores)

Registro SP (Puntero de Pila): Aunque es un registro de uso general, debe utilizarse sólo como puntero de pila, la cual sirve para almacenar las direcciones de retorno de subrutinas y los datos temporarios (mediante las instrucciones PUSH y POP). Al introducir (push) un valor en la pila a este registro se le resta dos, mientras que al extraer (pop) un valor de la pila este a registro se le suma dos.

Registro BP (Puntero de Base): Generalmente se utiliza para realizar direccionamiento indirecto dentro de la pila

Registro SI o Índice Fuente: Sirve como puntero fuente para las operaciones con cadenas. También sirve para realizar direccionamiento indirecto. Lleva la dirección donde comienza los datos que se quieren leer dentro de un segmento de datos.

Registro DI o Índice Destino: Sirve como puntero destino para las operaciones con cadenas. También sirve para realizar direccionamiento indirecto. Lleva la dirección donde terminan los datos que se quieren leer dentro de un segmento de datos.

SP -

Es un registro de uso general, solamente se lo debe utilizar para apuntar a una pila. Una pila sirve para saber las direcciones de retorno de una subrutina, y los datos temporales que están en la subrutina.

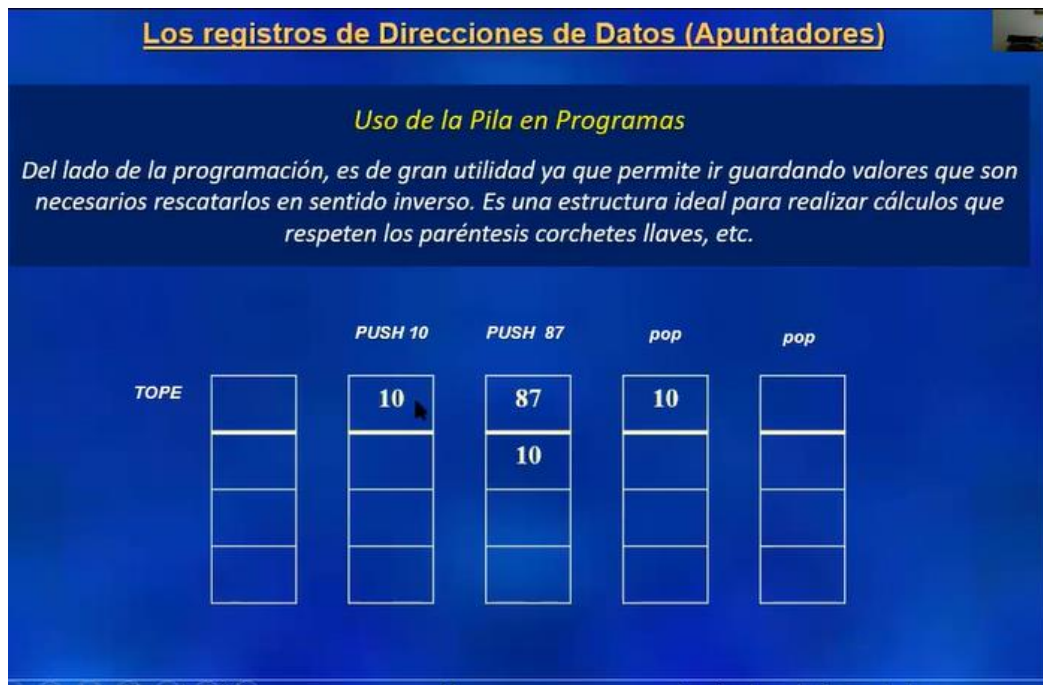
Las va apilando, las va anidando, entonces cada rutina llamada por otra, se van almacenando una arriba de la otra.

Apilar datos y poder hacer calculos, y poder utilizar los datos cuando los registros de propósito general no dan abasto.

BP - Direccionamiento indirecto dentro de la Pila

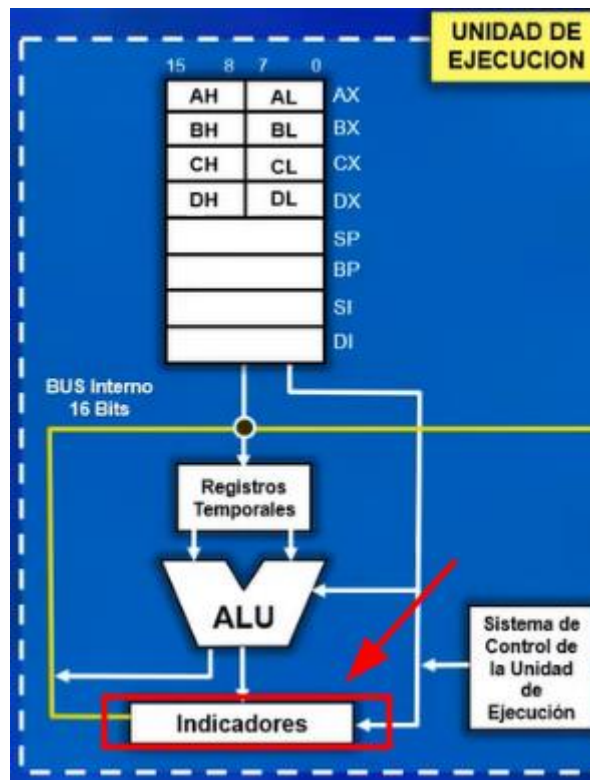
SI (Indice fuente), DI (Indice destino) -

Son utilizados para operaciones tipo cadena, sirven también para direccionamiento indirecto. Se puede direccionar con SI el inicio de una cadena, y DI puede indicar el puntero final de esa cadena de caracteres. Son utilizados específicamente para direccionamiento. Los dos ultimos se utilizan para cadenas/dedicados a direccionamiento



Una pila puede llegar a tener 16kb, pero conviene definirla con el menor tamaño posible para no utilizar demás el segmento de pila, para ahorrar memoria.

Registros Banderas (Flags)



Vinculado a la ALU y al banco de registros de propósito general

Contiene información de estado y de control de las operaciones del microprocesador. De los 16 bits del registro se utilizan solo 9 y cada uno de estos representa a una bandera. Permiten que instrucciones de bifurcación puedan respaldarse en resultados temporales de estos indicadores

Seis banderas de estado que registran el estado del procesador, normalmente asociado a una comparación a o una instrucción aritmética:

- CF - (Carry Flag)** Bandera de acarreo. Indica acarreo en las instrucciones aritméticas.
- OF - (Overflow Flag)** Bandera de desbordamiento aritmético.
- ZF - (Zero Flag)** Bandera de resultado cero o comparación igual.
- SF - (Sign Flag)** Bandera de resultado o comparación negativa.
- PF - (Parity Flag)** Bandera de paridad, utilizada en la verificación de la transferencia de bytes entre dos componentes de la computadora.
- AF - (Auxiliar Flag)** Bandera auxiliar. Indica si hay necesidad de ajuste en las operaciones aritméticas con números BCD.

Contiene información de estado y de control de las operaciones del microprocesador.

				OF	DF	IF	TF	SF	ZF		AF		PF		CF
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00

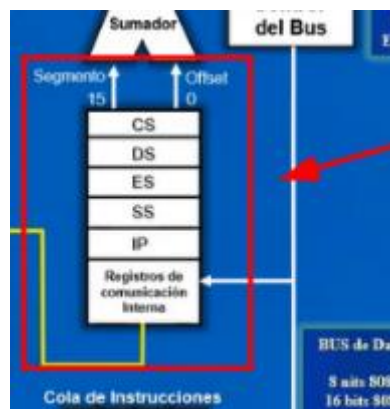
Tres banderas de control que registran el modo de funcionamiento de la computadora.

DF (Direction Flag) Bandera de dirección. Controla la dirección (hacia adelante o hacia atrás) en las operaciones con cadenas de caracteres incrementando o decrementando automáticamente los registros índices (SI y DI).

IF (Interrupt Flag) Bandera de interrupciones. Indica si están disponibles o no las interrupciones de los dispositivos externos.

TF (Trap Flag) Bandera de atrape. Controla la operación modo paso a paso (usada por el programa DEBUG).

Registros de direcciones de segmentos



La unidad de interfaz con el bus se encarga de comunicar al procesador con la memoria central con el objeto de llevar instrucciones y operandos dentro del ciclo de instrucción. 64kb es lo máximo que puede direccionar

Es el proceso en el que la memoria principal de la computadora se divide lógicamente en diferentes segmentos y cada segmento tiene su propia dirección base.

Básicamente se utiliza para mejorar la velocidad de ejecución del sistema informático, de modo que el procesador pueda recuperar y ejecutar los datos de la memoria de forma fácil y rápida.

La unidad de interfaz de bus (BIU) contiene cuatro registros de propósito especial de 16 bits denominados registros de segmento:

Registro de segmento de código (CS): se utiliza para direccionar la ubicación de la memoria en el segmento de código de la memoria, donde se almacena el programa ejecutable.

Registro de segmento de datos (DS): apunta al segmento de datos de la memoria donde se almacenan los datos.

Registro de segmento Extra (ES): también se refiere a un segmento en la memoria de la zona alta disponible para almacenar información.

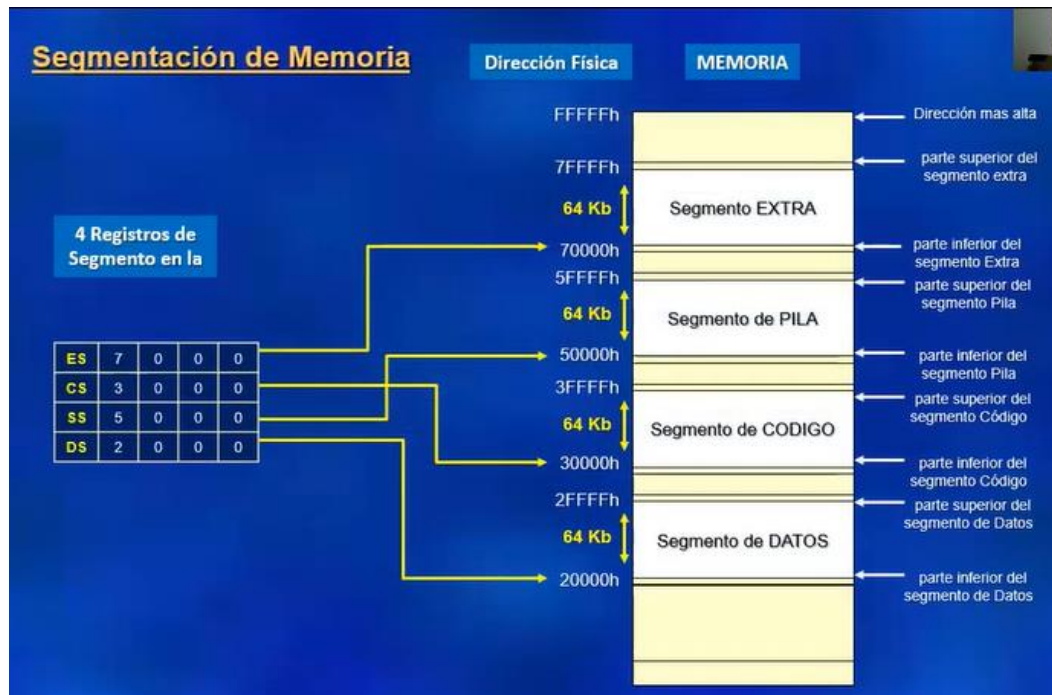
Registro de segmento de pila (SS): se utiliza para direccionar el segmento de pila de la memoria. El segmento de pila es el segmento de memoria que se utiliza para almacenar datos de pila.

Doble palabra → 16 bits. Son regs dedicados a direccionar la ubicación de los segmentos de memoria.

Puntero de Instrucciones



El apuntador de instrucciones IP de propósito especial especifica la siguiente instrucción de código máquina a ejecutarse, relativo al segmento localizado por CS es decir: [CS:IP].. Raramente se referirá directamente a ip. En lugar de eso, se usarán instrucciones para cambiar ip para alterar la localización de la siguiente instrucción a ejecutarse, así cambiando el flujo del programa. Puntero de instrucción (Instruction pointer).



IP solo va a tomar valores del segmento de código.

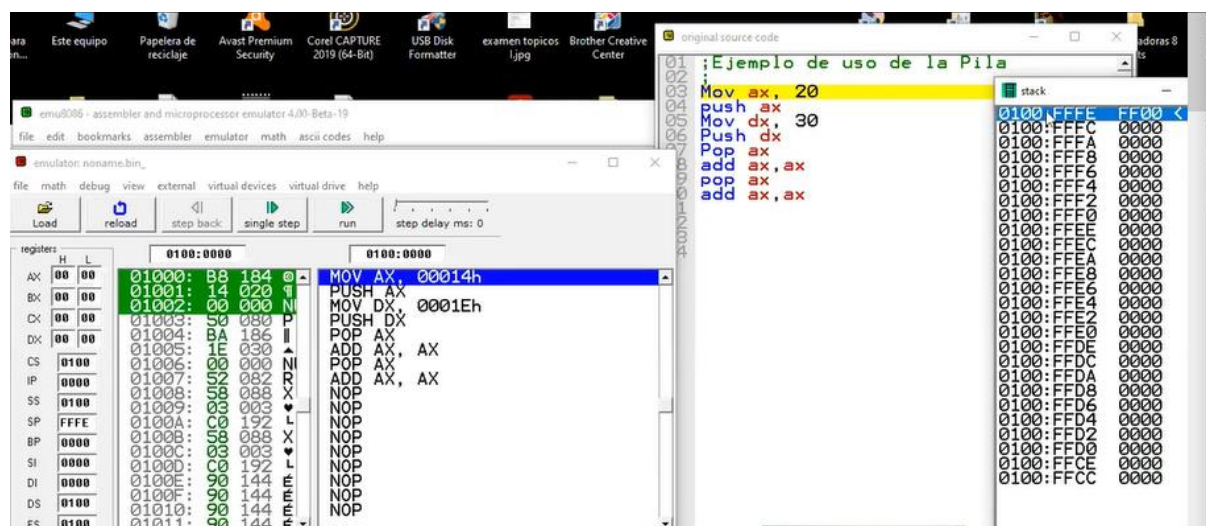
Ejemplo de uso de la pila:

```

emu8086 - assembler and microprocessor emulator 4.00-Beta-19
file edit bookmarks assembler emulator math ascii codes help
new open examples save compile emulate calculator convertor options help About

01 ;Ejemplo de uso de la Pila
02
03 Mov ax, 20
04 push ax
05 Mov dx, 30
06 push dx
07 Pop ax
08 add ax, ax
09 pop ax
10 add ax, ax
11
12

```



The screenshot shows the emu8086 emulator interface. The assembly code window displays the following code:

```

01 ;Ejemplo de uso de la Pila
02
03 Mov ax, 20
04 push ax
05 Mov dx, 30
06 push dx
07 Pop ax
08 add ax, ax
09 pop ax
10 add ax, ax
11
12

```

The registers window shows the following values:

Register	Value
AX	0000
BX	0000
CX	0000
DX	0000
ES	0100
CS	0100
IP	0000
SS	0100
SP	FFFE
BP	0000
SI	0000
DI	0000
DS	0100
ES	0100

The stack window shows the following values:

Address	Value
0100:FFFF	FF00
0100:FFFE	0000
0100:FFFD	0000
0100:FFFC	0000
0100:FFFB	0000
0100:FFFA	0000
0100:FFF9	0000
0100:FFF8	0000
0100:FFF7	0000
0100:FFF6	0000
0100:FFF5	0000
0100:FFF4	0000
0100:FFF3	0000
0100:FFF2	0000
0100:FFF1	0000
0100:FFF0	0000
0100:FFEF	0000
0100:FFEE	0000
0100:FFED	0000
0100:FFEC	0000
0100:FFEB	0000
0100:FFEA	0000
0100:FFE9	0000
0100:FFE8	0000
0100:FFE7	0000
0100:FFE6	0000
0100:FFE5	0000
0100:FFE4	0000
0100:FFE3	0000
0100:FFE2	0000
0100:FFE1	0000
0100:FFE0	0000
0100:FFDF	0000
0100:FFDE	0000
0100:FFDD	0000
0100:FFDC	0000
0100:FFDB	0000
0100:FFDA	0000
0100:FFD9	0000
0100:FFD8	0000
0100:FFD7	0000
0100:FFD6	0000
0100:FFD5	0000
0100:FFD4	0000
0100:FFD3	0000
0100:FFD2	0000
0100:FFD1	0000
0100:FFD0	0000
0100:FFCF	0000
0100:FFCE	0000
0100:FFCD	0000
0100:FFCC	0000

Modos de direccionamiento en 8086

Los modos de direccionamiento indican la manera de obtener los operandos de memoria. El tipo de direccionamiento se determina en función de los operandos de la instrucción, siendo el mas usado MOV destino, origen:

La instrucción MOV realiza transferencia de datos desde un operando origen a un operando destino. Su formato es el siguiente:

MOV destino, origen

- 1-Direccionamiento de registro
- 2-Direccionamiento inmediato
- 3-Direccionamiento directo
- 4-Direccionamiento indirecto mediante registro
- 5-Direccionamiento indirecto por registro base
- 6-Direccionamiento indexado
- 7-Direccionamiento indexado respecto a una base

DIRECCIONAMIENTO DE REGISTRO

Se lo utiliza cuando ambos operando son registros de la arquitectura.

MOV AX,BX ;transfiere el contenido de BX en AX

DIRECCIONAMIENTO INMEDIATO

Cuando el operando origen es una constante.

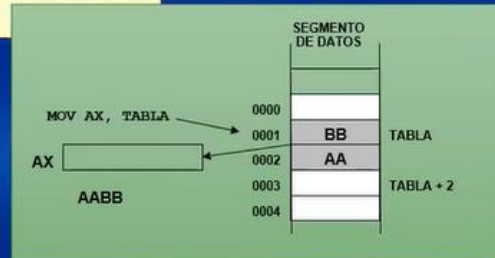
MOV AX,500 ;carga en AX el valor 500

DIRECCIONAMIENTO DIRECTO

Cuando el operando es una dirección de memoria. Ésta puede ser especificada con su valor entre [], o bien mediante una variable definida previamente (cómo definir etiquetas se verá más adelante).

MOV BX, [1000] ; almacena en BX el contenido de la dirección de memoria DS:1000.

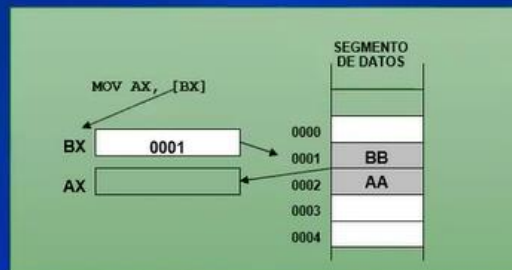
MOV AX, TABLA ; almacena en AX el contenido de la dirección de memoria DS:TABLA



DIRECCIONAMIENTO INDIRECTO MEDIANTE REGISTRO

Cuando el operando esta en memoria en una posición contenida en un registro (BX, BP, SI o DI).

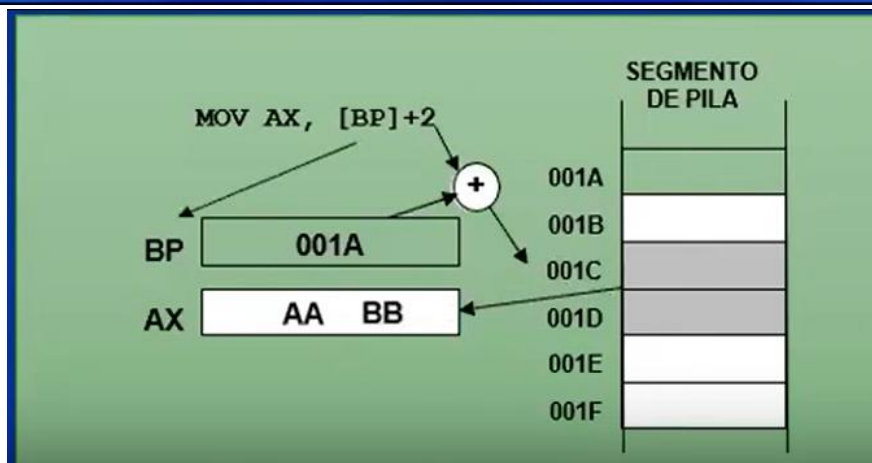
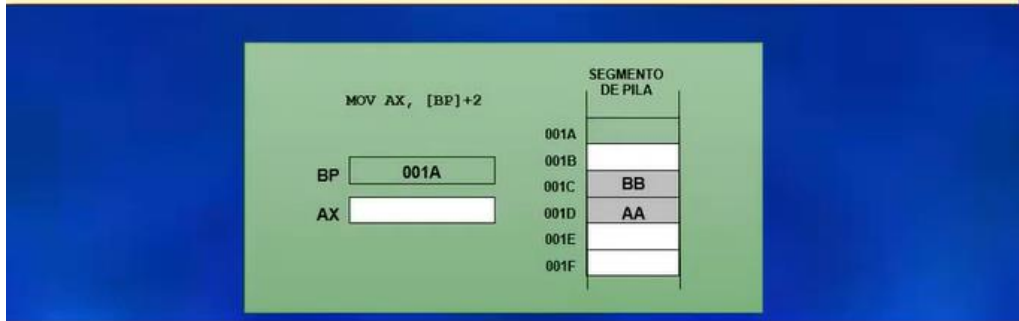
MOV AX,[BX] ; almacena en AX el contenido de la dirección de memoria DS:[BX]



DIRECCIONAMIENTO POR REGISTRO BASE

Cuando el operando esta en memoria en una posición apuntada por el registro BX o BP al que se le añade un determinado desplazamiento, permite acceder ,de una forma cómoda, a estructuras de datos que se encuentran en memoria.

MOV AX, [BP] + 2 ; almacena en AX el contenido de la posición de memoria que resulte de sumar 2 al contenido de BP (dentro de segmento de pila). Equivalente a MOV AX, [BP + 2]

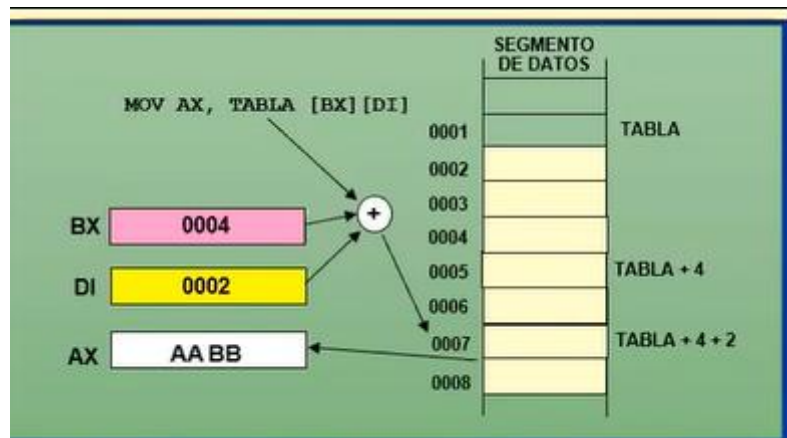
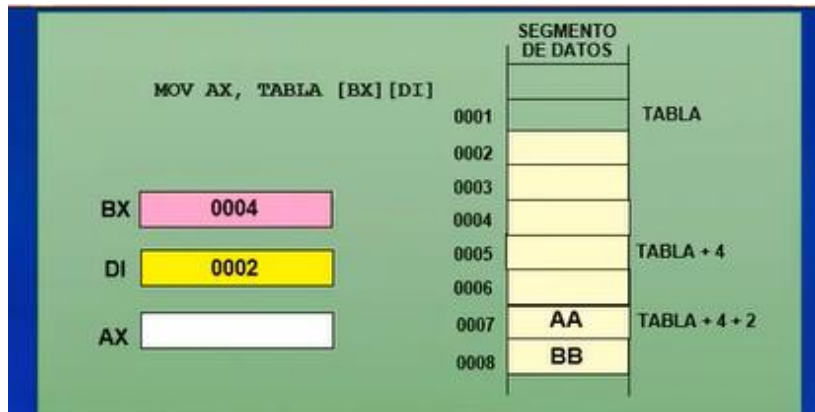


DIRECCIONAMIENTO INDEXADO

Cuando la dirección del operando es obtenida como la suma de un desplazamiento más un índice (DI, SI).

MOV AX, TABLA[DI] ; almacena en AX el contenido de la posición de memoria apuntada por el resultado de sumarle a TABLA el contenido de DI

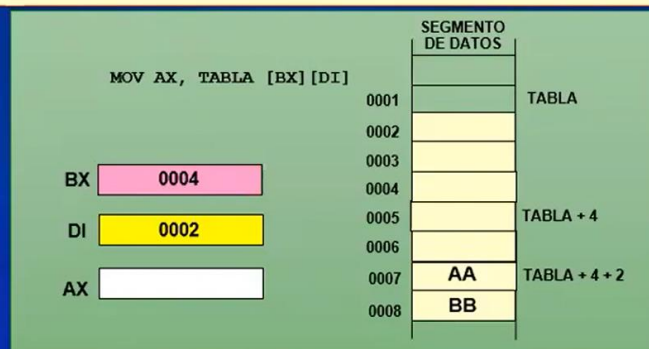




DIRECCIONAMIENTO INDEXADO RESPECTO A UNA BASE

Cuando la dirección del operando se obtiene de la suma de un registro base (BP o BX), de un índice (DI, SI) y opcionalmente un desplazamiento.

MOV AX, TABLA[BX][DI] ; almacena en AX el contenido de la posición de memoria apuntada por la suma de TABLA, el contenido de BX y el contenido de DI



Resumen de los Modos de Direcccionamiento

#	MODO	OPERANDO	REG	EJEMPLOS
1	Registro	Registro	---	MOV AX,BX
2	Valor	Valor	---	MOV AX,500H
3	Directo	Variable	DS	MOV AX,TABLA
4	Indirecto mediante registro	[SI] [DI] [BX] [BP]	DS DS DS SS	MOV AX,[SI] MOV AX,[DI] MOV AX,[BX] MOV AX,[BP]
5	Relativo a base	[BX]+Desp [BP]+Desp	DS SS	MOV AX,VALNUM[BX] MOV AX,[BP]+4
6	Directo indexado	[DI]+Desp [SI]+Desp	DS SS	MOV AX,TABLA[DI] MOV AX,VALNUM[SI]
7	Indexado a base	[BX][SI]+Desp [BX][DI]+Desp [BP][SI]+Desp [BP][DI]+Desp	DS DS SS SS	MOV AX,TABLA[BX][SI] MOV AX,TABLA[BX][DI] MOV AX,TABLA[BP][SI] MOV AX,TABLA[BP][DI]

FELICIDADES, LLEGASTE ☐